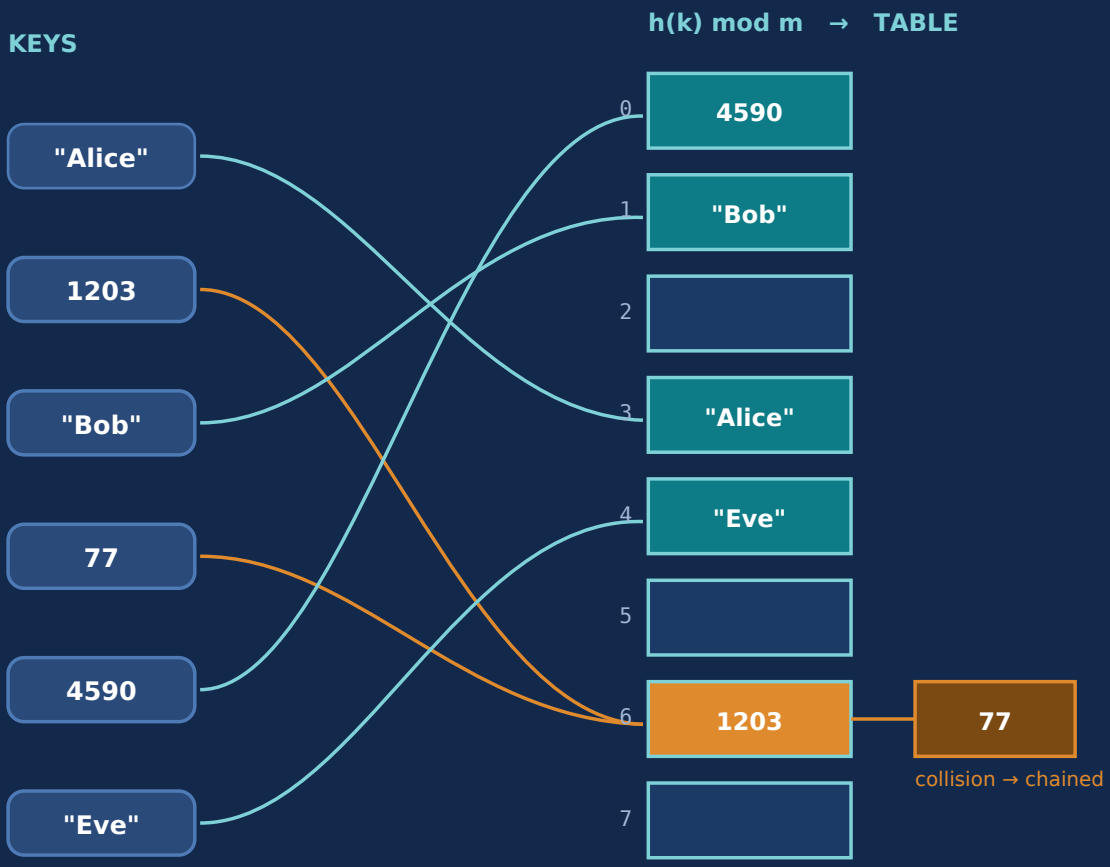


DATA STRUCTURES • C++ • FROM SCRATCH

Hashing in Data Structures

The Complete Chapter — Beginner to Exam-Ready

Hash Functions • Collisions • Separate Chaining • Linear / Quadratic Probing • Double Hashing
Load Factor • Rehashing • Advanced Hashing • C++ STL (unordered_map) • Complexity

7 complete programs (compiled and run) • 69 viva questions • 21 coding questions • 18 parts + revision sheet

All C++ in this book targets ISO C++17 (g++ -std=c++17). Outputs shown were produced by running the code.

Contents

A complete Hashing chapter for an engineering Data Structures course: learn from scratch, then revise for exams, labs and viva.

PART 1 — Introduction to Hashing 8

1.1 What is Hashing? 8
1.2 Why Do We Need Hashing? 8
1.3 Hashing versus Direct Addressing 9
1.4 Dictionaries, Maps, Symbol Tables and Associative Arrays 9
1.5 What is a Hash Table? 10
1.6 Hash Functions: Key, Hash Value and Table Index 10
1.7 How Insertion, Searching and Deletion Work 11
1.8 Average-case versus Worst-case Performance 12
1.9 Real-World Applications of Hashing 12

PART 2 — Hash Table Fundamentals and Internal Representation 13

2.1 Internal Structure of a Hash Table 13
2.2 Array-based Representation, Buckets and Slots 13
2.3 Mapping Keys to Indexes 13
2.4 Table Size and Capacity 14
2.5 Empty, Occupied and Deleted States 14
2.6 Static and Dynamically Resized Hash Tables 14
2.7 Integer Keys, String Keys and Other Key Types 14
2.8 How Hash-table Operations Modify the Table 15
2.9 Advantages, Limitations and Applications 16

PART 3 — Hash Functions: Types, Properties and Design 17

3.1 What Makes a Good Hash Function? 17
3.2 Division (Remainder) Method 17
3.3 Multiplication Method 18
3.4 Mid-Square Method 18
3.5 Folding Method 19
3.6 Digit Extraction (Digit Analysis) 19
3.7 Truncation Method 19
3.8 Polynomial Rolling Hash for Strings 20
3.9 Hashing Strings Character by Character 20
3.10 Hashing Composite Keys 21
3.11 Hashing Negative Integers Safely in C++ 21
3.12 Choosing the Table Size; Prime Sizes 21
3.13 Uniform Distribution, Clustering and Collision Frequency 22
3.14 Deterministic Hashing 22
3.15 Poor Hash Functions and Their Consequences 23
3.16 Hashing Numeric and Non-numeric Keys 23

3.17 Limitations of Simple Hash Functions . . . 23
3.18 A Complete C++ Program for All the Methods . . . 24

PART 4 — Collisions: Causes, Consequences and Fundamentals . . . 27

4.1 What is a Collision? . . . 27
4.2 Why Collisions are Unavoidable . . . 27
4.3 Multiple Keys Mapping to the Same Index: Small Examples . . . 27
4.4 Collision Probability and the Birthday-Paradox Intuition . . . 29
4.5 Collision Frequency and its Effect on Performance . . . 29
4.6 Collision Avoidance versus Collision Resolution . . . 30
4.7 Primary and Secondary Clustering . . . 30
4.8 Collision versus Duplicate Key . . . 31
4.9 A Map of Collision-Resolution Strategies . . . 31
4.10 Collision Handling in Real-world Hash Tables . . . 32

PART 5 — Separate Chaining . . . 33

5.1 Definition and Underlying Idea . . . 33
5.2 Linked-list-based Chaining: Array of Linked Lists . . . 33
5.3 Alternative Bucket Representations . . . 34
5.4 Insertion into a Chain . . . 34
5.5 Searching within a Chain . . . 34
5.6 Deletion from a Chain . . . 35
5.7 Updating an Existing Key and Handling Duplicate Keys . . . 35
5.8 Collision Examples and Complete Dry Run . . . 35
5.9 C++ Implementation Using Nodes and Pointers . . . 37
5.10 Memory Management and Pointer Safety . . . 39
5.11 C++ Implementation Using Standard Library Containers . . . 40
5.12 Effect of Load Factor on Chain Length . . . 41
5.13 Handling an Empty Bucket and a Missing Key . . . 41
5.14 Performance under Heavy Collisions . . . 42
5.15 Advantages and Disadvantages . . . 42
5.16 Time and Space Complexity . . . 42

PART 6 — Open Addressing: Fundamentals . . . 43

6.1 Definition and Working Principle . . . 43
6.2 Searching for an Available Slot and Insertion . . . 43
6.3 Unsuccessful Searches . . . 45
6.4 Deletion Complications: Why EMPTY Cannot Replace DELETED . . . 45
6.5 Empty, Occupied and Deleted Markers; Tombstones . . . 45
6.6 Probe Sequence Termination and Full-table Detection . . . 46
6.7 Load-factor Restrictions . . . 46
6.8 Probe Counts and Performance . . . 46
6.9 Comparison with Separate Chaining . . . 47
6.10 Primary and Secondary Clustering (Recap) . . . 47

PART 7 — Linear Probing 48

7.1 Definition and Formula 48
7.2 Step-by-step Insertion 48
7.3 Searching: Present and Absent Keys 49
7.4 Deletion Using Tombstones 49
7.5 Why Direct Deletion Can Break Searches 50
7.6 Primary Clustering: How Clusters Form and Grow 50
7.7 Successful versus Unsuccessful Search; Load Factor and Performance 51
7.8 Wrap-around and Full-table Conditions 51
7.9 Complete C++ Implementation 52
7.10 Edge Cases 54
7.11 Advantages, Disadvantages and Complexity 54

PART 8 — Quadratic Probing 55

8.1 Definition and Motivation 55
8.2 The Probing Formula 55
8.3 Special Cases, Table Size and Coverage 55
8.4 Step-by-step Probing: Triangular Offsets, $m = 8$ 56
8.5 Worked Insertion, Search and Deletion 57
8.6 Primary versus Secondary Clustering 58
8.7 Why Quadratic Probing May Fail to Visit Every Slot: Experiment B 58
8.8 Handling a Failed Insertion when Slots Remain Empty 58
8.9 Tombstones and Deletion 59
8.10 Complete C++ Implementation 59
8.11 Advantages, Limitations and Comparison with Linear Probing 61

PART 9 — Double Hashing 62

9.1 Definition and Motivation 62
9.2 The Formula: Primary and Secondary Hash Functions 62
9.3 Choosing h_1 and h_2 62
9.4 Why h_2 Must Never Be Zero 63
9.5 Coprimality: Why the Step Size Must Share No Factor with m 63
9.6 Worked Example with Complete Probe Sequences 63
9.7 Insertion, Search and Deletion 65
9.8 Tombstones, Clustering and Table-full Detection 65
9.9 Comparison with Linear and Quadratic Probing 66
9.10 A Poor Secondary Hash Function: Experiment 66
9.11 Complete C++ Implementation 66
9.12 Advantages, Limitations and Complexity 69

PART 10 — Load Factor, Resizing and Rehashing 70

10.1 Load Factor 70
10.2 How Load Factor Affects Collisions and Performance 70

10.3 Choosing a Resize Threshold and Growth Policy . . . 71
10.4 Rehashing versus Merely Copying the Array . . . 71
10.5 Step-by-step Resizing: A Complete Run . . . 71
10.6 Why Old Indexes Generally Cannot Be Reused . . . 73
10.7 Rehashing in Separate Chaining and in Open Addressing . . . 74
10.8 Clearly Distinguishing Collision Resolution from Resize/Rehash . . . 75
10.9 Shrinking a Table . . . 75
10.10 Amortized Complexity of Resizing . . . 75
10.11 Memory Overhead and Resize Costs . . . 76
10.12 Complete C++ Implementation of Automatic Resizing . . . 76

PART 11 — Comparison of Collision-Resolution Methods . . . 79

11.1 Master Comparison Table . . . 79
11.2 A Worked Comparison on the Same Keys . . . 79
11.3 Worst Case: All Keys Collide . . . 81
11.4 When to Choose Each Method . . . 81

PART 12 — Advanced Hashing Techniques . . . 82

12.1 Universal Hashing (core idea) . . . 82
12.2 Randomized Hashing and its Purpose (core idea) . . . 82
12.3 Perfect and Minimal Perfect Hashing (advanced) . . . 83
12.4 Static versus Dynamic Hashing . . . 83
12.5 Extendible Hashing (advanced, disk-based) . . . 83
12.6 Linear Hashing (advanced, disk-based) . . . 85
12.7 Cuckoo Hashing (core idea in modern practice) . . . 85
12.8 Robin Hood Hashing (advanced) . . . 88
12.9 Coalesced Hashing (advanced) . . . 89
12.10 Hopscotch Hashing (advanced) . . . 89
12.11 Consistent Hashing (core idea in distributed systems) . . . 90
12.12 Cryptographic versus Non-cryptographic Hash Functions . . . 91
12.13 Content Hashing and Hash-based Deduplication . . . 91
12.14 Hashing versus Checksums . . . 91
12.15 Hash Tables versus Balanced Search Trees . . . 92
12.16 When Are Advanced Techniques Useful, and When Unnecessary? . . . 92

PART 13 — Hashing in C++: Implementation and Library Usage . . . 93

13.1 Implementing Hashing Manually versus Using the Standard Library . . . 93
13.2 Where Each Hand-written Implementation Is . . . 93
13.3 Keys, Policies and Edge Cases in Hand-written Tables . . . 93
13.4 Exception and Resource Safety . . . 94
13.5 `std::unordered_map` and `std::unordered_set` . . . 94
13.6 `std::hash` . . . 96
13.7 Custom Hash Functions and Custom Key Equality . . . 96
13.8 Hashing Pairs and Composite Keys . . . 98

13.9 Bucket Interface and Load-factor Functions . . . 98
13.10 Iterator Invalidation and Usage Caveats . . . 99
13.11 Common Implementation Mistakes . . . 99
13.12 Debugging Incorrect Hash-table Behaviour . . . 100

PART 14 — Time Complexity, Space Complexity and Performance Analysis . . . 102

14.1 Three Kinds of Cost: Worst, Average and Amortized . . . 102
14.2 Complexity of Each Operation . . . 102
14.3 Separate Chaining: Derivation . . . 103
14.4 Open Addressing: Derivation Sketch . . . 103
14.5 Impact of Load Factor: Theory versus Simulation . . . 103
14.6 Cost of Unsuccessful Searches, Long Probe Sequences and Clustering . . . 104
14.7 Space Complexity . . . 104
14.8 Rehashing Costs and Amortization (Recap) . . . 104
14.9 Adversarial and Pathological Inputs . . . 104
14.10 Comparison with Linear Search, Binary Search and Balanced Trees . . . 105
14.11 Consolidated Complexity Table . . . 105
14.12 Why Average-case $O(1)$ Does Not Mean Worst-case $O(1)$ . . . 105
14.13 Numerical and Exam-style Complexity Questions . . . 105

PART 15 — Complete C++ Programs . . . 107

15.1 Program 1: Basic Hash Table with the Division Method . . . 107
15.2 Program 2: Hash Table with Separate Chaining . . . 111
15.3 Program 3: Hash Table with Linear Probing . . . 114
15.4 Program 4: Hash Table with Quadratic Probing . . . 118
15.5 Program 5: Hash Table with Double Hashing . . . 123
15.6 Program 6: Dynamic Hash Table with Automatic Resizing and Rehashing . . . 126
15.7 Program 7: String Keys: Polynomial Hashing and Word Frequency . . . 130

PART 16 — Important Theory Questions and Viva . . . 133

16.1 Basic Definitions . . . 133
16.2 Hash Functions . . . 133
16.3 Hash-table Representation . . . 134
16.4 Collision Fundamentals . . . 134
16.5 Separate Chaining . . . 134
16.6 Linear Probing . . . 134
16.7 Quadratic Probing . . . 135
16.8 Double Hashing . . . 135
16.9 Load Factor and Rehashing . . . 135
16.10 Complexity Analysis . . . 135
16.11 Advanced Hashing . . . 136
16.12 C++ Standard Library Hashing . . . 136
16.13 Practical Applications and Edge Cases . . . 136

PART 17 — Coding Questions: Easy to Difficult 137

17.1 Easy Questions 137
17.2 Medium Questions 142
17.3 Difficult Questions 150

PART 18 — Numerical, Dry-Run and Case-Study Questions 156

18.1 Worked Problems 156
18.2 Practice Problems with Answers 160
18.3 Case Studies 160

PART 19 — Last-Minute Revision Sheet 162

19.1 Essential Definitions 162
19.2 Important Formulas 162
19.3 Rehashing Workflow 163
19.4 Comparison of Collision-resolution Techniques 163
19.5 Complexity Table 163
19.6 Important C++ Library Functions 164
19.7 Common Mistakes and Exam Traps 164
19.8 Frequently Confused Concepts 164
19.9 High-Value Viva Facts 165
19.10 Syllabus Checklist 165

PART 1

INTRODUCTION TO HASHING

What hashing is, why we need it, and how a hash table works at a high level

1.1 What is Hashing?

Imagine a huge cupboard with 1000 numbered drawers. Instead of searching every drawer for your notebook, you follow a rule: "my notebook always goes into the drawer whose number is the last three digits of my roll number." To find it again you apply the same rule and open exactly one drawer. That rule-based jump to the right place is the heart of hashing.

DEFINITION. **Hashing** is a technique that converts a key (a number, string, or other value) into an array position using a **hash function**, so that the item can be stored and found directly, without searching through all items.

A structure that stores data this way is called a **hash table**. Hashing is one of the most useful ideas in computer science because it lets us insert, search and delete items in **average-case constant time**, written $O(1)$, under assumptions that we will state carefully in later parts.

MEMORY TRICK. Hashing = "compute the address instead of searching for it." A library catalogue number tells you the shelf; you do not read every book spine.

1.2 Why Do We Need Hashing?

Suppose a university stores 1,000,000 student records and must answer the question "does roll number 20417 exist, and what is the student's name?" thousands of times per second. Let us compare the options.

Approach	Search	Insert	Delete	Problem
Unsorted array / list	$O(n)$	$O(1)$ (append)	$O(n)$	Must scan up to n items
Sorted array + binary search	$O(\log n)$	$O(n)$	$O(n)$	Shifting elements on insert/delete
Balanced BST (AVL, red-black)	$O(\log n)$	$O(\log n)$	$O(\log n)$	Pointer-heavy, slower constants
Hash table (average case)	$O(1)$	$O(1)$	$O(1)$	Needs good hash function; no ordering

For n = 1,000,000: a linear scan may need 1,000,000 comparisons, binary search about 20, and a well-designed hash table typically needs one or two. The gap grows as data grows.

Problems with linear searching and plain arrays

- **Linear search** looks at every item in the worst case, so doubling the data doubles the time.
- **Binary search** is fast but requires sorted data. Keeping an array sorted makes every insertion and deletion shift many elements.
- **Plain index access** `a[i]` is instant, but only if you already know the index. A roll number such as 20417 is not a position we were given; we need a rule that produces the position.

1.3 Hashing versus Direct Addressing

The simplest "compute the address" idea is **direct addressing**: use the key itself as the array index. If keys are integers in the range 0–9, an array of size 10 is perfect: key 7 lives in `a[7]`.

Direct addressing breaks down when the **universe of possible keys** is huge but the number of keys actually stored is small. A 10-digit mobile number has 10^{10} possible values; a phone book of 5,000 people would waste an array of ten billion slots. Strings are worse: there are infinitely many possible names.

See Figure 1.1: it contrasts direct addressing, where every possible key owns a slot, with hashing, which squeezes a large key universe into a small table.

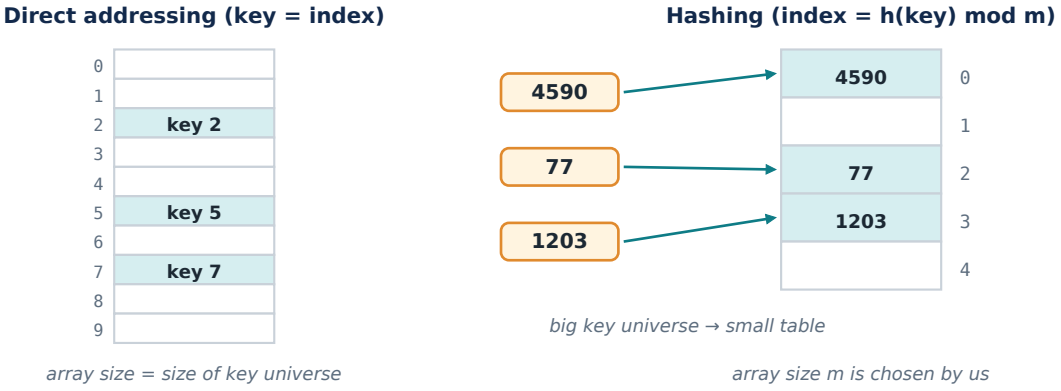


Figure 1.1 — Direct addressing needs one slot per possible key; hashing squeezes a large key universe into a small table of size m.

Hashing solves this by **compressing** the key universe: a hash function turns any key into a small index in the range 0 ... m-1. The price is that two different keys can land on the same index. That event is called a **collision**, and handling it is the main subject of Parts 4–11.

Feature	Direct addressing	Hashing
Index of key k	k itself	$h(k)$ mod m
Array size	Size of the whole key universe	Chosen by us (about the number of keys)
Collisions	Impossible (one slot per key)	Possible, must be handled
Memory	Huge if universe is large	Proportional to stored keys
Key types	Small integers only	Integers, strings, pairs, any hashable type

1.4 Dictionaries, Maps, Symbol Tables and Associative Arrays

Programmers rarely say "hash table" when they describe what they want; they describe the **behaviour**: store a value under a key and get it back later. The abstract data type (ADT) that offers this behaviour has several names.

Name	Where you meet it	Notes
Dictionary	Python `dict`, textbooks	Words (keys) and meanings (values)
Map	C++ `std::map` / `std::unordered_map`, Java `Map`	`std::map` is tree-based and ordered; `unordered_map` is hash-based
Symbol table	Compilers and interpreters	Identifier name → type, address, scope
Associative array	PHP arrays, JavaScript objects, awk	An array indexed by strings or other keys

The ADT supports: `insert(key, value)`, `search(key)` (also called find or lookup), `delete(key)` (erase), and often `update`. A hash table is **one way to implement** this ADT; a balanced tree is another. This book studies the hash-table implementation.

1.5 What is a Hash Table?

A **hash table** is an array of m positions plus a rule (the hash function) that tells us which position a key belongs to. Each stored item is usually a **key–value pair**. The key identifies the item; the value is the data attached to it.

Term	Meaning	Example
Key	The identifier used to find an item	Roll number 20417
Value	The data stored with the key	Student name "Rishi"
Slot / cell	One position of the underlying array, numbered 0 ... m-1	Slot 7
Bucket	A position that may hold one or more items (one in open addressing, a whole list in chaining)	Bucket 7 holding a list
Table size / capacity (m)	Number of slots / buckets	$m = 10$
Number of keys (n)	How many items are currently stored	$n = 4$
Hash function	Rule that converts a key into an index	$h(k) = k \bmod 10$
Collision	Two different keys mapped to the same index	17 and 47 when m=10

NOTE. In this book "slot" means one array cell. "Bucket" means the place a key is mapped to; with separate chaining a bucket is a list that can hold many keys, while with open addressing a bucket holds at most one. Many books use the two words interchangeably, so check the definition in your syllabus.

1.6 Hash Functions: Key, Hash Value and Table Index

A **hash function** h takes a key and produces a number. In practice this happens in two stages, and it is important to keep them apart:

1. **Hash value** (or hash code): a number computed from the key, often large. It does not depend on the table size. For an integer key it might be the key itself; for a string it might be a polynomial sum of the characters.
2. **Index** (or compression): the hash value is brought into the valid range 0 ... m-1, most commonly with the remainder operator.

$$\text{index} = \text{hash value} \bmod m \text{ so that } 0 \le \text{index} \le m - 1$$

See Figure 1.2: it shows the journey from key to slot as a pipeline, with the hash value and the index kept apart.



Figure 1.2 — The pipeline from key to slot. The hash value can be any size; the index must lie in the range 0 to m − 1.

COMMON MISTAKE. Confusing the hash value with the table index. A hash value of 20417 is not an index in a table of size 10. Always reduce it modulo the table size before using it to access the array.

A hash function must be **deterministic**: the same key must always give the same hash value (within one run of the program). Otherwise a key stored today could never be found tomorrow.

1.7 How Insertion, Searching and Deletion Work

We use a table with $m = 10$ slots and the hash function $h(k) = k \bmod 10$ (the last digit). The three operations are:

Operation	Conceptual steps
Insert (k)	1. Compute $i = h(k) \bmod m$. 2. Place the key in slot i (if the slot is free).
Search (k)	1. Compute $i = h(k) \bmod m$. 2. Look in slot i. 3. Found if the key is there; absent if the slot is empty.
Delete (k)	1. Search for the key. 2. If found, remove it from its slot.

Figure 1.3 shows the table after each insertion: keys 17, 24, 31 and 58 go to slots 7, 4, 1 and 8. Now we search.

See Figure 1.3 for the table after each of the four insertions.

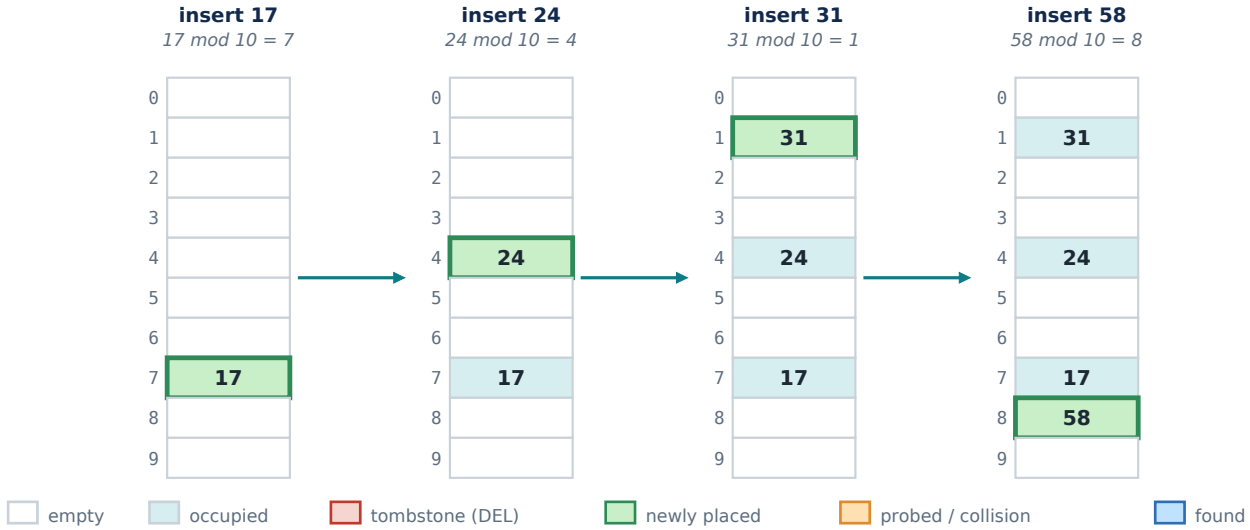


Figure 1.3 — Four insertions into a table of size 10 with h(k) = k mod 10 (new key highlighted in green).

- **search(24):** $24 \bmod 10 = 4$. Slot 4 holds 24, so the key is **found** after looking at one slot.
- **search(40):** $40 \bmod 10 = 0$. Slot 0 is empty, so 40 is **absent**; again just one slot was examined.
- **delete(24):** locate slot 4, then clear it. The table now has keys 17, 31, 58 only.

Notice that no loop ever ran over the whole table: each operation touched one slot. That is the source of the speed. The picture changes when two keys want the same slot. Try inserting 47:

- $47 \bmod 10 = 7$, but slot 7 already holds 17. This is a **collision**.
- The table must decide where 47 goes (or how to store two keys at slot 7). The strategies are studied from Part 5.

> **NOTE.** This section is deliberately simple. It ignores collisions and uses the identity-like hash $h(k)=k$. Real tables need collision handling, good hash functions, and resizing; all are covered in later parts.

1.8 Average-case versus Worst-case Performance

Hash-table speed depends on **how evenly the keys are spread** over the slots.

Case	What happens	Cost per operation
Average / expected	Keys spread out; each slot holds about n/m keys; the load stays bounded by resizing	$O(1)$ (expected, under assumptions)
Worst	All n keys fall into the same slot, so the table degenerates into a list or one long probe sequence	$O(n)$

WARNING. Hash tables do **not** guarantee $O(1)$. The $O(1)$ bound is an **expected** (average) bound that holds when the hash function spreads keys well, and when the load factor is kept under control. A bad hash function or adversarial input can give $O(n)$ operations.

The assumptions behind average-case $O(1)$ are developed in Parts 10 and 14. For now remember the headline: **fast on average, slow only if keys pile up.**

1.9 Real-World Applications of Hashing

Application	How hashing is used
Programming-language runtimes	`dict`, `HashMap`, `unordered_map`, JavaScript objects and sets are hash tables
Compilers and interpreters	Symbol tables map identifiers to types and addresses
Databases	Hash indexes, hash joins and hash partitioning of large tables
Caches and memoization	URL → page, function arguments → result, so repeated work is skipped
Duplicate detection	Insert each item into a hash set; a failed insert means a duplicate
Counting and frequency analysis	Word counts, log analysis, voting tallies
Spell checkers and dictionaries	A hash set of valid words gives instant membership tests
Network routers and firewalls	Fast lookups of addresses and connection records
Version control and storage	Content hashes identify files and commits; deduplication of backups
Load balancing and distributed storage	Hash a request or key to choose a server (consistent hashing, Part 12)
Security	Cryptographic hashes for integrity checks and password storage (not the same as table hashing; see Part 12)

EXAM TRAP. Examiners often ask "give two applications of hashing". Safe answers: symbol table in a compiler and duplicate detection / caching. Do not say "password storage" without adding that it needs a cryptographic (and slow, salted) hash, not a plain table hash.

PART 2

Hash Table Fundamentals and Internal Represent

Array-based storage, slot states, key types, and how the array changes during every operation

2.1 Internal Structure of a Hash Table

Inside, a hash table is nothing more than an **ordinary array** together with a few extra numbers. Understanding the array is the key to understanding every algorithm in this book.

Part	What it is	Typical C++ member
Storage array	The m slots that actually hold items	`std::vector<Entry> table;`
Capacity m	Number of slots in the array	`table.size()` or `int m;`
Count n	How many keys are stored right now	`int n;`
Hash function	Rule that maps a key to an index	`int hash(int key) const;`
Collision policy	What to do when the target slot is taken	chaining, or a probing rule
Resize policy	When and how to grow the array	load-factor threshold

2.2 Array-based Representation, Buckets and Slots

The array positions are numbered $0, 1, \ldots, m\text{-}1$. A position can be thought of in two ways:

- **As a slot:** a single cell that holds at most one entry (used by open addressing).
- **As a bucket:** a container attached to the position that can hold several entries (used by separate chaining, where it is a list).

Each entry is a **key–value pair**. In C++ a pair can be represented by a small structure:

```
struct Entry {
    int         key;     // identifies the record
    std::string value;   // data attached to the key
    // state: needed only for open addressing (see below)
};
std::vector<Entry> table(7);   // m = 7 slots
```

The code above stores one entry per slot. For separate chaining we would instead store `std::vector<std::list<Entry>>`, one list per bucket (Part 5).

2.3 Mapping Keys to Indexes

Mapping a key to an index is a two-step calculation (Part 1): compute a hash value, then reduce it with the remainder operator. For integer keys with the simplest hash function:

$$\text{index}(k) = k \bmod m$$

Example with $m = 7$ and student roll numbers as keys. Because $7 \times 14 = 98$ and $101 - 98 = 3$, we get $101 \bmod 7 = 3$.

Key k	Working	k mod 7 (index)
101	101 = 7 × 14 + 3	3
205	205 = 7 × 29 + 2	2
312	312 = 7 × 44 + 4	4
119	119 = 7 × 17 + 0	0

NOTE. For m=7 the only possible indexes are 0, 1, 2, 3, 4, 5, 6. If a program ever computes index 7 or −1, there is a bug: an out-of-range index in C++ is undefined behaviour.

2.4 Table Size and Capacity

The **capacity** m is the number of slots. It is the most important design parameter after the hash function:

- A table that is **too small** gets crowded: many collisions, long chains, long probe sequences.
- A table that is **too large** wastes memory (and, for large arrays, CPU cache).
- The **load factor** $\alpha = n/m$ measures crowding; Part 10 explains how to choose m and when to grow the table.
- The best choice of m also depends on the hash function. For the division method a **prime** m that is not close to a power of two is a good default (Part 3).

2.5 Empty, Occupied and Deleted States

When a table stores entries in its own array (open addressing) each slot is in exactly one of three states. Separate chaining needs only "bucket empty or not", because deletions simply unlink a node.

State	Meaning	Drawn as	Search behaviour
EMPTY	Never used (or reset by rebuilding the table)	blank cell	Search can **stop**: key cannot be further along its probe path
OCCUPIED	Holds a live key–value pair	cell with the key	Compare keys; if different keep going
DELETED (tombstone)	Held a key that has been removed	cell marked DEL	Search must **continue past it**; insert may reuse it

WARNING. DELETED is not the same as EMPTY. Why this distinction matters is explained in Part 6; for now just remember that there are **three** states, not two.

2.6 Static and Dynamically Resized Hash Tables

Feature	Static (fixed-size) table	Dynamic (resizable) table
Capacity	Chosen once, never changes	Grows (and sometimes shrinks) as n changes
Load factor	Rises without limit as items are added	Kept below a threshold by resizing
Open addressing when full	Insertion fails	Table is rebuilt larger before it fills
Operation cost	Predictable, but degrades if overloaded	Occasional expensive rebuild, cheap on average (amortized)
Use when	Number of keys is known in advance	Number of keys is unknown or varies

2.7 Integer Keys, String Keys and Other Key Types

The table works with array indexes, so every key type must be turned into an integer first. A key type needs two things:

1. A **hash function** that converts the key to an integer (Part 3 shows how for integers, strings, and composite keys).
2. An **equality test**, because two different keys may share a slot and we must be able to tell whether the stored key is the one we want.

Key type	Typical hash value	Notes
`int`	the integer itself	Negative values need care with `%` (Part 3)
`std::string`	polynomial of the characters	Order of characters matters: "ab" ≠ "ba"
`char`	its ASCII/Unicode code	At most 256 values: often direct addressing is better
pair / struct	combination of member hashes	Must combine, not just add (Part 13)
`double`	hash of its bit pattern	Be careful with −0.0 and 0.0, and avoid NaN keys

2.8 How Hash-table Operations Modify the Table

We follow one table, $m = 7$ and $h(k) = k \bmod 7$, through insertions, a search, a deletion, and (in a preview) a resize. The table below shows exactly which slot is touched by each operation. Keys are student roll numbers.

In Figure 2.1 the array of size 7 is followed through four insertions, a search and a deletion.

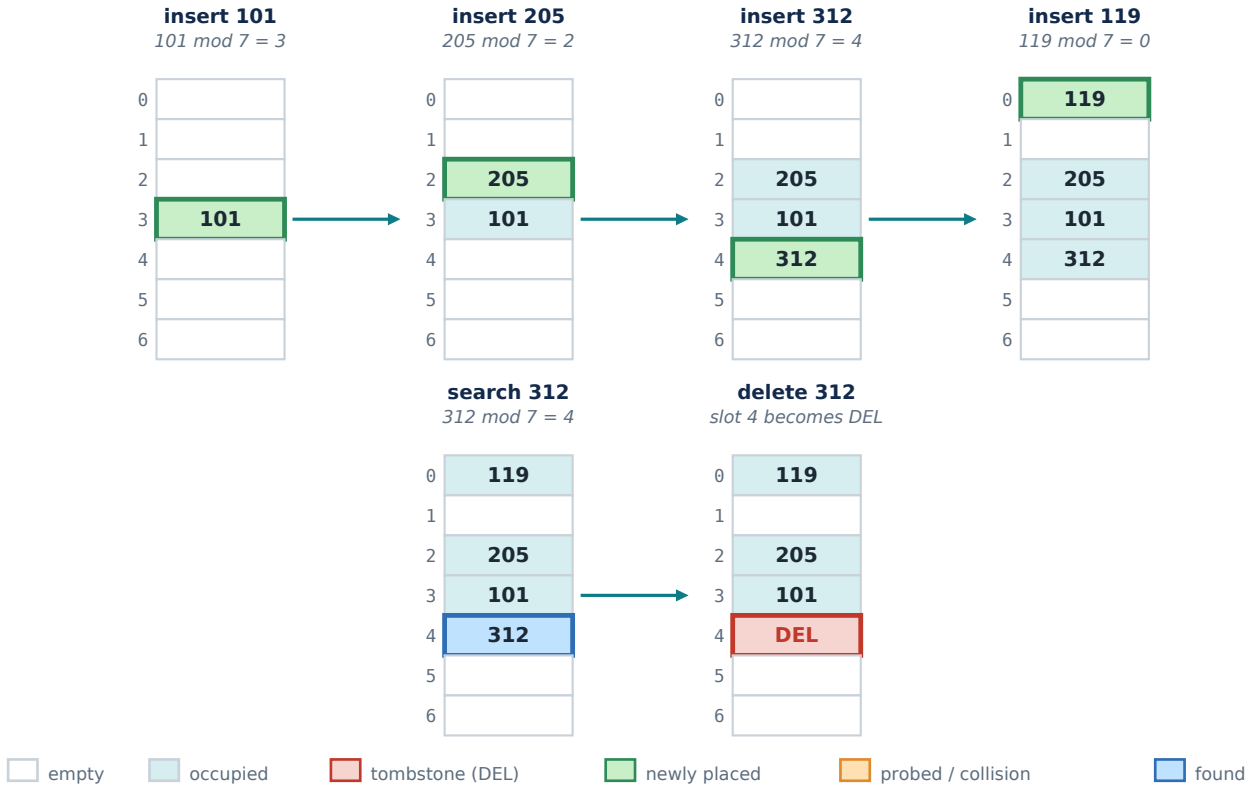


Figure 2.1 — The underlying array of size 7 after each operation. Each operation reads or writes exactly one slot.

Operation	Index computed	What happens to the array	Slots examined
insert 101	3	Slot 3 was empty, so 101 is stored there	1
insert 205	2	Slot 2 stores 205	1
insert 312	4	Slot 4 stores 312	1
insert 119	0	Slot 0 stores 119	1
search 312	4	Slot 4 contains 312: found. The array is unchanged	1
delete 312	4	Slot 4 is changed from OCCUPIED to DELETED	1

COMMON MISTAKE. Declaring a key present just because the slot is **occupied**. If we search for 150 we compute $150 \bmod 7 = 3$, and slot 3 holds 101. The slot is occupied but the key stored there is **not** 150, so 150 is not found there. Always compare the key, never only the occupancy.

How resizing changes the array

When the table becomes crowded we allocate a **new, larger array** and re-insert every key using the **new** size. The array of size 5 below holds 4 keys ($\alpha = 0.8$, crowded). We grow to $m = 11$. Every key gets a new index because m changed.

In Figure 2.2 the same four keys move to new slots when the array grows from 5 to 11 slots.

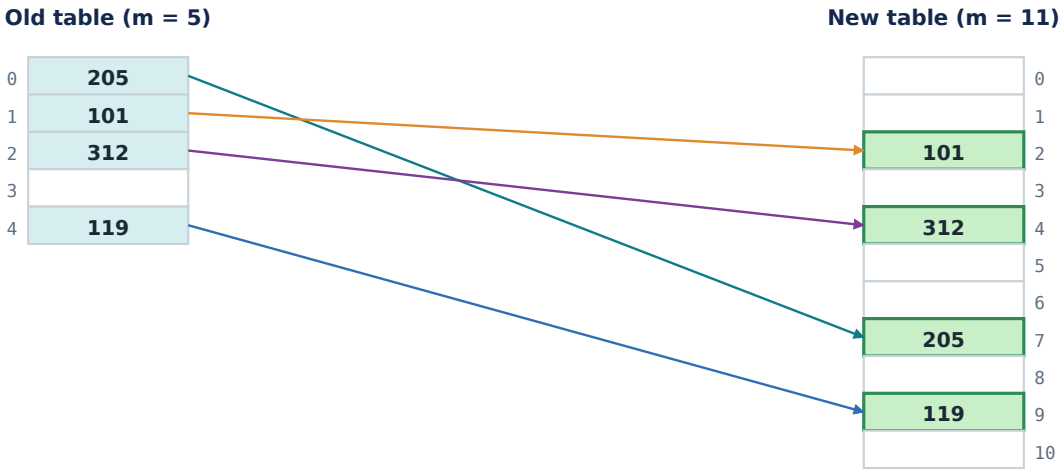


*Figure 2.2 — Resizing from m = 5 to m = 11. Each key is **recomputed** (k mod 11) and placed in the new array; the old indexes are not reused.*

Key	Old index (k mod 5)	New index (k mod 11)
101	1	2
205	0	7
312	2	4
119	4	9

Resizing is covered in detail in Part 10. The important idea for now: the array is replaced, and every key is hashed again.

2.9 Advantages, Limitations and Applications

Advantages	Limitations
Average-case $O(1)$ insert, search and delete	Worst case is $O(n)$ (all keys collide)
Works with keys of almost any type	No built-in ordering: cannot get keys in sorted order cheaply
Simple to implement	Needs a good hash function and collision handling
Excellent for membership tests and counting	Resizing causes occasional expensive rebuilds
Space proportional to the number of keys (plus slack)	Range queries (all keys between a and b) are inefficient
Fast in practice due to few memory accesses	Performance depends on load factor, which must be monitored

Applications include symbol tables, caches, sets and maps in language libraries, de-duplication, routing and database indexes (see Section 1.9). When you also need sorted order, nearest-neighbour queries or guaranteed worst-case bounds, prefer a balanced search tree.

PART 3

Hash Functions: Types, Properties and Design

Division, multiplication, mid-square, folding, digit extraction, truncation, string and composite hashing

3.1 What Makes a Good Hash Function?

A hash function is judged by how well it spreads keys over the slots and how cheaply it can be computed. The following properties are the checklist used throughout this book.

Property	Meaning	Why it matters
Deterministic	Same key always gives the same hash value (within a run)	Otherwise a stored key can never be found again
Uniform	Every slot is about equally likely	Uneven use means long chains / long probes
Fast	Cheap to compute: $O(L)$ for a key of length L	The hash is computed on every operation
Uses the whole key	Every part of the key influences the result	Ignoring parts creates avoidable collisions
Pattern-resistant	Similar keys (1001, 1002, 1003) do not cluster	Real keys are rarely random
Consistent with equality	If $a = b$ then $h(a) = h(b)$	Equal keys must find each other

DEFINITION. The ideal model used for analysis is **simple uniform hashing**: each key is equally likely to hash to any of the m slots, independently of where other keys hash. Real functions only approximate this; the average-case $O(1)$ results of Part 14 assume it.

Hash value versus table index

Every method in this part first produces an integer **hash value** that may be far larger than the table, and only then is the value reduced to an **index** in $0 \ldots m\text{-}1$ (usually with `mod m`). The two numbers are different things; Section 3.8 shows how the same key has one hash value but different indexes for different table sizes.

3.2 Division (Remainder) Method

The most common method for integer keys: take the remainder after dividing by the table size.

$$h(k) = k \bmod m$$

Worked example, $m = 11$:

Key k	Division	Remainder = index
100	100 = 11 × 9 + 1	1
250	250 = 11 × 22 + 8	8
1234	1234 = 11 × 112 + 2	2

- **Advantages:** one instruction, works for any m, easy to analyse.
- **Limitation:** quality depends heavily on m. If m is a power of two ($m = 2^p$) only the lowest p bits of the key are used. If m is a power of 10, only the last digits are used.
- **Recommendation:** choose m as a prime not too close to a power of two (see Section 3.12).

Keys (multiples of 8)	Index with $m = 8$	Index with $m = 7$
16, 24, 32, 40, 48, 56	0, 0, 0, 0, 0, 0	2, 3, 4, 5, 6, 0

With $m = 8$ every key lands in slot 0: a perfect collision disaster. With the prime $m = 7$ the six keys use six different slots.

3.3 Multiplication Method

The multiplication method works well for any table size, including powers of two. Multiply the key by a constant A with $0 < A < 1$, take the **fractional part**, then scale it up to the table size.

$$h(k) = \lfloor\, m \cdot (\, k\,A \bmod 1\,)\,\rfloor \text{ where } A \approx (\sqrt{5}-1)\,/\,2 = 0.6180339887$$

The notation $\mathrm{kA} \bmod 1$ means "the fractional part of kA". The constant 0.618... (the reciprocal of the golden ratio) was suggested by Knuth because it tends to spread consecutive keys evenly. **Worked example**, $m = 10$:

Key k	$k \times A$	Fractional part	$m \times$ fraction	Index (floor)
123	76.0181806	0.0181806	0.18181	0
124	76.6362146	0.6362146	6.36215	6
125	77.2542486	0.2542486	2.54249	2
126	77.8722826	0.8722826	8.72283	8

Consecutive keys 123, 124, 125, 126 are scattered to slots 0, 6, 2, 8; with the division method they would have gone to adjacent slots.

Integer version for $m = 2^p$. Use w-bit unsigned arithmetic: multiply by $\lfloor\, A \cdot 2^w \,\rfloor$ (wrapping modulo 2^w automatically) and keep the top p bits. For $w = 32$ the constant is 2654435769. This avoids floating point completely:

```
std::uint32_t hashMulShift(std::uint32_t k, int p) {
    return (k * 2654435769u) >> (32 - p);     // top p bits of the 32-bit product
}
```

- **Advantage:** the value of m is not critical, so powers of two (cheap to compute) are acceptable.
- **Limitation:** floating-point rounding can differ between machines, and the quality depends on A. The integer version is portable and exact.

3.4 Mid-Square Method

Square the key and extract the **middle digits** (or bits). The middle of a square depends on all digits of the key, which mixes them. For 4-digit keys the square has up to 8 digits and we take the middle two (digits 4 and 5 of the 8-digit square, counting from the left).

$$h(k) = \text{middle } r \text{ digits of } k^2$$

Key	k^2 (8 digits)	Middle 2 digits	Index
4567	20857489	57	57
5732	32855824	55	55
1234	01522756	22	22

Step by step for 4567: $4567^2 = 20{,}857{,}489$; the 8 digits are 2-0-8-**5-7**-4-8-9; the middle two digits are 57, so $h(4567) = 57$ (for a table with at least 100 slots).

- **Advantages:** every digit of the key affects the result; works for keys without obvious patterns.
- **Limitations:** keys with many leading or trailing zeros give squares with many zeros in the middle; the digit count of squares needs care and large keys overflow `int` (use `long long`).

3.5 Folding Method

Split the key into equal-sized pieces, then **add** the pieces. It is used for long keys such as phone numbers, ID numbers and credit-card-like digit strings. Two variants exist:

- **Shift folding:** add the pieces as they are.
- **Boundary folding:** reverse every second piece before adding, which breaks the symmetry between pieces.

$$h(k) = (\text{piece}_1 + \text{piece}_2 + \ldots + \text{piece}_n) \bmod m$$

Worked example. Key 123456789 with pieces of 3 digits: 123, 456, 789; table size $m = 1000$.

Method	Calculation	Sum	Index (mod 1000)
Shift folding	123 + 456 + 789	1368	368
Boundary folding	123 + **654** + 789 (middle piece reversed)	1566	566

NOTE. Folding turns a long key into a small number by combining all digits. Remember the final `mod m`, otherwise the sum (here 1368) can exceed the table size.

3.6 Digit Extraction (Digit Analysis)

If the **set of keys is known in advance** (a static table), study which digit positions vary and use only those. Constant positions carry no information. Example: 8-digit student IDs where the first three digits are a fixed batch code.

ID	Digits 1–3 (constant)	Digits 4, 6, 8	Index
25104127	251	017	17
25104389	251	039	39
25107456	251	046	46

- **Advantages:** very effective when the key pattern is known; can give almost no collisions.
- **Limitation:** if the key set changes, the chosen positions may become useless. Not suitable for general-purpose tables.

3.7 Truncation Method

The simplest method: **discard** part of the key and keep the rest, for example the last three digits. Truncating a 7-digit key to its last 3 digits:

Key	Keep	Index
7234591	last 3 digits = 591	591
5000123	last 3 digits = 123	123

WARNING. Truncation ignores most of the key. If keys share their last digits (all invoice numbers ending in 000) everything collides. It is equivalent to k mod 1000 and shares the weaknesses of a power-of-10 table size.

3.8 Polynomial Rolling Hash for Strings

A string is a sequence of characters, and the **order** matters ("ab" is not "ba"). The standard approach treats the string as a number written in base B using character codes as digits:

$$H(s) = (s_0 B^{n-1} + s_1 B^{n-2} + \ldots + s_{n-1}) \bmod M$$

Computed efficiently with **Horner's rule**: start with 0; for every character do `h = (h * B + c) mod M`. Typical choices are $B = 31$ or 131 and M a large prime such as 1,000,000,007.

Worked example. String "cat" with $B = 31$ (ASCII: c = 99, a = 97, t = 116):

Step	Character	Calculation	h
start			0
1	c (99)	0 × 31 + 99	99
2	a (97)	99 × 31 + 97 = 3069 + 97	3166
3	t (116)	3166 × 31 + 116 = 98146 + 116	98262

The **hash value** is 98262. The **index** depends on the table size: with $m = 11$ it is 98262 mod 11 = 10; with $m = 101$ it is 98262 mod 101 = 90. Because the character order is encoded in the powers of B, "ab" and "ba" differ:

String	Computation	Hash value
ab	97 × 31 + 98	3105
ba	98 × 31 + 97	3135

3.9 Hashing Strings Character by Character

Different loops give different quality. Compare three common choices:

Method	Update rule	Quality
Sum of characters	`h += c`	**Poor**: anagrams collide ("listen" and "silent" both give 655); short strings use only a narrow range of values
Polynomial / Horner	`h = h * B + c`	**Good** for general use; order-sensitive
FNV-1a	`h ^= c; h *= prime`	**Good** non-cryptographic mixing, 64-bit

```
std::uint64_t fnv1a(const std::string& s) {
    std::uint64_t h = 14695981039346656037ULL;        // offset basis
    for (unsigned char c : s) { h ^= c; h *= 1099511628211ULL; }   // FNV prime
    return h;
}
```

COMMON MISTAKE. Declaring the loop variable as plain `char`. On many platforms `char` is signed, so a character with code above 127 becomes **negative** and ruins the hash. Use `unsigned char`.

NOTE. In C++ unsigned arithmetic wraps around modulo 2^{64} by design. **Signed** overflow is undefined behaviour, so hash computations should use unsigned types.

3.10 Hashing Composite Keys

A composite key has several fields, for example `(x, y)` grid coordinates or (name, birth year). The hash must depend on **every** field and on their **order**. Adding the fields is wrong:

Key	Naive $x + y$	$h = 31x + y$, $m = 101$
(1, 2)	3	33
(2, 1)	3 → collision	63

General recipe: compute a hash for each field, then **combine** them with multiplication by a constant and addition (or XOR with shifts, as `boost::hash_combine` does). Section 13.8 shows the C++ standard-library way.

3.11 Hashing Negative Integers Safely in C++

In C++ the result of `%` has the **same sign as the left operand** (since C++11 division truncates toward zero). So `-7 % 5` is **−2**, not 3. Using it directly as an array index is a bug.

k	`k % 5` in C++	Safe: `((k % 5) + 5) % 5`	Mathematical k mod 5
-12	-2	3	3
-7	-2	3	3
-1	-1	4	4
0	0	0	0
12	2	2	2

```
int hashDivision(long long k, int m) {
    return static_cast<int>(((k % m) + m) % m);   // always in [0, m-1]
}
```

COMMON MISTAKE. Writing `abs(k) % m` to fix negative keys. `abs(INT_MIN)` overflows (undefined behaviour), and `k` and `-k` would also collide for no reason. Use the double-modulo formula or convert to an unsigned type first.

3.12 Choosing the Table Size; Prime Sizes

The best table size depends on the hash function, because patterns in the keys interact with m. If every key is a multiple of d, the keys can only reach m / gcd(d, m) different slots under k mod m:

Keys	m	gcd(d, m)	Slots reachable	Indexes produced
multiples of 10 (10...100)	10	10	1	0
multiples of 10 (10...100)	11	1	10	1, 2, 3, 4, 5, 6, 7, 8, 9, 10
multiples of 4 (4...40)	12	4	3	0, 4, 8
multiples of 4 (4...40)	13	1	10	1, 2, 3, 4, 6, 7, 8, 10, 11, 12

Using a **prime** m makes gcd(d, m) = 1 for every $d < m$, so regular patterns in the keys cannot concentrate on few slots. Practical guidelines:

- Pick m so that the expected load factor is about 0.5–0.75 (open addressing) or 1 (chaining). For $n = 1000$ and $\alpha \approx 0.7$, $m \approx 1430$; the next prime is 1433.
- Avoid primes close to powers of two or ten, because numbers of the form $2^p \pm 1$ and $10^p \pm 1$ interact with common key patterns.
- Powers of two are acceptable only when the hash function already mixes all bits well (multiplication method, FNV, library hashes). Library implementations make different choices: the GNU `libstdc++ unordered_map` uses prime bucket counts, while Java's `HashMap` uses powers of two and scrambles the hash bits first. These are implementation details.

- For double hashing (Part 9) a prime m is almost mandatory, so that every step size is coprime with m.

3.13 Uniform Distribution, Clustering and Collision Frequency

To see why a good function matters, hash 16 fruit names into $m = 11$ slots with two functions: (a) the code of the first letter, and (b) the polynomial hash with $B = 31$.

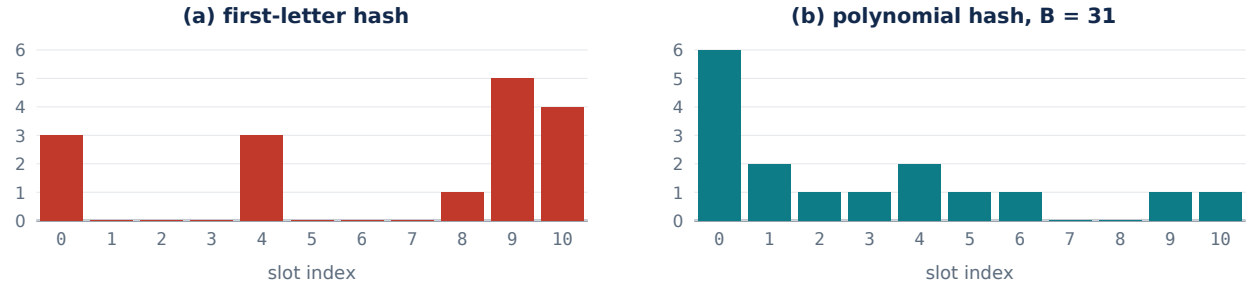


Figure 3.1 — Keys per slot for 16 fruit names, m = 11. Hash (a) uses only 5 slots with up to 5 keys in one slot; hash (b) uses 9 slots with at most 6 keys in a slot.

Measure	(a) first letter	(b) polynomial
Empty slots	6	2
Longest chain (max keys in a slot)	5	6
Keys that share a slot with another key	15	10
Ideal (perfectly even) for 16 keys, 11 slots	max 2, 0 empty	max 2, 0 empty

Even a good function cannot be perfect: when n keys are placed at random into m slots we **expect** about m · e^(−n/m) slots to stay empty. For $n = 16$, $m = 11$ that is about 2.6 empty slots. What we avoid is **systematic clustering**: many keys landing in the same few slots because of a pattern.

- **Clustering:** keys bunch together in some region of the table or in a few slots.
- **Collision frequency:** how often two keys share a slot; for n random keys in m slots the expected number of colliding pairs is $n(n-1)/(2m)$ (Part 4).

3.14 Deterministic Hashing

A hash function used inside a table must be deterministic: **equal keys produce equal hash values every time the function is called during the life of the table**. Things that break this rule:

- Hashing a **pointer address** of an object when equality compares the object contents (two equal strings at different addresses).
- Hashing a mutable key and then **changing the key** while it is stored in the table.
- Using `rand()` or the current time inside the hash function.

> **NOTE.** Determinism is required only within one table's lifetime. Some systems deliberately **randomize** the hash seed for each run (to resist attacks, Section 12.2). C++ guarantees `std::hash` is consistent within a program execution but not across executions, so never store `std::hash` values in files and expect them to match next time.

3.15 Poor Hash Functions and Their Consequences

Poor function	Why it is poor	Consequence
`h(k) = 0` (constant)	No information used	Every key collides: all operations $O(n)$
First character only	Uneven letter frequencies (Figure 3.1a)	Long chains for common letters
Sum of characters	Order ignored; sums of short strings fall in a small range	Anagrams collide; clustering
`k mod 2^p` on patterned keys	Uses only low bits	Keys differing only in high bits collide
`k mod 10` on prices ending in .00	Uses only last digit	Most keys in slot 0
Floating-point `(int)x`	Many values truncate to the same integer	Massive collisions

EXAM TRAP. A very common exam question: "Why is h(k) = k mod m with m = 100 a bad choice when all keys end in 00?" Answer: all keys map to slot 0, because only the last digits are examined; a prime such as 101 fixes it.

3.16 Hashing Numeric and Non-numeric Keys

Key type	Recommended approach
Small integers	Division method with prime m, or multiplication method
Large integers / 64-bit IDs	Mix the bits first (multiply by a large odd constant, XOR-shift), then reduce
Strings	Polynomial rolling hash or FNV-1a over `unsigned char`
Single characters	Direct addressing with an array of 256 entries
Dates (day, month, year)	Composite key; or convert to a single integer day number
Floating point numbers	Hash the bit pattern (`memcpy` to `uint64_t`); treat +0.0 and −0.0 as equal; never use NaN as a key
Pairs / structs	Combine member hashes in order (Part 13)
Enumerations	Cast to the underlying integer type

3.17 Limitations of Simple Hash Functions

- **Collisions are unavoidable** for any function mapping a larger key set into a smaller table (Part 4).
- **Predictability:** simple functions can be reverse-engineered, so an attacker can craft many keys that collide ("hash flooding"), making operations $O(n)$. Remedies: randomized seeds and universal hashing (Part 12).
- **Pattern sensitivity:** division with a bad m, or sum-of-characters, cluster on structured data.
- **Not secure:** table hash functions are not designed to resist deliberate collision search; do not use them for passwords or integrity checks (Section 12.12).
- **Method-specific limits:** mid-square needs big integers for long keys, digit extraction needs a fixed key set, folding ignores digit position inside a piece.

Method	Best for	Main weakness
Division	General integer keys	Poor m ruins it
Multiplication	Any m including 2^p	Needs a good constant A; float rounding
Mid-square	Keys without patterns	Zeros, overflow
Folding	Long digit keys	Position inside a piece is lost
Digit extraction	Fixed, known key set	Breaks if the key set changes
Truncation	Quick, rough hashing	Ignores most of the key
Polynomial / FNV	Strings	Overflow care, not secure

3.18 A Complete C++ Program for All the Methods

The program below implements every method of this part in standard C++17. All outputs quoted in the examples above were produced by this program.

```cpp
// Part 3 companion: classic hash functions in C++17
// Compile: g++ -std=c++17 -Wall -Wextra t03_hashfuncs.cpp -o hashfuncs
#include <cmath>
#include <cstdint>
#include <iostream>
#include <string>

// 1. Division method (safe for negative keys): result is always in [0, m-1]
int hashDivision(long long k, int m) {
    return static_cast<int>(((k % m) + m) % m);
}

// 2. Multiplication method: floor(m * frac(k * A)), A = (sqrt(5) - 1) / 2
int hashMultiplication(unsigned long long k, int m) {
    const double A = 0.6180339887498949;
    double product = static_cast<double>(k) * A;
    double frac = product - std::floor(product);      // fractional part
    return static_cast<int>(m * frac);
}

// 2b. Integer version for m = 2^p: multiply by floor(A * 2^32), keep the top p bits
std::uint32_t hashMulShift(std::uint32_t k, int p) {
    return (k * 2654435769u) >> (32 - p);              // wraps modulo 2^32 on purpose
}

// 3. Mid-square for 4-digit keys: middle two digits of the (up to) 8-digit square
int hashMidSquare(int k) {
    long long sq = static_cast<long long>(k) * k;
    return static_cast<int>((sq / 1000) % 100);
}

// 4. Folding: split the key into 3-digit groups and add them
int reverse3(int g) {                                  // 120 -> 021 -> 21
    return (g % 10) * 100 + ((g / 10) % 10) * 10 + (g / 100);
}
int hashFoldShift(long long k, int m) {
    long long sum = 0;
    while (k > 0) { sum += k % 1000; k /= 1000; }
    return static_cast<int>(sum % m);
}
int hashFoldBoundary(long long k, int m) {
    long long sum = 0;
    bool flip = false;                                 // reverse every second group
    while (k > 0) {
        int g = static_cast<int>(k % 1000);
        sum += flip ? reverse3(g) : g;
        flip = !flip;
        k /= 1000;
    }
    return static_cast<int>(sum % m);
}

// 5. Digit extraction: keep the digits at chosen positions (1 = leftmost)
int hashDigits(const std::string& key, int p1, int p2, int p3) {
    std::string s{key[p1 - 1], key[p2 - 1], key[p3 - 1]};
    return std::stoi(s);
}

// 6. Truncation: keep only the last three digits
int hashTruncate(long long k) { return static_cast<int>(k % 1000); }

// 7. Polynomial rolling hash (Horner's rule): s[0]*B^(n-1) + ... + s[n-1]
std::uint64_t polyHash(const std::string& s, std::uint64_t B, std::uint64_t M) {
    std::uint64_t h = 0;
    for (unsigned char c : s) h = (h * B + c) % M;
    return h;
}

// A deliberately poor hash: sum of the character codes (anagrams collide)
std::uint64_t sumHash(const std::string& s) {
    std::uint64_t h = 0;
    for (unsigned char c : s) h += c;
```

```cpp
    return h;
}

// 8. FNV-1a, a well-known non-cryptographic 64-bit string hash
std::uint64_t fnv1a(const std::string& s) {
    std::uint64_t h = 14695981039346656037ULL;          // offset basis
    for (unsigned char c : s) { h ^= c; h *= 1099511628211ULL; }   // prime
    return h;
}

// 9. Composite keys: combine member hashes, never just add them
int hashPair(int x, int y, int m) {
    long long h = 31LL * x + y;
    return static_cast<int>(((h % m) + m) % m);
}

int main() {
    std::cout << "== Division (m = 11) ==\n";
    for (long long k : {100LL, 250LL, 1234LL, -7LL})
        std::cout << "division(" << k << ") = " << hashDivision(k, 11) << "\n";
    std::cout << "plain C++ -7 % 5 = " << (-7 % 5)
              << ", safe = " << hashDivision(-7, 5) << "\n";

    std::cout << "== Multiplication (m = 10) ==\n";
    for (unsigned long long k : {123ULL, 124ULL, 125ULL, 126ULL})
        std::cout << "mult(" << k << ") = " << hashMultiplication(k, 10) << "\n";
    std::cout << "mulshift(123, p=4) = " << hashMulShift(123, 4) << "\n";

    std::cout << "== Mid-square ==\n";
    for (int k : {4567, 5732, 1234})
        std::cout << "midsquare(" << k << ") = " << hashMidSquare(k) << "\n";

    std::cout << "== Folding (m = 1000) ==\n";
    std::cout << "shift(123456789) = " << hashFoldShift(123456789LL, 1000) << "\n";
    std::cout << "boundary(123456789) = " << hashFoldBoundary(123456789LL, 1000) << "\n";

    std::cout << "== Digit extraction (digits 4, 6, 8) ==\n";
    for (const char* key : {"25104127", "25104389", "25107456"})
        std::cout << "digits(" << key << ") = " << hashDigits(key, 4, 6, 8) << "\n";

    std::cout << "== Truncation (last 3 digits) ==\n";
    std::cout << "truncate(7234591) = " << hashTruncate(7234591LL) << "\n";

    std::cout << "== Strings ==\n";
    std::cout << "poly(cat, 31) = " << polyHash("cat", 31, 1000000007ULL) << "\n";
    std::cout << "poly(ab) = " << polyHash("ab", 31, 1000000007ULL)
              << ", poly(ba) = " << polyHash("ba", 31, 1000000007ULL) << "\n";
    std::cout << "sum(listen) = " << sumHash("listen") << ", sum(silent) = "
              << sumHash("silent") << "\n";
    std::cout << "poly(listen) = " << polyHash("listen", 31, 1000000007ULL)
              << ", poly(silent) = " << polyHash("silent", 31, 1000000007ULL) << "\n";
    std::cout << std::hex << "fnv1a(a) = " << fnv1a("a") << std::dec << "\n";

    std::cout << "== Composite keys (m = 101) ==\n";
    std::cout << "naive (1,2) -> " << (1 + 2) << ", naive (2,1) -> " << (2 + 1) << "\n";
    std::cout << "pair(1,2) = " << hashPair(1, 2, 101) << ", pair(2,1) = "
              << hashPair(2, 1, 101) << "\n";
    return 0;
}
```

Output of t03_hashfuncs.cpp (g++ -std=c++17)

```
== Division (m = 11) ==
division(100) = 1
division(250) = 8
division(1234) = 2
division(-7) = 4
plain C++ -7 % 5 = -2, safe = 3
== Multiplication (m = 10) ==
mult(123) = 0
mult(124) = 6
mult(125) = 2
mult(126) = 8
mulshift(123, p=4) = 0
== Mid-square ==
midsquare(4567) = 57
midsquare(5732) = 55
midsquare(1234) = 22
== Folding (m = 1000) ==
```

```
shift(123456789) = 368
boundary(123456789) = 566
== Digit extraction (digits 4, 6, 8) ==
digits(25104127) = 17
digits(25104389) = 39
digits(25107456) = 46
== Truncation (last 3 digits) ==
truncate(7234591) = 591
== Strings ==
poly(cat, 31) = 98262
poly(ab) = 3105, poly(ba) = 3135
sum(listen) = 655, sum(silent) = 655
poly(listen) = 192458674, poly(silent) = 392640064
fnv1a(a) = af63dc4c8601ec8c
== Composite keys (m = 101) ==
naive (1,2) -> 3, naive (2,1) -> 3
pair(1,2) = 33, pair(2,1) = 63
```

Line	Code	What it does
10	`return static_cast<int>(((k % m) + m) % m);`	Double-modulo trick: `k % m` may be negative; adding `m` makes it non-negative; the second `% m` fixes values that were already positive.
16	`double product = static_cast<double>(k) * A;`	Convert the key to double and multiply by the constant A.
17	`double frac = product - std::floor(product);…`	Fractional part of kA: the number minus its integer part.
23	`return (k * 2654435769u) >> (32 - p); …`	Unsigned 32-bit multiplication wraps modulo 2^{32}; shifting right keeps the top p bits.
29	`return static_cast<int>((sq / 1000) % 100);`	Dividing by 1000 drops the last 3 digits of the square, then `% 100` keeps two digits: the middle of an 8-digit number.
38	`while (k > 0) { sum += k % 1000; k /= 1000; }`	Peels off 3-digit groups from the right with `k % 1000` and `k /= 1000`.
46	`sum += flip ? reverse3(g) : g;`	Boundary folding: every second group is reversed before it is added.
55	`std::string s{key[p1 - 1], key[p2 - 1], key[…`	Builds a 3-character string from the chosen digit positions (1-based from the left).
65	`for (unsigned char c : s) h = (h * B + c) % M;`	Horner's rule: shift previous result by one base-B digit and add the new character. Reducing mod M each step keeps the number small.
79	`for (unsigned char c : s) { h ^= c; h *= 109…`	FNV-1a step: XOR the byte in, then multiply by the FNV prime (wraps modulo 2^{64}).
85	`long long h = 31LL * x + y;`	Composite key: multiplying by 31 makes the pair order-sensitive; sum alone would not be.

PART 4

Collisions: Causes, Consequences and Fundament

Why collisions happen, how likely they are, how they hurt, and a map of all resolution strategies

4.1 What is a Collision?

DEFINITION. A **collision** occurs when two **different** keys $k_1 \neq k_2$ are mapped to the **same table index**: $h(k_1) \bmod m = h(k_2) \bmod m$.

Take $m = 10$ and $h(k) = k \bmod 10$. The keys 17 and 47 are different keys, but both give index 7, so after 17 has been stored in slot 7 the key 47 finds its target slot **already occupied by someone else**.

Key	k mod 10	Slot status when inserted
17	7	empty: stored in slot 7
47	7	**occupied by 17: collision**

4.2 Why Collisions are Unavoidable

The key universe U is usually far larger than the table: there are millions of possible roll numbers but only m slots. By the **pigeonhole principle**, if more than m different keys are possible, at least two of them must share a slot under any hash function.

- Even if we store only $n \leq m$ keys, a specific unlucky set of keys can still collide (the adversary picks keys with equal remainders).
- Making m larger reduces collisions but cannot remove the possibility, and memory is limited.
- The only collision-free situations are **static** key sets with a specially constructed **perfect hash function** (Part 12).

KEY POINT. Because collisions cannot be avoided in general, every practical hash table needs a **collision-resolution strategy**. The quality of that strategy decides the real-world speed of the table.

4.3 Multiple Keys Mapping to the Same Index: Small Examples

Here is a running example used throughout Parts 4–9: a table with $m = 10$ slots, $h(k) = k \bmod 10$, and the key sequence **17, 24, 47, 57, 34**. The hash indexes are:

Key	17	24	47	57	34
Index $h(k)$	7	4	7	7	4

Slot 7 is wanted by three keys (17, 47, 57) and slot 4 by two (24, 34). Table of collisions that occur as the keys arrive in order:

Insert	Index	Slot status	Collision?
17	7	empty	no
24	4	empty	no
47	7	holds 17	**yes**
57	7	holds 17 (and 47 nearby)	**yes**
34	4	holds 24	**yes**

Before and after one collision: exactly what the algorithm does

When inserting 47 the table looks like the **before** panel in Figure 4.1. Slot 7 is occupied by a different key (17). The algorithm must now choose between two designs:

1. **Separate chaining:** keep both keys in slot 7 by attaching a list. 47 is added to the list of bucket 7. No other slot is touched.
2. **Open addressing (linear probing shown):** slot 7 is full, so **probe** the next slot, 8. It is empty, so 47 is stored there. The table now contains the key in a slot that is not its home index.

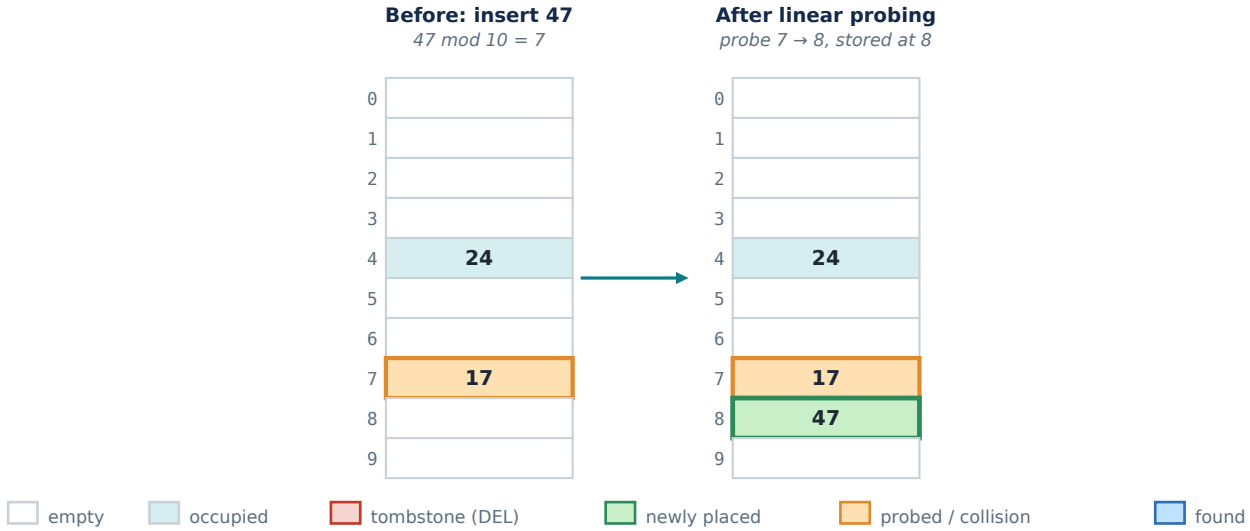


Figure 4.1 — Collision of 47 with 17 under open addressing (linear probing). Orange = slot examined and found occupied; green = where the key is placed.

In Figure 4.2 the same collision is handled by separate chaining: the array slots do not move.

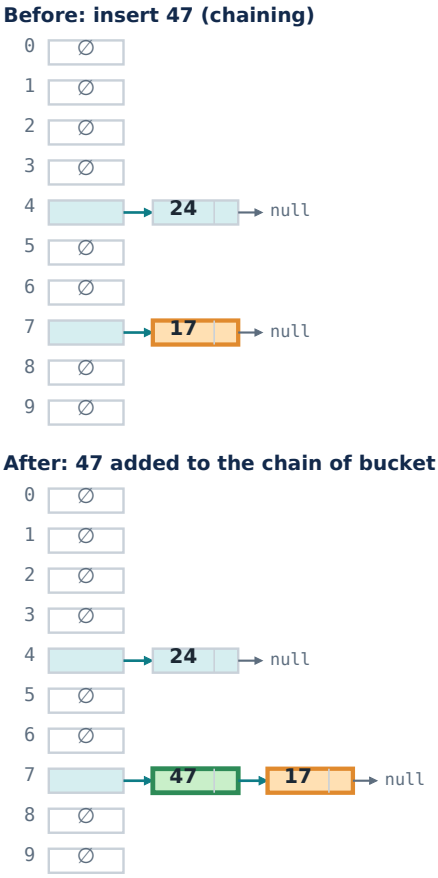


Figure 4.2 — The same collision under separate chaining: the new key is linked into bucket 7 and the array slots do not move.

NOTE. Both strategies keep every key retrievable. The cost is different: chaining pays with extra memory (list nodes) and pointer chasing; open addressing pays with longer probe sequences and more complex deletion.

4.4 Collision Probability and the Birthday-Paradox Intuition

How soon should we expect the first collision? Most people guess "when the table is nearly full." The truth is much sooner, a fact known as the **birthday paradox**: in a group of 23 people the chance that two share a birthday (among 365 days) is already about 51%.

For n keys placed independently and uniformly into m slots:

$$P(\text{no collision}) = (1 - 0/m)(1 - 1/m)(1 - 2/m) \dots (1 - (n-1)/m) \approx e^{-n(n-1)/(2m)}$$

$$\text{expected number of colliding pairs} = n(n-1) / (2m)$$

Table size m	Keys needed for P(collision) ≥ 50%	Approx. $1.18\sqrt{m}$
10	5	3.7
100	13	11.8
1000	38	37.2
10000	119	117.7

Worked example: table of 10 slots, 5 keys. P(no collision) = (10/10)(9/10)(8/10)(7/10)(6/10) = 0.3024, so a collision has probability 0.6976, about 70%, although the table is only half full.

Keys n	Slots m	Load n/m	Expected colliding pairs $n(n-1)/2m$
10	100	0.10	0.45
50	100	0.50	12.25
100	1000	0.10	4.95
1000	1000	1.00	499.5

KEY POINT. Collisions start happening when the number of keys is only about $\sqrt{m}$, not m. Therefore collision handling is a core part of a hash table, never an afterthought.

4.5 Collision Frequency and its Effect on Performance

Operation	Without collisions	With collisions
Insert	Compute index, write to slot	Must find another place (walk a chain or a probe sequence) before writing
Search (found)	Compute index, compare one key	Compare against every key met on the path until the target appears
Search (absent)	Index empty → stop at once	Must follow the **whole** chain/probe path until its end (empty slot or null)
Delete	Clear the slot	First search; then unlink node (chaining) or mark a tombstone (open addressing)

The more collisions, the longer these paths. The **load factor** $\alpha = n/m$ is the main control knob: higher α means more collisions (Part 10). The number of **probes** (slots or nodes examined) is the standard measure of cost.

4.6 Collision Avoidance versus Collision Resolution

	Collision avoidance	Collision resolution
Idea	Make collisions **rare**	Make collisions **harmless**
Tools	Good hash function; larger table; prime m; keeping load factor low; perfect hashing for static sets	Separate chaining; open addressing; cuckoo, Robin Hood, hopscotch, coalesced hashing
Guarantee	Never complete (except perfect hashing)	Always correct, only speed varies
When used	Always, as the first line of defence	Always, as the safety net

Requirements for a collision-handling method

1. **Correctness:** every stored key must remain findable; an absent key must be reported absent.
2. **Determinism:** the same sequence of operations gives the same layout (the probe sequence depends only on the key).
3. **Termination:** an operation must not loop forever (for probing: the sequence must end, and a full table must be detected).
4. **Deletion support:** removing a key must not hide other keys.
5. **Efficiency:** expected cost should stay close to $O(1)$ at reasonable load factors.
6. **Predictable memory use** and a way to grow when load increases (Part 10).

4.7 Primary and Secondary Clustering

Open addressing suffers from **clustering**: occupied slots join into long runs, which makes new keys probe longer and extends the runs further. Two kinds are distinguished:

Kind	Definition	Seen in
Primary clustering	Keys with **different** home indexes join the same contiguous run, because any key that lands in the run walks to its end, making it longer	Linear probing
Secondary clustering	Keys with the **same** home index follow the **same** probe sequence, so they pile up along one path	Quadratic probing (and linear)
(No clustering of either kind, ideally)	Probe sequence depends on the whole key, not only the home index	Double hashing (approximately)

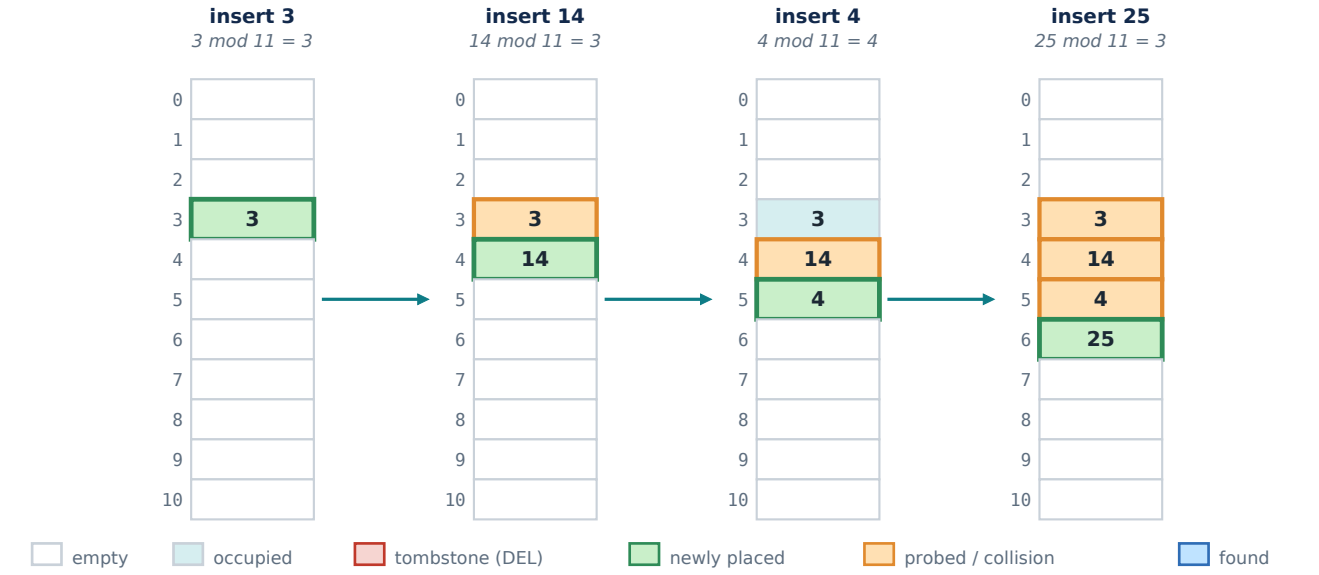


Figure 4.3 — Primary clustering with linear probing, m = 11. Keys 3, 14, 25 share home index 3 and 4 has home 4, yet they all join one growing run of slots 3–6.

In Figure 4.3, key 4 has home index 4 and was not related to the keys at home index 3, yet it ended up inside their run and extended it. A new key arriving anywhere in slots 3–6 must walk to slot 7. Details and analysis follow in Parts 7–9.

4.8 Collision versus Duplicate Key

> **EXAM TRAP.** These two ideas are often confused. **Collision**: two **different** keys, same index. **Duplicate key**: the **same** key inserted again. A collision is a normal event handled by the table; a duplicate is a **policy decision**.

Policy	Behaviour when key 101 is inserted twice with values "Asha" then "Anya"	C++ API
Reject	Keep "Asha"; report failure	`unordered_map::insert` (returns `false` flag)
Update (replace)	Store "Anya"; table size unchanged	`operator[]`, `insert_or_assign` (C++17)
Store multiple values	Keep both entries under key 101	`unordered_multimap`

Whatever policy is chosen, the insertion algorithm must **search for the key first**. In open addressing this means following the probe sequence until an empty slot is found (the key might exist further along) before deciding that the key is new. Forgetting this step creates silent duplicates.

4.9 A Map of Collision-Resolution Strategies

Figure 4.4 shows the strategies. Parts 5–9 study the first two families in depth; Part 12 introduces the advanced schemes.

Figure 4.4 summarises all collision-resolution strategies as a tree; the green leaves are studied in detail in Parts 5–9.

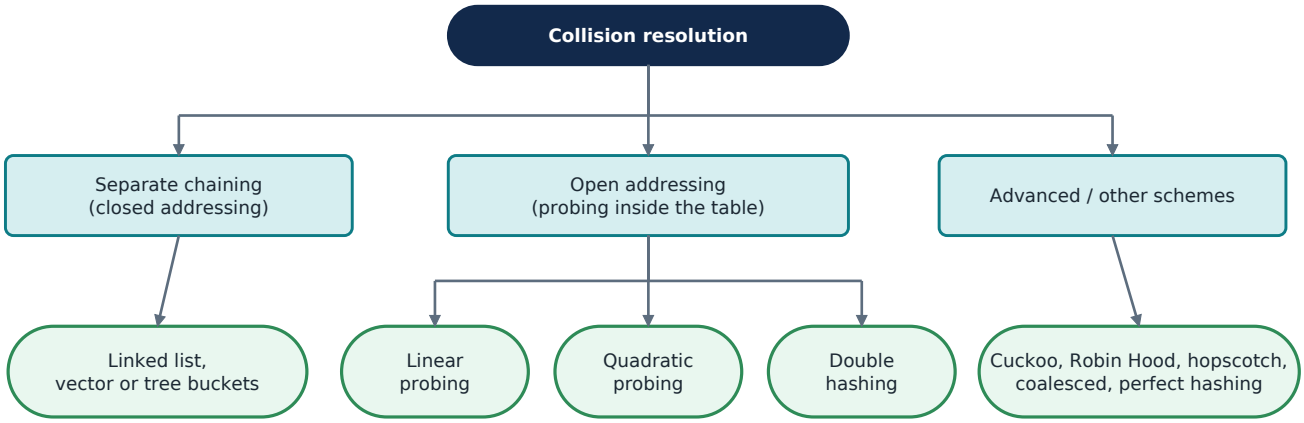


Figure 4.4 — Taxonomy of collision-resolution strategies. The green leaves are the schemes studied in detail in Parts 5–9.

Family	Where does the colliding key go?	Strength	Weakness
Separate chaining	Into a list attached to the home bucket	Simple deletion; tolerates $\alpha > 1$	Extra memory and pointers
Linear probing	Next free slot: home + 1, +2, ...	Cache-friendly, simple	Primary clustering
Quadratic probing	Slots home + offsets that grow quadratically	Less primary clustering	May miss slots; secondary clustering
Double hashing	Steps of size $h_2(k)$ from home	Close to ideal spreading	Two hash functions; poorer cache use

4.10 Collision Handling in Real-world Hash Tables

Library / language	Design (as commonly documented)
C++ `std::unordered_map`	Separate chaining in practice: the standard's bucket interface (`bucket_count`, `begin(n)`) assumes buckets that hold several elements
Java `HashMap`	Separate chaining; since Java 8 a long bucket may be converted to a balanced tree to avoid $O(n)$ chains
Python `dict`	Open addressing with a perturbed probe sequence and a compact entry array
Rust `HashMap`	Open addressing in the "Swiss table" design (SIMD-assisted probing)
Database hash indexes	Buckets/pages with overflow chains; extendible or linear hashing (Part 12)

NOTE. Implementation details change between library versions. The lesson to keep: **both** chaining and open addressing are used in production, and the choice depends on workload, memory behaviour and deletion needs (Part 11).

PART 5

SEPARATE CHAINING

Buckets that hold lists: insertion, search, deletion, complete C++ code and analysis

5.1 Definition and Underlying Idea

DEFINITION. Separate chaining (also *closed addressing* or *open hashing*) resolves collisions by letting every table position hold a **collection of keys**, usually a linked list. All keys that hash to index i are stored together in the chain of bucket i.

The array never overflows: a collision simply makes one chain longer. The table stores **pointers to the heads of lists**, not the keys themselves. Searching therefore has two phases: (1) hash the key to find the bucket in $O(1)$; (2) walk the chain comparing keys.

MEMORY TRICK. Think of lockers in a school corridor where several students share one locker, each with their own labelled box inside. The key tells you which locker; the label tells you which box.

5.2 Linked-list-based Chaining: Array of Linked Lists

Each array element is a pointer to the first node of a singly linked list (or `nullptr` for an empty bucket). A node holds a key, a value and a `next` pointer.

```
struct Node {
    int key;
    std::string value;
    Node* next;           // next node in the same bucket, or nullptr
};
Node** buckets;           // array of m head pointers
```

Figure 5.1 shows the array of lists for the example keys.

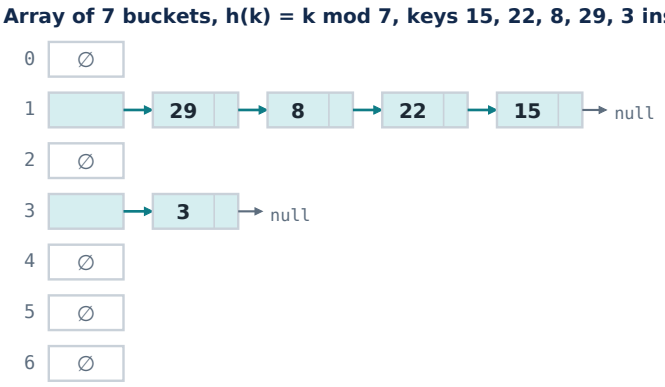


Figure 5.1 — Separate chaining. Bucket 1 holds four keys (15, 22, 8 and 29 all have remainder 1); bucket 3 holds one; the others are empty (∅).

5.3 Alternative Bucket Representations

Bucket type	Insert	Search	Notes
Singly linked list	$O(1)$ at head (after duplicate check)	$O(L)$	Simplest; one pointer per key; poor cache use
Doubly linked list	$O(1)$	$O(L)$	Faster erase when a node pointer is already known; extra pointer
Dynamic array (`std::vector`)	$O(1)$ amortized at end	$O(L)$	Cache-friendly; erase by swapping with last element
Sorted array / list	$O(L)$	$O(\log L)$ for arrays	Early exit on unsuccessful search
Balanced tree (e.g. red-black)	$O(\log L)$	$O(\log L)$	Used by Java `HashMap` for very long chains; needs ordering on keys
Move-to-front list	$O(1)$	frequent keys found sooner	Self-organising heuristic

Here L is the chain length. For the rest of this part we use the singly linked list, the textbook default.

5.4 Insertion into a Chain

Goal: add (k, v) without creating duplicate keys. Algorithm:

1. Compute $i = h(k) \bmod m$.
2. Walk the chain of bucket i. If a node has key k: apply the duplicate policy (here: **update** the value) and stop.
3. Otherwise create a new node and link it at the **head** of the chain: `new Node(k, v, buckets[i])`, then `buckets[i] = node`.
4. Increase the count n.

Insertion at the head is $O(1)$, but the duplicate check scans the whole chain, so an insertion costs $O(L)$ in total, where L is the chain length. Without a duplicate check (a table that allows repeated keys) it is truly $O(1)$.

> **COMMON MISTAKE.** Linking the new node but forgetting to make it the head (`buckets[i] = node`), or overwriting `buckets[i]` **before** reading it (losing the old chain). The constructor call that passes the old head as `next` and the assignment that replaces the head must happen in this order.

5.5 Searching within a Chain

1. Compute $i = h(k) \bmod m$.
2. Set `p = buckets[i]`.
3. While `p != nullptr`: if `p->key == k` return the node; else `p = p->next`.
4. If the loop ends, the key is **absent**.

The cost is the number of nodes compared. A successful search stops at the key; an unsuccessful search must scan the entire chain. An **empty bucket** is the best case for absent keys (zero comparisons).

5.6 Deletion from a Chain

Deletion is where chaining is easiest: unlink the node and free it. Two cases must be handled, because a singly linked list needs the **previous** node:

Case	Action
Node is the **head** (no previous node)	`buckets[i] = p->next;`
Node is in the **middle or end**	`prev->next = p->next;`
Key not found	Change nothing; return `false`

> **WARNING.** Always `delete p` **after** you have saved or used `p->next`. Reading `p->next` after `delete p` is a use-after-free bug (undefined behaviour).

5.7 Updating an Existing Key and Handling Duplicate Keys

Updating is search + assignment: `p->value = newValue`. The insertion routine above already does this, giving an **upsert** (insert-or-update). To implement the other policies from Section 4.8:

Policy	Change to `insert`
Reject	return `false` when the key is found, **without** changing the value
Update	assign the new value (done in our code)
Allow multiple	skip the search and always link a new node; search then returns the first match, and `equal_range`-style code must walk the whole chain

5.8 Collision Examples and Complete Dry Run

Table of $m = 7$ buckets, $h(k) = k \bmod 7$, nodes inserted at the head. Keys with values: 15 Asha, 22 Ravi, 8 Meera, 29 Kabir, 3 Isha, 10 Tara.

Operation	Index	Work done	Collision?
insert 15	1	bucket 1 was empty; node becomes the head	no
insert 22	1	bucket 1 had 1 node(s); new node linked at the head	**yes**
insert 8	1	bucket 1 had 2 node(s); new node linked at the head	**yes**
insert 29	1	bucket 1 had 3 node(s); new node linked at the head	**yes**
insert 3	3	bucket 3 was empty; node becomes the head	no
insert 10	3	bucket 3 had 1 node(s); new node linked at the head	**yes**

In Figure 5.2 the bucket contents are drawn after every insertion.

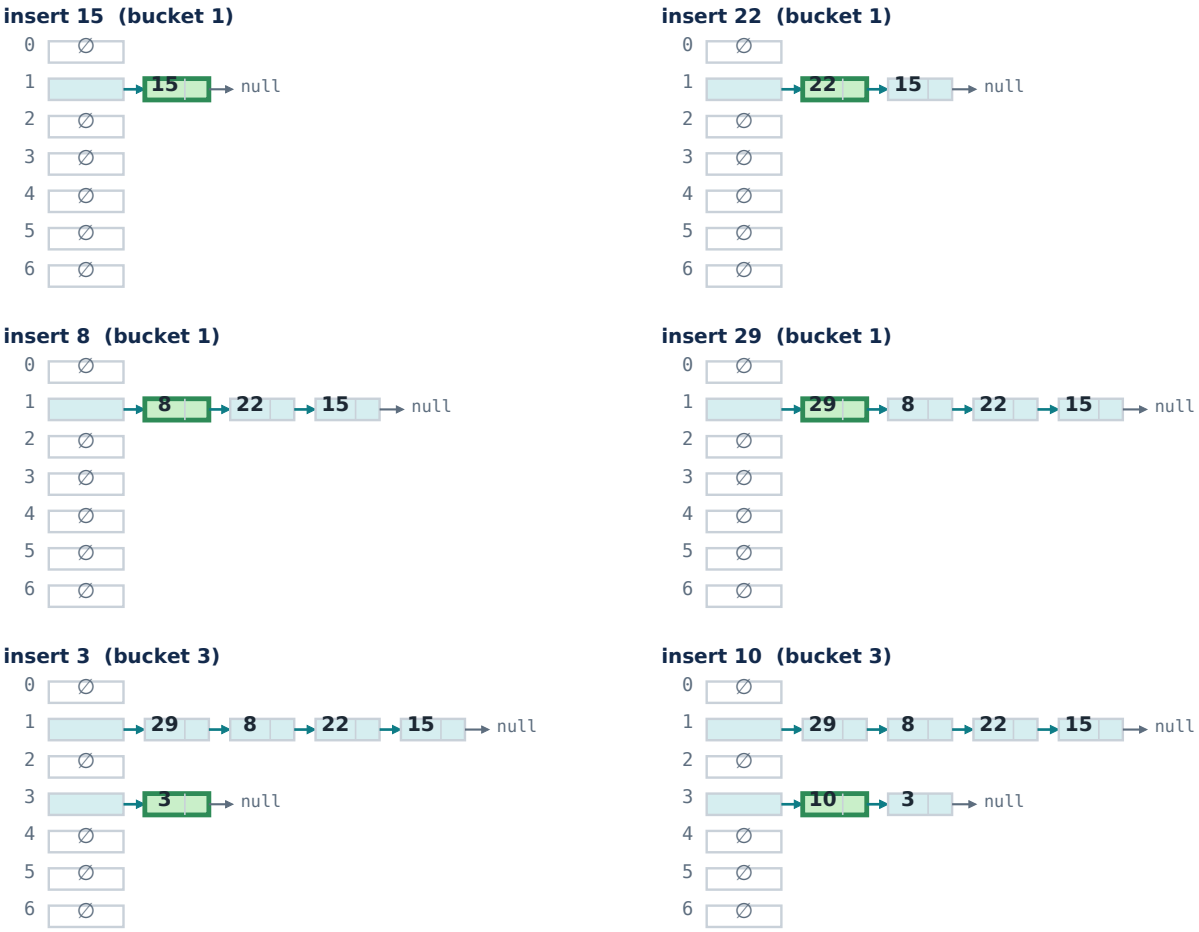


Figure 5.2 — Bucket contents after every insertion (new node in green). Chain of bucket 1 grows: 15 → 22,15 → 8,22,15 → 29,8,22,15.

Searching and deleting in the same table

Operation	Index	Nodes compared	Result
search 22	1	3 (29, 8, 22)	found
search 36	1	4 (29, 8, 22, 15)	absent: whole chain scanned
search 5	5	0	absent: bucket empty
insert 3 again (duplicate)	3	2 (10, 3)	value updated; n unchanged
erase 8	1	2 (29, 8)	middle node unlinked: 29 → 22 → 15
erase 29	1	1 (29)	head removed: 22 → 15
erase 99	1	2 (22, 15)	absent: nothing changes

Figure 5.3 shows what deleting a middle node and then a head node does to the chain of bucket 1.

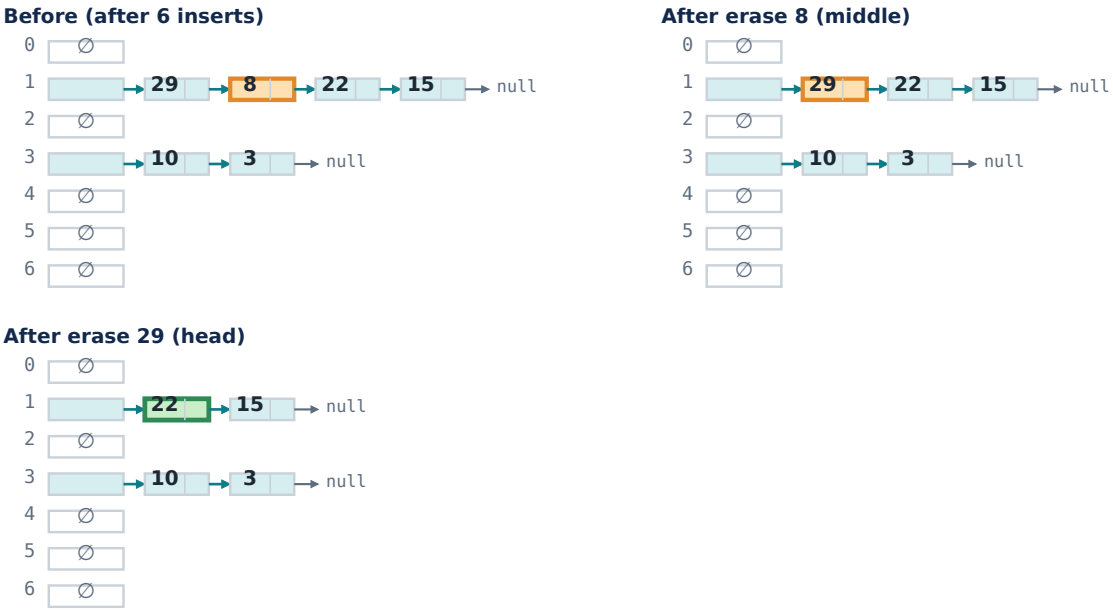


Figure 5.3 — Deletion in bucket 1. Erasing the middle node bypasses it; erasing the head moves the bucket pointer to the next node.

5.9 C++ Implementation Using Nodes and Pointers

The class below is a complete, compilable separate-chaining table with `insert`, `search`, `erase` and `display`. Compile with `g++ -std=c++17 file.cpp`.

```cpp
// Part 5: separate chaining with linked nodes (C++17)
#include <iostream>
#include <string>

struct Node {
    int key;
    std::string value;
    Node* next;
    Node(int k, const std::string& v, Node* n) : key(k), value(v), next(n) {}
};

class ChainedHashTable {
public:
    explicit ChainedHashTable(int capacity)
        : m_(capacity), n_(0), buckets_(new Node*[capacity]()) {}   // all nullptr
    ~ChainedHashTable() {
        for (int i = 0; i < m_; ++i) {
            Node* p = buckets_[i];
            while (p != nullptr) { Node* nx = p->next; delete p; p = nx; }
        }
        delete[] buckets_;
    }
    ChainedHashTable(const ChainedHashTable&) = delete;            // no shallow copies
    ChainedHashTable& operator=(const ChainedHashTable&) = delete;

    // true: new key added; false: key existed and its value was updated
    bool insert(int key, const std::string& value) {
        int i = indexFor(key);
        for (Node* p = buckets_[i]; p != nullptr; p = p->next) {
            if (p->key == key) { p->value = value; return false; }
        }
        buckets_[i] = new Node(key, value, buckets_[i]);            // insert at head
        ++n_;
        return true;
    }

    // returns the node or nullptr; 'steps' counts nodes compared
    const Node* search(int key, int& steps) const {
        steps = 0;
        for (const Node* p = buckets_[indexFor(key)]; p != nullptr; p = p->next) {
            ++steps;
            if (p->key == key) return p;
```

```cpp
        }
        return nullptr;
    }

    bool erase(int key) {
        int i = indexFor(key);
        Node* prev = nullptr;
        for (Node* p = buckets_[i]; p != nullptr; prev = p, p = p->next) {
            if (p->key == key) {
                if (prev == nullptr) buckets_[i] = p->next;         // removing the head
                else prev->next = p->next;                          // bypass the node
                delete p;
                --n_;
                return true;
            }
        }
        return false;
    }

    void display() const {
        for (int i = 0; i < m_; ++i) {
            std::cout << "[" << i << "]";
            for (const Node* p = buckets_[i]; p != nullptr; p = p->next)
                std::cout << " " << p->key << ":" << p->value << " ->";
            std::cout << " null\n";
        }
    }
    double loadFactor() const { return static_cast<double>(n_) / m_; }
    int size() const { return n_; }

private:
    int indexFor(int key) const { return ((key % m_) + m_) % m_; }
    int m_;
    int n_;
    Node** buckets_;
};

int main() {
    ChainedHashTable t(7);
    t.insert(15, "Asha"); t.insert(22, "Ravi"); t.insert(8, "Meera");
    t.insert(29, "Kabir"); t.insert(3, "Isha"); t.insert(10, "Tara");
    std::cout << "After 6 insertions (n = " << t.size() << ", load = "
              << t.loadFactor() << "):\n";
    t.display();

    int steps = 0;
    const Node* f = t.search(22, steps);
    std::cout << "search(22): " << (f ? f->value : "absent")
              << ", nodes compared = " << steps << "\n";
    f = t.search(36, steps);
    std::cout << "search(36): " << (f ? f->value : "absent")
              << ", nodes compared = " << steps << "\n";
    f = t.search(5, steps);
    std::cout << "search(5): " << (f ? f->value : "absent")
              << ", nodes compared = " << steps << "\n";

    std::cout << "insert(3, Isha2) new? " << t.insert(3, "Isha2") << " (0 = updated)\n";
    std::cout << "erase(8): " << t.erase(8) << ", erase(29): " << t.erase(29)
              << ", erase(99): " << t.erase(99) << "\n";
    std::cout << "After deletions (n = " << t.size() << "):\n";
    t.display();
    return 0;
}
```

Output (matches the dry run above)

```
After 6 insertions (n = 6, load = 0.857143):
[0] null
[1] 29:Kabir -> 8:Meera -> 22:Ravi -> 15:Asha -> null
[2] null
[3] 10:Tara -> 3:Isha -> null
[4] null
[5] null
[6] null
search(22): Ravi, nodes compared = 3
search(36): absent, nodes compared = 4
search(5): absent, nodes compared = 0
insert(3, Isha2) new? 0 (0 = updated)
erase(8): 1, erase(29): 1, erase(99): 0
```

```
After deletions (n = 4):
[0] null
[1] 22:Ravi -> 15:Asha -> null
[2] null
[3] 10:Tara -> 3:Isha2 -> null
[4] null
[5] null
[6] null
```

Line	Code	What it does
15	`: m_(capacity), n_(0), buckets_(new Node*[ca…`	Allocates m head pointers. The empty `()` value-initialises them to `nullptr`, so every bucket starts empty.
19	`while (p != nullptr) { Node* nx = p->next; d…`	Destructor: save `next` **before** `delete p`, then move on. Frees every node of every chain.
23	`ChainedHashTable(const ChainedHashTable&) = …`	Copying would duplicate raw pointers and cause a double free; the copy operations are disabled (Rule of Three/Five).
30	`if (p->key == key) { p->value = value; retur…`	Duplicate policy: update the value and return `false` (no new key).
32	`buckets_[i] = new Node(key, value, buckets_[…`	New node points to the old head, then becomes the head: $O(1)$ link.
38	`const Node* search(int key, int& steps) cons…`	Returns a pointer to the node or `nullptr`; `steps` reports how many nodes were compared.
52	`if (prev == nullptr) buckets_[i] = p->next; …`	Deleting the head: the bucket must point to the second node.
53	`else prev->next = p->next; …`	Deleting a later node: the previous node skips over it.
19	`while (p != nullptr) { Node* nx = p->next; d…`	Frees the node after it has been unlinked; `n_` is decremented.
74	`int indexFor(int key) const { return ((key %…`	Safe index even for negative keys (Section 3.11).
77	`Node** buckets_;`	The array of head pointers.

NOTE. Dry run for `erase(8)`: `i = 1`; walk `p = 29` (not 8, `prev = 29`), then `p = 8` (match). Since `prev != nullptr`, `prev->next = 22`; node 8 is deleted. Chain becomes 29 → 22 → 15.

5.10 Memory Management and Pointer Safety

- **Ownership:** the table owns all nodes. The destructor must free them; otherwise each insertion leaks memory.
- **Rule of Three/Five:** because the class owns raw memory, either implement copy/move correctly or **delete** them (as done here).
- **Dangling pointers:** do not return `Node*` from `search` and keep it after an `erase` or table destruction; the node may be gone.
- **Exception safety:** if `new` throws `std::bad_alloc` during insert the table is left unchanged because the node is created before any pointer is modified.
- **Safer alternatives:** use `std::unique_ptr<Node>` for `next` (watch out for recursive destruction of very long chains) or build the buckets from `std::list`/`std::vector`, as shown next.
- **Testing:** compile with `-fsanitize=address,undefined`; the programs in this book were run that way.

5.11 C++ Implementation Using Standard Library Containers

With `std::vector<std::list<std::pair<int, std::string>>>` the memory management disappears: destructors of the containers free everything. This version also uses a negative key (−3) to show the safe index.

```cpp
// Part 5: separate chaining built from standard containers (C++17)
#include <algorithm>
#include <iostream>
#include <list>
#include <string>
#include <utility>
#include <vector>

class StlChainTable {
    using Entry = std::pair<int, std::string>;
public:
    explicit StlChainTable(std::size_t m) : buckets_(m) {}

    bool insert(int key, const std::string& value) {
        auto& chain = buckets_[indexFor(key)];
        auto it = std::find_if(chain.begin(), chain.end(),
                               [key](const Entry& e) { return e.first == key; });
        if (it != chain.end()) { it->second = value; return false; }
        chain.emplace_front(key, value);
        return true;
    }
    const std::string* find(int key) const {
        for (const Entry& e : buckets_[indexFor(key)])
            if (e.first == key) return &e.second;
        return nullptr;
    }
    bool erase(int key) {
        auto& chain = buckets_[indexFor(key)];
        auto before = chain.size();
        chain.remove_if([key](const Entry& e) { return e.first == key; });
        return chain.size() != before;
    }
    void display() const {
        for (std::size_t i = 0; i < buckets_.size(); ++i) {
            std::cout << "[" << i << "]";
            for (const Entry& e : buckets_[i])
                std::cout << " " << e.first << ":" << e.second << " ->";
            std::cout << " null\n";
        }
    }
private:
    std::size_t indexFor(int key) const {
        long long m = static_cast<long long>(buckets_.size());
        return static_cast<std::size_t>(((key % m) + m) % m);
    }
    std::vector<std::list<Entry>> buckets_;        // memory freed automatically
};

int main() {
    StlChainTable t(5);
    for (int k : {12, 7, 22, 17, -3}) t.insert(k, "v" + std::to_string(k));
    t.display();
    const std::string* p = t.find(22);
    std::cout << "find(22) = " << (p ? *p : "absent") << "\n";
    std::cout << "erase(7) = " << t.erase(7) << ", erase(100) = " << t.erase(100) << "\n";
    t.display();
    return 0;
}
```

```
Output
[0] null
[1] null
[2] -3:v-3 -> 17:v17 -> 22:v22 -> 7:v7 -> 12:v12 -> null
[3] null
[4] null
find(22) = v22
erase(7) = 1, erase(100) = 0
[0] null
[1] null
[2] -3:v-3 -> 17:v17 -> 22:v22 -> 12:v12 -> null
[3] null
[4] null
```

Check: $12 \bmod 5 = 2$, $7 \bmod 5 = 2$, $22 \bmod 5 = 2$, $17 \bmod 5 = 2$ and $-3 \bmod 5 = 2$ (mathematically), so **all five keys collide in bucket 2**, in reverse insertion order because `emplace_front` inserts at the head.

5.12 Effect of Load Factor on Chain Length

In separate chaining n may exceed m, so the **load factor** $\alpha = n/m$ is the **average chain length**. Under simple uniform hashing the expected number of **nodes compared** is:

$$\text{unsuccessful search} \approx \alpha \quad \text{successful search} \approx 1 + \alpha/2 \text{ (target node included)}$$

$\alpha = n/m$	Average chain length	Unsuccessful search (nodes compared)	Successful search (nodes compared)
0.5	0.5	0.5	1.25
1	1	1	1.5
2	2	2	2.0
5	5	5	3.5

Because chains keep working when $\alpha > 1$, chaining degrades **gracefully**; practical implementations still resize when α passes about 1 (Part 10) to keep chains short. Even with $\alpha = 1$ the **longest** chain is typically longer than average: for $n = m$ random keys it is about $\ln n \,/\, \ln \ln n$ with high probability.

5.13 Handling an Empty Bucket and a Missing Key

Situation	Correct behaviour
Search in an **empty bucket**	`buckets[i] == nullptr`, loop body never runs, report absent (0 comparisons)
Search for a **missing key** in a non-empty chain	Scan to the end (`p == nullptr`), report absent
Erase from an **empty bucket**	Return `false`; do not dereference a null pointer
Erase a missing key	Return `false`, table unchanged, count unchanged
Insert into empty table	Works like any other insertion (head becomes the new node)

COMMON MISTAKE. Dereferencing `buckets[i]->key` without checking for `nullptr` crashes the program (segmentation fault) the first time a bucket is empty.

5.14 Performance under Heavy Collisions

If a poor hash function sends all n keys to one bucket, the table is just **one linked list**: insert (with duplicate check), search and delete all cost $O(n)$. Example: $n = 10\{,\}000$ keys all in bucket 0 require on average about 5,000 comparisons per successful search. Remedies:

- Improve the hash function (Part 3) and use a prime table size.
- Resize to keep α bounded (Part 10).
- Switch long chains to balanced trees (Java 8+ `HashMap`) so the worst case becomes $O(\log L)$.
- Use randomized hashing against adversarial inputs (Part 12).

5.15 Advantages and Disadvantages

Advantages	Disadvantages
Simple to implement and to reason about	Extra memory for pointers (about 8 bytes per node on 64-bit systems)
Deletion is easy: unlink a node	Pointer chasing is cache-unfriendly
Table never "fills up"; works with $\alpha > 1$	Dynamic allocation per key is slow (allocator overhead)
Performance degrades gradually with load	Chains can get long if the hash is poor
No clustering between different home indexes	Needs ordering or hashing for tree buckets

5.16 Time and Space Complexity

Operation	Expected (simple uniform hashing)	Worst case	Remark
Insert (with duplicate check)	$O(1 + \alpha)$	$O(n)$	Without the check: $O(1)$
Search	$O(1 + \alpha)$	$O(n)$	Absent keys scan the whole chain
Delete	$O(1 + \alpha)$	$O(n)$	Search + $O(1)$ unlink
Display / traverse all	$O(m + n)$	$O(m + n)$	Visit every bucket and node
Space	$O(m + n)$	$O(m + n)$	m head pointers + n nodes

If $\alpha = O(1)$ (kept constant by resizing) the expected time per operation is $O(1)$. The word "expected" is essential; see Part 14.

PART 6

OPEN ADDRESSING: FUNDAMENTALS

Probe sequences, tombstones, termination, full-table detection and probe-count analysis

6.1 Definition and Working Principle

DEFINITION. **Open addressing** (also *closed hashing*) stores every key **inside the table array itself**. When the home slot is taken, the algorithm **probes** other slots, in a fixed order that depends on the key, until it finds a free one. There are no lists and no extra nodes.

Each key therefore has a **probe sequence**: $h(k, 0)$, $h(k, 1)$, $h(k, 2)$, ..., $h(k, m\text{-}1)$. The first element is the key's home slot. Insertion, search and deletion all follow the **same** sequence, which is what makes the keys findable.

$$\text{probe sequence of key } k : h(k, 0), h(k, 1), \ldots, h(k, m-1) \text{ with } h(k, i) \in \{0, \ldots, m-1\}$$

Here i is the **probe number** (0 for the first look). A good probe sequence is a **permutation of all m slots**, so that if any free slot exists, the sequence will find it. The three standard families differ only in how $h(k, i)$ is computed:

Method	$h(k, i)$	Studied in
Linear probing	$(h(k) + i) \bmod m$	Part 7
Quadratic probing	$(h(k) + c_1\, i + c_2\, i^2) \bmod m$	Part 8
Double hashing	$(h_1(k) + i \cdot h_2(k)) \bmod m$	Part 9

6.2 Searching for an Available Slot and Insertion

Insertion follows the probe sequence and stops at the **first slot that can hold the key**:

1. Set $i = 0$ and remember no tombstone yet.
2. Compute $j = h(k, i)$.
3. If slot j is **EMPTY**: the key is not in the table. Store it (in the first tombstone seen on the way, otherwise in j) and stop.
4. If slot j is **OCCUPIED by** k: duplicate key. Apply the policy (reject or update) and stop.
5. If slot j is **DELETED** (tombstone): remember the first one, then continue.
6. Otherwise it is occupied by another key. Increase i and repeat from step 2.
7. If i reaches m without finding an EMPTY slot: use the remembered tombstone if there is one; otherwise report **table full**.

Figure 6.1 gives the flowchart of insertion in open addressing.

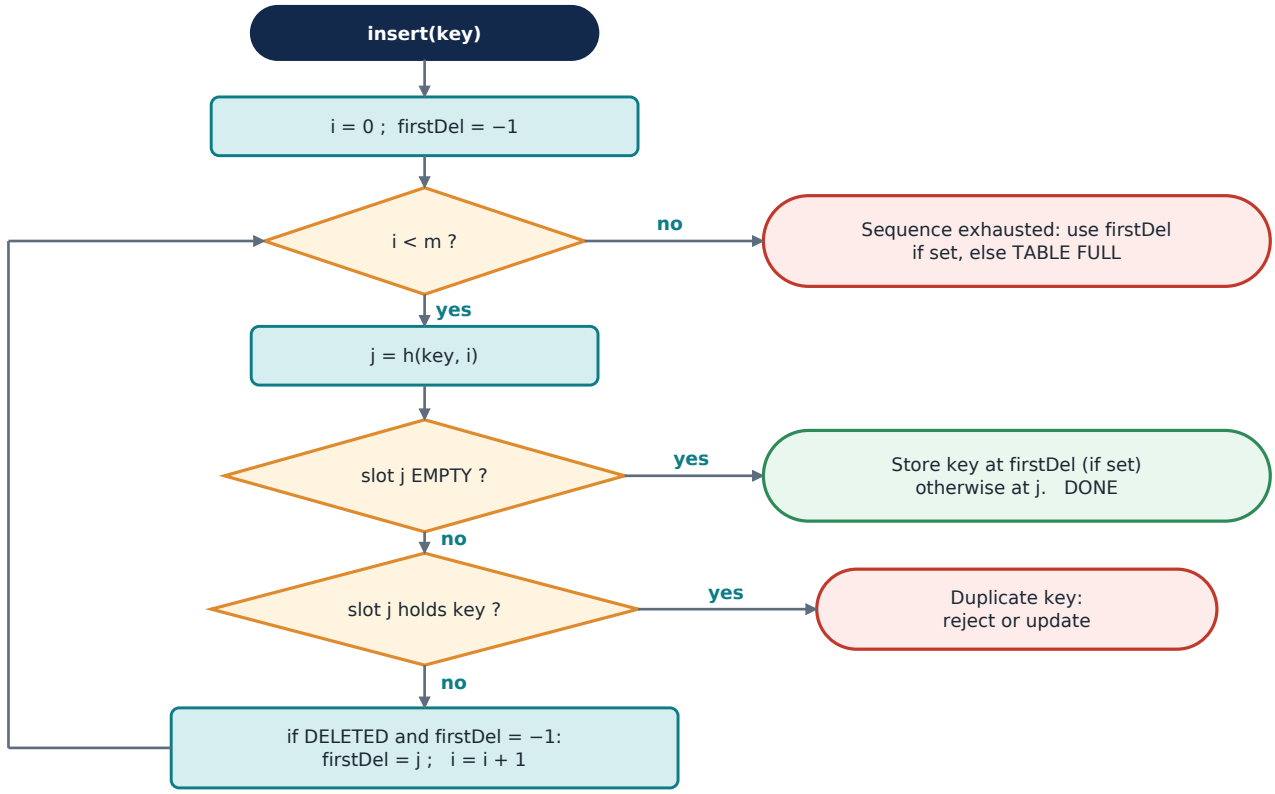


Figure 6.1 — Flowchart of insertion in open addressing. The loop ends at the first EMPTY slot, a duplicate, or after m probes.

Figure 6.2 gives the matching flowchart for search; note that only an EMPTY slot ends it early.

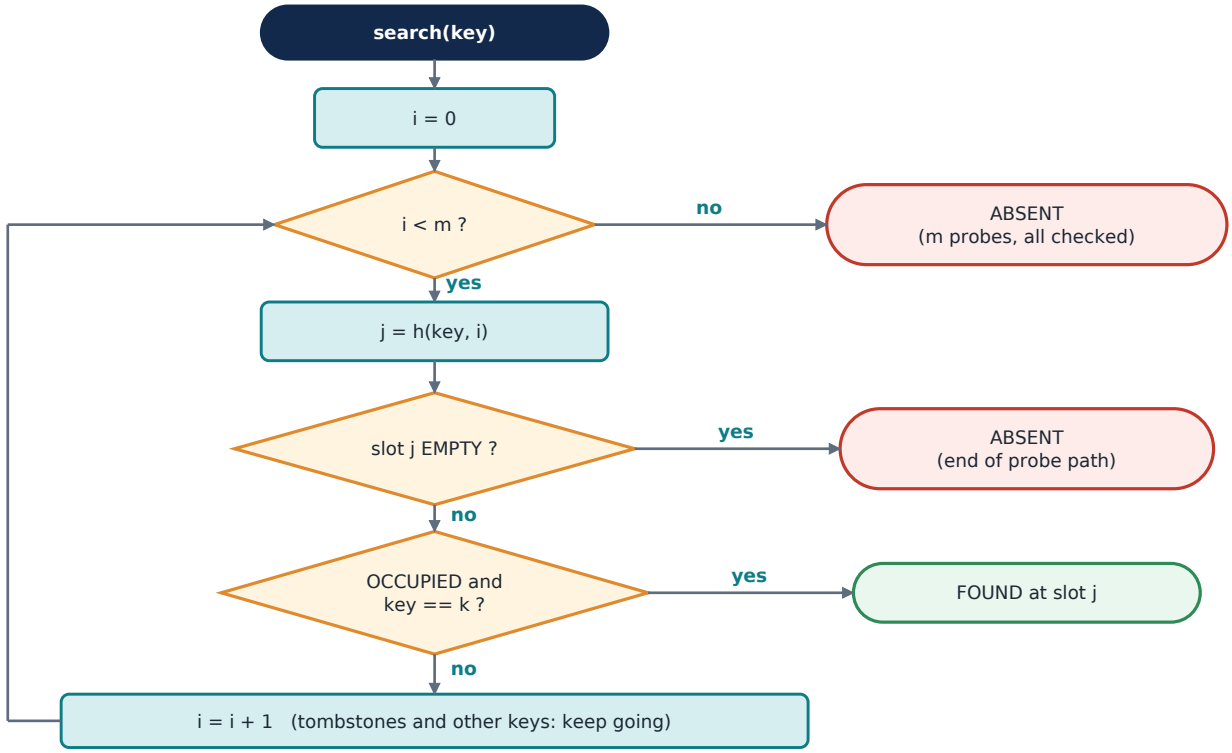


Figure 6.2 — Flowchart of search. Only an EMPTY slot proves absence; a tombstone must be skipped.

6.3 Unsuccessful Searches

An **unsuccessful search** (the key is absent) ends in one of two ways: it reaches an **EMPTY** slot (the key would have been placed there, so it is not further along), or it has examined all m slots. This is why open-addressing tables must always keep **at least one EMPTY slot** in practice: otherwise searches for absent keys never meet a stopping signal and cost m probes.

6.4 Deletion Complications: Why EMPTY Cannot Replace DELETED

Suppose we delete a key by simply setting its slot back to EMPTY. Later searches for other keys that were **placed beyond** that slot (because it was occupied at the time) will stop early and wrongly say "absent". Figure 6.3 shows this with linear probing: keys 25, 36, 47 all have home slot 3 (mod 11).

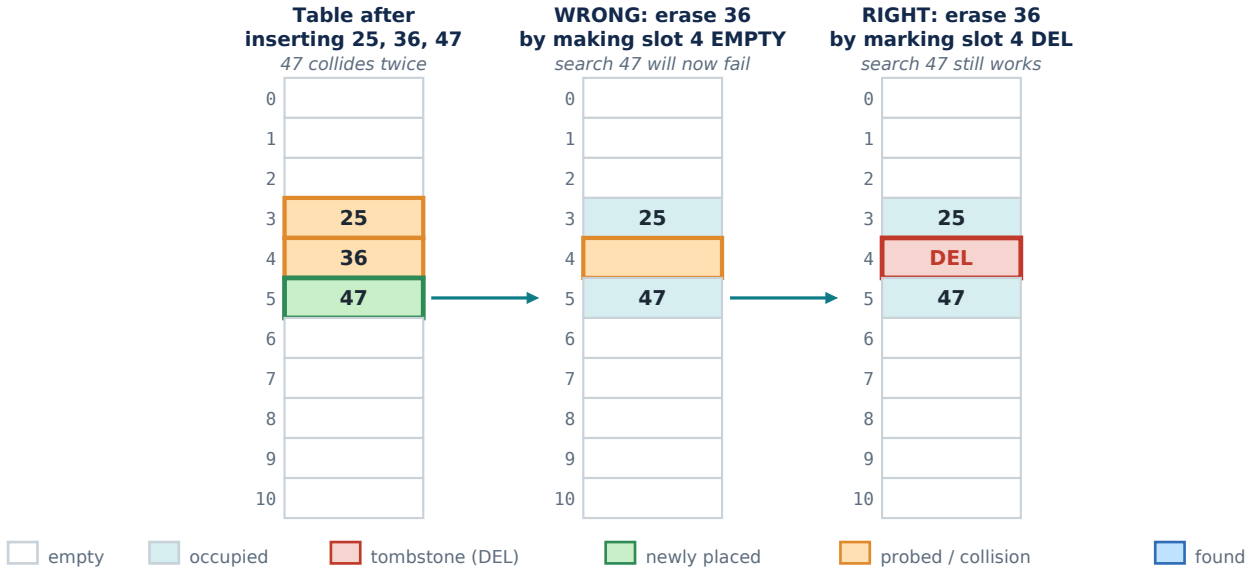


Figure 6.3 — Deletion in open addressing. Left: original table. Middle: slot 4 reset to EMPTY. Right: slot 4 turned into a tombstone.

Search for 47 (home 3)	Probe sequence	Result
With EMPTY at slot 4 (wrong)	3 (holds 25, not 47) → 4 (EMPTY: stop)	**ABSENT** — incorrect, 47 is in slot 5!
With tombstone at slot 4 (right)	3 (25) → 4 (DEL: skip) → 5 (holds 47)	**FOUND** at slot 5 after 3 probes

COMMON MISTAKE. Marking a deleted slot EMPTY breaks the probe chains that pass through it. This is the single most common bug in open-addressing code.

6.5 Empty, Occupied and Deleted Markers; Tombstones

DEFINITION. A **tombstone** (deleted marker) is a special slot state meaning "a key was here once". Searches treat it as **occupied-but-not-matching** (keep probing); insertions may **reuse** it.

Operation	EMPTY	OCCUPIED	DELETED (tombstone)
Search	stop: absent	compare key; match → found	skip, continue probing
Insert	place key (or first tombstone seen)	duplicate check; else continue	remember first; continue to check for duplicate
Delete	stop: absent	match → mark DELETED	skip, continue probing

Tombstones are not free: they **lengthen probe sequences** and make the table look fuller than it is. A table with many deletions and few EMPTY slots makes unsuccessful searches slow. The cure is to **rehash** (rebuild the table) when tombstones accumulate (Part 10), or to count tombstones together with live keys when deciding to resize. A tombstone can also be reused by a later insertion, as the dry runs in Parts 7–9 show.

EXAM TRAP. When reusing a tombstone for insertion you must **still search the rest of the probe sequence for a duplicate first**; only then store the new key in the tombstone. Otherwise two copies of the same key can exist.

6.6 Probe Sequence Termination and Full-table Detection

- **Bound the loop:** `for (int i = 0; i < m; ++i)`. A loop like `while (slot occupied) j = next(j)` runs forever on a full table or when the step size is 0.
- **Permutation property:** if the probe sequence is not a permutation of all slots (Part 8 quadratic, Part 9 bad h_2), the loop ends after m probes with failure **even though empty slots exist elsewhere**.
- **Count live keys:** keep n (number of live keys). If $n = m$ the table is full; refuse insertion. Better: never let n come close to m (resize).
- **Failed insertion is not the same as full:** in quadratic probing and double hashing a failure can happen when $n < m$; the correct response is to resize/rehash.

6.7 Load-factor Restrictions

For open addressing $n \le m$ always, so $\alpha = n/m \le 1$. Performance collapses as $\alpha \to 1$ (see below), therefore practical limits are:

Method	Typical maximum load factor	Reason
Linear probing	0.5 – 0.75	Primary clustering makes costs rise steeply
Quadratic probing	< 0.5 for the guarantee on prime m; up to about 0.7 in practice	Probe sequence may not cover the table
Double hashing	0.7 – 0.8	Close to uniform hashing, but cost still rises as $1/(1-\alpha)$
Separate chaining (for comparison)	1 – 2	Chains lengthen gracefully

6.8 Probe Counts and Performance

Expected number of probes under the **uniform hashing** assumption (every probe sequence equally likely among all permutations), for load factor $\alpha < 1$, taken from Knuth's classical analysis:

$$\text{unsuccessful search} \approx 1 / (1 - \alpha) \quad \text{successful search} \approx (1/\alpha) \cdot \ln(\, 1 / (1 - \alpha)\,)$$

For **linear probing**, which suffers from primary clustering, the classical results are larger:

$$\text{unsuccessful} \approx \tfrac{1}{2}\,[\, 1 + 1/(1 - \alpha)^2\,] \quad \text{successful} \approx \tfrac{1}{2}\,[\, 1 + 1/(1 - \alpha)\,]$$

Load factor α	Uniform / double hashing: unsuccessful	successful	Linear probing: unsuccessful	successful
0.5	2.00	1.39	2.50	1.50
0.7	3.33	1.72	6.06	2.17
0.8	5.00	2.01	13.00	3.00
0.9	10.00	2.56	50.50	5.50
0.95	20.00	3.15	200.50	10.50

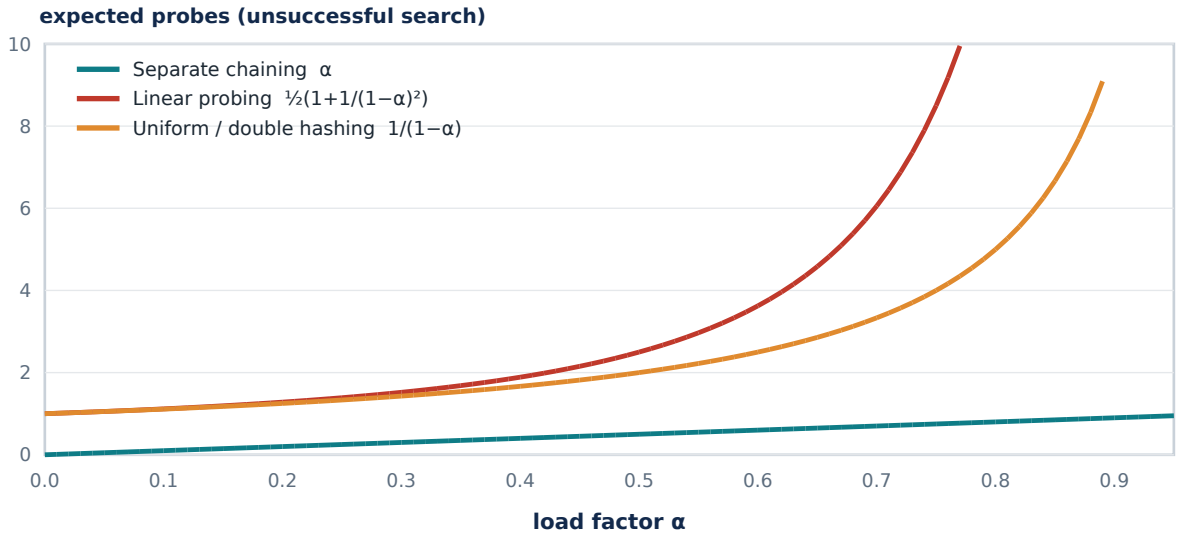


Figure 6.4 — Expected probes for an unsuccessful search as the load factor grows (formulas above; separate chaining shown for comparison, counting expected nodes compared).

WARNING. These are **expected** values for random keys and an idealised model; they are formulas from analysis, not measurements of a particular program. Worst-case behaviour (all keys colliding) is $O(n)$ for every method.

6.9 Comparison with Separate Chaining

Aspect	Open addressing	Separate chaining
Where keys live	In the table array	In nodes outside the array
Maximum α	$\alpha \le 1$; must stay well below 1	Can exceed 1
Extra memory	None for pointers; keep spare slots	One pointer per node plus allocation overhead
Deletion	Tombstones needed	Unlink node
Cache behaviour	Excellent (contiguous probes, especially linear)	Poor (pointer chasing)
Clustering	Primary / secondary clustering possible	No cross-bucket clustering
Resizing needed	Early (around 0.5–0.8)	Later (around 1)

6.10 Primary and Secondary Clustering (Recap)

Primary clustering (linear probing): runs of occupied slots grow because a key landing anywhere in a run joins its end. **Secondary clustering** (quadratic probing and linear): keys with the **same home slot** follow the **same probe sequence**, even if they have different keys. **Double hashing** avoids both by giving different step sizes to different keys. The next three parts study each method in turn.

PART 7

LINEAR PROBING

The simplest probing rule: try the next slot, then the next, wrapping around

7.1 Definition and Formula

DEFINITION. **Linear probing** resolves a collision by trying the **next slot** in the array, then the next, and so on, wrapping around from the last slot to slot 0, until a free slot is found.

$$h(k, i) = (h(k) + i) \bmod m \ , \ i = 0, 1, 2, \ldots, m-1$$

Symbol	Meaning
k	the key being inserted, searched or deleted
$h(k)$	the ordinary hash function (here k mod m): the **home slot**
i	the probe number: 0 = first look at the home slot, 1 = the next slot, ...
m	table size (number of slots)
$h(k, i)$	the slot examined on probe i
mod m	wrap-around: after slot m-1 come slot 0, 1, ...

7.2 Step-by-step Insertion

Table size $m = 11$, $h(k) = k$ mod 11. We insert **25, 36, 47, 14, 58**. All five keys have the **same home slot**: $25 \bmod 11 = 3$, $36 \bmod 11 = 3$, $47 \bmod 11 = 3$, $14 \bmod 11 = 3$, $58 \bmod 11 = 3$. This is the worst case for the method, and it shows every probe.

Insert	Home $h(k)$	Probe sequence (slot : status)	Probes	Stored at
25	3	3:free	1	3
36	3	3:occupied → 4:free	2	4
47	3	3:occupied → 4:occupied → 5:free	3	5
14	3	3:occupied → 4:occupied → 5:occupied → 6:free	4	6
58	3	3:occupied → 4:occupied → 5:occupied → 6:occupied → 7:free	5	7

In Figure 7.1 the table is drawn after each of the five insertions.

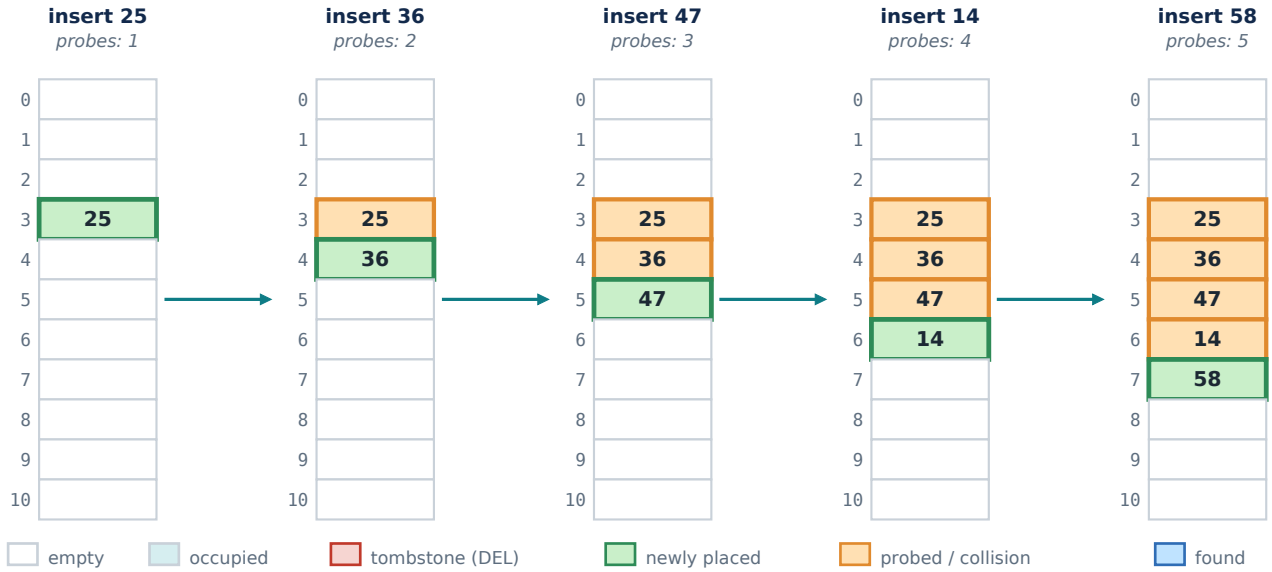


Figure 7.1 — Table after each insertion (orange = occupied slots probed, green = slot used). Each new key walks to the end of the growing run.

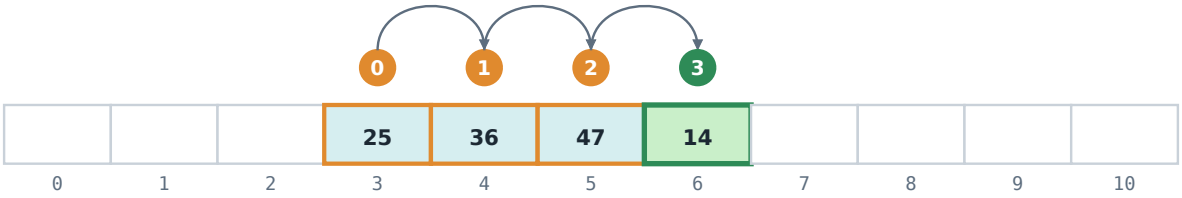


Figure 7.2 — Probe path for key 14 (home slot 3): probe 0 at slot 3, then 4, 5 and finally 6, the first free slot. Circles show the probe number.

Reading Figure 7.2: probe 0 examines the home slot 3 (holds 25), probe 1 examines slot 4 (36), probe 2 slot 5 (47), probe 3 slot 6, which is free: **14 is stored in slot 6 after 4 probes.** The key 58 will need 5 probes. In general, the j-th key of a group sharing a home slot needs j probes: collisions are costly.

7.3 Searching: Present and Absent Keys

Search	Home	Probe sequence	Outcome	Probes
58	3	3 → 4 → 5 → 6 → 7 (holds 58)	**found** in slot 7	5
69	3	3 → 4 → 5 → 6 → 7 → 8 (EMPTY)	**absent**: stopped at empty slot	6

- **Successful search** ends when the key is found: cost = position of the key in its probe sequence.
- **Unsuccessful search** ends at the first EMPTY slot: here 69 passes through the whole run 3–7 and stops at slot 8. The cost is the length of the run **plus one**, so absent keys are the expensive case.
- If no EMPTY slot existed, an absent key would need all m probes.

Figure 7.3 draws the probe path of the unsuccessful search for 69.

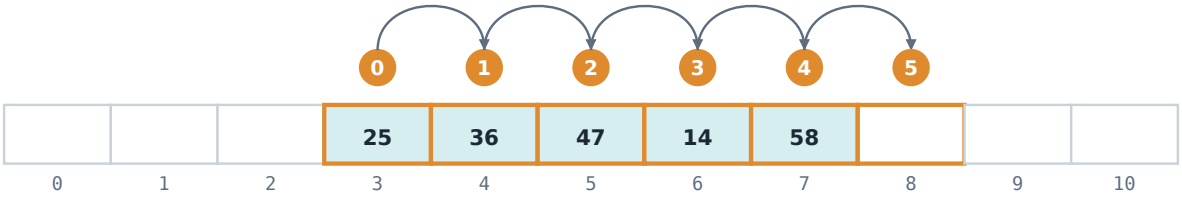


Figure 7.3 — Unsuccessful search for 69: six probes, the last one reaching the EMPTY slot 8 that proves 69 is absent.

7.4 Deletion Using Tombstones

We now delete 36 and look at what remains. Slot 4 becomes a **tombstone** (DEL), not EMPTY.

Figure 7.4 shows how a tombstone affects a later search and a later insertion.

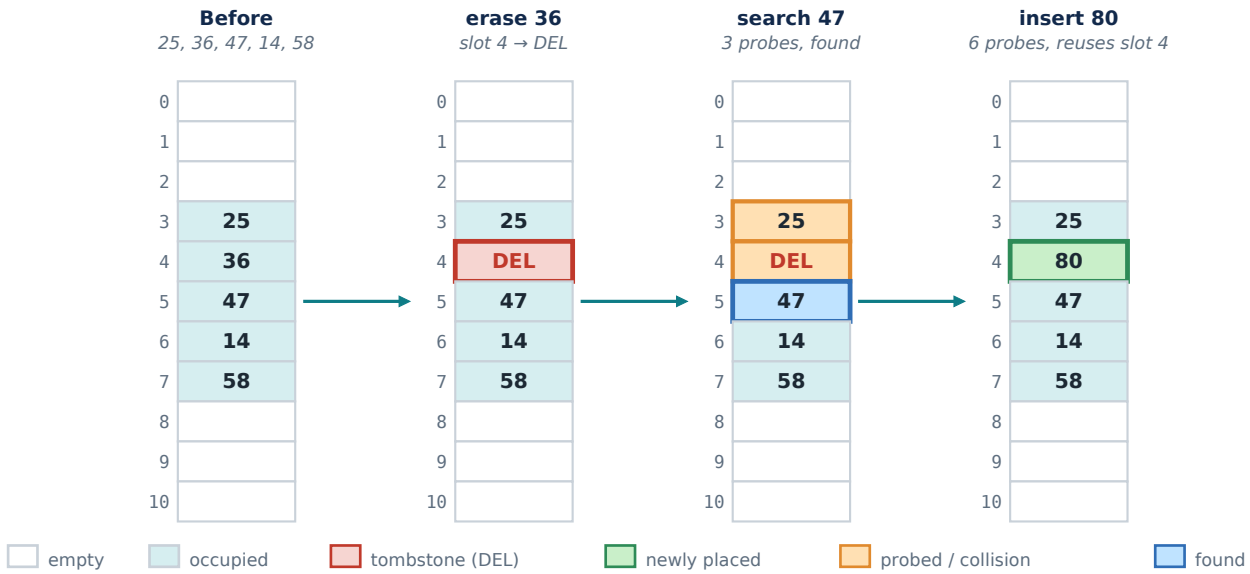


Figure 7.4 — Deletion with a tombstone. Search for 47 steps over the tombstone and still succeeds. Insert 80 (home 3) probes 3→4→5→6→7→8, remembering the tombstone at slot 4 and reusing it.

Operation	Probe sequence	Result
erase 36	3 → 4 (holds 36)	slot 4 := DELETED
search 47	3 (25) → 4 (DEL, skip) → 5 (47)	found in slot 5, 3 probes
insert 80	3 → 4 → 5 → 6 → 7 → 8 (slot 8 EMPTY)	stored in tombstone slot 4; 6 probes
insert 47 again	3 → 4 → 5	duplicate found in slot 5: rejected (status 0)

Note that inserting 80 had to look **all the way to the EMPTY slot 8** before placing the key in the tombstone at slot 4. If 80 had already been in the table at slot 7, placing a second copy at slot 4 would have created a duplicate.

7.5 Why Direct Deletion Can Break Searches

This was shown in Figure 6.3. Setting slot 4 to EMPTY would make the search for 47 stop at probe 1 and wrongly report "absent". The invariant of open addressing is: **every key is reachable from its home slot through occupied-or-deleted slots only.** An EMPTY slot inside a run destroys that invariant.

> **EXAM TRAP.** Alternative to tombstones for linear probing: **backward-shift deletion** moves later keys of the cluster back to fill the hole and restores the invariant without markers. It is correct only if each moved key is not shifted before its own home slot. Tombstones are the standard answer in exams.

7.6 Primary Clustering: How Clusters Form and Grow

A **cluster** (run) is a maximal block of consecutive occupied slots. With linear probing a key landing **anywhere inside** a cluster ends up at its **end**, so each insertion makes the cluster longer. Larger clusters are hit more often, which makes them grow even faster: a rich-get-richer effect.

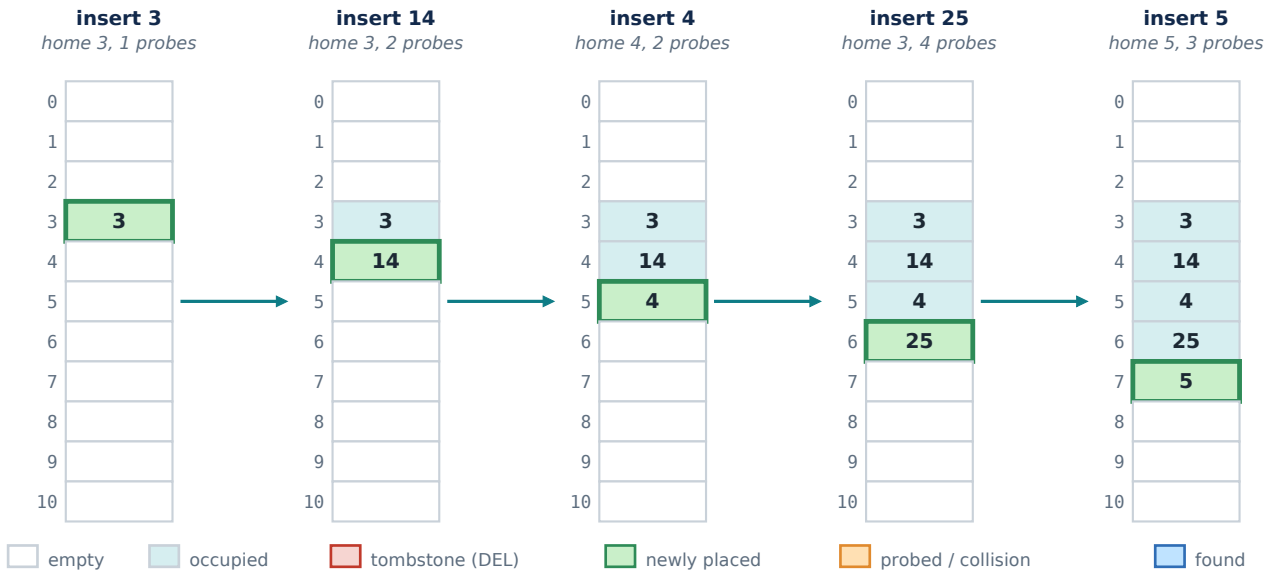


Figure 7.5 — Keys 3, 14, 4, 25, 5 have home slots 3, 3, 4, 3, 5. They merge into one cluster covering slots 3–7 and the later keys need 2, 2, 4 and 3 probes.

Key	Home	Probes	Why
3	3	1	slot 3 free
14	3	2	home 3 taken by 3
4	4	2	home 4 taken by 14 (a key from a different home!)
25	3	4	walks through the cluster 3,4,5
5	5	3	home 5 taken by 4 → slot 6 taken by 25 → slot 7

NOTE. Key 4 has home slot 4, which has nothing to do with home slot 3, but it still suffers because key 14 spilled over into slot 4. This is **primary clustering**: keys with different home slots interfere.

7.7 Successful versus Unsuccessful Search; Load Factor and Performance

Expected probes with linear probing (Knuth): successful $\approx \tfrac{1}{2}[1 + 1/(1-\alpha)]$, unsuccessful $\approx \tfrac{1}{2}[1 + 1/(1-\alpha)^2]$. For $\alpha = 0.5, 0.75, 0.9$ these are 1.5 / 2.5, 2.5 / 8.5 and 5.5 / 50.5. The unsuccessful case grows with the **square** of $1/(1-\alpha)$, which is why linear probing needs a low maximum load factor (about 0.5–0.7). The values in Part 6 (Figure 6.4) assume random keys.

7.8 Wrap-around and Full-table Conditions

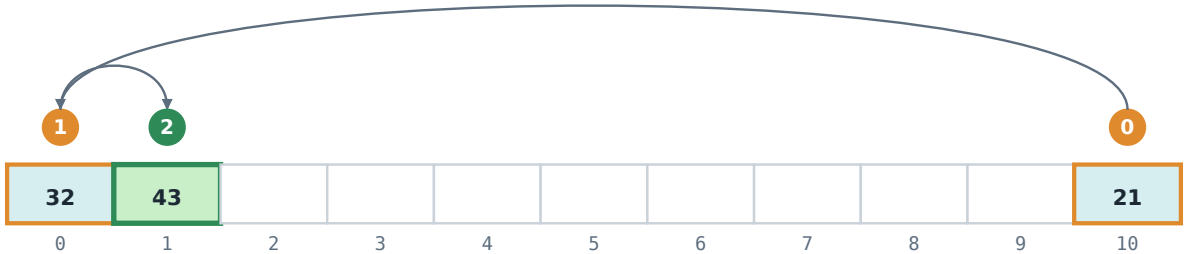


Figure 7.6 — Wrap-around: keys 21, 32, 43 all have home slot 10. 43 probes slot 10, wraps to slot 0, then slot 1 where it is stored.

- **Wrap-around** is the `mod m`: after slot 10 the next probe is slot 0.
- **Full table:** when all m slots are OCCUPIED, a new key makes m probes and fails. Without a loop bound the search would cycle for ever.
- **Full of tombstones:** a table with no EMPTY slot but several DELETED slots is not full for insertion (a tombstone can be reused) but is terrible for searching absent keys (m probes); rebuild it.

- **Never rely on finding EMPTY:** always bound the probe count by m.

7.9 Complete C++ Implementation

```cpp
// Part 7: linear probing with tombstones (C++17)
#include <iostream>
#include <vector>

enum class State { EMPTY, OCCUPIED, DELETED };

struct Slot {
    int key = 0;
    State state = State::EMPTY;
};

class LinearProbingTable {
public:
    explicit LinearProbingTable(int m) : m_(m), n_(0), t_(m) {}

    // 1 = inserted, 0 = duplicate, -1 = table full
    // 'where' = slot used, 'probes' = slots examined
    int insert(int key, int& where, int& probes) {
        int firstDeleted = -1;
        probes = 0;
        for (int i = 0; i < m_; ++i) {
            int j = probe(key, i);                      // h(k, i) = (h(k) + i) mod m
            ++probes;
            const Slot& s = t_[j];
            if (s.state == State::EMPTY) {              // key cannot exist further on
                where = (firstDeleted >= 0) ? firstDeleted : j;
                place(where, key);
                return 1;
            }
            if (s.state == State::DELETED) {
                if (firstDeleted < 0) firstDeleted = j;  // remember first reusable slot
            } else if (s.key == key) {
                where = j;
                return 0;                                // duplicate key
            }
        }
        if (firstDeleted >= 0) {                         // no EMPTY slot, but a tombstone
            where = firstDeleted;
            place(where, key);
            return 1;
        }
        where = -1;
        return -1;                                       // every slot occupied
    }

    int find(int key, int& probes) const {               // slot index or -1
        probes = 0;
        for (int i = 0; i < m_; ++i) {
            int j = probe(key, i);
            ++probes;
            if (t_[j].state == State::EMPTY) return -1;  // end of the probe path
            if (t_[j].state == State::OCCUPIED && t_[j].key == key) return j;
        }
        return -1;
    }

    bool erase(int key) {
        int probes = 0;
        int j = find(key, probes);
        if (j < 0) return false;
        t_[j].state = State::DELETED;                    // tombstone, NOT empty
        --n_;
        return true;
    }

    void display() const {
        std::cout << "slot:";
        for (int j = 0; j < m_; ++j) std::cout << " " << (j < 10 ? "  " : " ") << j;
        std::cout << "\nkey :";
        for (const Slot& s : t_) {
            if (s.state == State::EMPTY) std::cout << "   --";
            else if (s.state == State::DELETED) std::cout << "  DEL";
            else std::cout << " " << (s.key < 10 ? "  " : s.key < 100 ? " " : "") << s.key;
```

```
        }
        std::cout << "\n";
    }

private:
    int probe(int key, int i) const { return (((key % m_) + m_) % m_ + i) % m_; }
    void place(int j, int key) { t_[j].key = key; t_[j].state = State::OCCUPIED; ++n_; }
    int m_;
    int n_;
    std::vector<Slot> t_;
};

int main() {
    LinearProbingTable t(11);
    int where = 0, probes = 0;
    for (int k : {25, 36, 47, 14, 58}) {                // all have 25 mod 11 = 3
        int r = t.insert(k, where, probes);
        std::cout << "insert " << k << " -> slot " << where << " (" << probes
                  << " probes, status " << r << ")\n";
    }
    t.display();
    int j = t.find(58, probes);
    std::cout << "search 58 -> found at slot " << j << " (" << probes << " probes)\n";
    j = t.find(69, probes);
    std::cout << "search 69 -> " << (j < 0 ? "absent" : "found")
              << " (" << probes << " probes)\n";
    std::cout << "erase 36 -> " << t.erase(36) << "\n";
    t.display();
    j = t.find(47, probes);
    std::cout << "search 47 -> found at slot " << j << " (" << probes << " probes)\n";
    int r = t.insert(80, where, probes);
    std::cout << "insert 80 -> slot " << where << " (" << probes
              << " probes, status " << r << ")\n";
    r = t.insert(47, where, probes);
    std::cout << "insert 47 again -> slot " << where << " (" << probes
              << " probes, status " << r << ")\n";
    t.display();
    return 0;
}
```

Output (g++ -std=c++17)

```
insert 25 -> slot 3 (1 probes, status 1)
insert 36 -> slot 4 (2 probes, status 1)
insert 47 -> slot 5 (3 probes, status 1)
insert 14 -> slot 6 (4 probes, status 1)
insert 58 -> slot 7 (5 probes, status 1)
slot:   0   1   2   3   4   5   6   7   8   9  10
key :   --  --  --  25  36  47  14  58  --  --  --
search 58 -> found at slot 7 (5 probes)
search 69 -> absent (6 probes)
erase 36 -> 1
slot:   0   1   2   3   4   5   6   7   8   9  10
key :   --  --  --  25  DEL 47  14  58  --  --  --
search 47 -> found at slot 5 (3 probes)
insert 80 -> slot 4 (6 probes, status 1)
insert 47 again -> slot 5 (3 probes, status 0)
slot:   0   1   2   3   4   5   6   7   8   9  10
key :   --  --  --  25  80  47  14  58  --  --  --
```

Line	Code	What it does
5	`enum class State { EMPTY, OCCUPIED, DELETED };`	Three slot states; `enum class` keeps the names scoped and type-safe.
7	`struct Slot {`	One slot: a key and its state. Default state is EMPTY, so `std::vector<Slot> t_(m)` starts with an empty table.
19	`int firstDeleted = -1;`	Remembers the first tombstone met so that it can be reused after the duplicate check.
21	`for (int i = 0; i < m_; ++i) {`	Loop bound m guarantees termination, even for a full table.
22	`int j = probe(key, i); …`	Computes $h(k, i) = (h(k) + i) \bmod m$ (see `probe` below).
25	`if (s.state == State::EMPTY) { …`	End of the probe path: the key is absent, so place it (preferring the tombstone).
31	`if (firstDeleted < 0) firstDeleted = j; // …`	Only the **first** tombstone is remembered.

Line	Code	What it does
32	`} else if (s.key == key) {`	Occupied slot with the same key: duplicate; return 0 and report the slot.
37	`if (firstDeleted >= 0) { …`	Probe path exhausted without an EMPTY slot, but a tombstone exists: reuse it.
43	`return -1; …`	Nothing free: table full.
51	`if (t_[j].state == State::EMPTY) return -1; …`	In `find`: only an EMPTY slot proves absence.
52	`if (t_[j].state == State::OCCUPIED && t_[j].…`	A tombstone still stores an old key value; it must **not** be matched, hence the state test.
61	`t_[j].state = State::DELETED; …`	Erase = tombstone, never EMPTY.
79	`int probe(int key, int i) const { return (((…`	Safe home index (negative keys) plus the probe offset, wrapped by `% m_`.

NOTE. Dry-run check of `insert(80)` in the last table of the output: probes 3 (25), 4 (DEL; `firstDeleted = 4`), 5 (47), 6 (14), 7 (58), 8 (EMPTY): 6 probes. `where = firstDeleted = 4`; slot 4 receives 80.

7.10 Edge Cases

Case	Expected behaviour
Search in an empty table	First probe finds EMPTY: absent after 1 probe
Delete a missing key	Search ends at EMPTY, returns `false`, nothing changes
Insert duplicate	Detected on the probe path; rejected or updated; n unchanged
Key whose home slot is the last slot	Wrap-around to slot 0 (Figure 7.6)
Negative key	Use the double-modulo formula for the home slot
Table completely full	Insertion returns -1 after m probes; search for absent key costs m probes
Many tombstones	Searches get slower; rebuild (rehash) the table

7.11 Advantages, Disadvantages and Complexity

Advantages	Disadvantages
Very simple to code	**Primary clustering** hurts at high load
Excellent cache locality: probes touch adjacent memory	Needs low load factor (about 0.5–0.7)
No pointers or dynamic allocation per key	Deletion needs tombstones (or backward shift)
Often the fastest in practice for small keys	Sensitive to poor hash functions that create long runs

Operation	Expected (uniform keys, fixed $\alpha < 1$)	Worst case	Space
Insert	$O(1)$	$O(n)$ (or $O(m)$)	$O(m)$
Search	$O(1)$	$O(n)$	
Delete (tombstone)	$O(1)$	$O(n)$	

PART 8

QUADRATIC PROBING

Jumping by growing steps to avoid primary clustering, and why the probe sequence may not cover the table

8.1 Definition and Motivation

Linear probing puts a colliding key into the **next** slot, so clusters grow. **Quadratic probing** jumps by steps that **increase**: 1, then 3, then 6 ... (or 1, 4, 9 ...) away from the home slot. Keys that collide at nearby slots are therefore sent to different places, and large contiguous clusters do not build up.

DEFINITION. **Quadratic probing** examines the slots $h(k)$, $h(k)$ + offset$_1$, $h(k)$ + offset$_2$, ... where the offsets are a **quadratic polynomial in the probe number** i, all taken modulo m.

8.2 The Probing Formula

$$h(k, i) = (h(k) + c_1\, i + c_2\, i^2) \bmod m \text{ , } i = 0, 1, \ldots, m-1$$

Symbol	Meaning
$h(k)$	ordinary hash (home slot), e.g. k mod m
c_1, c_2	constants chosen by the designer ($c_2 \neq 0$, otherwise the method is linear probing)
i	probe number, starting at 0
m	table size

Implementation convention in this book. The two most useful choices need c = ½, which is not an integer. So the code computes the offset as an integer expression:

$$\text{offset}(i) = (C_1\, i + C_2\, i^2) / 2 \; i.e.\; c_1 = C_1/2 \text{ , } c_2 = C_2/2$$

Name	(C_1, C_2)	(c_1, c_2)	Offsets for i = 0, 1, 2, 3, 4, ...
Triangular probing	(1, 1)	(½, ½)	0, 1, 3, 6, 10, 15, ... (sums 1+2+3+...)
Pure squares	(0, 2)	(0, 1)	0, 1, 4, 9, 16, 25, ...
Mixed (e.g. textbooks)	(2, 2)	(1, 1)	0, 2, 6, 12, 20, ...

NOTE. Triangular offsets can also be generated **incrementally**: slot$_{i+1}$ = slot$_i$ + (i+1), mod m. No multiplication is needed: jump by 1, then 2, then 3, ...

8.3 Special Cases, Table Size and Coverage

The biggest question about any probing rule is: **does the sequence visit every slot?** If not, an insertion can fail although free slots exist. The answer depends on m and the constants. The table below was computed by running the probe formula for i = 0 ... m-1 from home slot 0.

m	Rule	Distinct slots visited	Visits all slots?
8	triangular	8	yes
16	triangular	16	yes
10	triangular	6	**no** (6 of 10)
7	triangular	4	**no** (4 of 7)
11	pure squares	6	**no** (6 of 11)
13	pure squares	7	**no** (7 of 13)
10	pure squares	6	**no** (6 of 10)
8	pure squares	3	**no** (3 of 8)

- **Triangular probing with m a power of two** visits **every** slot (a known result; the 8-slot example is traced below).
- **Pure squares with m an odd prime** visit exactly $(m+1)/2$ different slots, because i^2 and $(m-i)^2$ are equal modulo m. Consequence: if the table is **less than half full**, an insertion is **guaranteed** to find a free slot among those $(m+1)/2$ slots.
- **Other combinations** (pure squares with composite m, triangular with non-power-of-two m) may visit even fewer slots.
- No simple quadratic rule is guaranteed to visit all slots for **every** m. **Do not** write "quadratic probing always finds an empty slot" in an exam.

EXAM TRAP. Safe wording: "Quadratic probing can find a free slot only if its probe sequence reaches one. For a prime table size and load factor below 0.5 this is guaranteed for $h + i^2$; with $m = 2^p$ and triangular offsets every slot is visited; otherwise it may fail even when empty slots remain."

8.4 Step-by-step Probing: Triangular Offsets, $m = 8$

Eight keys 8, 16, 24, 32, 40, 48, 56, 64 all have k mod 8 = 0: the same home slot. Offsets for i = 0–7 are 0, 1, 3, 6, 10, 15, 21, 28; modulo 8 these are 0, 1, 3, 6, 2, 7, 5, 4, all different.

Key	Probe sequence (slots)	Stored at	Probes
8	0	0	1
16	0 → 1	1	2
24	0 → 1 → 3	3	3
32	0 → 1 → 3 → 6	6	4
40	0 → 1 → 3 → 6 → 2	2	5
48	0 → 1 → 3 → 6 → 2 → 7	7	6
56	0 → 1 → 3 → 6 → 2 → 7 → 5	5	7
64	0 → 1 → 3 → 6 → 2 → 7 → 5 → 4	4	8

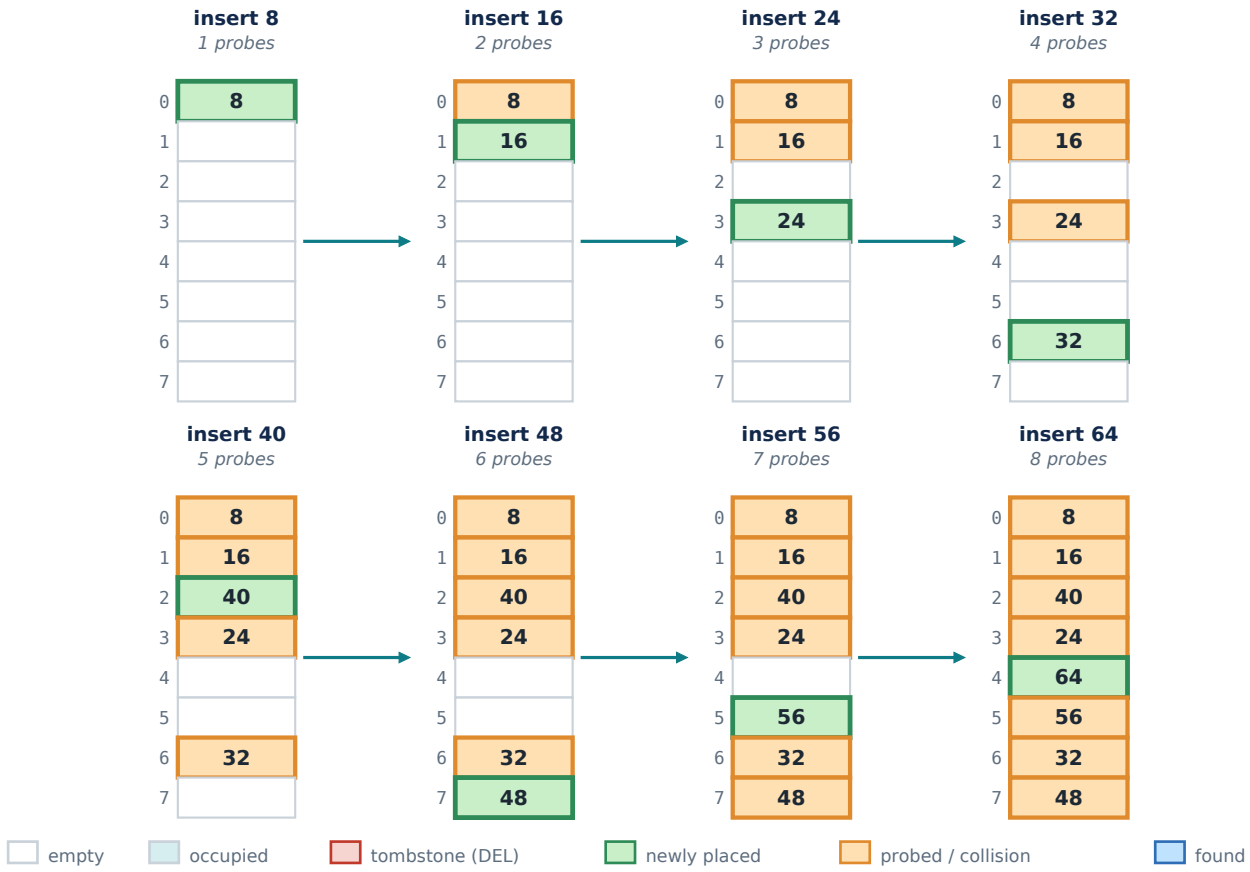


Figure 8.1 — Eight keys with the same home slot 0 fill the whole table, using the slots in the order 0, 1, 3, 6, 2, 7, 5, 4.

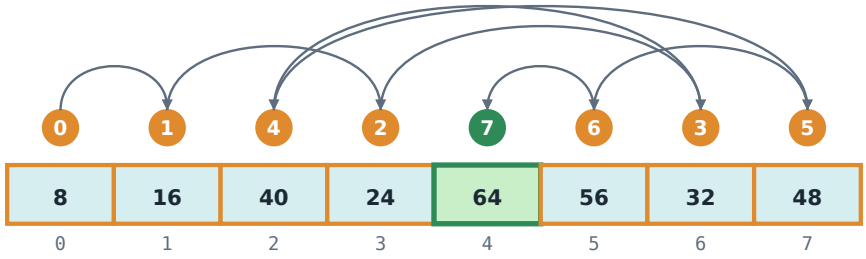


Figure 8.2 — Probe path of key 64: the slot visits are 0, 1, 3, 6, 2, 7, 5, 4. The last probe finds the only free slot.

The ninth key 72 (home 0) examines all eight slots, finds no EMPTY slot and must report failure: the table is **really full** (0 empty slots). Nothing is wrong with the method here; the table simply has no room.

8.5 Worked Insertion, Search and Deletion

Operation	Probe sequence	Result
insert 72 (table full)	0 → 1 → 3 → 6 → 2 → 7 → 5 → 4	all 8 slots occupied: **FAILED** (0 empty slots remain)
erase 24	0 → 1 → 3 (holds 24)	slot 3 := DELETED
search 64	0 → 1 → 3 → 6 → 2 → 7 → 5 → 4	found in slot 4 after 8 probes (tombstone at slot 3 skipped)
insert 72 again	0 → 1 → 3 → 6 → 2 → 7 → 5 → 4	no EMPTY slot, but tombstone 3 was remembered: **stored at slot 3**

COMMON MISTAKE. Using the number of probes as the **only** failure signal and then calling the table "full". In quadratic probing a failed insertion does **not** prove that all slots are occupied (see Experiment B). Check the count of EMPTY slots, or the load factor.

8.6 Primary versus Secondary Clustering

- **Primary clustering is reduced:** keys with different home slots follow different jump patterns, so they do not merge into a single run. In Figure 8.1 the occupied slots 0, 1, 3, 6 are not contiguous.
- **Secondary clustering remains:** all keys with the **same home slot** follow **exactly the same probe sequence** (Figure 8.2). Ten keys with home slot 5 all try 5, 6, 9, 4, 1, 0 … in the same order; each one has to step over all earlier ones.
- The cause: $h(k, i)$ depends on the key only through $h(k)$. Different keys with equal $h(k)$ cannot be told apart. Double hashing (Part 9) fixes this.

8.7 Why Quadratic Probing May Fail to Visit Every Slot: Experiment B

Take the **composite** size $m = 10$ with pure square offsets i^2. The offsets modulo 10 for $i = 0 \ldots 9$ are 0, 1, 4, 9, 6, 5, 6, 9, 4, 1: only the values {0, 1, 4, 5, 6, 9} occur. From home slot 5 the reachable slots are 5, 6, 9, 4 (= 14), 1 (= 11), 0 (= 10): **6 of 10**. Slots 2, 3, 7, 8 can **never** be reached from home slot 5. We insert 5, 15, 25, 35, 45, 55 (all home slot 5), then 65.

Key	Probe sequence	Outcome
5	5	stored at 5
15	5 → 6	stored at 6
25	5 → 6 → 9	stored at 9
35	5 → 6 → 9 → 4	stored at 4
45	5 → 6 → 9 → 4 → 1	stored at 1
55	5 → 6 → 9 → 4 → 1 → 0	stored at 0
65	5 → 6 → 9 → 4 → 1 → 0 → 1 → 4 → 9 → 6	**FAILED** (probes repeat; 4 slots still empty)

Figure 8.3 draws the probe sequence of the failing insertion of key 65.

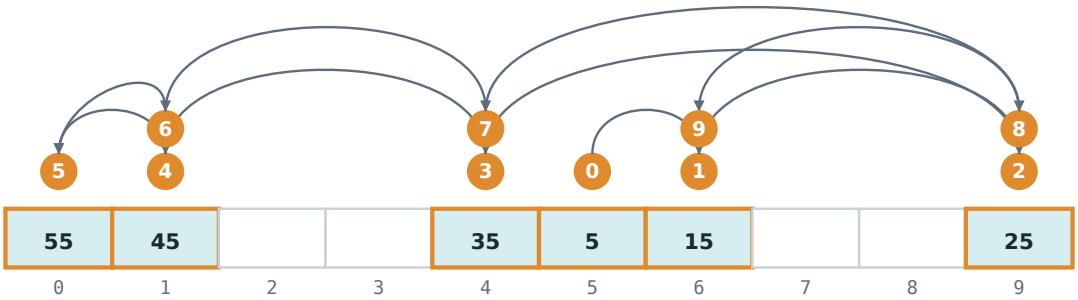


Figure 8.3 — Failure of key 65 (home slot 5) with pure square probing in a table of size 10. The sequence 5, 6, 9, 4, 1, 0 repeats; empty slots 2, 3, 7, 8 are never reached. Stacked circles show revisits.

8.8 Handling a Failed Insertion when Slots Remain Empty

When the loop of m probes ends without success but $n < m$, the correct reactions are:

1. **Report failure honestly** (return a status); never loop forever and never overwrite another key.
2. **Rehash into a larger table** (for example the next prime above $2m$) and insert again (Part 10). The new size changes the probe sequence.
3. **Prevent the situation** by keeping the load factor below 0.5 (prime m, pure squares) or by using $m = 2^p$ with triangular offsets.
4. As a last resort fall back to another method (for example linear probing) after a threshold number of probes.

8.9 Tombstones and Deletion

Deletion works exactly as in linear probing (Part 7): follow the probe sequence, stop at EMPTY, and turn the found slot into a tombstone. Searches skip tombstones; insertions remember the first one and reuse it after the full sequence has been checked for a duplicate (as in the trace above).

8.10 Complete C++ Implementation

The class is parameterised by (C_1, C_2). `main` runs Experiment A (triangular, $m = 8$) and Experiment B (pure squares, $m = 10$) and prints **every probe**.

```cpp
// Part 8: quadratic probing, h(k,i) = (h(k) + (C1*i + C2*i*i)/2) mod m  (C++17)
#include <iostream>
#include <vector>

enum class State { EMPTY, OCCUPIED, DELETED };
struct Slot { int key = 0; State state = State::EMPTY; };

class QuadraticTable {
public:
    // c1 = C1/2 and c2 = C2/2.  (C1,C2) = (1,1): triangular numbers; (0,2): plain i*i
    QuadraticTable(int m, int C1, int C2) : m_(m), C1_(C1), C2_(C2), t_(m) {}

    int slotFor(int key, int i) const {
        long long h = ((key % m_) + m_) % m_;
        long long ii = i;
        long long off = (C1_ * ii + C2_ * ii * ii) / 2;
        return static_cast<int>((h + off) % m_);
    }

    // 1 inserted, 0 duplicate, -1 failed (probe sequence exhausted); prints every probe
    int insert(int key) {
        int firstDeleted = -1;
        std::cout << "insert " << key << ": probes";
        for (int i = 0; i < m_; ++i) {
            int j = slotFor(key, i);
            std::cout << " " << j;
            if (t_[j].state == State::EMPTY) {
                int where = (firstDeleted >= 0) ? firstDeleted : j;
                t_[where].key = key;
                t_[where].state = State::OCCUPIED;
                std::cout << " -> stored at " << where << "\n";
                return 1;
            }
            if (t_[j].state == State::DELETED) {
                if (firstDeleted < 0) firstDeleted = j;
            } else if (t_[j].key == key) {
                std::cout << " -> duplicate\n";
                return 0;
            }
        }
        if (firstDeleted >= 0) {
            t_[firstDeleted].key = key;
            t_[firstDeleted].state = State::OCCUPIED;
            std::cout << " -> stored at tombstone " << firstDeleted << "\n";
            return 1;
        }
        std::cout << " -> FAILED after " << m_ << " probes, " << emptyCount()
                  << " empty slots remain\n";
        return -1;
    }

    int find(int key) const {
        for (int i = 0; i < m_; ++i) {
            int j = slotFor(key, i);
            if (t_[j].state == State::EMPTY) return -1;
            if (t_[j].state == State::OCCUPIED && t_[j].key == key) return j;
        }
        return -1;
    }

    bool erase(int key) {
        int j = find(key);
        if (j < 0) return false;
        t_[j].state = State::DELETED;
```

```cpp
        return true;
    }

    int emptyCount() const {
        int c = 0;
        for (const Slot& s : t_) if (s.state == State::EMPTY) ++c;
        return c;
    }

    int distinctSlots(int key) const {                  // slots reachable for this key
        std::vector<bool> seen(m_, false);
        int c = 0;
        for (int i = 0; i < m_; ++i) {
            int j = slotFor(key, i);
            if (!seen[j]) { seen[j] = true; ++c; }
        }
        return c;
    }

    void display() const {
        for (int j = 0; j < m_; ++j) {
            std::cout << "  [" << j << "] ";
            if (t_[j].state == State::EMPTY) std::cout << "--\n";
            else if (t_[j].state == State::DELETED) std::cout << "DEL\n";
            else std::cout << t_[j].key << "\n";
        }
    }

private:
    int m_, C1_, C2_;
    std::vector<Slot> t_;
};

int main() {
    std::cout << "=== A: m = 8, triangular probing (c1 = c2 = 1/2) ===\n";
    QuadraticTable a(8, 1, 1);
    for (int k : {8, 16, 24, 32, 40, 48, 56, 64}) a.insert(k);      // all home slot 0
    std::cout << "distinct slots reachable from home 0: " << a.distinctSlots(8) << " of 8\n";
    a.insert(72);                                                   // table is really full
    std::cout << "erase 24 -> " << a.erase(24) << ", find 64 -> slot " << a.find(64) << "\n";
    a.insert(72);                                                   // reuses the tombstone

    std::cout << "=== B: m = 10, plain i*i probing (c1 = 0, c2 = 1) ===\n";
    QuadraticTable b(10, 0, 2);
    std::cout << "distinct slots reachable from home 5: " << b.distinctSlots(5) << " of 10\n";
    for (int k : {5, 15, 25, 35, 45, 55, 65}) b.insert(k);          // all home slot 5
    b.display();
    return 0;
}
```

Output (g++ -std=c++17)

```
=== A: m = 8, triangular probing (c1 = c2 = 1/2) ===
insert 8: probes 0 -> stored at 0
insert 16: probes 0 1 -> stored at 1
insert 24: probes 0 1 3 -> stored at 3
insert 32: probes 0 1 3 6 -> stored at 6
insert 40: probes 0 1 3 6 2 -> stored at 2
insert 48: probes 0 1 3 6 2 7 -> stored at 7
insert 56: probes 0 1 3 6 2 7 5 -> stored at 5
insert 64: probes 0 1 3 6 2 7 5 4 -> stored at 4
distinct slots reachable from home 0: 8 of 8
insert 72: probes 0 1 3 6 2 7 5 4 -> FAILED after 8 probes, 0 empty slots remain
erase 24 -> 1, find 64 -> slot 4
insert 72: probes 0 1 3 6 2 7 5 4 -> stored at tombstone 3
=== B: m = 10, plain i*i probing (c1 = 0, c2 = 1) ===
distinct slots reachable from home 5: 6 of 10
insert 5: probes 5 -> stored at 5
insert 15: probes 5 6 -> stored at 6
insert 25: probes 5 6 9 -> stored at 9
insert 35: probes 5 6 9 4 -> stored at 4
insert 45: probes 5 6 9 4 1 -> stored at 1
insert 55: probes 5 6 9 4 1 0 -> stored at 0
insert 65: probes 5 6 9 4 1 0 1 4 9 6 -> FAILED after 10 probes, 4 empty slots remain
  [0] 55
  [1] 45
  [2] --
  [3] --
  [4] 35
```

```
  [5] 5
  [6] 15
  [7] --
  [8] --
  [9] 25
```

Line	Code	What it does
11	`QuadraticTable(int m, int C1, int C2) : m_(m…`	Constants are stored doubled ($C = 2c$) so that ½ can be used without floating point.
14	`long long h = ((key % m_) + m_) % m_;`	Safe home slot for negative keys; `long long` avoids overflow in the next lines.
16	`long long off = (C1_ * ii + C2_ * ii * ii) /…`	Offset $(C_1\, i + C_2\, i^2)/2$ in 64-bit arithmetic; the numerator is always even for the chosen constants, so the division is exact.
17	`return static_cast<int>((h + off) % m_);`	Final wrap into 0 ... m-1.
26	`std::cout << " " << j;`	Prints each probe so the trace in the book can be compared line by line.
27	`if (t_[j].state == State::EMPTY) {`	EMPTY ends the probe path: store the key (at the first tombstone if one was seen).
47	`std::cout << " -> FAILED after " << m_ << " …`	All m probes used: reports how many EMPTY slots remain, proving that a failure is not the same as a full table.
74	`int distinctSlots(int key) const { …`	Counts how many different slots the probe sequence of a key reaches (coverage test).
100	`QuadraticTable a(8, 1, 1);`	Experiment A: triangular probing, covers all 8 slots.
108	`QuadraticTable b(10, 0, 2);`	Experiment B: pure squares in a composite table, covers only 6 slots.

8.11 Advantages, Limitations and Comparison with Linear Probing

Aspect	Linear probing	Quadratic probing
Probe step	always 1	grows with i
Primary clustering	**yes**	largely avoided
Secondary clustering	yes	yes
Visits every slot?	always	**only for suitable m and constants**
Cache behaviour	best (adjacent slots)	good at first, worse for later probes
Implementation	simplest	slightly more complex; care with overflow
Deletion	tombstones	tombstones
Safe maximum load factor	0.5–0.7	< 0.5 guaranteed (prime m, pure squares)

Operation	Expected (suitable m, $\alpha < 0.5$)	Worst case	Space
Insert	$O(1)$	$O(m)$ probes, may fail	$O(m)$
Search	$O(1)$	$O(m)$	
Delete	$O(1)$	$O(m)$	

PART 9

DOUBLE HASHING

A second hash function chooses the step size, giving every key its own probe sequence

9.1 Definition and Motivation

Linear probing always steps by 1 and quadratic probing always follows the same pattern for a given home slot. Both therefore give **identical probe sequences to keys that share a home slot** (secondary clustering). **Double hashing** removes this by letting a **second hash function** decide the **step size**.

DEFINITION. **Double hashing** probes the slots $h_1(k)$, $h_1(k) + h_2(k)$, $h_1(k) + 2h_2(k)$, ... (mod m). The first function gives the **starting slot**, the second gives the **jump length**. Two keys with the same home slot usually get different jumps.

9.2 The Formula: Primary and Secondary Hash Functions

$$h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m \ , \ i = 0, 1, ..., m-1$$

Symbol	Role	Typical choice (m prime)
$h_1(k)$	primary hash: **home slot**	k mod m
$h_2(k)$	secondary hash: **step size**	1 + (k mod (m-1))
i	probe number	0, 1, 2, ...
m	table size	a prime number such as 11, 13, 101

NOTE. Linear probing is the special case $h_2(k) = 1$. Double hashing generalises it by making the step depend on the key.

9.3 Choosing h_1 and h_2

A good pair satisfies four conditions:

1. $h_2(k) \neq 0$ **for every key**, otherwise the probe sequence never moves.
2. $h_2(k)$ **is coprime to** m (shares no factor with m), so that the sequence visits every slot (Section 9.5).
3. h_2 **is as independent as possible from** h_1, otherwise keys with the same h_1 also get the same h_2 and the benefit is lost.
4. Both are cheap to compute and use the whole key.

Design	m	$h_1(k)$	$h_2(k)$	Range of h_2 / why coprime
A (used in this book)	prime	k mod m	1 + (k mod (m-1))	1 ... m-1: every value is coprime to a prime m
B	prime	k mod m	R - (k mod R), R a prime < m	1 ... R: nonzero and < m
C	2^p	k mod m	any **odd** number, e.g. 2(k mod (m/2)) + 1	odd numbers are coprime to 2^p

9.4 Why h_2 Must Never Be Zero

If $h_2(k) = 0$ then $h(k, i) = h_1(k)$ for every i: the sequence examines **one slot over and over**. The loop can then never find a free slot, even in an almost empty table. The program in this chapter shows it: `m = 11, step 0: visits 1 of 11 slots`.

> **COMMON MISTAKE.** Using $h_2(k) = k$ mod $(m\text{-}1)$ or k mod m directly. Any key that is a multiple of the modulus yields **step 0**. The standard fix is the shift by one: $1 + (k \bmod (m\text{-}1))$ guarantees a value from 1 to m-1.

9.5 Coprimality: Why the Step Size Must Share No Factor with m

Starting from slot s with step d, the probe slots are s, s+d, s+2d, ... (mod m). The sequence returns to the starting slot after exactly m / gcd(d, m) probes, so it visits only that many different slots:

$$\text{number of distinct slots visited} = m \,/\, \gcd(d, m)$$

m	step d	gcd(d, m)	slots visited	Program output
10	5	5	2	m = 10, step 5: visits 2 of 10 slots
10	4	2	5	m = 10, step 4: visits 5 of 10 slots
10	3	1	10	m = 10, step 3: visits 10 of 10 slots
11	5	1	11	m = 11, step 5: visits 11 of 11 slots

- **Prime m:** every step 1 ... m-1 is coprime to m, so every probe sequence is a **permutation of all slots**. This is why double hashing is normally used with a prime table size.
- $m = 2^p$**:** every **odd** step is coprime to m; even steps are not.
- **Composite** m with an arbitrary h_2 is dangerous (Section 9.10).

9.6 Worked Example with Complete Probe Sequences

Table size m = 11 (prime), $h_1(k) = k$ mod 11, $h_2(k) = 1 + (k \bmod 10)$. We insert the **same five keys as in linear probing** (Part 7): 25, 36, 47, 14, 58. All have home slot 3, but different steps:

Key	h_1	$h_2 = 1 + k$ **mod 10**	Probe sequence	Stored at	Probes
25	3	1 + 5 = 6	3	3	1
36	3	1 + 6 = 7	3 → 10	10	2
47	3	1 + 7 = 8	3 → 0	0	2
14	3	1 + 4 = 5	3 → 8	8	2
58	3	1 + 8 = 9	3 → 1	1	2

Check key 36: slot 3 is occupied by 25; second probe is $(3 + 1 \cdot 7)$ mod 11 = 10, which is free. Check key 47: $(3 + 8)$ mod 11 = 0. Check key 58: $(3 + 9)$ mod 11 = 1.

In Figure 9.1 the five keys of Part 7 are inserted by double hashing; compare with Figure 7.1.



Figure 9.1 — Double hashing, same five keys as Figure 7.1. Keys scatter over slots 0, 1, 3, 8, 10 instead of forming a run 3–7.

Figure 9.2 compares the final tables of linear probing and double hashing.

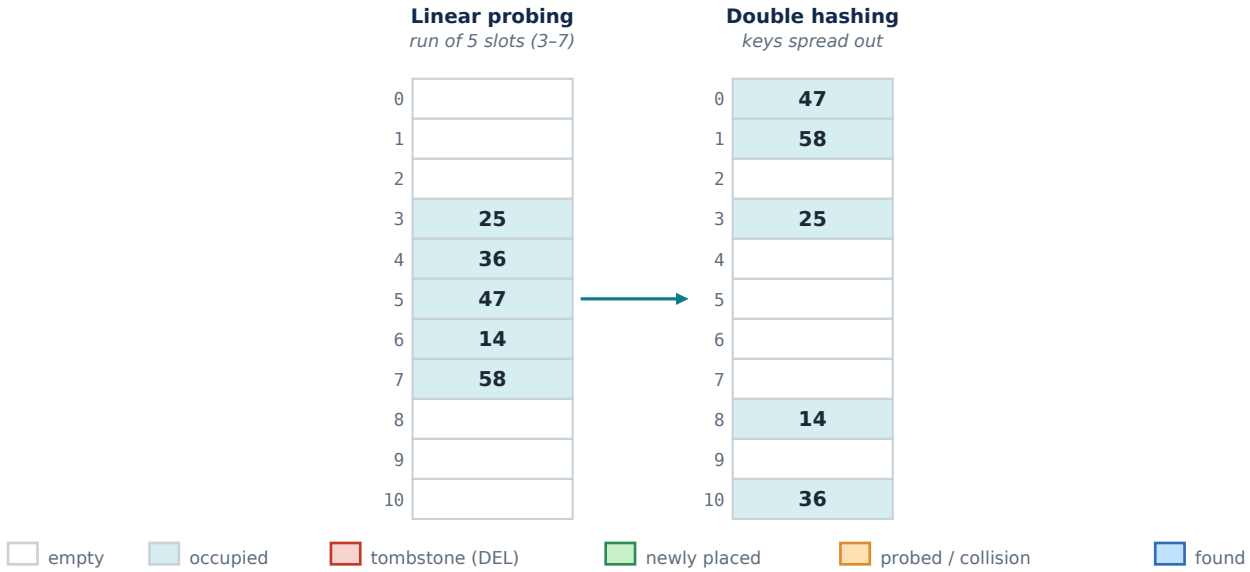


Figure 9.2 — Final tables compared. Total probes for the five insertions: linear 1+2+3+4+5 = 15; double hashing 1+2+2+2+2 = 9.

Figure 9.3 shows that searching key 58 now takes two probes.

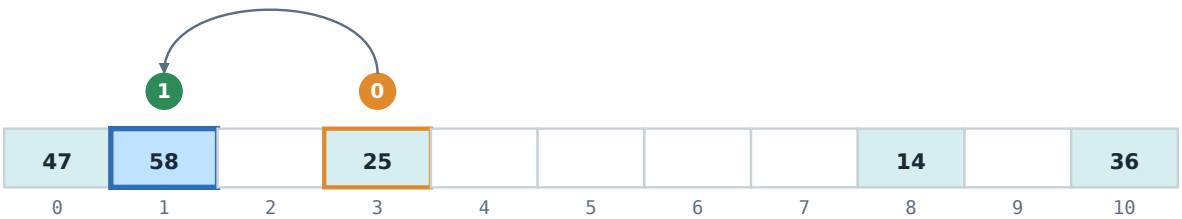


Figure 9.3 — Searching key 58 under double hashing takes two probes: slot 3 (holds 25), then slot 1 (holds 58). Linear probing needed five probes.

9.7 Insertion, Search and Deletion

Operation	Probe sequence	Result
search 58	3 → 1	found at slot 1: 2 probes
search 69 (absent)	$h_1 = 3$, $h_2 = 10$: 3 → 2	slot 2 is EMPTY: **absent** after 2 probes
erase 36	3 → 10 (holds 36)	slot 10 := DELETED
search 47	3 → 0	found at slot 0: 2 probes (tombstone not on its path)
insert 80	$h_1 = 3$, $h_2 = 1$: 3 → 4	stored at slot 4, an EMPTY slot
insert 21	$h_1 = 10$, $h_2 = 2$: 10 → 1 → 3 → 5	slot 10 is the tombstone (remembered); slots 1 and 3 occupied; slot 5 EMPTY → key stored in **tombstone slot 10**, 4 probes

Figure 9.4 shows deletion and tombstone reuse under double hashing.



Figure 9.4 — Deletion and tombstone reuse under double hashing.

9.8 Tombstones, Clustering and Table-full Detection

- **Tombstones** work exactly as in Parts 6–8: skip while searching, remember the first one while inserting, reuse it after the probe path has been checked for duplicates.
- **Clustering:** primary clustering is absent (a key does not join the run of another home slot) and secondary clustering is nearly absent because keys with equal h_1 usually get different steps. Only keys with **both** h_1 and h_2 equal share a probe sequence.
- **Table-full detection:** with prime m and a nonzero step every sequence is a permutation, so after m probes **all** slots have been examined: failure then really means "no EMPTY slot and no tombstone". With a bad h_2 this conclusion is **false**.

9.9 Comparison with Linear and Quadratic Probing

Feature	Linear	Quadratic	Double hashing
Probe step	1	grows with i	$h_2(k)$, depends on the key
Primary clustering	yes	no	no
Secondary clustering	yes	yes	practically no
Covers all slots	always	only for special m	if $h_2(k)$ coprime to m (prime m: always)
Cost per probe	lowest	low	highest (second hash, multiplication)
Cache locality	best	medium	worst (jumps)
Expected probes at $\alpha = 0.8$ (unsuccessful)	$\frac{1}{2}[1 + 1/(1-\alpha)^2] = 13.0$	between the two	$1/(1-\alpha) = 5.0$
Typical use	small, fast tables	teaching, moderate loads	high load, many collisions

The last numbers are the classical expected-probe formulas of Part 6; they apply to random keys and are not measurements.

9.10 A Poor Secondary Hash Function: Experiment

Take **composite** $m = 10$ and $h_2(k) = 5 - (k \bmod 5)$, which is nonzero but not always coprime to 10. For keys 10, 20, 30 (home slot 0, $h_2 = 5$) the probe sequence is 0, 5, 0, 5, ...:

Figure 9.5 shows how a poor step size traps the probe sequence in two slots.

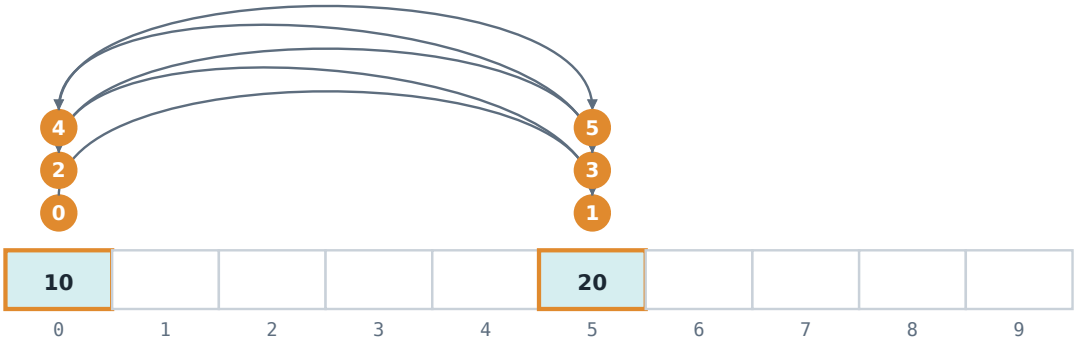


Figure 9.5 — With step 5 in a table of 10 slots, key 30 can only ever probe slots 0 and 5. Both are taken by 10 and 20, so the third key fails although 8 slots are empty.

WARNING. Result: key 30 cannot be inserted while 8 of 10 slots are free. A poor h_2 (or composite m) silently turns double hashing into a method that reaches only $m/\gcd(h_2, m)$ slots.

9.11 Complete C++ Implementation

```cpp
// Part 9: double hashing, h(k,i) = (h1(k) + i*h2(k)) mod m  (C++17)
#include <iostream>
#include <vector>

enum class State { EMPTY, OCCUPIED, DELETED };
struct Slot { int key = 0; State state = State::EMPTY; };

class DoubleHashTable {
public:
    explicit DoubleHashTable(int m) : m_(m), t_(m) {}       // m should be prime

    int h1(int key) const { return ((key % m_) + m_) % m_; }
    int h2(int key) const { return 1 + (((key % (m_ - 1)) + (m_ - 1)) % (m_ - 1)); }  // 1..m-1
    int slotFor(int key, int i) const {
        return static_cast<int>((h1(key) + static_cast<long long>(i) * h2(key)) % m_);
    }

    int insert(int key) {                                   // 1 inserted, 0 duplicate, -1 full
        int firstDeleted = -1;
```

```
        std::cout << "insert " << key << " (h1=" << h1(key) << ", h2=" << h2(key) << "): probes";
        for (int i = 0; i < m_; ++i) {
            int j = slotFor(key, i);
            std::cout << " " << j;
            if (t_[j].state == State::EMPTY) {
                int where = (firstDeleted >= 0) ? firstDeleted : j;
                t_[where].key = key;
                t_[where].state = State::OCCUPIED;
                std::cout << " -> stored at " << where << "\n";
                return 1;
            }
            if (t_[j].state == State::DELETED) {
                if (firstDeleted < 0) firstDeleted = j;
            } else if (t_[j].key == key) {
                std::cout << " -> duplicate\n";
                return 0;
            }
        }
        if (firstDeleted >= 0) {
            t_[firstDeleted].key = key;
            t_[firstDeleted].state = State::OCCUPIED;
            std::cout << " -> stored at tombstone " << firstDeleted << "\n";
            return 1;
        }
        std::cout << " -> FULL\n";
        return -1;
    }

    int find(int key, int& probes) const {
        probes = 0;
        for (int i = 0; i < m_; ++i) {
            int j = slotFor(key, i);
            ++probes;
            if (t_[j].state == State::EMPTY) return -1;
            if (t_[j].state == State::OCCUPIED && t_[j].key == key) return j;
        }
        return -1;
    }

    bool erase(int key) {
        int probes = 0;
        int j = find(key, probes);
        if (j < 0) return false;
        t_[j].state = State::DELETED;
        return true;
    }

    void display() const {
        for (int j = 0; j < m_; ++j) {
            std::cout << "  [" << j << "] ";
            if (t_[j].state == State::EMPTY) std::cout << "--\n";
            else if (t_[j].state == State::DELETED) std::cout << "DEL\n";
            else std::cout << t_[j].key << "\n";
        }
    }

private:
    int m_;
    std::vector<Slot> t_;
};

// How many different slots does a probe sequence with this home and step visit?
int coverage(int m, int home, int step) {
    std::vector<bool> seen(m, false);
    int c = 0;
    for (int i = 0; i < m; ++i) {
        int j = (home + i * step) % m;
        if (!seen[j]) { seen[j] = true; ++c; }
    }
    return c;
}

int main() {
    std::cout << "=== m = 11 (prime), h1 = k mod 11, h2 = 1 + k mod 10 ===\n";
    DoubleHashTable t(11);
    for (int k : {25, 36, 47, 14, 58}) t.insert(k);         // all have home slot 3
    t.display();
    int probes = 0;
```

```
    int j = t.find(58, probes);
    std::cout << "search 58 -> slot " << j << " (" << probes << " probes)\n";
    j = t.find(69, probes);
    std::cout << "search 69 -> " << (j < 0 ? "absent" : "found") << " (" << probes << " probes)\n";
    std::cout << "erase 36 -> " << t.erase(36) << "\n";
    j = t.find(47, probes);
    std::cout << "search 47 -> slot " << j << " (" << probes << " probes)\n";
    t.insert(80);
    t.insert(21);                                          // home slot 10 holds the tombstone
    t.display();

    std::cout << "=== Why the step size matters ===\n";
    std::cout << "m = 10, step 5: visits " << coverage(10, 0, 5) << " of 10 slots\n";
    std::cout << "m = 10, step 4: visits " << coverage(10, 0, 4) << " of 10 slots\n";
    std::cout << "m = 10, step 3: visits " << coverage(10, 0, 3) << " of 10 slots\n";
    std::cout << "m = 11, step 5: visits " << coverage(11, 0, 5) << " of 11 slots\n";
    std::cout << "m = 11, step 0: visits " << coverage(11, 0, 0) << " of 11 slots\n";

    std::cout << "=== A poor h2 on m = 10: h2(k) = 5 - k mod 5 ===\n";
    for (int k : {10, 20, 30}) {
        int step = 5 - k % 5;
        std::cout << "key " << k << ": home " << k % 10 << ", step " << step << ", probes";
        for (int i = 0; i < 6; ++i) std::cout << " " << (k % 10 + i * step) % 10;
        std::cout << " ...\n";
    }
    return 0;
}
```

Output (g++ -std=c++17)

```
=== m = 11 (prime), h1 = k mod 11, h2 = 1 + k mod 10 ===
insert 25 (h1=3, h2=6): probes 3 -> stored at 3
insert 36 (h1=3, h2=7): probes 3 10 -> stored at 10
insert 47 (h1=3, h2=8): probes 3 0 -> stored at 0
insert 14 (h1=3, h2=5): probes 3 8 -> stored at 8
insert 58 (h1=3, h2=9): probes 3 1 -> stored at 1
  [0] 47
  [1] 58
  [2] --
  [3] 25
  [4] --
  [5] --
  [6] --
  [7] --
  [8] 14
  [9] --
  [10] 36
search 58 -> slot 1 (2 probes)
search 69 -> absent (2 probes)
erase 36 -> 1
search 47 -> slot 0 (2 probes)
insert 80 (h1=3, h2=1): probes 3 4 -> stored at 4
insert 21 (h1=10, h2=2): probes 10 1 3 5 -> stored at 10
  [0] 47
  [1] 58
  [2] --
  [3] 25
  [4] 80
  [5] --
  [6] --
  [7] --
  [8] 14
  [9] --
  [10] 21
=== Why the step size matters ===
m = 10, step 5: visits 2 of 10 slots
m = 10, step 4: visits 5 of 10 slots
m = 10, step 3: visits 10 of 10 slots
m = 11, step 5: visits 11 of 11 slots
m = 11, step 0: visits 1 of 11 slots
=== A poor h2 on m = 10: h2(k) = 5 - k mod 5 ===
key 10: home 0, step 5, probes 0 5 0 5 0 5 ...
key 20: home 0, step 5, probes 0 5 0 5 0 5 ...
key 30: home 0, step 5, probes 0 5 0 5 0 5 ...
```

Line	Code	What it does
10	`explicit DoubleHashTable(int m) : m_(m), t_(…`	Constructor; m must be prime for the coprimality guarantee.
12	`int h1(int key) const { return ((key % m_) +…`	Primary hash: safe remainder (non-negative even for negative keys).
13	`int h2(int key) const { return 1 + (((key % …`	Secondary hash 1 + (k mod (m-1)): always between 1 and m-1, never 0.
15	`return static_cast<int>((h1(key) + static_ca…`	64-bit multiplication prevents overflow for large i and large steps.
20	`std::cout << "insert " << key << " (h1=" << …`	Shows h_1, h_2 and each probe, which reproduces the tables above.
32	`if (firstDeleted < 0) firstDeleted = j;`	Remember the first tombstone on the path (reused later).
44	`std::cout << " -> FULL\n";`	All m probes failed: with prime m this means the table is truly full.
82	`int coverage(int m, int home, int step) {`	Counts distinct slots visited by `home + i*step`: demonstrates the gcd rule.
118	`int step = 5 - k % 5;`	The deliberately poor h_2 of Section 9.10.

9.12 Advantages, Limitations and Complexity

Advantages	Limitations
Nearly ideal spread: closest to "uniform hashing"	Needs two good hash functions and (normally) prime m
No primary clustering, almost no secondary clustering	Poor cache locality: probes jump around memory
Handles higher load factors than linear probing	More expensive per probe
Probe sequence is a permutation for prime m	Bad h_2 or composite m can break coverage
Deletion by tombstones is straightforward	Resizing needs a new prime and a fresh h_2 range

Operation	Expected (α fixed < 1, uniform hashing)	Worst case
Insert	$O(1)$ (about $1/(1-\alpha)$ probes)	$O(m)$
Search (successful)	$O(1)$ (about $(1/\alpha)\ln(1/(1-\alpha))$ probes)	$O(m)$
Search (unsuccessful)	$O(1)$ (about $1/(1-\alpha)$ probes)	$O(m)$
Space	$O(m)$	$O(m)$

PART 10

LOAD FACTOR, RESIZING AND REHASHING

Keeping tables sparse enough to be fast: when to grow, how to rebuild, and what it costs

10.1 Load Factor

DEFINITION. The **load factor** of a hash table is the average number of keys per slot: $\alpha = n / m$, where n = number of keys stored and m = number of slots (buckets).

$$\alpha = n / m$$

Symbol	Meaning	Example
n	number of **live** keys currently stored (not counting deleted ones)	7
m	table capacity: number of slots or buckets	23
α	load factor	7/23 = 0.30

- **Open addressing:** $0 \le \alpha \le 1$. At $\alpha = 1$ the table is full. Tombstones occupy slots too, so the **effective load** is $(n + d)/m$ with d the number of tombstones; this is the number that controls search time.
- **Separate chaining:** α can be larger than 1; it equals the **average chain length**.

Method	α range	What α measures	Typical resize threshold
Separate chaining	0 … ∞	average keys per bucket	$\alpha > 1$ (C++ default `max_load_factor` is 1.0); Java uses 0.75
Linear probing	0 … 1	fraction of slots in use	0.5 – 0.7
Quadratic probing	0 … 1	fraction of slots in use	< 0.5 (guarantee for prime m)
Double hashing	0 … 1	fraction of slots in use	0.7 – 0.8

NOTE. Thresholds of real libraries are implementation choices that differ between libraries and versions; the table lists commonly documented defaults and typical textbook values.

10.2 How Load Factor Affects Collisions and Performance

More keys per slot means more collisions: the expected number of colliding pairs is $n(n\text{-}1)/(2\mathrm{m}) \approx \alpha\, n/2$, so it grows in proportion to α. For open addressing the expected number of probes explodes as $\alpha \to 1$ (Part 6, Figure 6.4):

α	Linear probing: unsuccessful	Double hashing: unsuccessful	Chaining: nodes compared (unsuccessful)
0.25	1.4	1.3	0.25
0.5	2.5	2.0	0.5
0.75	8.5	4.0	0.75
0.9	50.5	10.0	0.9

The **cost of space** works the other way: a low α wastes memory. The designer balances time against space, and the resizing policy keeps α inside a chosen band.

10.3 Choosing a Resize Threshold and Growth Policy

1. **Choose** $\alpha_{\max}$ (for example 0.5 for linear probing, 1.0 for chaining).
2. **Before inserting**, check whether the insertion would push the load above $\alpha_{\max}$: `(n + 1) / m > αmax`. If so, grow first.
3. **Choose the new capacity:** roughly **double** it, and for the division method round up to a **prime** (`nextPrime(2 * m)`). Doubling is what makes the average cost constant (Section 10.10).
4. **Rehash** all keys into the new array (Section 10.5).
5. **Optionally shrink** when α falls far below the band (Section 10.9).

COMMON MISTAKE. Growing by a **constant** amount (m + 10) instead of a **constant factor**. Then the total rebuilding cost over n insertions is $\mathrm{O}(n^2)$, not $\mathrm{O}(n)$.

10.4 Rehashing versus Merely Copying the Array

When the capacity changes from m_{old} to m_{new}, the home slot of every key changes, because it is computed with k mod m. **Copying** the old array into a bigger one leaves keys in slots that no longer match their hash. Example: the table of size 5 with keys 25 (slot 0) and 12 (slot 2) is grown to size 11.

Figure 10.1 contrasts copying the array with rehashing it.

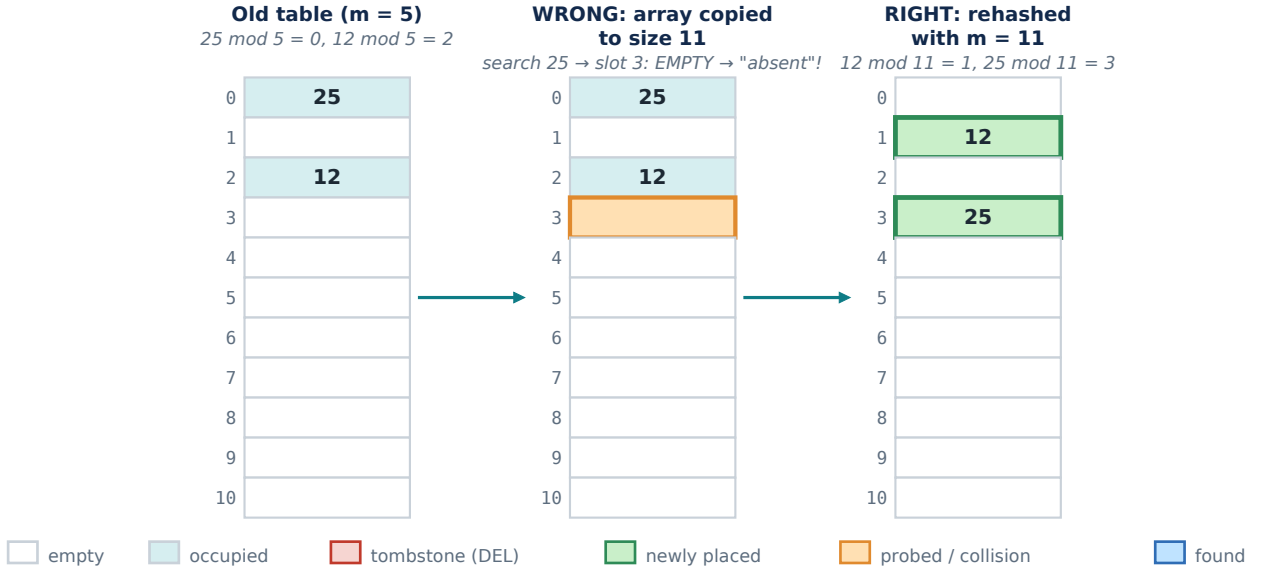


Figure 10.1 — Copying versus rehashing. After a plain copy, searching for 25 looks at slot 25 mod 11 = 3 and wrongly reports the key as missing.

EXAM TRAP. Rehashing = **recompute the hash index of every stored key with the new capacity and insert it into a fresh table.** It is not memcpy.

10.5 Step-by-step Resizing: A Complete Run

Rule used: table starts with $m = 5$; before an insertion that would make (used slots + 1) exceed $0.5 \cdot m$ the table is rebuilt with `nextPrime(2m)`. Linear probing. We insert 12, 25, 7, 18, 31, 44, 9. The program output (Section 10.12) is the reference; the Python check below reproduces it exactly.

Insert	Check	Action	Capacity after
12	used 0 + 1 ≤ 2.5	place in slot 2	5
25	used 1 + 1 ≤ 2.5	place in slot 0	5
7	used 2 + 1 = 3 > 2.5	**rehash** 5 → 11, then place 7 in slot 7	11
18, 31	used 3, 4 (+1 ≤ 5.5)	18: home 7 taken → slot 8; 31 → slot 9	11
44	used 5 + 1 = 6 > 5.5	**rehash** 11 → 23, then place 44 in slot 21	23
9	used 6 + 1 ≤ 11.5	place in slot 9	23

Figure 10.2 draws the first rehash of the example (5 to 11 slots).

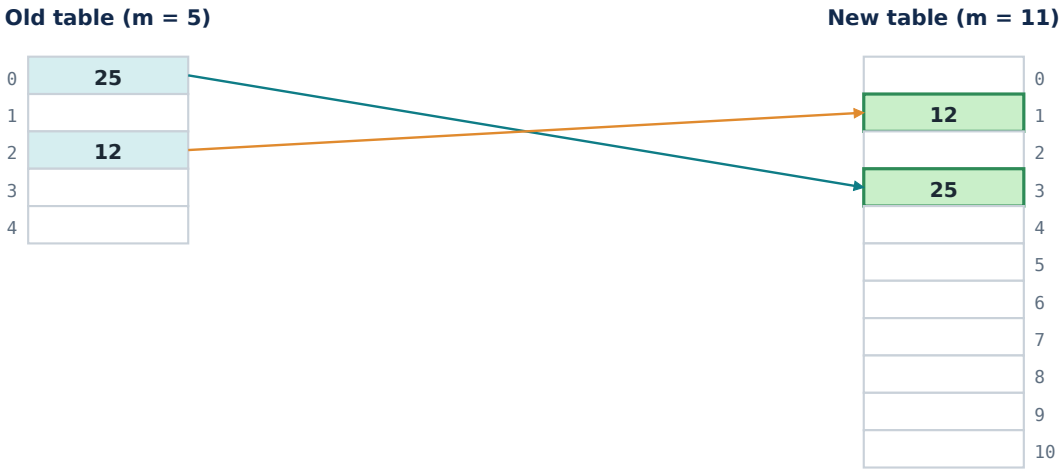


Figure 10.2 — First rehash (5 → 11). Key 25 moves from slot 0 to slot 3 and key 12 from slot 2 to slot 1. Only afterwards is the new key 7 inserted.

Figure 10.3 draws the second rehash (11 to 23 slots).

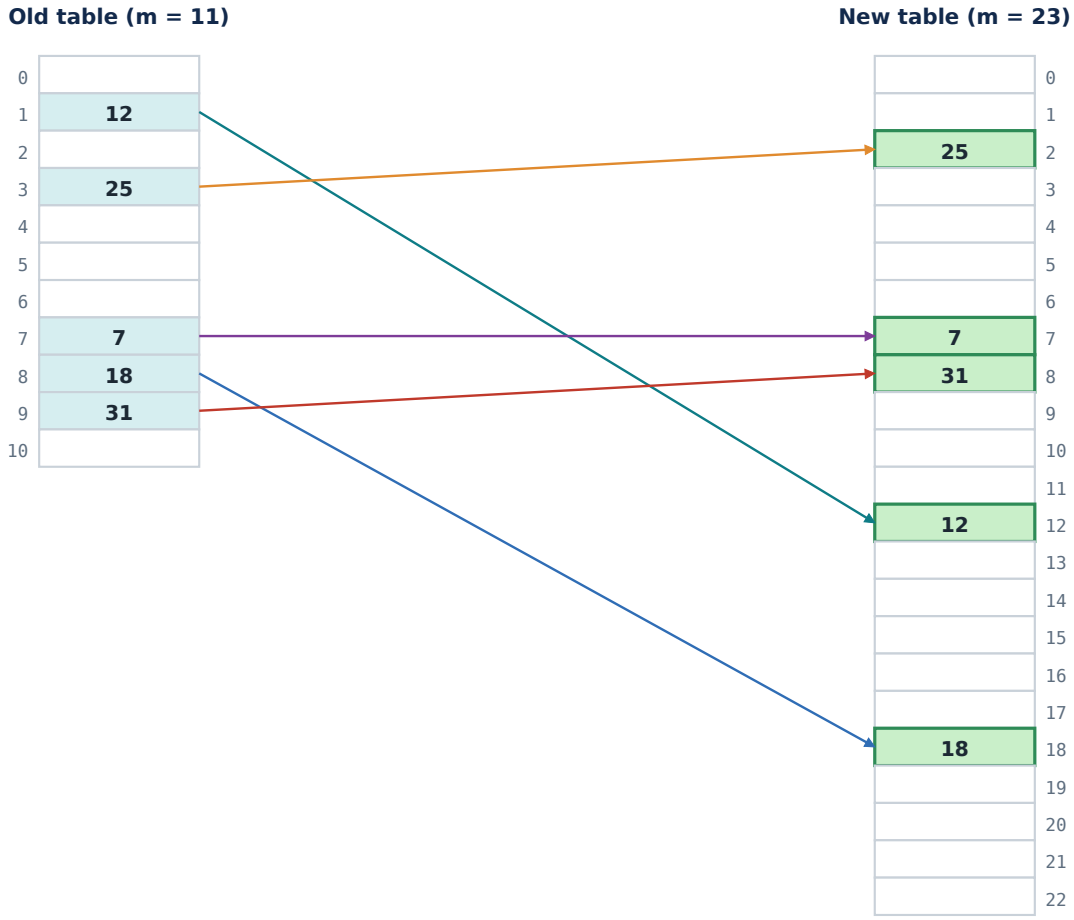


*Figure 10.3 — Second rehash (11 → 23). Keys are re-inserted in **old slot order**; 25 moves from slot 3 to slot 2, 31 from slot 9 to slot 8, 12 from slot 1 to slot 12. Key 18 moves to slot 18.*

Key	Old slot (m = 11)	Index k mod 11	New k mod 23	New slot
12	1	1	12	12
25	3	3	2	2
7	7	7	7	7
18	8	7	18	18
31	9	9	8	8

Notice key 18: its home in the old table was 18 mod 11 = 7 (taken by key 7), so it was displaced to slot 8; in the new table its home is 18 mod 23 = 18, so it moved back to its **home** slot. Rehashing therefore also **removes accumulated collisions** and tombstones.

10.6 Why Old Indexes Generally Cannot Be Reused

The new index k mod m_{new} equals the old index k mod m_{old} only by coincidence (for instance when $k < \min(m_{\text{old}}, m_{\text{new}})$ both equal k). For prime table sizes the two remainders are unrelated, so no shortcut exists. Even the displaced position of a collided key depends on the **whole contents** of the table at insertion time. Special case: if both sizes are powers of two and $m_{\text{new}} = 2\ m_{\text{old}}$, then k mod m_{new} is either the old index or old index + m_{old}; extendible and linear hashing (Part 12) exploit this.

10.7 Rehashing in Separate Chaining and in Open Addressing

	Separate chaining	Open addressing
Elements moved	Existing **nodes are relinked** into new buckets (no new allocation per key)	Keys are **re-inserted** one by one into a new array
Order of re-insertion	Any; chain order may reverse	Matters: collisions are resolved afresh; use a complete probe from home
Tombstones	None	Discarded: rehashing is the cleanup operation
New array	Array of head pointers	Array of slots
Cost	$O(m_{\text{old}} + n)$ to scan buckets and nodes	$O(m_{\text{old}} + n \cdot \text{probes})$

The chaining version relinks every node. The complete program below doubles the table when the load exceeds 2 (the 7th key with $m = 3$).

```cpp
// Part 10: rehashing a separate-chaining table by RELINKING nodes (no new allocation)
#include <iostream>
#include <vector>

struct Node {
    int key;
    Node* next;
};

class ChainTable {
public:
    explicit ChainTable(int m) : buckets_(m, nullptr), n_(0) {}
    ~ChainTable() { clear(); }
    ChainTable(const ChainTable&) = delete;
    ChainTable& operator=(const ChainTable&) = delete;

    void insert(int key) {
        std::size_t i = static_cast<std::size_t>(key) % buckets_.size();
        buckets_[i] = new Node{key, buckets_[i]};        // head insertion
        ++n_;
        if (n_ > 2 * buckets_.size()) rehash(2 * buckets_.size() + 1);   // load > 2
    }

    void rehash(std::size_t newM) {
        std::vector<Node*> fresh(newM, nullptr);
        for (Node* head : buckets_) {
            while (head != nullptr) {
                Node* next = head->next;                 // save before relinking
                std::size_t j = static_cast<std::size_t>(head->key) % newM;   // NEW index
                head->next = fresh[j];
                fresh[j] = head;                         // node moved, not copied
                head = next;
            }
        }
        buckets_.swap(fresh);                            // old (now empty) array is discarded
        std::cout << "rehashed to " << newM << " buckets\n";
    }

    void display() const {
        for (std::size_t i = 0; i < buckets_.size(); ++i) {
            std::cout << "[" << i << "]";
            for (const Node* p = buckets_[i]; p != nullptr; p = p->next) std::cout << " " << p->key;
            std::cout << "\n";
        }
    }

private:
    void clear() {
        for (Node* head : buckets_) {
            while (head != nullptr) { Node* nx = head->next; delete head; head = nx; }
        }
    }
    std::vector<Node*> buckets_;
    std::size_t n_;
};

int main() {
    ChainTable t(3);
    for (int k : {10, 20, 30, 40, 50, 60}) t.insert(k);
```

```
    std::cout << "before the 7th key:\n";
    t.display();
    t.insert(70);                                          // n = 7 > 2 * 3 triggers the rehash
    std::cout << "after:\n";
    t.display();
    return 0;
}
```

Output

```
before the 7th key:
[0] 60 30
[1] 40 10
[2] 50 20
rehashed to 7 buckets
after:
[0] 70
[1] 50
[2] 30
[3] 10
[4] 60
[5] 40
[6] 20
```

Dry run: before the rehash the chains are [0] 60→30, [1] 40→10, [2] 50→20. Each node is moved to bucket k mod 7: 60→4, 30→2, 40→5, 10→3, 50→1, 20→6; then 70 is inserted into bucket 0.

10.8 Clearly Distinguishing Collision Resolution from Resize/Rehash

	Collision resolution	Resize / rehash
Trigger	Home slot of **one** key is taken	Load factor crosses the threshold
Scope	One key, a few probes or one chain	**All** keys
Changes m?	No	**Yes**
Changes hash index?	No: $h(k)$ unchanged, only the final position	Yes: indexes recomputed with new m
Cost	$O(1)$ expected	$O(m_{new} + n)$
Purpose	Store the key correctly	Keep future collisions rare

10.9 Shrinking a Table

If many keys are deleted the table may become mostly empty and waste memory. Shrink (rehash into a smaller prime, e.g. `nextPrime(m / 2)`) when α drops **well below** the growth threshold. Use **hysteresis**: grow at $\alpha > 0.5$ but shrink only below $\alpha < 0.125$ (one quarter of the growth threshold), so that a table hovering around one threshold does not rebuild again and again (thrashing). After doubling at $\alpha = 0.5$ the load is 0.25; after halving at $\alpha = 0.125$ the load is 0.25: both land safely in the middle of the band. Many libraries never shrink automatically.

10.10 Amortized Complexity of Resizing

One insertion that triggers a rehash costs $O(n)$, but this is rare. If the capacity **doubles** each time, the rebuilds happen at sizes c, $2c$, $4c$, … and move c, $2c$, $4c$, … keys. Over n insertions the total moving cost is a geometric series bounded by $2n$:

$$\text{total work} \le n \text{ (inserts)} + (\, n/2 + n/4 + n/8 + \ldots\,) \le n + n = 2n \Rightarrow \text{amortized cost per insertion} = O(1)$$

n keys inserted	Rehash moves with capacity doubling from 1 at $\alpha = 1$	Total moves / n
8	7	0.88
64	63	0.98
1,000	1,023	1.02
100,000	131,071	1.31

NOTE. **Amortized** means "averaged over a whole sequence of operations", not "averaged over random inputs". A single insertion can still cost $O(n)$, which matters for real-time systems. Some systems rebuild incrementally, moving a few keys per operation.

10.11 Memory Overhead and Resize Costs

- During a rehash both arrays exist: peak memory is about $m_{\text{old}} + m_{\text{new}} \approx 3\, m_{\text{old}}$ slots when doubling.
- A rehash touches every slot and keys can be scattered through memory (cache misses); in chaining it can also free and re-allocate nodes if implemented by copying.
- If n is known in advance, **reserve** capacity once (`reserve(n)` in C++) and avoid all intermediate rehashes (Part 13).
- References, pointers and iterators to elements can be **invalidated** by a rehash (Part 13).

10.12 Complete C++ Implementation of Automatic Resizing

```cpp
// Part 10: automatic resizing and rehashing for a linear-probing table (C++17)
#include <iostream>
#include <utility>
#include <vector>

bool isPrime(int n) {
    if (n < 2) return false;
    for (int d = 2; d * d <= n; ++d)
        if (n % d == 0) return false;
    return true;
}
int nextPrime(int n) {
    while (!isPrime(n)) ++n;
    return n;
}

enum class State { EMPTY, OCCUPIED, DELETED };
struct Slot { int key = 0; State state = State::EMPTY; };

class DynamicTable {
public:
    explicit DynamicTable(int m = 5, double maxLoad = 0.5)
        : m_(m), n_(0), used_(0), maxLoad_(maxLoad), t_(m) {}

    bool insert(int key) {                      // false if the key already exists
        if (contains(key)) return false;
        if ((used_ + 1) > maxLoad_ * m_) {      // would exceed the load limit
            rehash(nextPrime(2 * m_));
        }
        placeNew(key);
        return true;
    }

    bool contains(int key) const {
        for (int i = 0; i < m_; ++i) {
            const Slot& s = t_[(home(key) + i) % m_];
            if (s.state == State::EMPTY) return false;
            if (s.state == State::OCCUPIED && s.key == key) return true;
        }
        return false;
    }

    bool erase(int key) {
        for (int i = 0; i < m_; ++i) {
            Slot& s = t_[(home(key) + i) % m_];
            if (s.state == State::EMPTY) return false;
```

```cpp
            if (s.state == State::OCCUPIED && s.key == key) {
                s.state = State::DELETED;        // tombstone: 'used_' stays the same
                --n_;
                return true;
            }
        }
        return false;
    }

    int capacity() const { return m_; }
    int size() const { return n_; }
    double loadFactor() const { return static_cast<double>(n_) / m_; }

    void display() const {
        for (int j = 0; j < m_; ++j) {
            if (t_[j].state == State::OCCUPIED) std::cout << " [" << j << "]=" << t_[j].key;
        }
        std::cout << "\n";
    }

private:
    int home(int key) const { return ((key % m_) + m_) % m_; }

    int placeNew(int key) {                      // key is known to be absent
        int j = home(key);
        while (t_[j].state == State::OCCUPIED) j = (j + 1) % m_;   // reuse EMPTY or DELETED
        if (t_[j].state == State::EMPTY) ++used_;
        t_[j].key = key;
        t_[j].state = State::OCCUPIED;
        ++n_;
        return j;
    }

    void rehash(int newCap) {
        std::vector<Slot> old = std::move(t_);   // keep the old array
        std::cout << "  rehash: capacity " << m_ << " -> " << newCap << ", moving "
                  << n_ << " keys\n";
        int oldCap = m_;
        t_.assign(newCap, Slot());               // brand-new empty array
        m_ = newCap;                             // from now on home() uses the NEW size
        n_ = 0;
        used_ = 0;                               // tombstones are discarded
        for (int j = 0; j < oldCap; ++j) {
            if (old[j].state != State::OCCUPIED) continue;
            int k = placeNew(old[j].key);
            std::cout << "    key " << old[j].key << ": old slot " << j << " -> new slot "
                      << k << "\n";
        }
    }

    int m_;
    int n_;
    int used_;                                   // OCCUPIED + DELETED slots
    double maxLoad_;
    std::vector<Slot> t_;
};

int main() {
    DynamicTable t(5, 0.5);
    for (int k : {12, 25, 7, 18, 31, 44, 9}) {
        std::cout << "insert " << k << "\n";
        t.insert(k);
        std::cout << "  capacity " << t.capacity() << ", n = " << t.size()
                  << ", load = " << t.loadFactor() << "\n";
    }
    t.display();
    std::cout << "contains(31) = " << t.contains(31) << ", contains(32) = " << t.contains(32)
              << "\n";
    return 0;
}
```

Output (g++ -std=c++17)

```
insert 12
  capacity 5, n = 1, load = 0.2
insert 25
  capacity 5, n = 2, load = 0.4
insert 7
  rehash: capacity 5 -> 11, moving 2 keys
```

```
    key 25: old slot 0 -> new slot 3
    key 12: old slot 2 -> new slot 1
  capacity 11, n = 3, load = 0.272727
insert 18
  capacity 11, n = 4, load = 0.363636
insert 31
  capacity 11, n = 5, load = 0.454545
insert 44
  rehash: capacity 11 -> 23, moving 5 keys
    key 12: old slot 1 -> new slot 12
    key 25: old slot 3 -> new slot 2
    key 7: old slot 7 -> new slot 7
    key 18: old slot 8 -> new slot 18
    key 31: old slot 9 -> new slot 8
  capacity 23, n = 6, load = 0.26087
insert 9
  capacity 23, n = 7, load = 0.304348
 [2]=25 [7]=7 [8]=31 [9]=9 [12]=12 [18]=18 [21]=44
contains(31) = 1, contains(32) = 0
```

Line	Code	What it does
12	`int nextPrime(int n) {`	Finds the smallest prime $\geq n$ by trial division, used to choose prime capacities.
23	`: m_(m), n_(0), used_(0), maxLoad_(maxLoad),…`	`used_` counts non-EMPTY slots (live keys plus tombstones): the effective load.
26	`if (contains(key)) return false;`	Duplicate check first, so a duplicate never triggers a pointless rehash.
27	`if ((used_ + 1) > maxLoad_ * m_) { // …`	Grow **before** inserting, so the load never exceeds $\alpha_{\max}$.
28	`rehash(nextPrime(2 * m_));`	New capacity: smallest prime at least double the old one.
48	`s.state = State::DELETED; // tombston…`	Erase leaves a tombstone; `used_` is unchanged, which is why rehash is also a cleanup.
72	`while (t_[j].state == State::OCCUPIED) j = (…`	Linear probing for the first free (EMPTY or DELETED) slot.
81	`std::vector<Slot> old = std::move(t_); // …`	Keeps the old array for reading while `t_` is rebuilt.
85	`t_.assign(newCap, Slot()); // …`	A fresh array of EMPTY slots with the **new** size.
86	`m_ = newCap; // …`	From this line `home()` uses the new capacity: this is what makes it a rehash and not a copy.
90	`if (old[j].state != State::OCCUPIED) continue;`	Only live keys are moved; EMPTY and DELETED slots are dropped.
91	`int k = placeNew(old[j].key);`	Re-inserts a key with the new modulus; reports its new slot.

PART 11

Comparison of Collision-Resolution Methods

Chaining, linear probing, quadratic probing and double hashing side by side

11.1 Master Comparison Table

Feature	Separate chaining	Linear probing	Quadratic probing	Double hashing
Core idea	colliding keys share a list in the bucket	try next slot: $h + i$	try $h + c_1\, i + c_2\, i^2$	try $h_1 + i \cdot h_2$
Data representation	array of heads + nodes	one array of slots	one array of slots	one array of slots
Insertion	duplicate check along chain, link at head: $O(1+\alpha)$	probe to first free slot	probe along quadratic path; can fail early	probe along key-specific path
Successful search	$\approx 1 + \alpha/2$ nodes	$\approx \frac{1}{2}[1+1/(1-\alpha)]$ probes	between linear and double	$\approx (1/\alpha)\ln(1/(1-\alpha))$ probes
Unsuccessful search	$\approx \alpha$ nodes (whole chain)	$\approx \frac{1}{2}[1+1/(1-\alpha)^2]$ probes	between linear and double	$\approx 1/(1-\alpha)$ probes
Deletion	unlink node: simple	tombstone (or backward shift)	tombstone	tombstone
Clustering	none across buckets	**primary**	secondary	minimal
Load-factor behaviour	works for $\alpha > 1$; degrades gently	needs $\alpha \lesssim 0.7$; degrades sharply	needs $\alpha < 0.5$ for guarantee	needs $\alpha \lesssim 0.8$; degrades faster near 1
Memory overhead	pointer per node + allocator	none (but spare slots)	none (spare slots)	none (spare slots)
Cache locality	poor	**best**	medium	poor
Implementation difficulty	easy	easiest	medium (coverage issues)	medium (two hashes, prime m)
Average case	$O(1+\alpha)$	$O(1)$ for fixed $\alpha<1$	$O(1)$ for fixed small α	$O(1)$ for fixed $\alpha<1$
Worst case	$O(n)$	$O(n)$	$O(n)$ or failure	$O(n)$
Table full?	never	at $n = m$	may fail earlier	at $n = m$ (prime m)

The formulas are the expected-case results for random keys from Parts 5–9; they are **not guarantees**. Every method has $O(n)$ worst case when the hash function is bad or the input is adversarial.

11.2 A Worked Comparison on the Same Keys

Insert the eight keys **23, 47, 12, 89, 34, 56, 71, 5** into a table of $m = 11$ slots, $h(k) = k$ mod 11 (home slots 1, 3, 1, 1, 1, 1, 5, 5). Double hashing uses $h_2 = 1 + k$ mod 10; quadratic probing uses pure squares $h + i^2$. Cost convention: **one unit = one slot (or bucket) accessed or one node compared**; chaining counts the bucket access plus the nodes compared.

Method	Total cost to insert 8 keys	Average cost, successful search	Average cost, unsuccessful search (one absent key per home slot)
Separate chaining	19	3.38	1.73
Linear probing	26	3.25	4.27
Quadratic probing	25	3.12	3.64
Double hashing	16	2.00	3.27

Figure 11.1 shows the final tables of the three open-addressing methods for the same eight keys.

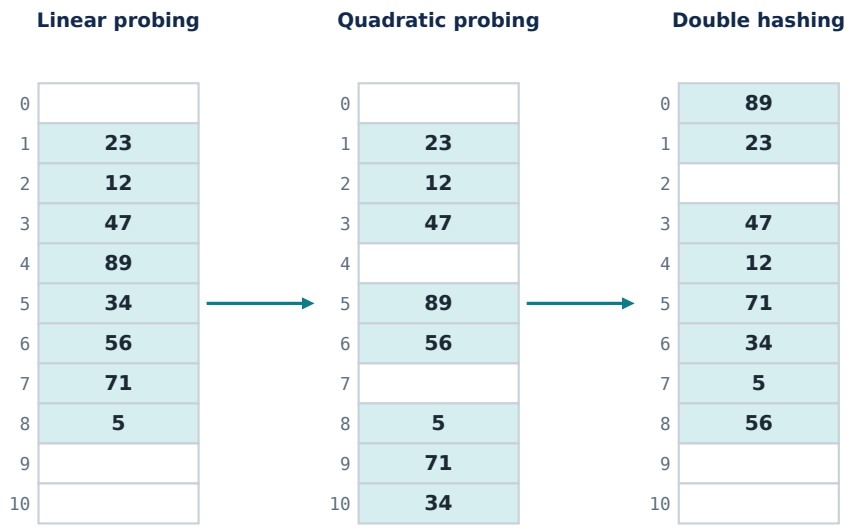


Figure 11.1 — Final tables for the same eight keys. Linear probing forms one long run of consecutive occupied slots; quadratic probing and double hashing scatter the keys.

Figure 11.2 shows the chained table for the same keys.

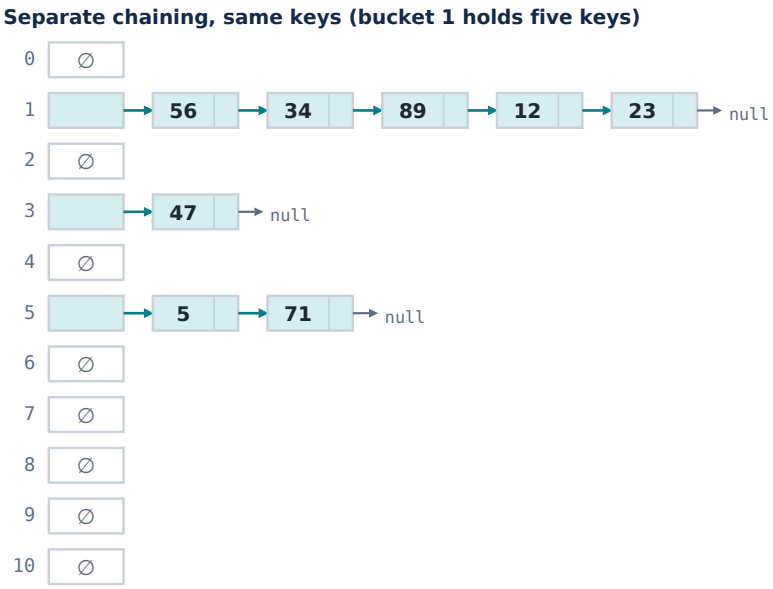


Figure 11.2 — Separate chaining table for the same keys.

This is only a **toy illustration** with eight keys, not a benchmark. Five of the keys share home slot 1, which is an unlucky pattern: chaining gets one long chain, linear probing one long run, and double hashing (with its key-dependent steps) copes best here. Unsuccessful searches in chaining are cheap on average because 8 of the 11 buckets are short or empty. Do not generalise from eight keys; use the analysis of Parts 5–9 and measure real workloads.

11.3 Worst Case: All Keys Collide

If all n keys share one home slot, the cost of inserting the j-th key is j for every method, so the total cost is $1 + 2 + \ldots + n = n(n+1)/2$ and a single search can cost n. This is the situation in Part 7 (five keys, 15 probes). Hence $O(1)$ **is an average-case statement that rests on assumptions**: (1) the hash function spreads keys roughly uniformly, (2) the load factor is bounded by resizing, (3) no adversary chooses the keys.

Keys with the same home slot	Linear probing total probes	Chaining total (bucket + nodes)
5	15	15
10	55	55
100	5050	5050

11.4 When to Choose Each Method

Scenario	Good choice	Why
General-purpose library, unknown workload, frequent deletions	Separate chaining	Simple, tolerant of high load, easy deletion
Small keys (ints), read-heavy, memory-sensitive, performance-critical	Linear probing (low α)	Cache-friendly; very fast probes
Moderate load, want less clustering than linear but still array-based	Quadratic probing (prime m, $\alpha<0.5$)	Avoids primary clustering; simple formula
High load factor needed or poor locality of keys	Double hashing	Probe sequence nearly random; lowest probe counts
Many deletions	Chaining	Tombstones accumulate in open addressing and need periodic rebuilds
Load factor above 1 expected	Chaining	Open addressing cannot exceed 1
Values are large objects	Chaining or open addressing storing pointers	Large slots hurt cache use of open addressing
Real-time system with strict latency	Any, with pre-reserved capacity	Avoid rehash pauses (reserve in advance or rehash incrementally)

NOTE. Rules of thumb, not laws: measure with realistic keys before deciding. A well-implemented chained table is a safe default; open addressing is usually chosen to save memory and gain speed with small keys and few deletions.

PART 12

ADVANCED HASHING TECHNIQUES

Universal, perfect, extendible, linear, cuckoo, Robin Hood, hopscotch, consistent hashing and related ideas

ADVANCED. This part is a guided tour for an undergraduate course. Sections are marked **core idea** (likely in exams) or **advanced** (understand the principle, not every proof). More space is given to the ideas that build directly on Parts 5–11.

12.1 Universal Hashing (core idea)

Motivation. For any fixed hash function an adversary who knows it can pick keys that all collide. **Universal hashing** defeats this by choosing the function **at random** from a family H when the table is created.

DEFINITION. A family H of functions $U \to \{0, \ldots, m\text{-}1\}$ is **universal** if for any two **different** keys $x \neq y$, the probability (over the random choice of $h \in H$) that $h(x) = h(y)$ is at most $1/m$.

Classic family (Carter–Wegman). Choose a prime p larger than every key, then pick $a \in \{1, \ldots, p\text{-}1\}$ and $b \in \{0, \ldots, p\text{-}1\}$ at random:

$$h_{a,b}(k) = ((a\,k + b) \bmod p) \bmod m$$

Key k	$h_{3,4}(k)$	$h_{5,2}(k)$
2	((3·2 + 4) mod 17) mod 5 = 0	((5·2 + 2) mod 17) mod 5 = 2
7	((3·7 + 4) mod 17) mod 5 = 3	((5·7 + 2) mod 17) mod 5 = 3
12	((3·12 + 4) mod 17) mod 5 = 1	((5·12 + 2) mod 17) mod 5 = 1

Two members of the family ($p = 17$, $m = 5$) give different tables for the same keys, so no key set is bad for **all** of them. **Benefit:** expected chain length of a chained table is at most $1 + n/m$ for **any** key set, with the expectation over the random choice of h, not over the keys. **Limitation:** needs a modular multiplication per hash, and a good random source.

12.2 Randomized Hashing and its Purpose (core idea)

A **randomized** (seeded) hash function mixes a secret random value chosen at program start into the computation, for example `h = hash(seed, key)`. The purpose:

- **Defence against hash flooding:** attackers cannot precompute keys that collide (a real attack on web servers whose hash tables used a fixed, known function).
- **Stable average-case behaviour** independent of the input pattern.
- Several languages hash strings with a keyed hash (for example SipHash) and a per-process random seed, and some give the seed through an environment variable for reproducibility. Details vary by language and version.

NOTE. Consequence: iteration order of such tables can change from run to run, and hash values must never be stored and reused after the program exits.

12.3 Perfect and Minimal Perfect Hashing (advanced)

Motivation. If the set of keys is **fixed in advance** (reserved words of a language, a read-only dictionary) we can search for a hash function with **no collisions at all**.

- **Perfect hash function:** injective on the key set S: different keys always give different indexes. Lookup needs exactly one probe, worst case $\mathrm{O}(1)$.
- **Minimal perfect hash function (MPHF):** perfect **and** the table has exactly n slots ($m = n$): zero wasted space.

Key set S = {12, 25, 31, 47, 56}	Result
$h(k) = k$ mod m, smallest m with no collision	$m = 10$: indexes 2, 5, 1, 7, 6 (perfect, but 5 wasted slots)
$h(k) = ((a\ k)$ mod 101) mod 5, search a	$a = 10$: indexes 4, 3, 2, 1, 0 (**minimal** perfect: 5 keys, 5 slots)

For small sets a brute-force search over a parameter works (as computed above). For large sets practical algorithms construct perfect functions in expected linear time; the classical **FKS scheme** (Fredman–Komlós–Szemerédi) uses **two levels**: a first-level hash splits the keys into buckets; a bucket with b keys gets its own second-level table of size about b^2 with a randomly chosen function that is collision-free with good probability. Total space is $\mathrm{O}(n)$ in expectation and lookup is two hash computations in the worst case.

- **Benefits:** worst-case $\mathrm{O}(1)$ lookup, tiny memory for MPHF.
- **Limitations:** keys must be **static**; construction is more expensive; membership of keys **outside** S is not detected (a non-key also hashes to some slot), so the stored key must still be compared.

12.4 Static versus Dynamic Hashing

	Static hashing	Dynamic hashing
Number of buckets	fixed at creation	grows and shrinks with the data
Overflow	overflow chains or pages	bucket splitting keeps buckets short
Rebuild	whole file when it outgrows the table	incremental (one bucket at a time)
Examples	the fixed arrays in Parts 5–9 without resizing; fixed bucket pages in a simple database file	extendible hashing, linear hashing (below)
Typical use	small known data sets	disk-based files and databases whose size changes

Parts 5–10 resized a **whole in-memory table** at once. On **disk**, rebuilding everything is far too expensive, so databases use dynamic schemes that split **one bucket (one disk page) at a time**.

12.5 Extendible Hashing (advanced, disk-based)

Idea. Keep a **directory** of 2^d pointers indexed by the **low-order** d **bits** of the hash value (d = *global depth*). Several directory entries may point to the same **bucket**. Each bucket records its own **local depth** $d' \le d$ = number of bits that really distinguish its keys. A bucket holds at most B keys (here $B = 2$).

Insert algorithm. (1) Use the last d bits of the hash to find the directory entry and bucket. (2) If the bucket has room, store the key. (3) Otherwise **split**: if the bucket's local depth equals the global depth first **double the directory** ($d \to d{+}1$, each pointer duplicated); then create a new bucket, raise the local depth of both to $d'{+}1$, redistribute the keys by bit d', and repoint half of the directory entries to the new bucket; retry the insertion.

Figure 12.1 shows extendible hashing when a directory doubling is needed.

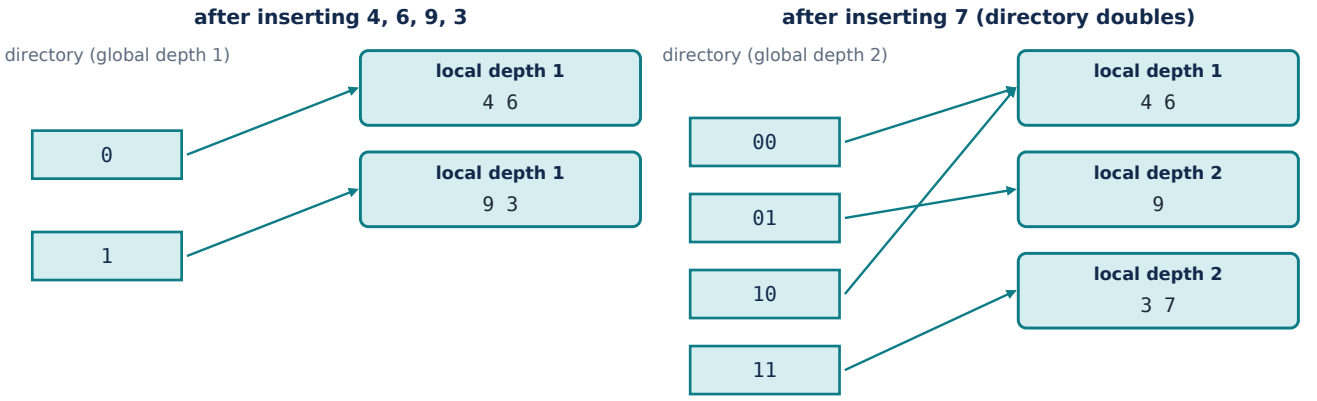


Figure 12.1 — Extendible hashing, bucket capacity 2, hash = key itself, directory index = last bits of the key. Key 7 overflows the odd bucket {9, 3}: because local depth = global depth = 1, the directory doubles to 4 entries and the bucket splits on bit 1.

Figure 12.2 shows a bucket split that does not need a doubling.

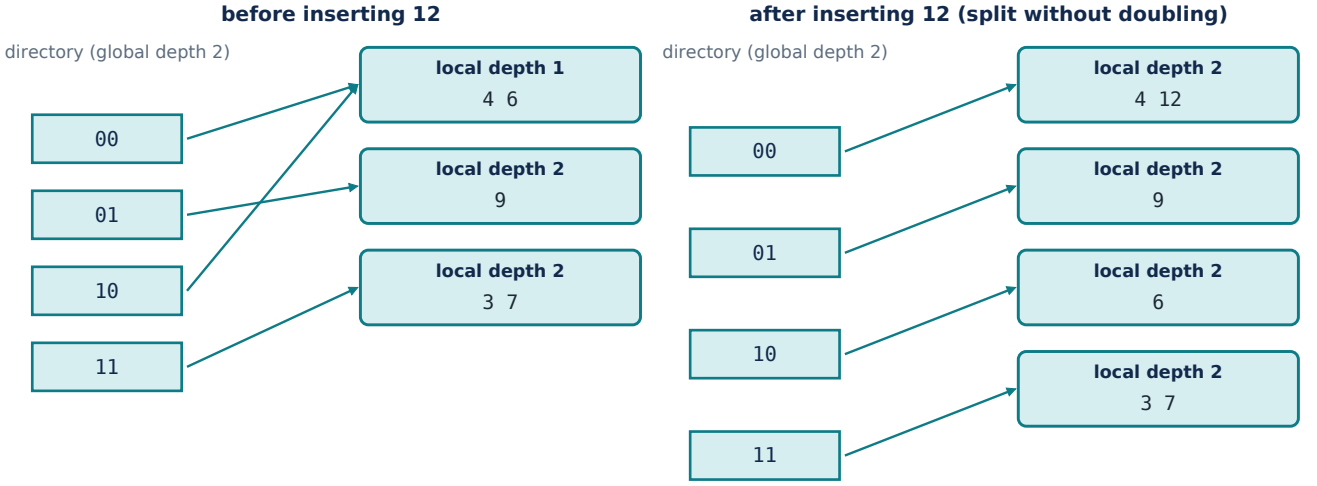


Figure 12.2 — Key 12 (binary 1100) overflows the even bucket {4, 6} whose local depth 1 is smaller than the global depth 2. Only the bucket splits (on bit 1): 4 and 12 stay, 6 moves to a new bucket; the directory keeps 4 entries.

Key	Binary	Last 1 bit	Last 2 bits
4	00100	0	00
6	00110	0	10
9	01001	1	01
3	00011	1	11
7	00111	1	11
12	01100	0	00

- **Benefits:** a lookup costs one directory access plus **one bucket access** (a single disk read when the directory is in memory); growth is gradual; no full rebuild.
- **Limitations:** the directory can become large when keys cluster in their low bits (many keys with the same suffix force repeated doubling); extra indirection.
- **Complexity:** lookup $\mathrm{O}(1)$ bucket accesses; a split costs $\mathrm{O}(B)$ plus possible directory doubling $\mathrm{O}(2^d)$.

12.6 Linear Hashing (advanced, disk-based)

Idea. Avoid a directory. Keep N initial buckets and a **split pointer** `next` that walks through the buckets **in order**, splitting the bucket it points to **regardless of which bucket overflowed**. Hash with $h_L(k) = k \bmod (N \cdot 2^L)$ at *level* L; buckets before `next` already split and use h_{L+1}:

$$\text{address}(k) = h_L(k) \text{ ; if address} < \text{next then address} = h_{L+1}(k)$$

Splitting bucket `next` redistributes its keys between itself and a new bucket appended at the end by using h_{L+1}; then `next` increases. When `next` reaches $N \cdot 2^L$ the level L increases and `next` returns to 0. Here: $N = 2$, bucket capacity 2 (overflowing keys go to an overflow chain, shown inside the bucket), and a split is triggered whenever an insertion finds its bucket full.

Insert	Bucket contents (index: keys)	level L	next	Event
4	0: 4 1: -	0	0	
9	0: 4 1: 9	0	0	
6	0: 4 6 1: 9	0	0	
11	0: 4 6 1: 9 11	0	0	
8	0: 4 8 1: 9 11 2: 6	0	1	split bucket 0 -> new bucket 2
15	0: 4 8 1: 9 2: 6 3: 11 15	1	0	split bucket 1 -> new bucket 3
7	0: 8 1: 9 2: 6 3: 11 15 7 4: 4	1	1	split bucket 0 -> new bucket 4
12	0: 8 1: 9 2: 6 3: 11 15 7 4: 4 12	1	1	

Reading the trace: inserting 8 hits the full bucket 0 ({4, 6}); bucket 0 is split with $h_1(k) = k \bmod 4$, so 6 moves to the new bucket 2. Inserting 15 hits the full bucket 1, and since `next` = 1 that bucket is the one that splits; `next` then wraps and level 1 begins.

- **Benefits:** no directory, buckets are added one at a time, load stays under control; simple address arithmetic.
- **Limitations:** the bucket that splits is not necessarily the bucket that overflowed, so some overflow chains exist temporarily.
- **Complexity:** expected $O(1)$ per access; each split costs $O(B)$.

12.7 Cuckoo Hashing (core idea in modern practice)

Idea. Each key has **two** candidate slots, one in each of two tables (h_1 in T_1, h_2 in T_2), and **always lives in one of them**. A lookup inspects at most **two slots**: worst-case $O(1)$.

Insert. Try $T_1[h_1(k)]$. If occupied, **kick out** the resident, put k there, and move the evicted key to its **other** table; repeat. If the chain is too long (a cycle), rebuild the tables with **new hash functions** (rehash).

Key	$h_1 = k$ **mod 7**	$h_2 = \lfloor k/7 \rfloor$ **mod 7**
10	3	1
17	3	2
3	3	0
24	3	3

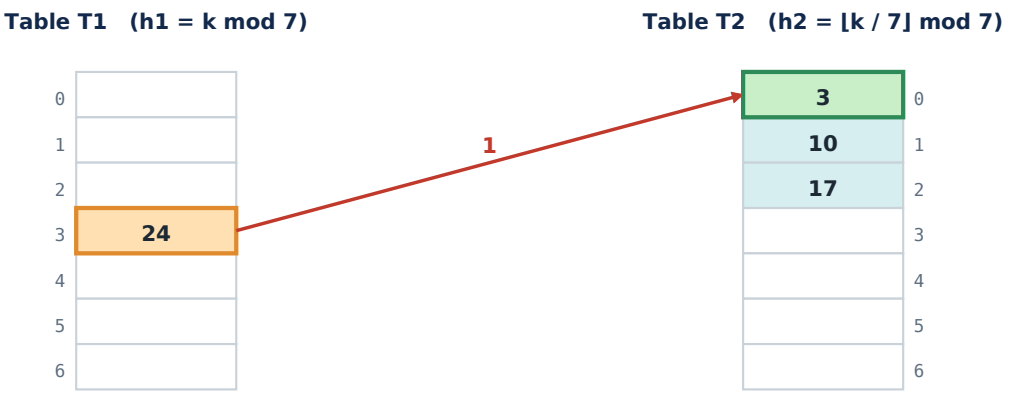


Figure 12.3 — Cuckoo hashing after inserting 10, 17, 3 and then 24. All four keys want T1[3]. Insert 24 (step 1) evicts 3, which moves to its other slot T2[0] (step 2); there it finds a free slot and the chain stops.

Lookup of 3 inspects $T_1[3]$ (holds 24) and then $T_2[0]$ (holds 3): found with two probes. Lookup of 4 inspects $T_1[4]$ and $T_2[0]$ ... both do not hold 4: absent after exactly two probes.

- **Benefits:** worst-case two probes per lookup; deletion is trivial (just clear the slot); no tombstones.
- **Limitations:** load factor must stay below about 0.5 for two tables with high probability of success (variants with more hash functions or bucketed slots reach higher loads); insertion can trigger a **full rehash**; every insertion must handle cycles.
- **Complexity:** lookup and delete $O(1)$ worst case; insert expected amortized $O(1)$ (under assumptions on the hash functions and load).

```cpp
// Part 12 companion: three advanced hashing ideas in C++17
#include <iostream>
#include <map>
#include <string>
#include <utility>
#include <vector>

// ---- 1. Universal hashing: h_{a,b}(k) = ((a*k + b) mod p) mod m, p prime > largest key
struct UniversalHash {
    long long a, b, p;
    int m;
    int operator()(long long k) const { return static_cast<int>(((a * k + b) % p) % m); }
};

// ---- 2. Cuckoo hashing with two tables of size 7 (stores only keys)
class Cuckoo {
public:
    Cuckoo() : t_{std::vector<int>(7, -1), std::vector<int>(7, -1)} {}
    static int h(int table, int k) { return table == 0 ? k % 7 : (k / 7) % 7; }

    bool contains(int k) const { return t_[0][h(0, k)] == k || t_[1][h(1, k)] == k; }

    // returns false if the kick chain gets too long (a real table would rehash)
    bool insert(int k) {
        int table = 0;
        for (int kicks = 0; kicks < 10; ++kicks) {
            int i = h(table, k);
            std::swap(k, t_[table][i]);                  // place k, pick up the old occupant
            std::cout << "  T" << table + 1 << "[" << i << "] <- "
                      << t_[table][i] << (k == -1 ? "" : ", evicted " + std::to_string(k)) << "\n";
            if (k == -1) return true;                    // nothing evicted: done
            table = 1 - table;                           // evicted key goes to its other table
        }
        return false;
    }
private:
    std::vector<int> t_[2];
};

// ---- 3. Consistent hashing: servers on a ring of positions 0..99
class Ring {
public:
    void addServer(const std::string& name, int pos) { ring_[pos] = name; }
```

```
    void removeServer(int pos) { ring_.erase(pos); }
    const std::string& serverFor(int keyPos) const {
        auto it = ring_.lower_bound(keyPos);             // first server at or after the key
        if (it == ring_.end()) it = ring_.begin();       // wrap around the ring
        return it->second;
    }
private:
    std::map<int, std::string> ring_;
};

int main() {
    std::cout << "== Universal hashing, p = 17, m = 5 ==\n";
    UniversalHash f1{3, 4, 17, 5}, f2{5, 2, 17, 5};
    for (int k : {2, 7, 12})
        std::cout << "key " << k << ": h(3,4) = " << f1(k) << ", h(5,2) = " << f2(k) << "\n";

    std::cout << "== Cuckoo hashing ==\n";
    Cuckoo c;
    for (int k : {10, 17, 3, 24}) {
        std::cout << "insert " << k << "\n";
        c.insert(k);
    }
    std::cout << "contains(3) = " << c.contains(3) << ", contains(4) = " << c.contains(4) << "\n";

    std::cout << "== Consistent hashing, ring size 100 ==\n";
    Ring r;
    r.addServer("A", 10); r.addServer("B", 40); r.addServer("C", 75);
    for (int k : {5, 35, 50, 60, 90})
        std::cout << "key " << k << " -> " << r.serverFor(k) << "\n";
    r.addServer("D", 55);
    std::cout << "after adding D at 55:\n";
    for (int k : {5, 35, 50, 60, 90})
        std::cout << "key " << k << " -> " << r.serverFor(k) << "\n";
    return 0;
}
```

Output (universal hashing, cuckoo hashing and consistent hashing demos)

```
== Universal hashing, p = 17, m = 5 ==
key 2: h(3,4) = 0, h(5,2) = 2
key 7: h(3,4) = 3, h(5,2) = 3
key 12: h(3,4) = 1, h(5,2) = 1
== Cuckoo hashing ==
insert 10
  T1[3] <- 10
insert 17
  T1[3] <- 17, evicted 10
  T2[1] <- 10
insert 3
  T1[3] <- 3, evicted 17
  T2[2] <- 17
insert 24
  T1[3] <- 24, evicted 3
  T2[0] <- 3
contains(3) = 1, contains(4) = 0
== Consistent hashing, ring size 100 ==
key 5 -> A
key 35 -> B
key 50 -> C
key 60 -> C
key 90 -> A
after adding D at 55:
key 5 -> A
key 35 -> B
key 50 -> D
key 60 -> C
key 90 -> A
```

The first block of the program is the universal-hashing family above, the second implements the cuckoo insertion loop (compare its output with Figure 12.3), and the third is the consistent-hashing ring of Section 12.11.

12.8 Robin Hood Hashing (advanced)

Idea. Linear probing in which every key remembers its **probe distance** from its home slot. When an inserting key has travelled **farther** than the resident of the current slot, it **takes the slot** ("steal from the rich") and the resident continues probing. The effect: keys that are far from home get closer, probe lengths become **more uniform** (lower variance) and searches can **stop early**: if the probed resident is closer to home than the searcher has travelled, the key is not present.

Figure 12.4 compares linear probing with Robin Hood hashing on the same keys.

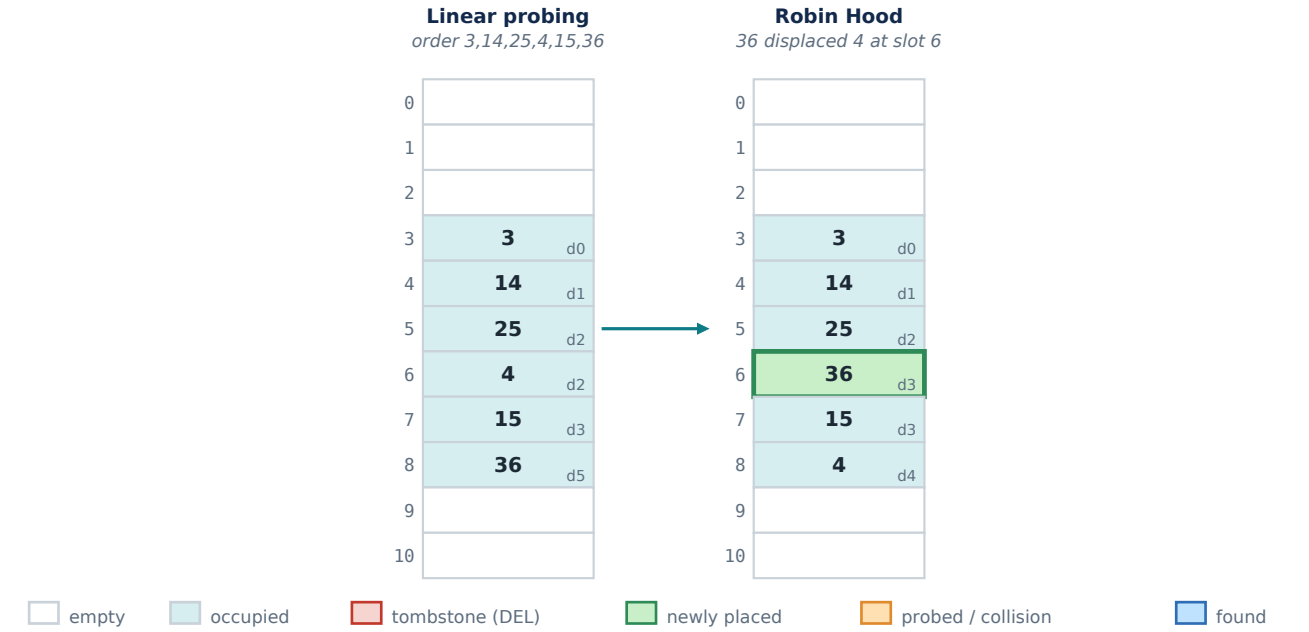


Figure 12.4 — The same six keys (home slots 3, 3, 3, 4, 4, 3) inserted by linear probing and by Robin Hood hashing. Small text in each cell = probe distance d from the home slot.

The only difference arises when 36 is inserted. Trace (slot: action):

- slot 3: 3 (dist 0) stays, 36 moves on (dist 0)
- slot 4: 14 (dist 1) stays, 36 moves on (dist 1)
- slot 5: 25 (dist 2) stays, 36 moves on (dist 2)
- slot 6: 36 (dist 3) displaces 4 (dist 2)
- slot 7: 15 (dist 3) stays, 4 moves on (dist 3)
- slot 8: place 4 (distance 4)

Measure	Linear probing	Robin Hood
Distances of the six keys	0, 1, 2, 2, 3, 5	0, 1, 2, 3, 3, 4
Largest distance	5	4
Sum of distances	13	13

The **sum** (hence the average) is the same, but Robin Hood avoids the worst offender (distance 5 becomes 4 here). With larger tables the reduction of the **variance and maximum** is substantial. Deletion is done by **backward shift** (move the following keys one slot back until an EMPTY slot or a key at distance 0) instead of tombstones.

- **Benefits:** low variance of probe lengths, early termination of unsuccessful searches, no tombstones.
- **Limitations:** insertion moves keys (more writes); needs a distance field per slot.
- **Complexity:** expected $\mathrm{O}(1)$ for fixed load; worst case $\mathrm{O}(n)$.

12.9 Coalesced Hashing (advanced)

Idea. Separate-chaining-style **chains stored inside the table**: every slot has a `next` field holding the index of the next slot of its chain. A colliding key is stored in the **free slot with the largest index** (a pointer `free` moves downward) and linked to the end of the chain of its home slot. No external nodes are allocated. Chains that start at different homes can **merge** (coalesce).

Insert	Home	Action
10	3	home slot free: stored in slot 3
3	3	home slot occupied: stored in free slot 6, slot 3 now points to 6
17	3	home slot occupied: stored in free slot 5, slot 6 now points to 5
5	5	home slot occupied: stored in free slot 4, slot 5 now points to 4

Slot	0	1	2	3	4	5	6
Key	-	-	-	10	5	17	3
next	-	-	-	6	-	4	5

Searching 5: home slot 5 holds 17 (not 5), follow `next` to slot 4, which holds 5: found. The chain 3 → 6 → 5 → 4 mixes keys with homes 3 and 5: that is the **coalescing**. **Benefits:** no pointer allocation, no clustering as bad as linear probing, load can approach 1. **Limitations:** deletion is complicated; analysis is harder; a reserved *cellar* area (extra slots not used as home addresses) improves the performance.

12.10 Hopscotch Hashing (advanced)

Idea. Linear probing with a **guarantee**: every key is stored within a fixed **neighbourhood** of H slots after its home slot (for example $H = 4$ here; 32 or 64 in practice). Each home slot keeps a bitmap telling which of its H neighbours hold keys that belong to it, so a lookup inspects at most H slots that are close together in memory.

Insert. Find the nearest free slot f by linear probing. While f is **outside** the neighbourhood of the home slot, **hop** an earlier key into f (a key may move to f if f is still within **its own** neighbourhood), which frees a slot closer to the home; repeat. If no key can hop, resize the table.

Figure 12.5 shows the hop that frees a slot inside the neighbourhood of key 45.

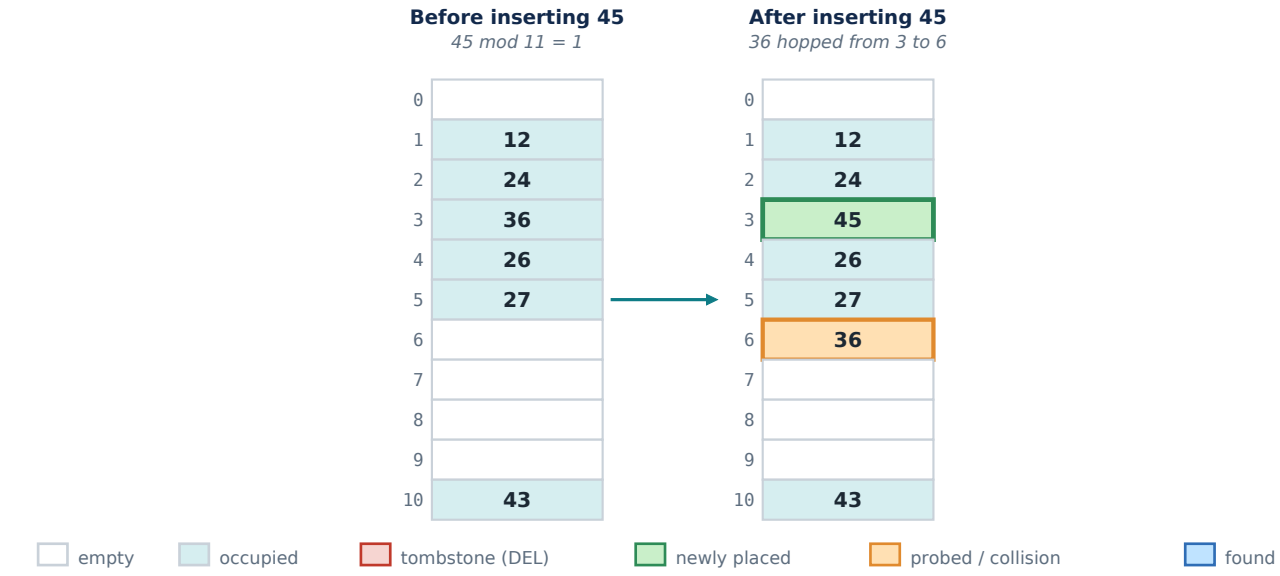


Figure 12.5 — Hopscotch hashing with neighbourhood size H = 4. Key 45 (home 1) finds the first free slot at 6, which is 5 steps away (> 3). Key 36 (home 3) may move to slot 6 because 6 is within 3 slots of its home; that frees slot 3, which is within 45's neighbourhood.

Home slot	1	2	3	4	5	10
Bitmap of neighbours (4 bits)	1010	1000	0001	1000	1000	1000

After the hop, the bitmap for home slot 1 is 1010: its keys 12 and 45 sit at offsets 0 and 2. **Benefits:** at most H slots checked per lookup, excellent cache use, good for concurrent tables. **Limitations:** insertion may fail if no hop is possible (requires resizing); more complicated code. **Complexity:** lookup $\mathrm{O}(H) = \mathrm{O}(1)$ worst case; insertion expected $\mathrm{O}(1)$ for moderate load.

12.11 Consistent Hashing (core idea in distributed systems)

Problem. To spread keys over N servers the obvious rule is `server = hash(key) mod N`. When N changes (a server is added or fails), **almost every key changes server**, which means an enormous data migration or a flush of all caches.

Idea. Hash **both servers and keys onto the same circle** (the *hash ring*, positions 0 ... 2^{32}-1 in practice; 0–99 here). A key belongs to the **first server found clockwise** from its position. Adding or removing a server affects only the keys in the **arc next to it**.

Figure 12.6 shows the hash ring before and after server D joins.

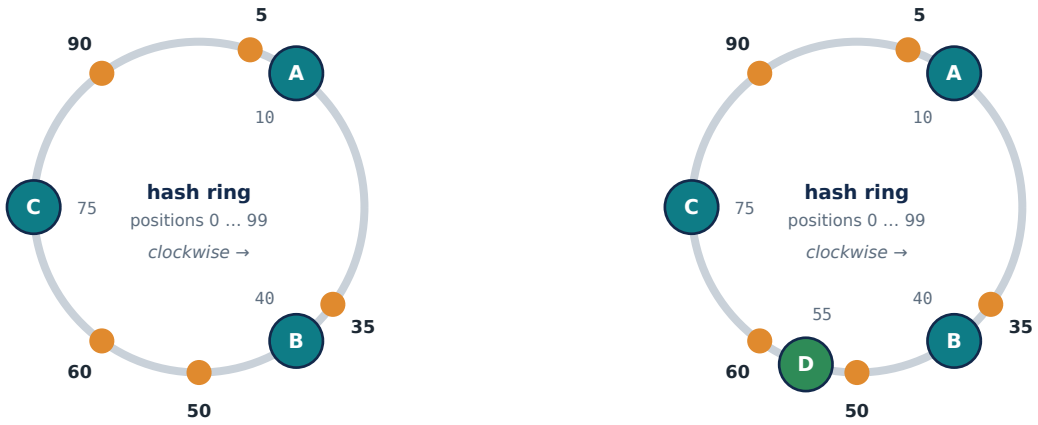


Figure 12.6 — Ring of 100 positions. Left: servers A (10), B (40), C (75). Right: server D added at 55. Only key 50 (in the arc 41–55 that D now covers) moves from C to D; every other key keeps its server.

Key position	5	35	50	60	90
Server before	A	B	C	C	A
Server after adding D	A	B	D	C	A

Quantitatively, for all 100 key positions: adding D to the ring moves **15** keys (15%), all of them to D; changing a `mod N` scheme from $N = 3$ to $N = 4$ servers changes the server of **73** of 100 keys. On average a ring with N servers moves only about $1/N$ of the keys when one server joins.

- **Virtual nodes:** each physical server is placed at many pseudo-random ring positions, which evens out the load and lets stronger servers take more positions.
- **Lookup:** keep server positions in a sorted structure (`std::map`, as in the program above) and use `lower_bound(key position)`, wrapping to the first element: $\mathrm{O}(\log N)$.
- **Uses:** distributed caches, key–value stores, content delivery networks, load balancers.
- **Limitations:** uneven load without virtual nodes; replication and failure handling need extra design.

12.12 Cryptographic versus Non-cryptographic Hash Functions

	Non-cryptographic (table hashing)	Cryptographic
Examples	division / polynomial hash, FNV-1a, MurmurHash-family	SHA-256, SHA-3, BLAKE2
Goal	speed and even spread of keys	resist attacks
Pre-image resistance (cannot find input from hash)	no	yes
Collision resistance (cannot find two inputs with same hash)	no: collisions easy to construct	yes (computationally infeasible)
Speed	nanoseconds per key	slower
Output size	machine word, reduced mod m	128–512 bits
Use in hash tables	yes	rarely: too slow, overkill

WARNING. Ordinary data-structure hash functions are NOT password-hashing algorithms. They are fast and easy to invert or collide, so stolen hashes could be cracked immediately. Even a fast cryptographic hash (plain SHA-256) is unsuitable for passwords because attackers can try billions of guesses per second. Password storage uses a **slow, salted, memory-hard** algorithm designed for the purpose (bcrypt, scrypt, Argon2, PBKDF2) with a unique random **salt** per user.

12.13 Content Hashing and Hash-based Deduplication

A **content hash** is a cryptographic digest of the **bytes** of a file or block; equal content gives equal digests, and different content gives different digests with overwhelming probability. Systems use it as an **identifier**:

- **Deduplication:** a backup or cloud storage system splits files into chunks, hashes each chunk and stores a chunk only if its digest is new. Duplicates cost a table lookup instead of storage.
- **Version control and content-addressable storage:** objects are named by the digest of their content (Git historically used SHA-1 and has been moving to SHA-256).
- **Integrity checks:** download a file, hash it, compare with the published digest.
- **Algorithm:** `digest = H(chunk); if (!seen.contains(digest)) { store(chunk); seen.insert(digest); }`. Cautious systems also compare the bytes when digests match.

12.14 Hashing versus Checksums

	Checksum / CRC	Hash (table)	Cryptographic hash
Purpose	detect **accidental** corruption	index into a table	integrity against **deliberate** tampering, identifiers
Resists a malicious change?	no (easy to forge)	no	yes
Typical size	8–64 bits	machine word	256 bits or more
Examples	parity bit, CRC-32, Adler-32	FNV, polynomial hash	SHA-256

12.15 Hash Tables versus Balanced Search Trees

Feature	Hash table	Balanced BST (AVL, red-black)
Search / insert / delete	$O(1)$ expected, $O(n)$ worst	$O(\log n)$ worst case
Ordered traversal	no	yes, in sorted order
Range queries, predecessor, min/max	inefficient	$O(\log n + k)$
Key requirement	hash function + equality	ordering (`<`)
Worst-case guarantee	none without special schemes	yes
Memory / cache	compact (open addressing), occasional resize pauses	pointer-based, no resize pauses
C++ library	`unordered_map`, `unordered_set`	`map`, `set`

Hashing versus tries

Feature	Hash table of strings	Trie (prefix tree)
Lookup time	$O(L)$ to hash a key of length L + expected $O(1)$ compare	$O(L)$ worst case, no hashing
Prefix search (autocomplete)	not supported	natural
Sorted order of keys	no	yes (alphabetical)
Memory	compact	can be large (many child pointers)
Collisions	yes	none

12.16 When Are Advanced Techniques Useful, and When Unnecessary?

Situation	Technique	Comment
Typical application code	plain `std::unordered_map`	Library hashing is enough; do not hand-roll
Input from untrusted users (web servers)	randomized / keyed hashing (universal family)	Prevents hash flooding
Fixed keyword set, lookup speed critical	perfect hashing	Compilers, protocol parsers
Worst-case lookup bound required	cuckoo or hopscotch hashing	Bounded probes
Very high-performance in-memory table	Robin Hood / open addressing variants	Cache-friendly, low variance
Disk-based, growing file	extendible or linear hashing	One bucket (page) split at a time
Keys spread over changing set of machines	consistent hashing	Minimal key movement
Need sorted / range access	balanced tree or B-tree, **not** hashing	Wrong tool

> **NOTE.** For almost all coursework and everyday programs the techniques of Parts 5–10 plus the standard library are sufficient. Advanced schemes solve specific problems: adversaries, hard worst-case bounds, disk files, and distributed systems.

PART 13

Hashing in C++: Implementation and Library Usa

Writing tables by hand, then using unordered_map, unordered_set, std::hash and custom hash functions correctly

NOTE. All programs in this part are standard C++17 (`g++ -std=c++17 -Wall -Wextra`). They were compiled and run with the sanitizers enabled; the outputs shown are the real outputs.

13.1 Implementing Hashing Manually versus Using the Standard Library

Aspect	Hand-written table (Parts 5-10, 15)	`std::unordered_map` / `unordered_set`
Purpose	learning, exams, special requirements	production code
Collision handling	you choose and write it	chaining-style buckets, implementation defined
Resizing	you write `rehash`	automatic; controlled by `max_load_factor`, `reserve`, `rehash`
Hash function	you design it	`std::hash<Key>` (or your own functor)
Deletion	tombstones or unlink nodes	`erase`, always correct
Safety	bugs are easy (Section 13.11)	tested, exception-safe
Iteration order	whatever you implement	**unspecified**, may change after any insertion
Guarantees	none unless you prove them	average $O(1)$, worst case $O(n)$ in the standard

Use the library unless you are asked to implement a table (coursework, interviews) or need behaviour it does not give (a specific probing scheme, a fixed memory layout). The hand-written tables teach **why** the library behaves as it does.

13.2 Where Each Hand-written Implementation Is

Required implementation	Where in this book
Array-based hash table (division method, no collision handling)	Program 1 (Part 15)
Separate chaining with linked nodes	Section 5.9; Program 2
Separate chaining with standard containers	Section 5.11
Linear probing with tombstones	Section 7.9; Program 3
Quadratic probing with probe tracing and failure handling	Section 8.10; Program 4
Double hashing	Section 9.11; Program 5
Dynamic resizing and rehashing	Section 10.12; Program 6
String keys with polynomial hashing	Section 3.8; Program 7

13.3 Keys, Policies and Edge Cases in Hand-written Tables

Topic	Rule used throughout the book
Negative integer keys	home index = `((k % m) + m) % m`
String keys	polynomial hash over `unsigned char` with unsigned arithmetic, then `% m`
Duplicate keys	insertion searches the whole probe path/chain first; policy = update (or reject, see Section 4.8)
Empty table	search ends at the first probe (EMPTY) / empty bucket; nothing to delete
Full table	probe loop bounded by m; insertion returns a failure status; better: resize earlier
Tombstones	delete marks DELETED; search skips; insert remembers the first and reuses it after the duplicate check
Resizing	before inserting if (used + 1) > $\alpha_{\max} \cdot m$; new prime capacity; **re-hash all keys**

13.4 Exception and Resource Safety

- **RAII:** let containers (`std::vector`, `std::list`, smart pointers) own memory. A table made of `std::vector<Slot>` or `std::vector<std::list<...>>` needs no destructor, no copy constructor and no assignment operator (Rule of Zero).
- **Raw `new`:** if you must use it (Section 5.9) delete the copy operations or implement them, and free all nodes in the destructor.
- **Strong guarantee for rehash:** build the **new** table completely (in local variables) and only then **swap** it with the old one. If an allocation throws part-way, the original table is untouched. In Section 10.12 the old array is moved aside first; a production version would allocate before moving.
- **Throwing hash/equality functions:** if `hash(key)` or `==` throws in the middle of an insertion the table must remain valid. Mark simple hash functors `noexcept` when they cannot throw.
- **Do not modify a key** while it is stored in a table (for example through a pointer); its hash would change and the key becomes unreachable.
- **Resource keys:** if values are owning pointers, erase must destroy them; prefer `std::unique_ptr` values.

13.5 `std::unordered_map` and `std::unordered_set`

`std::unordered_map<Key, T>` stores key–value pairs; `std::unordered_set<Key>` stores only keys. Both live in `<unordered_map>` / `<unordered_set>`. Core operations:

Operation	unordered_map	unordered_set	Notes
insert	`m.insert({k, v})`, `m.emplace(k, v)`	`s.insert(k)`	returns `pair<iterator,bool>`; **does not overwrite**
insert or overwrite	`m[k] = v`, `m.insert_or_assign(k, v)` (C++17)	—	`operator[]` default-constructs a missing value
insert only if missing	`m.try_emplace(k, args...)` (C++17)	—	does not construct if key exists
lookup	`m.find(k)`, `m.count(k)`, `m.at(k)`	`s.find(k)`, `s.count(k)`	`at` throws `std::out_of_range`; C++20 adds `contains`
erase	`m.erase(k)`, `m.erase(it)`	`s.erase(k)`	by key returns count erased; by iterator returns next iterator
size / empty / clear	`size()`, `empty()`, `clear()`	same	
iterate	`for (auto& [k, v] : m)`	`for (auto& k : s)`	**order unspecified**

```cpp
// Part 13: std::unordered_map and std::unordered_set (C++17)
#include <algorithm>
#include <functional>
#include <stdexcept>
#include <iostream>
#include <string>
#include <unordered_map>
#include <unordered_set>
#include <vector>

int main() {
    std::unordered_map<std::string, int> stock;

    std::cout << "== insertion and duplicate policies ==\n";
    auto r1 = stock.insert({"apple", 5});              // pair<iterator, bool>
    auto r2 = stock.insert({"apple", 9});              // key exists: NOT overwritten
    std::cout << "insert apple 5: inserted=" << r1.second << "\n";
    std::cout << "insert apple 9: inserted=" << r2.second
              << ", value stays " << stock["apple"] << "\n";
    stock["banana"] = 7;                               // operator[] creates or overwrites
    stock.insert_or_assign("apple", 11);               // C++17: always assigns
    stock.emplace("cherry", 3);
    stock.try_emplace("cherry", 99);                   // C++17: does nothing if key exists
    std::cout << "apple=" << stock.at("apple") << " banana=" << stock.at("banana")
              << " cherry=" << stock.at("cherry") << " size=" << stock.size() << "\n";

    std::cout << "== lookup ==\n";
    auto it = stock.find("durian");
    std::cout << "find durian: " << (it == stock.end() ? "not found" : "found") << "\n";
```

```cpp
    std::cout << "count(banana) = " << stock.count("banana") << "\n";
    std::cout << "operator[] on a missing key inserts it: size before = " << stock.size();
    int zero = stock["fig"];                          // creates fig -> 0 !
    std::cout << ", after = " << stock.size() << " (value " << zero << ")\n";
    try {
        stock.at("grape");                            // at() throws instead of inserting
    } catch (const std::out_of_range&) {
        std::cout << "at(grape) threw std::out_of_range\n";
    }

    std::cout << "== erase while iterating ==\n";
    for (auto i = stock.begin(); i != stock.end();) {
        if (i->second < 5) i = stock.erase(i);        // erase returns the next iterator
        else ++i;
    }
    std::vector<std::string> names;
    for (const auto& kv : stock) names.push_back(kv.first);
    std::sort(names.begin(), names.end());            // iteration order is unspecified
    for (const auto& n : names) std::cout << n << "=" << stock[n] << " ";
    std::cout << "\n";

    std::cout << "== unordered_set ==\n";
    std::unordered_set<int> seen;
    int dups = 0;
    for (int x : {4, 8, 15, 16, 23, 42, 8, 4}) {
        if (!seen.insert(x).second) ++dups;           // insert fails for a duplicate
    }
    std::cout << "distinct = " << seen.size() << ", duplicates = " << dups
              << ", contains 15? " << seen.count(15) << "\n";

    std::cout << "== std::hash ==\n";
    std::hash<std::string> hs;
    std::cout << "equal strings, equal hashes: "
              << (hs("hashing") == hs(std::string("hash") + "ing")) << "\n";

    std::cout << "== bucket interface and load factor ==\n";
    std::unordered_map<int, int> m;
    std::cout << "max_load_factor default = " << m.max_load_factor() << "\n";
    m.reserve(100);                                   // room for 100 elements without rehash
    std::size_t buckets = m.bucket_count();
    std::cout << "bucket_count >= 100 after reserve(100): " << (buckets >= 100) << "\n";
    for (int i = 0; i < 100; ++i) m[i] = i * i;
    std::cout << "no rehash while inserting 100: " << (m.bucket_count() == buckets) << "\n";
    std::cout << "load_factor = size/bucket_count: "
              << (m.load_factor() ==
                  static_cast<float>(m.size()) / static_cast<float>(m.bucket_count()))
              << ", load_factor <= max_load_factor: "
              << (m.load_factor() <= m.max_load_factor()) << "\n";

    std::cout << "== references survive a rehash ==\n";
    int* p = &m[7];                                   // pointer to a stored value
    for (int i = 1000; i < 2000; ++i) m[i] = i;       // forces rehashing
    std::cout << "pointer still valid and equal to the element: " << (p == &m[7] && *p == 49) << "\n";
    return 0;
}
```

Output (g++ -std=c++17)

```
== insertion and duplicate policies ==
insert apple 5: inserted=1
insert apple 9: inserted=0, value stays 5
apple=11 banana=7 cherry=3 size=3
== lookup ==
find durian: not found
count(banana) = 1
operator[] on a missing key inserts it: size before = 3, after = 4 (value 0)
at(grape) threw std::out_of_range
== erase while iterating ==
apple=11 banana=7
== unordered_set ==
distinct = 6, duplicates = 2, contains 15? 1
== std::hash ==
equal strings, equal hashes: 1
== bucket interface and load factor ==
max_load_factor default = 1
bucket_count >= 100 after reserve(100): 1
no rehash while inserting 100: 1
load_factor = size/bucket_count: 1, load_factor <= max_load_factor: 1
== references survive a rehash ==
```

```
pointer still valid and equal to the element: 1
```

Line	Code	What it does
16	`auto r2 = stock.insert({"apple", 9}); …`	`insert` never overwrites: `second` is `false` and the stored value stays 5.
20	`stock["banana"] = 7; …`	`operator[]` creates the key if missing (or returns a reference to the existing value) and then assigns.
21	`stock.insert_or_assign("apple", 11); …`	C++17 call that always assigns and tells you whether it inserted.
23	`stock.try_emplace("cherry", 99); …`	Key exists, so the value 99 is **not** used or constructed.
32	`int zero = stock["fig"]; …`	**Trap:** merely *reading* with `operator[]` inserts `fig -> 0` and grows the map.
35	`stock.at("grape"); …`	`at` throws `std::out_of_range` for a missing key and never inserts.
42	`if (i->second < 5) i = stock.erase(i); …`	Correct erase-while-iterating idiom: `erase` returns the iterator to the next element.
47	`std::sort(names.begin(), names.end()); …`	Iteration order is unspecified, so sort before printing for reproducible output.
55	`if (!seen.insert(x).second) ++dups; …`	Duplicate detection with a set: insertion fails for a key already present.
68	`m.reserve(100); …`	Prepares buckets for 100 elements so that inserting them never rehashes.
80	`int* p = &m[7]; …`	Pointers/references to elements stay valid across a rehash (iterators do not).

13.6 `std::hash`

`std::hash<Key>` is a function object that returns a `std::size_t` hash value. The standard library provides it for integers, floating-point types, pointers, `bool`, `char` types, `std::string`, `std::string_view`, `std::bitset`, `std::unique_ptr` and others. Important facts:

- The **algorithm is not specified.** For integers many implementations return the value itself; for strings they use a mixing function. Never rely on particular numbers.
- Within one program run, equal keys give equal hashes. Values may **differ between runs or platforms**, so do not store them.
- There is **no** `std::hash` for `std::pair`, `std::tuple`, arrays or your own structs: you must supply one (Section 13.8).
- A table reduces the hash value to a bucket index itself (like Section 1.6): `hash % bucket_count` or an equivalent.

13.7 Custom Hash Functions and Custom Key Equality

A container of `Key` uses two policies: a **Hash** functor (`size_t operator()(const Key&) const`) and a **KeyEqual** functor (default `std::equal_to<Key>`). The contract between them is the most important rule:

> **KEY POINT. If `equal(a, b)` is true then `hash(a) == hash(b)` must be true.** The reverse need not hold (different keys may collide). If this rule is broken, equal keys land in different buckets and lookups fail even though the key is stored.

```cpp
// Part 13: custom hash functions, composite keys and key equality (C++17)
#include <cctype>
#include <cstddef>
#include <functional>
#include <iostream>
#include <string>
#include <unordered_map>
#include <unordered_set>
#include <utility>

// Combine a new value into a running hash (same idea as boost::hash_combine)
inline void hashCombine(std::size_t& seed, std::size_t v) {
    seed ^= v + 0x9e3779b97f4a7c15ULL + (seed << 6) + (seed >> 2);
```

```cpp
}

// 1. A struct key: needs operator== and a hash that uses ALL members
struct Point {
    int x, y;
    bool operator==(const Point& o) const { return x == o.x && y == o.y; }
};
struct PointHash {
    std::size_t operator()(const Point& p) const noexcept {
        std::size_t seed = 0;
        hashCombine(seed, std::hash<int>{}(p.x));
        hashCombine(seed, std::hash<int>{}(p.y));
        return seed;
    }
};

// 2. std::pair has no std::hash, so we supply one
struct PairHash {
    std::size_t operator()(const std::pair<int, int>& p) const noexcept {
        std::size_t seed = 0;
        hashCombine(seed, std::hash<int>{}(p.first));
        hashCombine(seed, std::hash<int>{}(p.second));
        return seed;
    }
};

// 3. Case-insensitive string keys: hash AND equality must agree on what "equal" means
struct CiHash {
    std::size_t operator()(const std::string& s) const {
        std::size_t h = 0;
        for (unsigned char c : s) h = h * 131 + static_cast<unsigned char>(std::tolower(c));
        return h;
    }
};
struct CiEqual {
    bool operator()(const std::string& a, const std::string& b) const {
        if (a.size() != b.size()) return false;
        for (std::size_t i = 0; i < a.size(); ++i)
            if (std::tolower(static_cast<unsigned char>(a[i])) !=
                std::tolower(static_cast<unsigned char>(b[i]))) return false;
        return true;
    }
};

// 4. A terrible but LEGAL hash: everything collides
struct ConstantHash {
    std::size_t operator()(int) const { return 0; }
};

int main() {
    std::unordered_set<Point, PointHash> visited;
    visited.insert({1, 2});
    visited.insert({2, 1});
    visited.insert({1, 2});                              // duplicate
    std::cout << "points stored: " << visited.size() << ", has (2,1): "
              << visited.count({2, 1}) << ", has (3,3): " << visited.count({3, 3}) << "\n";

    std::unordered_map<std::pair<int, int>, std::string, PairHash> grid;
    grid[{0, 0}] = "start";
    grid[{4, 7}] = "goal";
    std::cout << "grid(4,7) = " << grid.at({4, 7}) << ", size = " << grid.size() << "\n";

    std::unordered_map<std::string, int, CiHash, CiEqual> counts;
    for (const char* w : {"Hash", "hash", "HASH", "table"}) ++counts[w];
    std::cout << "case-insensitive: distinct = " << counts.size() << ", hash -> "
              << counts["hAsH"] << "\n";

    std::unordered_map<int, int, ConstantHash> bad;
    for (int i = 0; i < 50; ++i) bad[i] = i;
    std::size_t b = bad.bucket(7);
    std::cout << "constant hash: all 50 keys in one bucket: " << (bad.bucket_size(b) == 50)
              << ", lookup still correct: " << (bad.at(33) == 33) << "\n";
    return 0;
}
```

```
Output (g++ -std=c++17)
points stored: 2, has (2,1): 1, has (3,3): 0
grid(4,7) = goal, size = 2
case-insensitive: distinct = 2, hash -> 3
constant hash: all 50 keys in one bucket: 1, lookup still correct: 1
```

Line	Code	What it does
12	`inline void hashCombine(std::size_t& seed, s…`	Mixes one more value into `seed`. Order matters, so (1,2) and (2,1) give different results.
19	`bool operator==(const Point& o) const { retu…`	Key equality for the struct; every member that takes part in equality must take part in the hash.
22	`std::size_t operator()(const Point& p) const…`	Hash functor for `Point`: `noexcept` because it cannot throw.
31	`struct PairHash {`	`std::pair` has no standard hash; this functor is passed as the third template argument.
44	`for (unsigned char c : s) h = h * 131 + stat…`	The case-insensitive hash **lower-cases** before combining, so "Hash", "hash" and "HASH" hash equally.
48	`struct CiEqual {`	Equality that ignores case; consistent with `CiHash` (equal under `CiEqual` ⇒ equal hash).
59	`struct ConstantHash {`	A legal but terrible hash: everything collides, but the table still works correctly, only slowly.
84	`std::cout << "constant hash: all 50 keys in …`	With a constant hash all 50 elements share one bucket: the $O(n)$ worst case of Part 14, visible through the bucket interface.

Specialising `std::hash` is the alternative to passing a functor: write `namespace std { template<> struct hash<Point> { size_t operator()(const Point&) const noexcept; }; }` once, and then `std::unordered_set<Point>` works without extra template arguments. Specialise only for **your own** types.

13.8 Hashing Pairs and Composite Keys

- **Combine**, never add or XOR the member hashes naively: `h(x) + h(y)` is symmetric ((1,2) and (2,1) collide) and `h(x) ^ h(y)` is zero when $x = y$.
- Use a **mixing combine** such as `seed ^= v + 0x9e3779b97f4a7c15 + (seed << 6) + (seed >> 2)` (the same idea as `boost::hash_combine`), repeated for every field.
- Alternative for small integer pairs: pack into one 64-bit integer (`(uint64_t(x) << 32) | uint32_t(y)`) and hash that.
- All members used in `operator==` must be hashed; extra members must **not** be hashed (equal objects would then hash differently).

13.9 Bucket Interface and Load-factor Functions

Function	Meaning	Typical use
`bucket_count()`	number of buckets m currently allocated	diagnostics
`size()`	number of elements n	
`load_factor()`	n / m = `size() / bucket_count()`	check crowding
`max_load_factor()`	returns the threshold; `max_load_factor(f)` sets it (default 1.0)	tune speed versus memory
`rehash(n)`	makes the bucket count at least `n` and at least `size() / max_load_factor()`; **rehashes all elements**	force a rebuild
`reserve(n)`	same as `rehash(ceil(n / max_load_factor()))`: room for `n` elements	**call before bulk insertion**
`bucket(k)`, `bucket_size(b)`	bucket index of a key; number of elements in bucket `b`	debugging collisions
`begin(b)`, `end(b)`	iterate the elements of one bucket (local iterators)	proof that buckets hold several elements

When an insertion would make `load_factor()` exceed `max_load_factor()` the container **rehashes** automatically (Part 10). The exact new bucket count is implementation defined; the program in this part therefore tests **properties** (`bucket_count() >= 100`, load factor never above the maximum) and never prints a bucket count.

13.10 Iterator Invalidation and Usage Caveats

Operation	Iterators	References / pointers to elements
`insert` / `emplace` / `operator[]` **without** rehash	remain valid	remain valid
`insert` / `emplace` / `operator[]` **with** rehash	**invalidated**	remain valid
`rehash`, `reserve` (when they rehash)	**invalidated**	remain valid
`erase(it)`	only `it` invalidated	only the erased element
`clear()`	all invalidated	all invalidated

- **Erase while iterating:** use `it = m.erase(it);` never `m.erase(it); ++it;`.
- **Do not insert while holding iterators** (a hidden rehash invalidates them); store keys and re-find, or `reserve` beforehand so no rehash can happen.
- `operator[]` **inserts:** `if (m[k] == 0)` on a missing key silently adds `k` (see `zero = stock["fig"]` above). Use `find`, `count` or `contains`.
- **Iteration order is unspecified** and can change after any insertion or rehash; never depend on it for output (sort first).
- **Modifying a key** through a pointer breaks the table; keys are `const` in a map for this reason.
- **Hash-quality problems** show up as `bucket_size` spikes; fix the hash, not the table.

13.11 Common Implementation Mistakes

#	Mistake	Consequence	Cure
1	`%` with negative keys	negative index, undefined behaviour	`((k % m) + m) % m`
2	using the hash value as an index without `% m`	out-of-range access	always reduce
3	EMPTY instead of tombstone on delete	unreachable keys (Section 13.12)	DELETED marker
4	searching tombstone slots as matches	finds deleted keys	compare key only when OCCUPIED
5	unbounded probe loop	infinite loop on a full table / bad step	loop at most m times
6	duplicate check skipped	two copies of one key	scan whole probe path first
7	second hash can return 0	probe never moves	1 + (k mod (m−1))
8	resizing by copying the array	keys unreachable	recompute every index
9	resize by a constant increment	$O(n^2)$ total	multiply capacity
10	forgetting to free nodes / copying raw-pointer tables	leaks, double free	destructor, delete copy, RAII
11	hash and equality disagree	lookups fail	equal ⇒ same hash
12	signed overflow in hash arithmetic	undefined behaviour	unsigned types
13	assuming O(1) is guaranteed	performance bugs under bad inputs	know the assumptions (Part 14)

13.12 Debugging Incorrect Hash-table Behaviour

Hash-table bugs are usually **silent**: the program runs but keys vanish or searches fail. A systematic approach:

1. **Reproduce with a tiny table** (m = 7 or 11) and keys that all share a home slot, as in Parts 7–9. Collision bugs appear at once.
2. **Print the table** after every operation (slot, state, key) and compare with a hand dry run.
3. **Check invariants** after each operation: (a) every stored key is found by `contains`; (b) the stored count equals the number of OCCUPIED slots; (c) no key appears twice; (d) no index is out of range.
4. **Use sanitizers** (`-fsanitize=address,undefined`) to catch out-of-range indexes, use-after-free and signed overflow.
5. **Test the boundaries:** empty table, one element, full table, negative keys, wrap-around, deleting the head of a chain, deleting then re-inserting the same key.

The program below contains the classic bug (a deleted slot is reset to EMPTY). The invariant checker `unreachableKeys()` exposes it immediately, and the tombstone version passes:

```
// Part 13: finding a hash-table bug with an invariant checker (C++17)
#include <iostream>
#include <vector>

enum class State { EMPTY, OCCUPIED, DELETED };
struct Slot { int key = 0; State state = State::EMPTY; };

class Table {
public:
    Table(int m, bool useTombstones) : m_(m), tomb_(useTombstones), t_(m) {}

    void insert(int key) {                              // assumes the table is not full
        int j = key % m_;
        while (t_[j].state == State::OCCUPIED) j = (j + 1) % m_;
        t_[j].key = key;
        t_[j].state = State::OCCUPIED;
    }
    bool contains(int key) const {
        for (int i = 0; i < m_; ++i) {
            const Slot& s = t_[(key % m_ + i) % m_];
            if (s.state == State::EMPTY) return false;
            if (s.state == State::OCCUPIED && s.key == key) return true;
        }
        return false;
    }
    void erase(int key) {
        for (int i = 0; i < m_; ++i) {
            Slot& s = t_[(key % m_ + i) % m_];
            if (s.state == State::EMPTY) return;
            if (s.state == State::OCCUPIED && s.key == key) {
                s.state = tomb_ ? State::DELETED : State::EMPTY;   // BUG when tomb_ is false
                return;
            }
        }
    }
    // Invariant: every stored key must be findable by contains()
    int unreachableKeys() const {
        int bad = 0;
        for (const Slot& s : t_)
            if (s.state == State::OCCUPIED && !contains(s.key)) {
                std::cout << "  unreachable key " << s.key << " (home slot " << s.key % m_ << ")\n";
                ++bad;
            }
        return bad;
    }
private:
    int m_;
    bool tomb_;
    std::vector<Slot> t_;
};

int main() {
    for (bool tombstones : {false, true}) {
        std::cout << (tombstones ? "deletion with tombstone:\n" : "deletion by resetting to EMPTY:\n");
        Table t(11, tombstones);
```

```
56          for (int k : {25, 36, 47}) t.insert(k);         // all home slot 3
57          t.erase(36);
58          std::cout << "  contains(47) = " << t.contains(47) << "\n";
59          int bad = t.unreachableKeys();                  // prints each offender first
60          std::cout << "  unreachable keys: " << bad << "\n";
61      }
62      return 0;
63  }
```

Output

```
deletion by resetting to EMPTY:
  contains(47) = 0
  unreachable key 47 (home slot 3)
  unreachable keys: 1
deletion with tombstone:
  contains(47) = 1
  unreachable keys: 0
```

With keys 25, 36, 47 (home slot 3) the table has them in slots 3, 4, 5. Erasing 36 and writing EMPTY into slot 4 cuts the probe path: `contains(47)` stops at slot 4 and the checker reports 47 as **unreachable** although it is still stored.

> **MEMORY TRICK.** A checker like `unreachableKeys()` takes ten lines and finds more bugs than staring at the code. Run it inside `assert` in debug builds.

PART 14

Time Complexity, Space Complexity and Performa

Average, amortized and worst case; derivations, simulation, adversarial inputs and comparison with other structures

14.1 Three Kinds of Cost: Worst, Average and Amortized

Kind	Meaning	Example in hashing
Worst case	the largest cost over **all** inputs of a given size	all n keys collide: search costs n
Average / expected case	cost averaged over a **probability model** (random hash function or random keys)	search costs about $1 + \alpha$ with simple uniform hashing
Amortized	total cost of a **sequence** of operations divided by its length, no probability involved	doubling the table makes n insertions cost $O(n)$ in total

KEY POINT. The statement "hash tables have $O(1)$ insert, search and delete" means: **expected** $O(1)$ for search and delete, **expected amortized** $O(1)$ for insert, **under assumptions**. It is **not** a worst-case guarantee.

Assumptions behind the expected bounds:

1. **Simple uniform hashing:** each key is equally likely to go to any slot, independently of the others (or the hash function is chosen at random from a universal family).
2. **Bounded load factor:** $\alpha = n/m = O(1)$, maintained by resizing.
3. **Constant-time hash computation and key comparison** (for strings of length L both cost $O(L)$).
4. **Keys are not chosen by an adversary** who knows the hash function.

14.2 Complexity of Each Operation

Operation	Chaining (expected)	Open addressing (expected, $\alpha < 1$ fixed)	Worst case (all methods)
Insert (with duplicate check)	$O(1 + \alpha)$	$O(1/(1-\alpha))$ probes	$O(n)$
Search, successful	$O(1 + \alpha)$	$O(1)$	$O(n)$
Search, unsuccessful	$O(1 + \alpha)$	$O(1/(1-\alpha))$ probes	$O(n)$
Delete	$O(1 + \alpha)$	same as search	$O(n)$
Update value	same as search	same as search	$O(n)$
Rehash / resize	$O(m_{\text{old}} + n)$	$O(m_{\text{old}} + n \cdot \text{probes})$	$O(n^2)$ if all keys collide
Traverse all keys	$O(m + n)$	$O(m)$	same

The factor $1 + \alpha$ for chaining is "1 for hashing and reaching the bucket, plus α for walking the chain". If resizing keeps $\alpha \le$ a constant, both columns become $O(1)$.

14.3 Separate Chaining: Derivation

Let n keys be hashed to m buckets with simple uniform hashing. For a fixed bucket, each key lands there with probability $1/m$, so the **expected chain length** is $n/m = \alpha$.

- **Unsuccessful search:** examine the whole chain of the key's bucket: α **nodes** expected, plus $O(1)$ for the hash and bucket access: $\Theta(1 + \alpha)$.
- **Successful search:** the sought key is equally likely to be any stored key. Keys inserted at the tail of a chain: finding the i-th inserted key examines the keys placed in its bucket before it, about $1 + (i\text{-}1)/m$ nodes. Averaging over $i = 1 \ldots n$ gives $1 + (n\text{-}1)/(2m) \approx 1 + \alpha/2$ nodes.
- **Delete:** a search plus $O(1)$ unlinking.

$$\text{unsuccessful} \approx \alpha \quad \text{successful} \approx 1 + \alpha/2 \text{ (nodes compared)}$$

14.4 Open Addressing: Derivation Sketch

Assume **uniform hashing**: each probe sequence is equally likely to be any permutation of the m slots. In an unsuccessful search each probe finds an occupied slot with probability about α (when n of m slots are occupied), independently enough for this estimate:

$$E[\text{probes, unsuccessful}] \approx 1 + \alpha + \alpha^2 + \alpha^3 + \ldots = 1 \,/\, (1 - \alpha)$$

The geometric series converges only for $\alpha < 1$; as $\alpha \to 1$ the cost grows without bound. Inserting a key costs the same as an unsuccessful search for it. A **successful** search for the key that was inserted when the table held i keys costs $1/(1 - i/m)$; averaging over i:

$$E[\text{probes, successful}] \approx (1/\alpha) \cdot \ln(\, 1 \,/\, (1 - \alpha)\,)$$

Linear probing is not uniform hashing because of primary clustering; Knuth's classical analysis gives larger values: unsuccessful $\approx \tfrac{1}{2}[1 + 1/(1\text{-}\alpha)^2]$, successful $\approx \tfrac{1}{2}[1 + 1/(1\text{-}\alpha)]$. **Quadratic probing** lies between linear probing and uniform hashing. These formulas hold for random keys in an idealised model.

14.5 Impact of Load Factor: Theory versus Simulation

Below, the theoretical values are compared with a **simulation** run for this book (not a timing benchmark of a machine): table size $m = 1009$ (prime), random keys, and **2000 absent and all stored keys** searched; the average number of **probes** (slots examined) for open addressing, or **nodes compared** for chaining, is reported. Quadratic probing is omitted because pure-square probing fails above $\alpha = 0.5$ on some keys.

α	Method	Unsuccessful: theory	Unsuccessful: simulated	Successful: theory	Successful: simulated
0.5	linear probing	2.50	2.55	1.50	1.53
0.5	double hashing	2.00	1.98	1.39	1.42
0.5	chaining (nodes compared)	0.50	0.51	1.25	1.25
0.7	linear probing	6.04	5.44	2.17	2.05
0.7	double hashing	3.33	3.43	1.72	1.76
0.7	chaining (nodes compared)	0.70	0.69	1.35	1.34
0.9	linear probing	50.40	32.29	5.50	4.29
0.9	double hashing	9.99	10.14	2.56	2.57
0.9	chaining (nodes compared)	0.90	0.90	1.45	1.44

The simulated values agree well with the formulas for $\alpha \le 0.7$. At $\alpha = 0.9$ the asymptotic formula overestimates linear probing for a finite table of 1009 slots and one random sample, but the pattern is the lesson: for an absent key linear probing needs tens of probes, double hashing about 10, chaining under 1.

14.6 Cost of Unsuccessful Searches, Long Probe Sequences and Clustering

- **Unsuccessful searches are the expensive case** in open addressing: they only stop at an EMPTY slot, so every clustered or tombstone slot on the path is paid for.
- **Insertion costs the same as an unsuccessful search**, so a table that is full of tombstones is slow to insert into as well.
- **Primary clustering** (linear probing) makes the cost grow with $1/(1\text{-}\alpha)^2$ instead of $1/(1\text{-}\alpha)$.
- **Long probe sequences are rare on average but not impossible:** the maximum probe length over n keys is noticeably larger than the average; real-time systems care about this tail.
- **Tombstones** count as occupied for searching: rebuild the table when (live + deleted) / m gets large.

14.7 Space Complexity

Structure	Space	Example: n = 1,000,000 int keys, 4-byte key, 8-byte pointers (illustration)
Open addressing, load 0.5	$O(m)$ with $m = 2n$	about $2n \times (4 + 1)$ bytes if a 1-byte state is stored ≈ 10 MB (alignment may add more)
Separate chaining, load 1	$O(m + n)$	bucket array $m \times 8$ = 8 MB plus n nodes of (4 key + 4 padding + 8 pointer) = 16 MB ≈ 24 MB plus allocator overhead
Sorted array	$O(n)$	4 MB
Balanced BST	$O(n)$	n nodes with key, 2 child pointers and balance data ≈ 32 MB

The byte counts are **back-of-envelope estimates** under the stated assumptions; real sizes depend on the compiler, alignment and the memory allocator.

14.8 Rehashing Costs and Amortization (Recap)

A single rehash costs $O(m_{\text{old}} + n)$. With geometric growth the **total** rehash work over n insertions is at most about $2n$ (Section 10.10), so the **amortized** insertion cost stays $O(1)$, but one unlucky insertion pays $O(n)$. Reserving capacity in advance removes the pauses.

14.9 Adversarial and Pathological Inputs

- **Patterned keys:** keys that are all multiples of m (or of a factor shared with m) go to a few slots under k mod m (Section 3.12).
- **Hash flooding:** an attacker who knows the hash function sends many colliding keys (for example specially crafted strings) to a server. A chained table of n colliding keys needs $n(n+1)/2$ comparisons to insert them all.
- **Cures:** randomized/keyed hash functions or universal hashing (Section 12.1–12.2), tree-structured buckets, limits on input size.

Colliding keys n	Comparisons for all insertions $n(n+1)/2$	Comparisons if spread evenly ($\alpha = 1$, about 1 each)
1,000	500,500	1,000
10,000	50,005,000	10,000
100,000	5,000,050,000	100,000

14.10 Comparison with Linear Search, Binary Search and Balanced Trees

Method	Search cost	Comparisons for n = 1,000,000	Insert	Ordered ?
Linear search (unsorted array)	$O(n)$	about 500,000 (average, successful), 1,000,000 (absent)	$O(1)$ append	no
Binary search (sorted array)	$O(\log_2 n)$	about 20	$O(n)$ shifting	yes
AVL tree	$O(\log n)$	at most about 28	$O(\log n)$	yes
Hash table, $\alpha \le 0.75$	$O(1)$ expected	about 1–3 slots or nodes	$O(1)$ expected amortized	no

Hash tables win on pure lookup and insertion speed; trees win when ordering, range queries or **worst-case guarantees** matter. Hashing also needs a hash function and spare memory.

14.11 Consolidated Complexity Table

Structure	Search	Insert	Delete	Space	Notes
Separate chaining	$O(1+\alpha)$ avg, $O(n)$ worst	$O(1+\alpha)$ avg	$O(1+\alpha)$ avg	$O(m+n)$	α may exceed 1
Linear probing	$O(1)$ avg ($\alpha<1$), $O(n)$ worst	same	same (tombstone)	$O(m)$	primary clustering
Quadratic probing	$O(1)$ avg, $O(n)$ worst	same, may fail	same	$O(m)$	coverage conditions
Double hashing	$O(1)$ avg, $O(n)$ worst	same	same	$O(m)$	prime m
Cuckoo hashing	$O(1)$ **worst**	$O(1)$ expected amortized	$O(1)$ worst	$O(m)$	$\alpha < 0.5$
Perfect hashing (static)	$O(1)$ **worst**	not applicable	not applicable	$O(n)$	fixed key set
Sorted array	$O(\log n)$	$O(n)$	$O(n)$	$O(n)$	ordered
Balanced BST	$O(\log n)$	$O(\log n)$	$O(\log n)$	$O(n)$	ordered

14.12 Why Average-case $O(1)$ Does Not Mean Worst-case $O(1)$

1. The hash function is a **fixed deterministic function**: for any such function there exist sets of keys that all collide (pigeonhole), so some input forces $O(n)$.
2. The expectation is taken over a **model** (random keys or a random choice of hash function). Individual operations can be far above the mean.
3. Resizing makes **single** insertions cost $O(n)$ even though the amortized cost is $O(1)$.
4. Open-addressing costs diverge as $\alpha \to 1$; the $O(1)$ claim needs α bounded away from 1.

14.13 Numerical and Exam-style Complexity Questions

Q1. A chained table has $n = 3000$ keys and $m = 1000$ buckets. Expected nodes compared in an unsuccessful and a successful search? **A.** $\alpha = 3$: unsuccessful ≈ 3 nodes; successful $\approx 1 + 3/2 = 2.5$ nodes.

Q2. Linear probing, $\alpha = 0.8$. Expected probes for unsuccessful search? **A.** $\tfrac{1}{2}[1 + 1/(1\text{-}0.8)^2] = \tfrac{1}{2}[1 + 25] = 13.0$ probes. For double hashing: $1/(1\text{-}0.8) = 5$.

Q3. Uniform hashing, $\alpha = 0.5$. Successful-search probes? **A.** $(1/0.5)\ln(1/0.5) = 2 \times 0.693 = 1.39$.

Q4. Why does open addressing with $\alpha = 0.99$ perform poorly even with a perfect hash function? **A.** Only 1% of slots are free, so an unsuccessful search needs about $1/(1\text{-}\alpha) = 100$ probes (more for linear probing).

Q5. A table doubles from 8 to 16 to 32 … slots as 1,000 keys are added one by one. About how many key moves happen in total due to rehashing? **A.** If each rehash happens when the table is full, the moves are $8 + 16 + 32 + \ldots + 512 = 1\{,\}016$ (always less than twice the last size); that is about one move per insertion, so the amortized cost is $O(1)$.

Q6. State the worst-case time of searching in a hash table with n keys, and give an input that causes it. **A.** $O(n)$: for example, $m = 10$ and keys 10, 20, 30, … which all have remainder 0.

PART 15

COMPLETE C++ PROGRAMS

Seven independent, compilable programs with algorithm, line-by-line explanation, exact output, dry run, complexity and pitfalls

Every program in this part is **self-contained** (it includes everything it uses), is written in standard **C++17**, was compiled with `g++ -std=c++17 -Wall -Wextra -pedantic -fsanitize=address,undefined`, and was executed; the output blocks are the captured output, copied without editing. The sample "input" is the list of operations hard-coded in `main`, so every program is deterministic and can be re-run by the reader. Each program case-checks empty, absent, duplicate, deletion, collision-heavy and full-table situations wherever they apply.

#	Program	Main idea
1	Basic hash table (division method)	index = key mod m, one entry per slot
2	Separate chaining	insert, search, delete, display with buckets of lists
3	Linear probing	tombstones, duplicates, full table
4	Quadratic probing	probe tracing and failure handling by rehash
5	Double hashing	correct second hash, full detection
6	Dynamic table	automatic grow, shrink and tombstone cleanup
7	String keys	polynomial hash and word-frequency counting

15.1 Program 1: Basic Hash Table with the Division Method

Objective	Store (key, value) pairs in an array using $h(k) = k \bmod m$, with insertion, searching and display. A colliding key is **reported and not stored**, which motivates Parts 5–9.
Concepts demonstrated	division method, index computation, safe modulo for negative keys, comparing the key (not only occupancy), duplicate update, collision detection

Algorithm

Pseudocode

```
insert(key, value):
    i = ((key mod m) + m) mod m
    if slot i is free:                 store (key, value); n = n + 1
    else if slot i holds the same key: replace the value        (update)
    else:                              report COLLISION; do not store
search(key):
    i = ((key mod m) + m) mod m
    if slot i is used AND its key == key: return its value
    else: return "not found"
```

Source code

```cpp
// Program 1: basic hash table, division method, one entry per slot (C++17)
#include <iostream>
#include <string>
#include <vector>

class BasicHashTable {
public:
    enum class Result { Inserted, Updated, Collision };

    explicit BasicHashTable(int m) : m_(m), used_(m, false), keys_(m, 0), values_(m) {}

    int indexFor(long long key) const {                 // division method, safe for negatives
        return static_cast<int>(((key % m_) + m_) % m_);
    }

    Result insert(long long key, const std::string& value) {
        int i = indexFor(key);
        if (!used_[i]) {                                // free slot: store the pair
            used_[i] = true;
            keys_[i] = key;
```

```
            values_[i] = value;
            ++n_;
            return Result::Inserted;
        }
        if (keys_[i] == key) {                          // same key again: update
            values_[i] = value;
            return Result::Updated;
        }
        return Result::Collision;                       // different key owns the slot
    }

    const std::string* search(long long key) const {
        int i = indexFor(key);
        if (used_[i] && keys_[i] == key) return &values_[i];   // compare the KEY, not just used_
        return nullptr;
    }

    void display() const {
        for (int i = 0; i < m_; ++i) {
            std::cout << "  [" << i << "] ";
            if (used_[i]) std::cout << keys_[i] << " : " << values_[i] << "\n";
            else std::cout << "(empty)\n";
        }
    }
    int size() const { return n_; }

private:
    int m_;
    int n_ = 0;
    std::vector<bool> used_;
    std::vector<long long> keys_;
    std::vector<std::string> values_;
};

static const char* name(BasicHashTable::Result r) {
    switch (r) {
        case BasicHashTable::Result::Inserted: return "inserted";
        case BasicHashTable::Result::Updated: return "updated";
        default: return "COLLISION (not stored)";
    }
}

int main() {
    BasicHashTable t(10);
    const std::pair<long long, const char*> input[] = {
        {17, "Asha"}, {24, "Ravi"}, {31, "Meera"}, {58, "Kabir"},
        {47, "Isha"}, {-5, "Tara"}, {24, "Ravi2"}};
    for (const auto& kv : input) {
        auto r = t.insert(kv.first, kv.second);
        std::cout << "insert " << kv.first << " (index " << t.indexFor(kv.first) << "): "
                  << name(r) << "\n";
    }
    std::cout << "table (n = " << t.size() << "):\n";
    t.display();
    for (long long k : {31LL, 99LL, 47LL, -5LL}) {
        const std::string* v = t.search(k);
        std::cout << "search " << k << ": " << (v ? *v : "not found") << "\n";
    }
    return 0;
}
```

Line-by-line explanation of the important logic

Line	Code	What it does
8	`enum class Result { Inserted, Updated, Colli…`	Three possible outcomes of an insertion, so the caller can react.
13	`return static_cast<int>(((key % m_) + m_) % …`	Division method; the double modulo keeps the index in $0 \ldots m\text{-}1$ for negative keys such as -5.
18	`if (!used_[i]) { …`	Free slot: store key and value, count the entry.
25	`if (keys_[i] == key) { …`	Slot taken by the **same** key: this is a duplicate, not a collision; the policy is update.
29	`return Result::Collision; …`	Slot taken by a **different** key: this is a collision; this basic table cannot store the key.
34	`if (used_[i] && keys_[i] == key) return &val…`	Search compares the **key**; `used_[i]` alone would wrongly "find" 47 in the slot of 17.
55	`static const char* name(BasicHashTable::Resu…`	Helper that converts the enum to text for printing.

Sample input (hard-coded operations)

Step	Operation	Step	Operation
1	insert 17, "Asha"	5	insert 47, "Isha" (same index as 17)
2	insert 24, "Ravi"	6	insert −5, "Tara" (negative key)
3	insert 31, "Meera"	7	insert 24, "Ravi2" (duplicate key)
4	insert 58, "Kabir"	8	search 31, 99, 47, −5

Exact output

Output of Program 1

```
insert 17 (index 7): inserted
insert 24 (index 4): inserted
insert 31 (index 1): inserted
insert 58 (index 8): inserted
insert 47 (index 7): COLLISION (not stored)
insert -5 (index 5): inserted
insert 24 (index 4): updated
table (n = 5):
  [0] (empty)
  [1] 31 : Meera
  [2] (empty)
  [3] (empty)
  [4] 24 : Ravi2
  [5] -5 : Tara
  [6] (empty)
  [7] 17 : Asha
  [8] 58 : Kabir
  [9] (empty)
search 31: Meera
search 99: not found
search 47: not found
search -5: Tara
```

Dry run: table changes

Operation	Index	Slot before	Action
insert 17	17 mod 10 = 7	empty	stored
insert 24	24 mod 10 = 4	empty	stored
insert 31	31 mod 10 = 1	empty	stored
insert 58	58 mod 10 = 8	empty	stored
insert 47	47 mod 10 = 7	17	COLLISION: slot owned by 17, key not stored
insert -5	-5 mod 10 = 5	empty	stored
insert 24	24 mod 10 = 4	24	value updated

Figure 15.1 follows the table of Program 1 through its insertions.

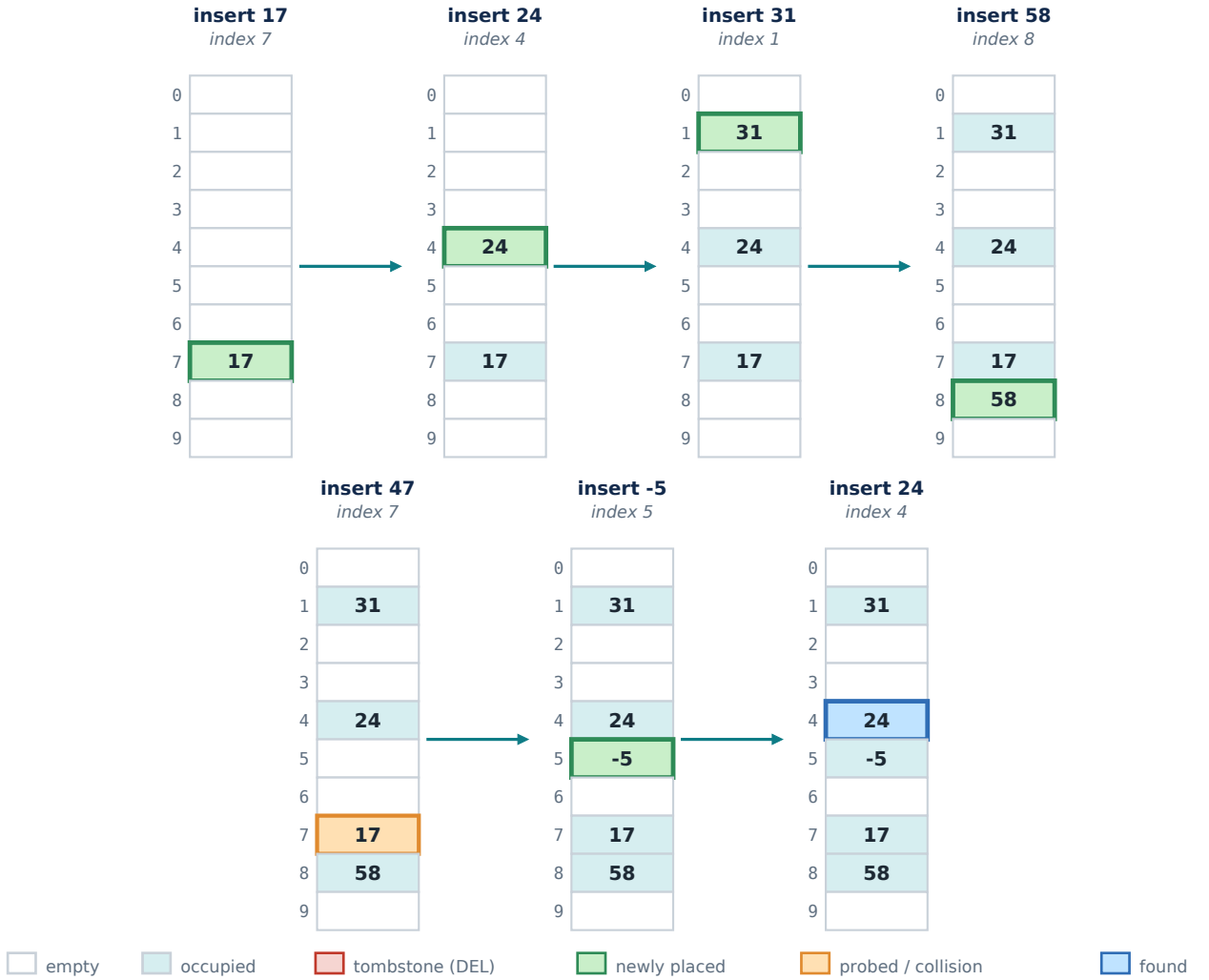


Figure 15.1 — Program 1 after each insertion. Insert 47 is orange (slot 7 owned by 17: collision, nothing changes); insert 24 again is blue (value updated in place).

Complexity analysis

Operation	Time	Space
insert / search	$O(1)$ (one index computation and one comparison); no probing or chains	$O(m)$ for the three arrays
display	$O(m)$	

Common mistakes and edge cases

- `key % m` with a negative key gives a negative index (here $-5 \% 10 = -5$ in C++); the double modulo prevents it.
- Treating "slot used" as "key found" (search 47 correctly reports **not found**).
- **Collision = data loss** in this program: 47 is silently not stored unless the caller checks the result. Real tables must resolve collisions.
- Empty table: every search returns not found at once. Duplicate key: update, never a second copy.

15.2 Program 2: Hash Table with Separate Chaining

Objective	Resolve collisions by keeping a list of entries in each bucket; provide insert (with update), search (counting nodes compared), delete and display.
Concepts demonstrated	array of lists, head insertion, duplicate update, deletion of head/middle/absent keys, load factor, longest chain, empty bucket and empty table

Algorithm

Pseudocode

```
insert(key, value):
    chain = bucket[key mod m]
    for each entry e in chain: if e.key == key: e.value = value; return "updated"
    add (key, value) at the FRONT of chain; n = n + 1; return "inserted"
search(key):   walk the chain of bucket[key mod m] counting entries compared
erase(key):    find the entry in its chain, unlink it, n = n - 1;  absent -> false
load factor = n / m        longest chain = max length over all buckets
```

Source code

```cpp
// Program 2: separate chaining with insert, search, delete, display (C++17)
#include <algorithm>
#include <iostream>
#include <list>
#include <string>
#include <utility>
#include <vector>

class ChainedTable {
    using Entry = std::pair<int, std::string>;
public:
    explicit ChainedTable(int m) : buckets_(m) {}

    bool insert(int key, const std::string& value) {     // true = new key, false = updated
        auto& chain = buckets_[indexFor(key)];
        for (Entry& e : chain)
            if (e.first == key) { e.second = value; return false; }
        chain.emplace_front(key, value);                 // new node at the head
        ++n_;
        return true;
    }

    const std::string* search(int key, int& compared) const {
        compared = 0;
        for (const Entry& e : buckets_[indexFor(key)]) {
            ++compared;
            if (e.first == key) return &e.second;
        }
        return nullptr;
    }

    bool erase(int key) {
        auto& chain = buckets_[indexFor(key)];
        for (auto it = chain.begin(); it != chain.end(); ++it) {
            if (it->first == key) { chain.erase(it); --n_; return true; }
        }
        return false;
    }

    void display() const {
        for (std::size_t i = 0; i < buckets_.size(); ++i) {
            std::cout << "  [" << i << "]";
            for (const Entry& e : buckets_[i]) std::cout << " " << e.first << ":" << e.second << " ->";
            std::cout << " null\n";
        }
    }

    double loadFactor() const { return static_cast<double>(n_) / buckets_.size(); }
    std::size_t longestChain() const {
        std::size_t best = 0;
        for (const auto& c : buckets_) best = std::max(best, c.size());
        return best;
    }
    int size() const { return n_; }

```

```cpp
private:
    std::size_t indexFor(int key) const {
        long long m = static_cast<long long>(buckets_.size());
        return static_cast<std::size_t>(((key % m) + m) % m);
    }
    std::vector<std::list<Entry>> buckets_;
    int n_ = 0;
};

int main() {
    ChainedTable t(7);
    int compared = 0;
    std::cout << "search in empty table: "
              << (t.search(5, compared) ? "found" : "absent") << ", nodes compared = "
              << compared << "\n";
    const std::pair<int, const char*> input[] = {
        {15, "Asha"}, {22, "Ravi"}, {8, "Meera"}, {29, "Kabir"}, {3, "Isha"}, {10, "Tara"}};
    for (const auto& kv : input) t.insert(kv.first, kv.second);
    std::cout << "after 6 insertions: n = " << t.size() << ", load = " << t.loadFactor()
              << ", longest chain = " << t.longestChain() << "\n";
    t.display();
    for (int k : {22, 36, 5}) {
        const std::string* v = t.search(k, compared);
        std::cout << "search " << k << ": " << (v ? *v : "absent")
                  << ", nodes compared = " << compared << "\n";
    }
    std::cout << "insert 3 again (update): new key? " << t.insert(3, "Isha2") << "\n";
    std::cout << "erase 8: " << t.erase(8) << ", erase 29: " << t.erase(29)
              << ", erase 99: " << t.erase(99) << "\n";
    std::cout << "after deletions: n = " << t.size() << "\n";
    t.display();
    return 0;
}
```

Line-by-line explanation of the important logic

Line	Code	What it does
61	`std::vector<std::list<Entry>> buckets_;`	The table: a vector of m lists. All memory is owned by the containers (no destructor needed).
16	`for (Entry& e : chain)`	Duplicate check scans the whole chain before inserting.
18	`chain.emplace_front(key, value); …`	New entries go to the head: $O(1)$ once the duplicate check is done.
26	`++compared;`	Counts nodes examined; an empty bucket gives 0, an absent key in a chain gives the chain length.
35	`if (it->first == key) { chain.erase(it); --n…`	Unlinks the node from the list: works for head, middle and tail alike.
51	`for (const auto& c : buckets_) best = std::m…`	Longest chain: the quantity that decides the worst-case search time.
59	`return static_cast<std::size_t>(((key % m) +…`	Safe bucket number for negative keys.

Sample input (hard-coded operations)

Step	Operation
1	search 5 in the **empty** table
2	insert 15 Asha, 22 Ravi, 8 Meera, 29 Kabir, 3 Isha, 10 Tara ($m = 7$; keys 15, 22, 8, 29 share bucket 1)
3	search 22 (present), 36 (absent, bucket 1), 5 (absent, empty bucket)
4	insert 3 again as Isha2 (duplicate), then erase 8 (middle), 29 (head) and 99 (absent)

Exact output

```
Output of Program 2
search in empty table: absent, nodes compared = 0
after 6 insertions: n = 6, load = 0.857143, longest chain = 4
  [0] null
  [1] 29:Kabir -> 8:Meera -> 22:Ravi -> 15:Asha -> null
  [2] null
  [3] 10:Tara -> 3:Isha -> null
  [4] null
  [5] null
  [6] null
search 22: Ravi, nodes compared = 3
search 36: absent, nodes compared = 4
search 5: absent, nodes compared = 0
insert 3 again (update): new key? 0
erase 8: 1, erase 29: 1, erase 99: 0
after deletions: n = 4
  [0] null
  [1] 22:Ravi -> 15:Asha -> null
  [2] null
  [3] 10:Tara -> 3:Isha2 -> null
  [4] null
  [5] null
  [6] null
```

Dry run: table changes

Figure 15.2 shows the buckets of Program 2 after each insertion.

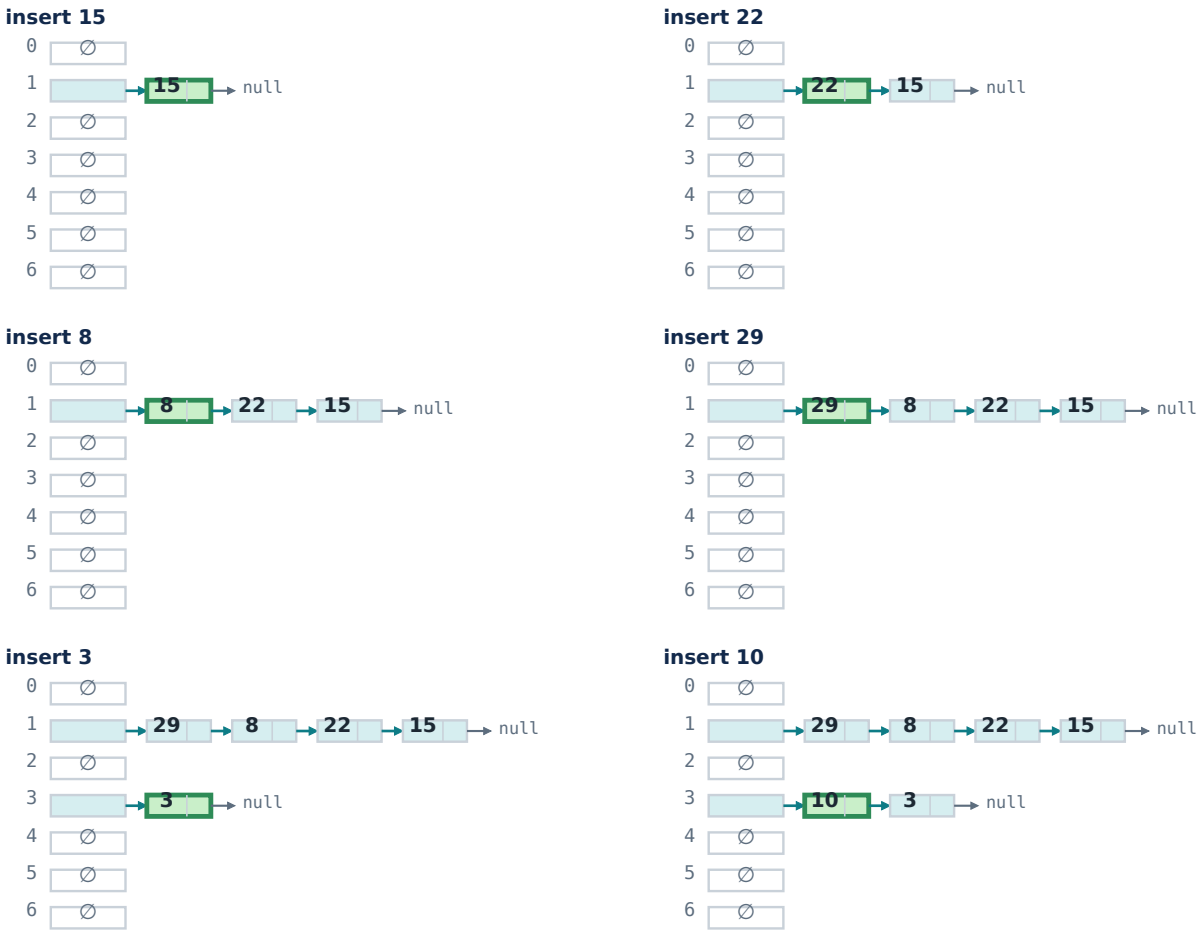


Figure 15.2 — Program 2: bucket contents after each insertion (new node in green). Four keys collide in bucket 1.

Operation	Bucket	Nodes compared	Result
search 5 (empty table)	5	0	absent
search 22	1	3	found
search 36	1	4	absent
search 5	5	0	absent
erase 8	1	2	middle node unlinked (29 → 22 → 15)
erase 29	1	1	head unlinked (22 → 15)
erase 99	1	2	absent

Complexity analysis

Operation	Expected	Worst	Space
insert, search, erase	$O(1 + \alpha)$	$O(n)$ (all keys in one chain)	$O(m + n)$
display / longestChain	$O(m + n)$	$O(m + n)$	

Common mistakes and edge cases

- Forgetting the duplicate scan, which creates two entries for one key.
- Erasing while iterating without using the iterator returned by `erase` (here the loop returns immediately after `erase`, which is safe).
- Searching an **empty bucket** must not dereference anything; the range-for handles it (0 comparisons).
- Load factor above 1 is fine in chaining (here $6/7 < 1$, but 20 keys in 7 buckets would also work, only slower).

15.3 Program 3: Hash Table with Linear Probing

Objective	Open addressing with linear probing: insertion with duplicate rejection, searching, deletion by tombstones with reuse, and full-table detection.
Concepts demonstrated	probe sequence $h(k,i) = (h(k)+i) \bmod m$, EMPTY/OCCUPIED/DELETED states, tombstone reuse after duplicate check, wrap-around, bounded loop, full table

Algorithm

Pseudocode

```
insert(key):                          search(key):
    firstDeleted = none                   for i = 0 .. m-1:
    for i = 0 .. m-1:                         j = (key mod m + i) mod m
        j = (key mod m + i) mod m             EMPTY    -> return absent
        EMPTY   : place at firstDeleted          OCCUPIED and key matches -> return j
                  (if set) else at j; done       (DELETED or other key: continue)
        DELETED : remember first one          return absent
        OCCUPIED, same key: duplicate
    if firstDeleted set: place there      erase(key): j = search(key); mark j DELETED
    else: FULL
```

Source code

```cpp
// Program 3: linear probing with tombstones, duplicate handling and full-table detection
#include <iomanip>
#include <iostream>
#include <vector>

class LinearTable {
public:
    enum class State { Empty, Occupied, Deleted };
    enum class Status { Inserted, Duplicate, Full };

    explicit LinearTable(int m) : m_(m), slots_(m) {}

    Status insert(int key, int& probes) {
        int firstDeleted = -1;
        probes = 0;
        for (int i = 0; i < m_; ++i) {
```

```
            int j = ((key % m_) + m_ + i) % m_;          // h(k, i) = (h(k) + i) mod m
            ++probes;
            Slot& s = slots_[j];
            if (s.state == State::Empty) {
                put(firstDeleted >= 0 ? firstDeleted : j, key);
                return Status::Inserted;
            }
            if (s.state == State::Deleted) {
                if (firstDeleted < 0) firstDeleted = j;
            } else if (s.key == key) {
                return Status::Duplicate;                // policy: reject duplicates
            }
        }
        if (firstDeleted >= 0) { put(firstDeleted, key); return Status::Inserted; }
        return Status::Full;
    }

    int search(int key, int& probes) const {             // slot or -1
        probes = 0;
        for (int i = 0; i < m_; ++i) {
            int j = ((key % m_) + m_ + i) % m_;
            ++probes;
            const Slot& s = slots_[j];
            if (s.state == State::Empty) return -1;
            if (s.state == State::Occupied && s.key == key) return j;
        }
        return -1;
    }

    bool erase(int key) {
        int probes = 0;
        int j = search(key, probes);
        if (j < 0) return false;
        slots_[j].state = State::Deleted;
        --n_;
        return true;
    }

    void display() const {
        std::cout << "  slot:";
        for (int j = 0; j < m_; ++j) std::cout << std::setw(5) << j;
        std::cout << "\n  key :";
        for (const Slot& s : slots_) {
            if (s.state == State::Empty) std::cout << std::setw(5) << "--";
            else if (s.state == State::Deleted) std::cout << std::setw(5) << "DEL";
            else std::cout << std::setw(5) << s.key;
        }
        std::cout << "\n";
    }
    int size() const { return n_; }

private:
    struct Slot { int key = 0; State state = State::Empty; };
    void put(int j, int key) { slots_[j].key = key; slots_[j].state = State::Occupied; ++n_; }
    int m_;
    int n_ = 0;
    std::vector<Slot> slots_;
};

static const char* text(LinearTable::Status s) {
    return s == LinearTable::Status::Inserted ? "inserted"
         : s == LinearTable::Status::Duplicate ? "duplicate (rejected)" : "FULL";
}

int main() {
    LinearTable t(7);
    int p = 0;
    int j = t.search(10, p);
    std::cout << "search 10 in empty table: " << (j < 0 ? "absent" : "found")
              << " (" << p << " probe)\n";
    for (int k : {10, 17, 24, 5, 12}) {
        auto s = t.insert(k, p);
        std::cout << "insert " << k << ": " << text(s) << " (" << p << " probes)\n";
    }
    t.display();
    j = t.search(12, p);
    std::cout << "search 12: slot " << j << " (" << p << " probes, wrapped around)\n";
    std::cout << "erase 17: " << t.erase(17) << ", erase 99 (absent): " << t.erase(99) << "\n";
```

```
    j = t.search(24, p);
    std::cout << "search 24 after erase: slot " << j << " (" << p
              << " probes, skipped the tombstone)\n";
    auto s = t.insert(31, p);
    std::cout << "insert 31: " << text(s) << " (" << p << " probes, reused the tombstone)\n";
    s = t.insert(10, p);
    std::cout << "insert 10 again: " << text(s) << " (" << p << " probe)\n";
    t.insert(8, p);
    t.insert(9, p);
    t.display();
    s = t.insert(100, p);
    std::cout << "insert 100 into a full table: " << text(s) << " after " << p << " probes\n";
    std::cout << "n = " << t.size() << "\n";
    return 0;
}
```

Line-by-line explanation of the important logic

Line	Code	What it does
8	`enum class State { Empty, Occupied, Deleted };`	Three slot states; DELETED is the tombstone.
14	`int firstDeleted = -1;`	First reusable slot met on the path.
17	`int j = ((key % m_) + m_ + i) % m_; …`	Linear probe with wrap-around; the home index is made non-negative first.
21	`put(firstDeleted >= 0 ? firstDeleted : j, ke…`	At an EMPTY slot the key is absent: use the remembered tombstone if there is one.
27	`return Status::Duplicate; // …`	Duplicate found anywhere on the probe path: rejected.
30	`if (firstDeleted >= 0) { put(firstDeleted, k…`	All m probes done, no EMPTY slot but a tombstone exists: reuse it.
31	`return Status::Full;`	No EMPTY slot and no tombstone: the table is full.
40	`if (s.state == State::Empty) return -1;`	Search stops only at EMPTY.
50	`slots_[j].state = State::Deleted;`	Erase = tombstone.
57	`for (int j = 0; j < m_; ++j) std::cout << st…`	Fixed-width columns so the printed table lines up.

Sample input (hard-coded operations)

Step	Operation
1	search 10 in the empty table
2	insert 10, 17, 24 (home 3), 5, 12 (home 5), $m = 7$
3	search 12 (wraps around); erase 17 and erase 99 (absent); search 24 over the tombstone
4	insert 31 (reuses the tombstone); insert 10 again (duplicate); insert 8 and 9 (table becomes full); insert 100 (FULL)

Exact output

```
Output of Program 3
search 10 in empty table: absent (1 probe)
insert 10: inserted (1 probes)
insert 17: inserted (2 probes)
insert 24: inserted (3 probes)
insert 5: inserted (2 probes)
insert 12: inserted (3 probes)
  slot:    0    1    2    3    4    5    6
  key :   12   --   --   10   17   24    5
search 12: slot 0 (3 probes, wrapped around)
erase 17: 1, erase 99 (absent): 0
search 24 after erase: slot 5 (3 probes, skipped the tombstone)
insert 31: inserted (6 probes, reused the tombstone)
insert 10 again: duplicate (rejected) (1 probe)
  slot:    0    1    2    3    4    5    6
  key :   12    8    9   10   31   24    5
insert 100 into a full table: FULL after 7 probes
n = 7
```

Dry run: table changes

Operation	Probe sequence	Result
insert 10	3	slot 3
insert 17	3 → 4	slot 4
insert 24	3 → 4 → 5	slot 5
insert 5	5 → 6	slot 6
insert 12	5 → 6 → 0	slot 0
search 12	5 → 6 → 0	found slot 0
erase 17	3 → 4	slot 4 := DEL
search 24	3 → 4 → 5	found slot 5, tombstone skipped
insert 31	3 → 4 → 5 → 6 → 0 → 1	stored in tombstone slot 4
insert 10	3	duplicate, rejected
insert 100	2 → 3 → 4 → 5 → 6 → 0 → 1	**FULL** after 7 probes

Figure 15.3 shows the table states of Program 3, including the tombstone and the full table.

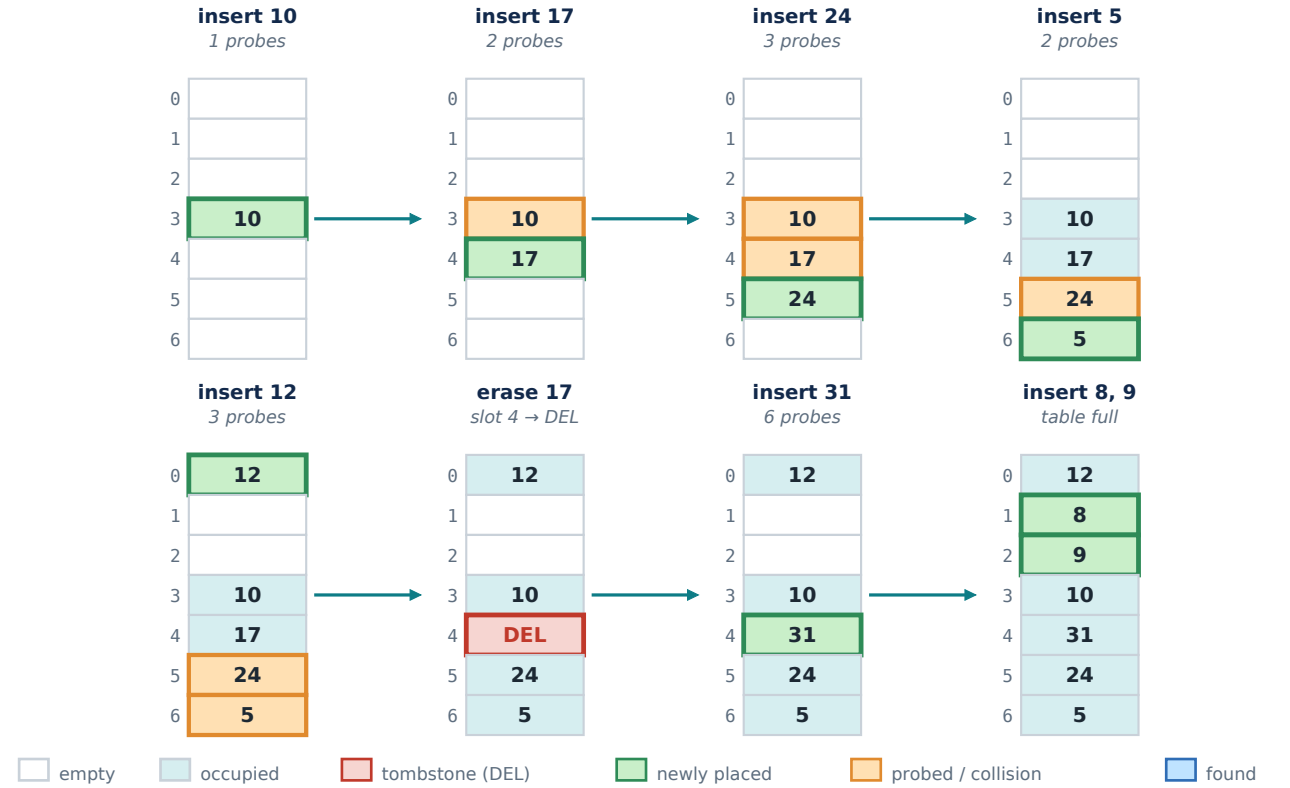


Figure 15.3 — Program 3: table states ($m = 7$). The wrap-around of key 12 (probes 5, 6, 0), the tombstone at slot 4 and its reuse by 31, and the final full table.

Complexity analysis

Operation	Expected ($\alpha \leq 0.7$)	Worst case	Space
insert, search, erase	$O(1)$ probes	$O(m)$ probes (also when full)	$O(m)$

Common mistakes and edge cases

- Resetting a deleted slot to EMPTY (Part 6); the tombstone avoids it.
- Reusing the tombstone **before** finishing the duplicate scan.
- Matching a tombstone because it still stores the old key value: the state is tested.
- Unbounded loop in a full table: the loop runs at most m times. A table with no EMPTY slot also makes searches for absent keys cost m probes.
- Empty table, wrap-around (key 12), duplicate key and absent key delete are all exercised above.

15.4 Program 4: Hash Table with Quadratic Probing

Objective	Quadratic probing with complete probe-sequence tracing and failure handling: detect an exhausted probe sequence, report how many slots are still empty, then grow the table and retry.
Concepts demonstrated	offsets $(C_1\, i + C_2\, i^2)/2$, triangular coverage for $m = 2^p$, incomplete coverage for composite m, failure versus full table, rehash and retry

Algorithm

Pseudocode

```
insert(key):
    for i = 0 .. m-1:
        j = (key mod m + (C1*i + C2*i*i) / 2) mod m        print j
        if slot j holds key:  duplicate -> done
        if slot j is empty:   store key at j -> done
    return FAILED            (probe sequence exhausted; empty slots may remain!)
insertOrGrow(key):
    if insert(key) fails:
        m = nextPrime(2m); use offsets i*i; re-insert every old key; insert(key) again
```

Source code

```cpp
// Program 4: quadratic probing with probe tracing and failure handling (rehash and retry)
#include <iostream>
#include <vector>

bool isPrime(int n) {
    if (n < 2) return false;
    for (int d = 2; d * d <= n; ++d) if (n % d == 0) return false;
    return true;
}
int nextPrime(int n) { while (!isPrime(n)) ++n; return n; }

class QuadraticTable {
public:
    // offset(i) = (C1*i + C2*i*i) / 2   ->  (1,1): triangular, (0,2): i*i
    QuadraticTable(int m, int C1, int C2) : m_(m), C1_(C1), C2_(C2), t_(m, kEmpty) {}

    int slotFor(int key, int i) const {
        long long h = ((key % m_) + m_) % m_;
        long long ii = i;
        return static_cast<int>((h + (C1_ * ii + C2_ * ii * ii) / 2) % m_);
    }

    // true on success; false if the probe sequence is exhausted (does NOT mean table full)
    bool insert(int key, bool trace) {
        if (trace) std::cout << "  insert " << key << ": probes";
        for (int i = 0; i < m_; ++i) {
            int j = slotFor(key, i);
            if (trace) std::cout << " " << j;
            if (t_[j] == key) { if (trace) std::cout << " -> duplicate\n"; return true; }
            if (t_[j] == kEmpty) {
                t_[j] = key;
                ++n_;
                if (trace) std::cout << " -> stored at " << j << "\n";
```

```
                return true;
            }
        }
        if (trace) std::cout << " -> FAILED (" << m_ - n_ << " empty slots remain)\n";
        return false;
    }

    // failure handling: grow to a larger prime table and retry
    void insertOrGrow(int key) {
        if (insert(key, true)) return;
        std::cout << "  growing " << m_ << " -> " << nextPrime(2 * m_) << " and rehashing "
                  << n_ << " keys\n";
        std::vector<int> old = t_;
        m_ = nextPrime(2 * m_);
        C1_ = 0; C2_ = 2;                              // plain i*i; prime m, load < 1/2 is safe
        t_.assign(m_, kEmpty);
        n_ = 0;
        for (int k : old) if (k != kEmpty) insert(k, false);
        insert(key, true);
    }

    int distinctSlots(int key) const {
        std::vector<bool> seen(m_, false);
        int c = 0;
        for (int i = 0; i < m_; ++i) if (!seen[slotFor(key, i)]) { seen[slotFor(key, i)] = true; ++c; }
        return c;
    }
    int capacity() const { return m_; }

    void display() const {
        std::cout << "  table:";
        for (int j = 0; j < m_; ++j) {
            if (t_[j] == kEmpty) std::cout << " .";
            else std::cout << " " << t_[j];
        }
        std::cout << "\n";
    }

private:
    static constexpr int kEmpty = -1;
    int m_, C1_, C2_;
    int n_ = 0;
    std::vector<int> t_;
};

int main() {
    std::cout << "A) m = 8, triangular offsets: probe sequence reaches "
              << QuadraticTable(8, 1, 1).distinctSlots(0) << " of 8 slots\n";
    QuadraticTable a(8, 1, 1);
    for (int k : {8, 16, 24, 32, 40, 48, 56, 64}) a.insert(k, true);
    a.display();
    std::cout << "  insert 72 into the full table:\n";
    a.insert(72, true);

    std::cout << "B) m = 10, offsets i*i: reaches "
              << QuadraticTable(10, 0, 2).distinctSlots(5) << " of 10 slots from home 5\n";
    QuadraticTable b(10, 0, 2);
    for (int k : {5, 15, 25, 35, 45, 55}) b.insert(k, true);
    b.display();
    b.insertOrGrow(65);
    b.display();
    std::cout << "  capacity is now " << b.capacity() << "\n";
    return 0;
}
```

Line-by-line explanation of the important logic

Line	Code	What it does
10	`int nextPrime(int n) { while (!isPrime(n)) +…`	Smallest prime $\geq n$, used when the table must grow.
15	`QuadraticTable(int m, int C1, int C2) : m_(m…`	Constants are doubled ($c = C/2$) so ½ is handled with integer arithmetic.
19	`long long ii = i;`	64-bit arithmetic prevents overflow in $C_2\, i^2$.

Line	Code	What it does
20	`return static_cast<int>((h + (C1_ * ii + C2_…`	The probe formula $h + (C_1\, i + C_2\, i^2)/2$ reduced mod m.
29	`if (t_[j] == key) { if (trace) std::cout << …`	Duplicate found on the probe path (treated as success).
37	`if (trace) std::cout << " -> FAILED (" << m_…`	Failure message includes the number of empty slots: proof that **failure is not the same as full**.
42	`void insertOrGrow(int key) {`	Failure handling: grow and retry.
48	`C1_ = 0; C2_ = 2; …`	After growing to a prime m the plain-square rule is used (safe for load below 0.5).
51	`for (int k : old) if (k != kEmpty) insert(k,…`	Rehash: every old key is re-inserted with the **new** size.
55	`int distinctSlots(int key) const {`	Coverage test: counts distinct slots in one probe sequence.

Sample input (hard-coded operations)

Part	Operations
A	$m = 8$, triangular offsets: insert 8, 16, 24, 32, 40, 48, 56, 64 (all home slot 0); insert 72 into the full table
B	$m = 10$, offsets i^2: insert 5, 15, 25, 35, 45, 55 (home slot 5); insert 65 with `insertOrGrow`

Exact output

Output of Program 4

```
A) m = 8, triangular offsets: probe sequence reaches 8 of 8 slots
  insert 8: probes 0 -> stored at 0
  insert 16: probes 0 1 -> stored at 1
  insert 24: probes 0 1 3 -> stored at 3
  insert 32: probes 0 1 3 6 -> stored at 6
  insert 40: probes 0 1 3 6 2 -> stored at 2
  insert 48: probes 0 1 3 6 2 7 -> stored at 7
  insert 56: probes 0 1 3 6 2 7 5 -> stored at 5
  insert 64: probes 0 1 3 6 2 7 5 4 -> stored at 4
  table: 8 16 40 24 64 56 32 48
  insert 72 into the full table:
  insert 72: probes 0 1 3 6 2 7 5 4 -> FAILED (0 empty slots remain)
B) m = 10, offsets i*i: reaches 6 of 10 slots from home 5
  insert 5: probes 5 -> stored at 5
  insert 15: probes 5 6 -> stored at 6
  insert 25: probes 5 6 9 -> stored at 9
  insert 35: probes 5 6 9 4 -> stored at 4
  insert 45: probes 5 6 9 4 1 -> stored at 1
  insert 55: probes 5 6 9 4 1 0 -> stored at 0
  table: 55 45 . . 35 5 15 . . 25
  insert 65: probes 5 6 9 4 1 0 1 4 9 6 -> FAILED (4 empty slots remain)
  growing 10 -> 23 and rehashing 6 keys
  insert 65: probes 19 -> stored at 19
  table: . . 25 . . 5 . . . 55 . . 35 . . 15 . . . 65 . . 45
  capacity is now 23
```

Dry run: table changes

Figure 15.4 shows part A of Program 4: eight keys with the same home slot.

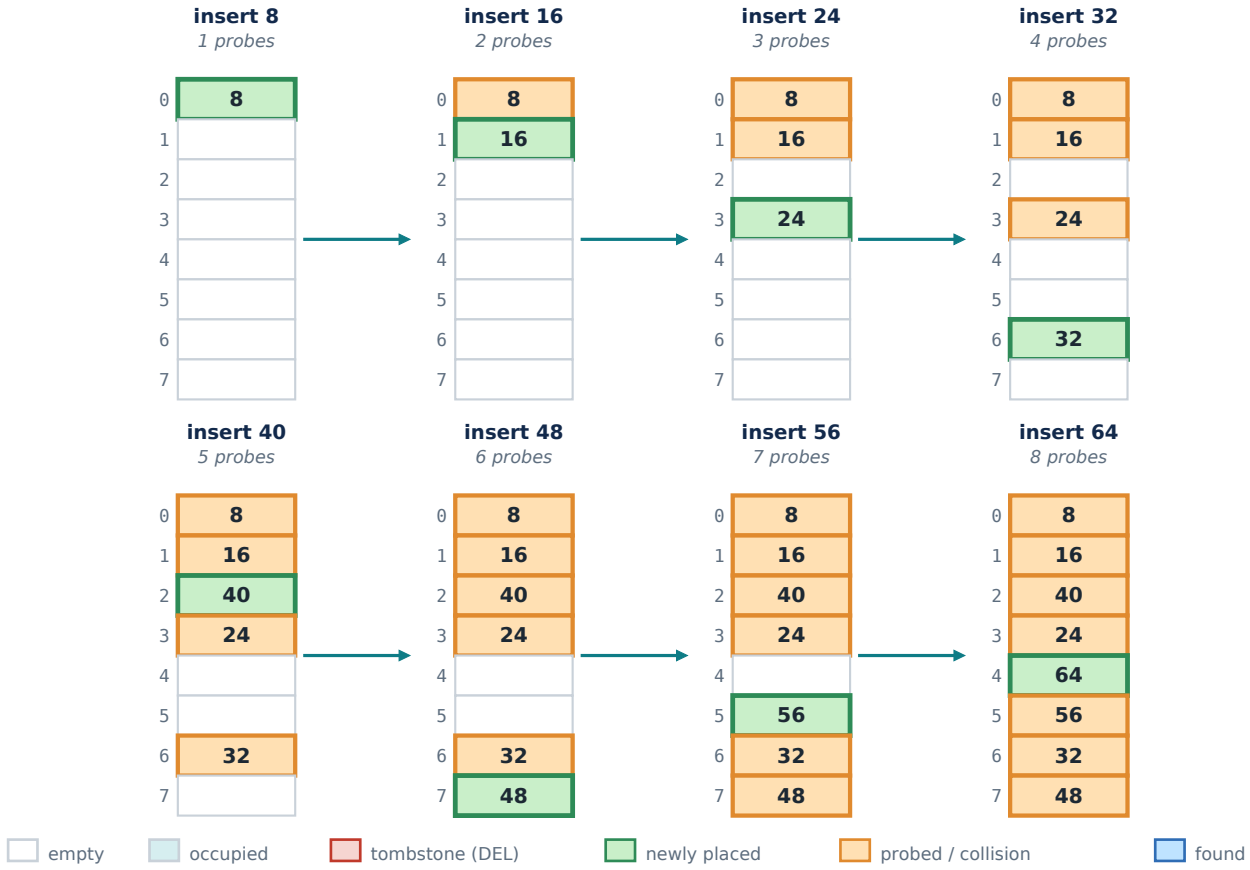


Figure 15.4 — Program 4, part A: eight keys with the same home slot use slots 0, 1, 3, 6, 2, 7, 5, 4 in this order.

Figure 15.5 shows part B of Program 4: the rehash that follows a failed insertion.

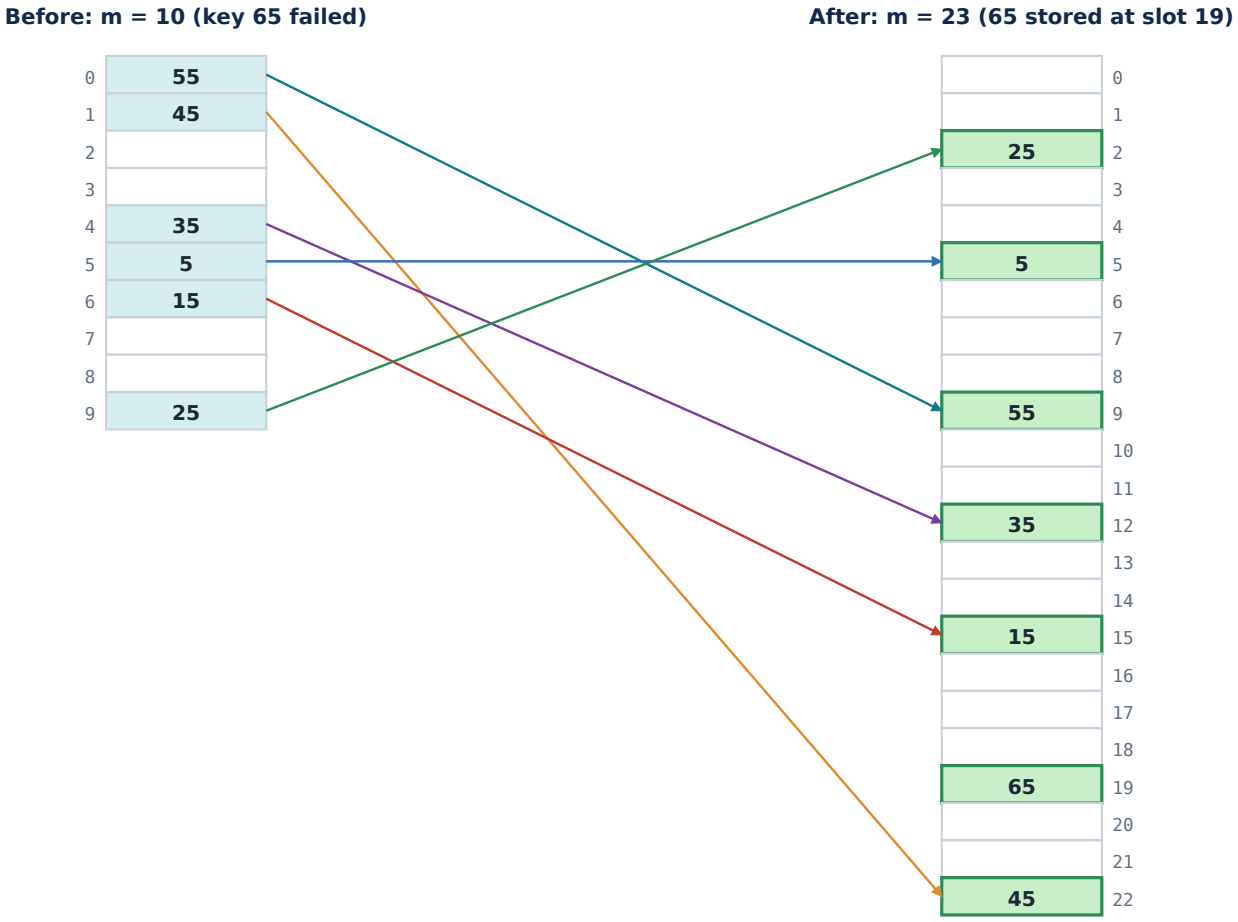


Figure 15.5 — Program 4, part B: the failed insertion of 65 triggers a rehash from 10 to 23 slots. Every key gets a new index; 65 now succeeds at its first probe.

Step	Event
1	Keys 5, 15, 25, 35, 45, 55 fill slots 5, 6, 9, 4, 1, 0 (offsets 0, 1, 4, 9, 6, 5).
2	Key 65: probes 5, 6, 9, 4, 1, 0, 1, 4, 9, 6 → **FAILED**, 4 slots (2, 3, 7, 8) still empty.
3	`insertOrGrow`: new capacity nextPrime(20) = 23; the 6 old keys are re-inserted in old slot order; then 65 is stored at slot 19.

Complexity analysis

Operation	Expected (suitable m, $\alpha < 0.5$)	Worst case	Space
insert / search	$O(1)$	$O(m)$ probes, and may fail	$O(m)$
insertOrGrow (failure path)	$O(m_{\text{new}} + n)$ for the rehash		$O(m_{\text{new}})$

Common mistakes and edge cases

- Believing failure means "full": the output shows 4 empty slots after the failure.
- Using pure squares with a composite m (coverage 6 of 10) or triangular offsets with a non-power-of-two m.
- Infinite loops: the loop is bounded by m.
- Forgetting that growing changes **every** index: keys must be re-inserted, not copied.
- Full table with triangular m=8: insertion of 72 fails after 8 probes with 0 empty slots, which is a genuine full table.

15.5 Program 5: Hash Table with Double Hashing

Objective	Open addressing with $h(k,i) = (h_1(k) + i \cdot h_2(k))$ mod m using a **correct** second hash $h_2(k) = 1 + (k \bmod (m\text{-}1))$ and a prime table size; includes search, tombstone deletion, duplicates and full detection.
Concepts demonstrated	primary/secondary hash, step size in $1 \ldots m\text{-}1$, coprimality with prime m, tombstones, duplicate rejection, full-table detection, self-check of h_2

Algorithm

Pseudocode

```
h1(k) = k mod m                      (m is PRIME)
h2(k) = 1 + (k mod (m-1))            (always 1 .. m-1, never 0)
insert(key):
    for i = 0 .. m-1:
        j = (h1 + i * h2) mod m
        EMPTY: store at first remembered tombstone, else j; done
        DELETED: remember the first one;  same key: duplicate
    tombstone remembered -> reuse it;  otherwise FULL
```

Source code

```cpp
// Program 5: double hashing with a correctly chosen second hash function
#include <cassert>
#include <iostream>
#include <vector>

class DoubleHashTable {
public:
    enum class State { Empty, Occupied, Deleted };

    explicit DoubleHashTable(int m) : m_(m), slots_(m) {}       // m must be prime

    int h1(int key) const { return ((key % m_) + m_) % m_; }
    // 1 + (k mod (m-1)) is in 1..m-1: never 0 and coprime to a prime m
    int h2(int key) const { return 1 + (((key % (m_ - 1)) + (m_ - 1)) % (m_ - 1)); }
    int slotFor(int key, int i) const {
        return static_cast<int>((h1(key) + static_cast<long long>(i) * h2(key)) % m_);
    }

    // 1 inserted, 0 duplicate, -1 full
    int insert(int key) {
        int firstDeleted = -1;
        std::cout << "  insert " << key << " (h1=" << h1(key) << ", h2=" << h2(key) << "): probes";
        for (int i = 0; i < m_; ++i) {
            int j = slotFor(key, i);
            std::cout << " " << j;
            Slot& s = slots_[j];
            if (s.state == State::Empty) {
                int where = firstDeleted >= 0 ? firstDeleted : j;
                slots_[where].key = key;
                slots_[where].state = State::Occupied;
                std::cout << " -> stored at " << where << "\n";
                return 1;
            }
            if (s.state == State::Deleted) { if (firstDeleted < 0) firstDeleted = j; }
            else if (s.key == key) { std::cout << " -> duplicate\n"; return 0; }
        }
        if (firstDeleted >= 0) {
            slots_[firstDeleted].key = key;
            slots_[firstDeleted].state = State::Occupied;
            std::cout << " -> stored at tombstone " << firstDeleted << "\n";
            return 1;
        }
        std::cout << " -> FULL (all " << m_ << " slots examined)\n";
        return -1;
    }

    int search(int key, int& probes) const {
        probes = 0;
        for (int i = 0; i < m_; ++i) {
            int j = slotFor(key, i);
            ++probes;
            const Slot& s = slots_[j];
            if (s.state == State::Empty) return -1;
```

```
            if (s.state == State::Occupied && s.key == key) return j;
        }
        return -1;
    }

    bool erase(int key) {
        int p = 0;
        int j = search(key, p);
        if (j < 0) return false;
        slots_[j].state = State::Deleted;
        return true;
    }

    void display() const {
        std::cout << "  table:";
        for (const Slot& s : slots_) {
            if (s.state == State::Empty) std::cout << " .";
            else if (s.state == State::Deleted) std::cout << " DEL";
            else std::cout << " " << s.key;
        }
        std::cout << "\n";
    }

private:
    struct Slot { int key = 0; State state = State::Empty; };
    int m_;
    std::vector<Slot> slots_;
};

int main() {
    DoubleHashTable t(13);
    bool stepOk = true;
    for (int k = -200; k <= 200; ++k) stepOk = stepOk && t.h2(k) >= 1 && t.h2(k) <= 12;
    std::cout << "self-check: h2(k) is always in 1..12 for k = -200..200: " << stepOk << "\n";

    std::cout << "keys 27, 40, 53, 66 all have h1 = 1 but different steps:\n";
    for (int k : {27, 40, 53, 66}) t.insert(k);
    t.display();
    int p = 0;
    int j = t.search(66, p);
    std::cout << "search 66: slot " << j << " (" << p << " probes)\n";
    j = t.search(79, p);
    std::cout << "search 79 (absent): " << (j < 0 ? "absent" : "found") << " (" << p << " probes)\n";
    std::cout << "erase 40: " << t.erase(40) << ", erase 1000 (absent): " << t.erase(1000) << "\n";
    j = t.search(53, p);
    std::cout << "search 53 after erase: slot " << j << " (" << p << " probes)\n";
    t.insert(14);                                   // home 1 holds 27, then probes on
    t.insert(27);                                   // duplicate
    t.display();

    std::cout << "full-table detection with m = 5 (prime):\n";
    DoubleHashTable small(5);
    for (int k : {1, 6, 11, 16, 21}) small.insert(k);   // all home slot 1
    small.display();
    small.insert(26);
    return 0;
}
```

Line-by-line explanation of the important logic

Line	Code	What it does
12	`int h1(int key) const { return ((key % m_) +…`	Home slot; safe modulo.
14	`int h2(int key) const { return 1 + (((key % …`	Step size from 1 to m-1: the `1 +` shift removes the possibility of step 0.
15	`int slotFor(int key, int i) const {`	The probe formula, with 64-bit multiplication.
22	`std::cout << " insert " << key << " (h1=" <…`	Prints h_1, h_2 and every probe for tracing.
34	`if (s.state == State::Deleted) { if (firstDe…`	Remember the first tombstone.
43	`std::cout << " -> FULL (all " << m_ << " slo…`	With prime m the sequence visits all slots, so after m probes FULL is exact.
86	`for (int k = -200; k <= 200; ++k) stepOk = s…`	Self-check: h_2 stays in 1..12 even for negative keys.
105	`DoubleHashTable small(5);`	Tiny prime table used to demonstrate full detection.

Sample input (hard-coded operations)

Part	Operations
1	self-check of h_2 for k = -200 ... 200; m = 13
2	insert 27, 40, 53, 66 (all h_1 = 1); search 66; search 79 (absent); erase 40; erase 1000 (absent); search 53; insert 14; insert 27 (duplicate)
3	m = 5: insert 1, 6, 11, 16, 21 (all home 1), then insert 26 into the full table

Exact output

```
Output of Program 5
self-check: h2(k) is always in 1..12 for k = -200..200: 1
keys 27, 40, 53, 66 all have h1 = 1 but different steps:
  insert 27 (h1=1, h2=4): probes 1 -> stored at 1
  insert 40 (h1=1, h2=5): probes 1 6 -> stored at 6
  insert 53 (h1=1, h2=6): probes 1 7 -> stored at 7
  insert 66 (h1=1, h2=7): probes 1 8 -> stored at 8
  table: . 27 . . . . 40 53 66 . . . .
search 66: slot 8 (2 probes)
search 79 (absent): absent (2 probes)
erase 40: 1, erase 1000 (absent): 0
search 53 after erase: slot 7 (2 probes)
  insert 14 (h1=1, h2=3): probes 1 4 -> stored at 4
  insert 27 (h1=1, h2=4): probes 1 -> duplicate
  table: . 27 . . 14 . DEL 53 66 . . . .
full-table detection with m = 5 (prime):
  insert 1 (h1=1, h2=2): probes 1 -> stored at 1
  insert 6 (h1=1, h2=3): probes 1 4 -> stored at 4
  insert 11 (h1=1, h2=4): probes 1 0 -> stored at 0
  insert 16 (h1=1, h2=1): probes 1 2 -> stored at 2
  insert 21 (h1=1, h2=2): probes 1 3 -> stored at 3
  table: 11 1 16 21 6
  insert 26 (h1=1, h2=3): probes 1 4 2 0 3 -> FULL (all 5 slots examined)
```

Dry run: table changes

Key	h_1	$h_2 = 1 + k$ mod 12	Probe sequence	Stored at
27	1	1 + 3 = 4	1	1
40	1	1 + 4 = 5	1 → 6	6
53	1	1 + 5 = 6	1 → 7	7
66	1	1 + 6 = 7	1 → 8	8

Figure 15.6 shows the probe paths of Program 5.

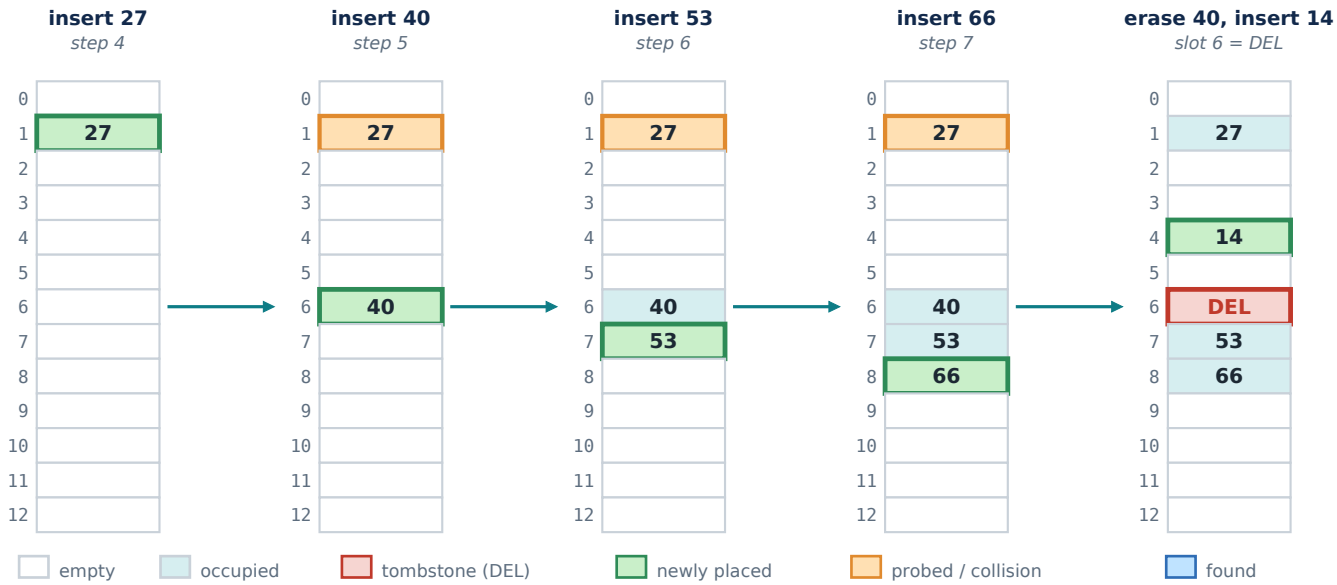


Figure 15.6 — Program 5, $m = 13$: four keys with the same home slot 1 follow four different probe paths (steps 4, 5, 6, 7), then slot 6 becomes a tombstone and key 14 is inserted.

Key (home 1, $m = 5$)	h_2	Probes	Stored at
1	2	see output	1
6	3	see output	4
11	4	see output	0
16	1	see output	2
21	2	see output	3
26	3	1 → 4 → 2 → 0 → 3	**FULL**: all 5 slots examined

Complexity analysis

Operation	Expected	Worst case	Space
insert, search, erase	$O(1)$; about $1/(1\text{-}\alpha)$ probes for an absent key	$O(m)$ probes	$O(m)$

Common mistakes and edge cases

- $h_2(k) = k \bmod m$ (can be 0) or a composite m: coverage fails (Part 9). The self-check proves $h_2 \in [1, 12]$ here.
- Not bounding the loop.
- Treating tombstones as EMPTY, or as matches. Forgetting that with prime m a failure really means no free slot.
- Negative keys: both h_1 and h_2 use the safe modulo.

15.6 Program 6: Dynamic Hash Table with Automatic Resizing and Rehashing

Objective	A linear-probing table that **grows** when crowded, **shrinks** when nearly empty, and **cleans up tombstones** by rehashing at the same capacity, with prime capacities and hysteresis between the grow and shrink thresholds.
Concepts demonstrated	load factor, resize policy, rehash versus copy, tombstone cleanup, shrinking with hysteresis, amortised cost, preserving all keys across rehashes

Algorithm

Pseudocode

```
insert(key):
    if key present: return false
    if usedSlots + 1 > 0.5 * m:                    # crowded (live keys + tombstones)
        if liveKeys + 1 > 0.4 * m: rehash(nextPrime(2m))   # really too many keys: grow
        else:                      rehash(m)               # mostly tombstones: cleanup
    place key (linear probing; reuse EMPTY or DELETED slot)
erase(key):
    mark DELETED; if m > 11 and liveKeys < 0.125 * m: rehash(nextPrime(m / 2))
rehash(newM): new array of newM EMPTY slots; for every OCCUPIED old slot: place(key) with the NEW m
```

Source code

```cpp
// Program 6: dynamic hash table, linear probing, automatic grow / shrink / tombstone cleanup
#include <iostream>
#include <vector>

bool isPrime(int n) {
    if (n < 2) return false;
    for (int d = 2; d * d <= n; ++d) if (n % d == 0) return false;
    return true;
}
int nextPrime(int n) { while (!isPrime(n)) ++n; return n; }

class DynamicTable {
public:
    DynamicTable() : m_(5), t_(5) {}

    bool insert(int key) {
        if (contains(key)) return false;
        if (used_ + 1 > 0.5 * m_) {                    // too crowded (live keys + tombstones)
            if (n_ + 1 > 0.4 * m_) rehash(nextPrime(2 * m_), "grow");   // many live keys
            else rehash(m_, "cleanup (tombstones)");                    // mostly tombstones
        }
        place(key);
        return true;
    }

    bool contains(int key) const {
        for (int i = 0; i < m_; ++i) {
            const Slot& s = t_[(home(key) + i) % m_];
            if (s.state == State::Empty) return false;
            if (s.state == State::Occupied && s.key == key) return true;
        }
        return false;
    }

    bool erase(int key) {
        for (int i = 0; i < m_; ++i) {
            Slot& s = t_[(home(key) + i) % m_];
            if (s.state == State::Empty) return false;
            if (s.state == State::Occupied && s.key == key) {
                s.state = State::Deleted;
                --n_;
                if (m_ > 11 && n_ < 0.125 * m_) rehash(nextPrime(m_ / 2), "shrink");
                return true;
            }
        }
        return false;
    }

    int capacity() const { return m_; }
    int size() const { return n_; }
    int rehashes() const { return rehashes_; }
    double load() const { return static_cast<double>(n_) / m_; }

private:
    enum class State { Empty, Occupied, Deleted };
    struct Slot { int key = 0; State state = State::Empty; };

    int home(int key) const { return ((key % m_) + m_) % m_; }

    void place(int key) {
        int j = home(key);
        while (t_[j].state == State::Occupied) j = (j + 1) % m_;
```

```
63          if (t_[j].state == State::Empty) ++used_;
64          t_[j].key = key;
65          t_[j].state = State::Occupied;
66          ++n_;
67      }
68
69      void rehash(int newCap, const char* why) {
70          std::cout << "  rehash #" << ++rehashes_ << " (" << why << "): " << m_ << " -> " << newCap
71                    << ", live keys " << n_ << ", slots in use before " << used_ << "\n";
72          std::vector<Slot> old = std::move(t_);
73          t_.assign(newCap, Slot());
74          m_ = newCap;
75          n_ = 0;
76          used_ = 0;
77          for (const Slot& s : old)
78              if (s.state == State::Occupied) place(s.key);   // recompute with the NEW size
79      }
80
81      int m_;
82      int n_ = 0;
83      int used_ = 0;
84      int rehashes_ = 0;
85      std::vector<Slot> t_;
86  };
87
88  int main() {
89      DynamicTable t;
90      std::cout << "inserting 1..20:\n";
91      for (int k = 1; k <= 20; ++k) t.insert(k);
92      std::cout << "capacity " << t.capacity() << ", n = " << t.size()
93                << ", load = " << t.load() << "\n";
94      bool all = true;
95      for (int k = 1; k <= 20; ++k) all = all && t.contains(k);
96      std::cout << "all 20 keys still found after the rehashes: " << all << "\n";
97
98      std::cout << "duplicate insert of 7: " << t.insert(7) << ", absent key 99 found? "
99                << t.contains(99) << "\n";
100
101     std::cout << "erasing 1..18:\n";
102     for (int k = 1; k <= 18; ++k) t.erase(k);
103     std::cout << "capacity " << t.capacity() << ", n = " << t.size() << ", 19 and 20 found: "
104               << (t.contains(19) && t.contains(20)) << "\n";
105
106     std::cout << "churn (insert then erase 100..111) to create tombstones:\n";
107     for (int k = 100; k < 112; ++k) { t.insert(k); t.erase(k); }
108     std::cout << "capacity " << t.capacity() << ", n = " << t.size() << ", rehashes so far = "
109               << t.rehashes() << "\n";
110     return 0;
111 }
```

Line-by-line explanation of the important logic

Line	Code	What it does
14	`DynamicTable() : m_(5), t_(5) {}`	Starts with the small prime capacity 5.
18	`if (used_ + 1 > 0.5 * m_) { …`	`used_` counts live keys **and** tombstones: the effective load that slows probing.
19	`if (n_ + 1 > 0.4 * m_) rehash(nextPrime(2 * …`	Many live keys: double (to a prime).
20	`else rehash(m_, "cleanup (tombstones)"); …`	Few live keys but many tombstones: rebuild at the same size to discard them.
42	`if (m_ > 11 && n_ < 0.125 * m_) rehash(nextP…`	Shrink only when very sparse. Because grow tests 0.4 and shrink tests 0.125 there is no thrashing.
63	`if (t_[j].state == State::Empty) ++used_;`	Reusing a DELETED slot does not increase `used_`.
72	`std::vector<Slot> old = std::move(t_);`	Takes the old array; `t_` becomes a fresh empty vector.
73	`t_.assign(newCap, Slot());`	New array of the new capacity, all EMPTY.
78	`if (s.state == State::Occupied) place(s.key)…`	Re-inserts live keys using the new capacity (tombstones disappear).

Sample input (hard-coded operations)

Step	Operation
1	insert 1, 2, …, 20 into the initial table of capacity 5
2	check that all 20 keys are found; insert 7 again (duplicate); search the absent key 99
3	erase 1 … 18 (the table becomes sparse)
4	churn: insert and immediately erase 100, 101, …, 111 to create tombstones

Exact output

Output of Program 6

```
inserting 1..20:
  rehash #1 (grow): 5 -> 11, live keys 2, slots in use before 2
  rehash #2 (grow): 11 -> 23, live keys 5, slots in use before 5
  rehash #3 (grow): 23 -> 47, live keys 11, slots in use before 11
capacity 47, n = 20, load = 0.425532
all 20 keys still found after the rehashes: 1
duplicate insert of 7: 0, absent key 99 found? 0
erasing 1..18:
  rehash #4 (shrink): 47 -> 23, live keys 5, slots in use before 20
  rehash #5 (shrink): 23 -> 11, live keys 2, slots in use before 5
capacity 11, n = 2, 19 and 20 found: 1
churn (insert then erase 100..111) to create tombstones:
  rehash #6 (cleanup (tombstones)): 11 -> 11, live keys 2, slots in use before 5
  rehash #7 (cleanup (tombstones)): 11 -> 11, live keys 2, slots in use before 5
  rehash #8 (cleanup (tombstones)): 11 -> 11, live keys 2, slots in use before 5
capacity 11, n = 2, rehashes so far = 8
```

Dry run: the sequence of rehashes

#	Reason	Capacity	Live keys	Slots in use before
1	grow	5 → 11	2	2
2	grow	11 → 23	5	5
3	grow	23 → 47	11	11
4	shrink	47 → 23	5	20
5	shrink	23 → 11	2	5
6	cleanup (tombstones)	11 → 11	2	5
7	cleanup (tombstones)	11 → 11	2	5
8	cleanup (tombstones)	11 → 11	2	5

How to read it. Inserting 1–20 doubles the table through the primes 5 → 11 → 23 → 47 (the first three rehashes). Erasing 1–18 shrinks it 47 → 23 → 11. In the churn phase every pair insert+erase leaves a tombstone; once five of eleven slots are in use while only two keys are live, the table is rebuilt **at the same capacity** (cleanup) instead of growing. Without the 0.4/0.125 gap between grow and shrink tests the churn phase would repeatedly grow and shrink (an earlier draft of this program did exactly that, 23 rehashes in the same loop).

Complexity analysis

Operation	Cost	Note
insert / search / erase	$O(1)$ expected	load kept between about 0.125 and 0.5
one rehash	$O(m_{\text{old}} + m_{\text{new}})$	scan old array, build new array
n insertions in a row	$O(n)$ total, $O(1)$ amortised	capacity at least doubles at each growth
Space	$O(m)$	during rehash old and new arrays coexist (about $3\,m_{\text{old}}$ slots)

Common mistakes and edge cases

- Copying the array without re-hashing: keys become unreachable (Part 10).
- Counting only live keys for the growth test and forgetting tombstones, so probing slows down.
- Equal grow and shrink thresholds (thrashing). Growing by a constant amount (quadratic total cost).
- Non-prime capacities with the division method.
- Duplicate check **before** growing so duplicate inserts never trigger a rehash (see `insert(7)` = 0).

15.7 Program 7: String Keys: Polynomial Hashing and Word Frequency

Objective	Hash strings with a polynomial rolling hash (Horner's rule, base 31, unsigned 64-bit arithmetic) and count word frequencies in a separate-chaining table.
Concepts demonstrated	string hashing, order sensitivity, unsigned overflow, empty string key, collisions between different words, absent words, hash value versus index

Algorithm

Pseudocode

```
polyHash(s): h = 0; for each character c (as unsigned): h = h * 31 + c   (mod 2^64, automatic)
add(word):   chain = bucket[polyHash(word) mod m]
             if word in chain: count = count + 1  else append (word, 1)
count(word): walk the chain; return the count or 0 if absent
```

Source code

```cpp
// Program 7: string keys, polynomial rolling hash, separate chaining, word frequency
#include <cstdint>
#include <iostream>
#include <list>
#include <string>
#include <utility>
#include <vector>

// Horner's rule over unsigned char with unsigned 64-bit arithmetic (wraps, never UB)
std::uint64_t polyHash(const std::string& s, std::uint64_t base = 31) {
    std::uint64_t h = 0;
    for (unsigned char c : s) h = h * base + c;
    return h;
}

class WordCounter {
public:
    explicit WordCounter(std::size_t m) : buckets_(m) {}

    std::size_t indexFor(const std::string& w) const { return polyHash(w) % buckets_.size(); }

    int add(const std::string& w) {                       // returns the new count
        auto& chain = buckets_[indexFor(w)];
        for (auto& e : chain)
            if (e.first == w) return ++e.second;
        chain.emplace_back(w, 1);                         // new word at the end of its chain
        ++distinct_;
        return 1;
    }
    int count(const std::string& w) const {
        for (const auto& e : buckets_[indexFor(w)])
            if (e.first == w) return e.second;
        return 0;                                         // absent word
    }
    void display() const {
        for (std::size_t i = 0; i < buckets_.size(); ++i) {
            std::cout << "  [" << i << "]";
            for (const auto& e : buckets_[i]) std::cout << " " << e.first << "(" << e.second << ")";
            std::cout << "\n";
        }
    }
    std::size_t distinct() const { return distinct_; }
    std::size_t collisions() const {                      // words sharing a bucket with an earlier word
        std::size_t c = 0;
        for (const auto& ch : buckets_) if (ch.size() > 1) c += ch.size() - 1;
```

```cpp
        return c;
    }

private:
    std::vector<std::list<std::pair<std::string, int>>> buckets_;
    std::size_t distinct_ = 0;
};

int main() {
    const std::vector<std::string> text = {"apple", "banana", "cherry", "apple", "date", "fig",
                                           "banana", "apple", "grape", "kiwi", "lemon", ""};
    WordCounter wc(11);
    std::cout << "word -> polynomial hash (B = 31) -> index (m = 11)\n";
    for (const std::string& w : text) {
        std::cout << "  \"" << w << "\" -> " << polyHash(w) << " -> " << wc.indexFor(w) << "\n";
        wc.add(w);
    }
    std::cout << "distinct words: " << wc.distinct() << ", collisions: " << wc.collisions() << "\n";
    wc.display();
    std::cout << "count(apple) = " << wc.count("apple") << ", count(banana) = " << wc.count("banana")
              << ", count(mango) = " << wc.count("mango")
              << ", count(\"\") = " << wc.count("") << "\n";
    std::cout << "anagrams hash differently: \"listen\" -> " << polyHash("listen")
              << ", \"silent\" -> " << polyHash("silent") << "\n";
    return 0;
}
```

Line-by-line explanation of the important logic

Line	Code	What it does
10	`std::uint64_t polyHash(const std::string& s,…`	Polynomial hash returning a 64-bit **hash value** (not yet an index).
12	`for (unsigned char c : s) h = h * base + c;`	Horner step. `unsigned char` avoids negative character codes; unsigned wrap-around is well defined.
20	`std::size_t indexFor(const std::string& w) c…`	Compression: hash value → index in $0 \ldots m\text{-}1$.
25	`if (e.first == w) return ++e.second;`	Known word: increment its count.
26	`chain.emplace_back(w, 1); …`	New word appended to its chain.
45	`for (const auto& ch : buckets_) if (ch.size(…`	Counts words that share a bucket with an earlier word (collisions).
33	`return 0; …`	Absent word has count 0.

Sample input (hard-coded operations)

Step	Operation
1	add the 12 tokens apple, banana, cherry, apple, date, fig, banana, apple, grape, kiwi, lemon and the **empty string**, with $m = 11$
2	count apple, banana, mango (absent), "" (empty)
3	hash the anagrams "listen" and "silent"

Exact output

Output of Program 7

```
word -> polynomial hash (B = 31) -> index (m = 11)
  "apple" -> 93029210 -> 10
  "banana" -> 2898612069 -> 1
  "cherry" -> 2933454233 -> 6
  "apple" -> 93029210 -> 10
  "date" -> 3076014 -> 7
  "fig" -> 101380 -> 4
  "banana" -> 2898612069 -> 1
  "apple" -> 93029210 -> 10
  "grape" -> 98615627 -> 0
  "kiwi" -> 3292336 -> 3
  "lemon" -> 102857459 -> 1
  "" -> 0 -> 0
distinct words: 9, collisions: 2
```

```
   [0] grape(1) (1)
   [1] banana(2) lemon(1)
   [2]
   [3] kiwi(1)
   [4] fig(1)
   [5]
   [6] cherry(1)
   [7] date(1)
   [8]
   [9]
   [10] apple(3)
count(apple) = 3, count(banana) = 2, count(mango) = 0, count("") = 1
anagrams hash differently: "listen" -> 3192458695, "silent" -> 3392640085
```

Dry run: hash values and indexes

Word	Hash value (B = 31)	Index = value mod 11
"apple"	93029210	10
"banana"	2898612069	1
"cherry"	2933454233	6
"date"	3076014	7
"fig"	101380	4
"grape"	98615627	0
"kiwi"	3292336	3
"lemon"	102857459	1
""	0	0

"fig" step by step: $f = 102$, $i = 105$, $g = 103$: $h = 0 \times 31 + 102 = 102$; $102 \times 31 + 105 = 3267$; $3267 \times 31 + 103 = 101380$, and $101380 \bmod 11 = 4$. Words "banana", "lemon" collide in bucket 1 and "grape" and "" in bucket 0: two collisions, resolved by chaining. The 64-bit value of "banana" (2898612069) exceeds 32 bits, which is why `std::uint64_t` is used.

Complexity analysis

Operation	Time	Space
hash a word of length L	$O(L)$	$O(1)$
add / count	$O(L \times (1 + \alpha))$ expected: hash plus compares along the chain	$O(m$ + total characters stored)

Common mistakes and edge cases

- `char` instead of `unsigned char` (negative codes for non-ASCII bytes); signed overflow (undefined behaviour).
- Summing characters (anagrams collide); the polynomial hash separates "listen" and "silent".
- Using the 64-bit hash value directly as an index. It must be reduced mod m.
- The **empty string** is a valid key (hash 0, bucket 0) and must be handled like any other.
- Collisions between different words are normal; equality on the full string, not the hash, decides whether a word is already stored.

PART 16

IMPORTANT THEORY QUESTIONS AND VIVA

Questions with concise, accurate answers, organised from basic definitions to traps

Answers are written to be **spoken**: one to four sentences, with the key term first. Questions marked **[N]** are numerical, **[C]** comparison, **[T]** common viva traps.

16.1 Basic Definitions

#	Question	Answer
1	What is hashing?	Hashing converts a key into an array index with a hash function so that items can be stored and found directly instead of searched for.
2	What is a hash table?	An array of m slots/buckets plus a hash function and a collision-handling method; it stores key–value pairs and supports insert, search and delete in expected $O(1)$ time.
3	Why is hashing better than linear or binary search for lookups?	Linear search is $O(n)$; binary search is $O(\log n)$ and needs sorted data, so insertions cost $O(n)$. Hashing needs no ordering and takes expected $O(1)$ for all three operations.
4	Define key, value, slot, bucket and capacity.	Key: identifier used to find an item. Value: data stored with the key. Slot: one array cell. Bucket: position that can hold one (open addressing) or several (chaining) entries. Capacity m: number of slots/buckets.
5	[T] What is the difference between a hash value and a table index?	The hash value is the (possibly huge) number the hash function returns; the index is that value reduced into $0 \dots m$-1, usually by `% m`. The value does not depend on m; the index does.
6	What is direct addressing and why is hashing needed?	Direct addressing uses the key itself as the index, which needs one slot per possible key. For large key universes (phone numbers, strings) that is impossible, so hashing compresses keys into a small table.

16.2 Hash Functions

#	Question	Answer
7	State the properties of a good hash function.	Deterministic, uniform spread over slots, fast to compute, uses the whole key, and consistent with equality (equal keys give equal hashes).
8	Explain the division method and a good choice of m.	$h(k) = k \bmod m$. Choose m prime and not near a power of two or ten; powers of two use only the low bits and powers of ten only the last digits.
9	[N] Using the division method with $m = 11$, find the index of 1234.	$1234 = 11 \times 112 + 2$, so the index is 2.
10	How does the multiplication method work?	$h(k) = \lfloor m\,(kA \bmod 1) \rfloor$ with $0 < A < 1$ (for example $A \approx 0.618034$): take the fractional part of kA and scale it by m. m need not be prime.
11	Explain the mid-square and folding methods.	Mid-square: square the key and take the middle digits. Folding: split the key into pieces and add them (shift folding) or add them with alternate pieces reversed (boundary folding), then reduce mod m.
12	[N] Shift-fold 123456789 into 3-digit pieces with $m = 1000$.	$123 + 456 + 789 = 1368$; $1368 \bmod 1000 = 368$.
13	How is a string hashed with a polynomial hash?	Treat the characters as digits in base B: $h = ((c_0 B + c_1) B + c_2) \dots$ (Horner's rule), usually modulo a large prime or 2^{64}. Order matters, so "ab" and "ba" differ.
14	[T] Why is "sum of characters" a poor string hash?	It ignores order (anagrams collide) and short strings give small values that cluster. Example: "listen" and "silent" both give 655.
15	[T] Why is `-7 % 5` dangerous in C++ and how do you fix it?	It equals −2 because the remainder takes the sign of the dividend, giving a negative index. Use `((k % m) + m) % m`.

16.3 Hash-table Representation

#	Question	Answer
16	Name the three states of a slot in open addressing.	EMPTY (never used), OCCUPIED (live key), DELETED (tombstone: a key was removed).
17	What does the underlying array look like in separate chaining?	An array of m head pointers (or containers); each points to the first node of the list of keys that hash to that bucket, or is null/empty.
18	Difference between static and dynamic hash tables?	A static table has fixed capacity; a dynamic one is resized (rehashed) when the load factor passes a threshold, keeping operations fast.

16.4 Collision Fundamentals

#	Question	Answer
19	What is a collision? Can it be avoided?	Two different keys with the same index. In general no: by the pigeonhole principle a large key universe maps into fewer slots. Only a perfect hash function on a **fixed** key set avoids them.
20	Explain the birthday-paradox intuition for hashing.	Collisions appear after only about $\sqrt{m}$ insertions: in a table of 365 slots 23 keys already give a 50% chance of a collision. So collision handling is always needed.
21	[T] Collision versus duplicate key?	Collision: different keys, same index (normal, handled by the table). Duplicate: the same key inserted again (a policy decision: reject, update or store multiple values).
22	Collision resolution versus collision avoidance?	Avoidance reduces collisions (good hash, larger table, low load). Resolution handles the ones that still occur (chaining, probing).
23	What are primary and secondary clustering?	Primary: runs of occupied slots grow because keys from different home slots join them (linear probing). Secondary: keys with the same home slot follow the same probe sequence (quadratic probing and linear).

16.5 Separate Chaining

#	Question	Answer
24	How does separate chaining work?	Each bucket holds a list of all keys with that index. Insert adds a node (after a duplicate check), search walks the chain, delete unlinks a node.
25	[N] $m = 7$, keys 15, 22, 8, 29 (insert at head). What does bucket 1 contain?	All four are $\equiv 1 \bmod 7$. Head insertion gives the chain 29 → 8 → 22 → 15.
26	Why is deletion easy in chaining?	Unlink the node (handling head versus non-head) and free it; no tombstones are needed.
27	What happens when $\alpha > 1$ in chaining?	It still works: average chain length is α. Expected search cost is about $1 + \alpha/2$ (successful) and α (unsuccessful) nodes, so resizing keeps α near 1.

16.6 Linear Probing

#	Question	Answer
28	Write the linear probing formula.	$h(k, i) = (h(k) + i) \bmod m$ for $i = 0, 1, \ldots, m\text{-}1$.
29	[N] $m = 10$, $h(k) = k \bmod 10$. Insert 25, 35, 45 and give the slots.	25 → 5. 35 home 5 is taken → 6. 45 home 5, 6 taken → 7.
30	[T] Why can't we simply empty a slot when deleting in linear probing?	An EMPTY slot inside a run ends searches early, making later keys unreachable. A tombstone keeps the probe path intact.
31	Why is linear probing cache-friendly but cluster-prone?	Probes touch adjacent memory, which is fast; but a key landing in a run joins its end, so runs grow (primary clustering) and expected probes rise as $1/(1\text{-}\alpha)^2$.

16.7 Quadratic Probing

#	Question	Answer
32	Write a quadratic probing formula and its motivation.	$h(k,i) = (h(k) + c_1\, i + c_2\, i^2)$ mod m. The growing step breaks up the runs of linear probing (less primary clustering).
33	[T] Does quadratic probing always find an empty slot if one exists?	No. It depends on m and the constants. For prime m with $h + i^2$ the first $(m+1)/2$ probes are distinct, so insertion is guaranteed when the table is less than half full; for $m = 2^p$ with triangular offsets every slot is visited. Otherwise it can fail with empty slots left.
34	[N] $m = 10$, offsets i^2, home 5: which slots are visited?	Offsets mod 10 are 0, 1, 4, 9, 6, 5, ..., so slots 5, 6, 9, 4, 1, 0: six of ten.
35	Does quadratic probing remove all clustering?	No. It removes primary clustering but keeps secondary clustering: keys with the same home slot follow identical probe sequences.

16.8 Double Hashing

#	Question	Answer
36	State the double hashing formula.	$h(k, i) = (h_1(k) + i \cdot h_2(k))$ mod m.
37	[T] Why must $h_2(k)$ never be 0?	With step 0 the probe sequence repeats the home slot and can never find a free slot.
38	Why should $h_2(k)$ be coprime to m?	The sequence visits m / gcd(h_2, m) distinct slots; coprimality (for example prime m and $1 \le h_2 < m$) makes it a permutation of all slots.
39	Give a standard pair h_1, h_2.	m prime, $h_1(k) = k$ mod m, $h_2(k) = 1 + (k$ mod $(m\text{-}1))$ (range 1 to m-1).
40	[N] $m = 11$, $h_2 = 1 + k$ mod 10. Probe sequence for 36 when slot 3 is full?	$h_1 = 3$, $h_2 = 7$; probes 3, 10, 6, 2, ... (add 7 each time, mod 11).

16.9 Load Factor and Rehashing

#	Question	Answer
41	Define the load factor.	$\alpha = n/m$, keys stored divided by slots. For open addressing $\alpha \le 1$; for chaining it is the average chain length.
42	Why do we rehash instead of copying the array?	The index is k mod m, so a new m changes every key's index. Copying leaves keys where the new function will not look. Rehashing recomputes every index.
43	[N] A table of 5 slots with 4 keys is resized to 11. What is the load factor before and after?	Before: $4/5 = 0.8$. After: $4/11 \approx 0.36$.
44	Why does doubling give amortized $O(1)$ insertion?	The rehash work at sizes c, $2c$, $4c$, ... is a geometric series bounded by about $2n$ over n insertions, so the average cost per insertion is constant (one insertion can still cost $O(n)$).
45	What is the difference between resolving a collision and rehashing?	Collision resolution deals with one key whose slot is taken, without changing m. Rehashing rebuilds the whole table with a new size because the load is too high.

16.10 Complexity Analysis

#	Question	Answer
46	Why is hash-table search average $O(1)$ but worst case $O(n)$?	On average keys spread out so chains/probe paths are short; in the worst case all keys collide and the table degenerates to a list. The average assumes uniform hashing and bounded load.
47	[N] Linear probing at $\alpha = 0.5$: expected probes for an unsuccessful search?	$\frac{1}{2}[1 + 1/(1\text{-}0.5)^2] = \frac{1}{2}(1 + 4) = 2.5$.
48	[N] Uniform hashing at $\alpha = 0.9$: expected probes for an absent key?	$1/(1\text{-}0.9) = 10$.
49	Space complexity of chaining and of open addressing?	Chaining $O(m + n)$ (array of heads plus nodes); open addressing $O(m)$ with $m \ge n$.
50	[C] Compare hash table, sorted array and balanced BST.	Hash: expected $O(1)$, no order. Sorted array: search $O(\log n)$, insert $O(n)$. Balanced BST: all $O(\log n)$ worst case and ordered traversal.

16.11 Advanced Hashing

#	Question	Answer
51	What is universal hashing?	Choosing the hash function at random from a family where any two distinct keys collide with probability at most $1/m$; it defeats adversarial key choices.
52	What is perfect hashing?	A collision-free hash function for a fixed key set; lookup takes one probe in the worst case. A minimal perfect hash uses exactly n slots for n keys.
53	Describe cuckoo hashing.	Two tables and two hash functions; a key lives in one of its two candidate slots. Insertion evicts the occupant to its other slot, repeating; a cycle triggers a rehash. Lookup needs at most two probes.
54	What is consistent hashing used for?	Distributing keys over servers on a hash ring so that adding or removing a server moves only about $1/N$ of the keys, unlike `hash mod N`.
55	What is extendible hashing?	A directory of 2^d pointers indexed by d bits of the hash; full buckets split (doubling the directory only if their local depth equals the global depth). Lookup takes one bucket access.
56	[T] Can an ordinary table hash function be used to store passwords?	No. It is fast and easy to invert or collide. Passwords need slow, salted, purpose-built algorithms (bcrypt, scrypt, Argon2, PBKDF2).
57	Hashing versus checksum?	A checksum (CRC) detects accidental errors; a cryptographic hash resists deliberate tampering; a table hash only spreads keys.

16.12 C++ Standard Library Hashing

#	Question	Answer
58	Which standard containers are hash tables?	`unordered_map`, `unordered_set`, `unordered_multimap`, `unordered_multiset`.
59	[T] What does `m[k]` do for a missing key in `unordered_map`?	It inserts `k` with a value-initialised value and returns a reference to it, so reading can grow the map. Use `find`, `count` or `at` to avoid it.
60	Difference between `insert` and `insert_or_assign`?	`insert` never overwrites an existing key and returns `false`; `insert_or_assign` (C++17) assigns the new value.
61	What do `reserve(n)` and `rehash(n)` do?	`reserve(n)` sizes the table for n elements without exceeding `max_load_factor`; `rehash(n)` sets the bucket count to at least n. Both rehash the elements.
62	[T] What must a custom hash and equality satisfy?	If two keys compare equal their hashes must be equal. Hash all members used by `==`, nothing else.
63	When are iterators and references invalidated?	A rehash invalidates **iterators** but not references/pointers to elements; erasing an element invalidates only that element.

16.13 Practical Applications and Edge Cases

#	Question	Answer
64	Give three applications of hashing.	Symbol tables in compilers, caches/memoization, duplicate detection, word-frequency counting, database hash indexes and joins, content-addressable storage.
65	[T] Why must insertion in open addressing check the whole probe path for a duplicate before reusing a tombstone?	The key might already be stored beyond the tombstone; placing a second copy at the tombstone creates a duplicate.
66	[T] An insertion into a quadratic-probing table fails although $n < m$. What do you do?	Report failure and rehash into a larger (prime) table, then retry; the probe sequence did not reach any free slot.
67	How do you test a hash table implementation?	Use a tiny table with keys sharing a home slot; test empty, full, absent, duplicate, negative and wrap-around cases; delete then search the keys behind the deleted one; run an invariant checker and sanitizers.
68	[T] Is `O(1)` insertion in a hash table guaranteed?	No: it is expected amortised $O(1)$ under uniform hashing and bounded load. Collisions make single operations $O(n)$, and a rehash costs $O(n)$.
69	[T] Why not use `h(k) = k mod 100` when all keys are multiples of 100?	Every key maps to slot 0. Use a prime modulus, which has no common factor with the key pattern.

NOTE. The book contains 69 viva questions with answers in this part.

PART 17

CODING QUESTIONS: EASY TO DIFFICULT

21 problems with constraints, hints, tested C++17 solutions, exact output and complexity

Each question gives the **problem**, the **input/output format**, **constraints**, **examples**, a **hint**, the **hashing concept** tested, the **difficulty** and the **complexity**. The solution follows, as a **complete program** that was compiled and run; the printed output is its real output. Inputs are hard-coded in `main` so that the output is deterministic. Try each problem yourself before reading the hint.

Level	Questions	Skills tested
Easy	Q1–Q7	implementing a basic table, frequency counting, duplicate detection, first non-repeating character, anagrams, Two Sum
Medium	Q8–Q15	grouping by a computed key, set-based $O(n)$ algorithms, prefix sums, correct deletion, custom hashing, caching (LRU), rehashing, top-k
Difficult	Q16–Q21	comparing probing methods, pathological collisions, pair-sum hashing, consistent hashing, cuckoo hashing, rolling hash

17.1 Easy Questions

Q1 [Easy] Implement a basic hash table

Problem	Implement a hash table with `put(key, value)` (insert or update), `get(key)` and `erase(key)` for integer keys. Collisions must not lose data.
Input / output	Operations hard-coded in `main`; print the results of `get` and `erase`.
Constraints	$m = 5$ buckets; keys are ints (may be negative); at most 10^5 operations.
Examples	put 5, 15, 25 (all collide), put 7; `get(15)` → 150; update 25; `erase(5)` → true; `erase(100)` → false.
Hint	Use a bucket per index holding a list of pairs; always scan the bucket for the key before adding.
Hashing concept	separate chaining, collisions, update versus insert
Difficulty	Easy
Complexity	$O(1 + \alpha)$ expected per operation; $O(m + n)$ space

```cpp
// Q1: basic hash table (separate chaining) with insert/get/erase
#include <iostream>
#include <list>
#include <utility>
#include <vector>

class HashTable {
public:
    explicit HashTable(int m) : b_(m) {}
    void put(int key, int value) {                      // insert or update
        for (auto& kv : b_[idx(key)])
            if (kv.first == key) { kv.second = value; return; }
        b_[idx(key)].emplace_back(key, value);
    }
    bool get(int key, int& value) const {
        for (const auto& kv : b_[idx(key)])
            if (kv.first == key) { value = kv.second; return true; }
        return false;
    }
    bool erase(int key) {
        auto& chain = b_[idx(key)];
        for (auto it = chain.begin(); it != chain.end(); ++it)
            if (it->first == key) { chain.erase(it); return true; }
        return false;
    }
private:
    std::size_t idx(int key) const {
        long long m = static_cast<long long>(b_.size());
        return static_cast<std::size_t>(((key % m) + m) % m);
    }
    std::vector<std::list<std::pair<int, int>>> b_;
```

```
};

int main() {
    HashTable t(5);
    t.put(5, 50); t.put(15, 150); t.put(25, 250); t.put(7, 70);   // 5, 15, 25 collide
    int v = 0;
    std::cout << "get(15) = " << (t.get(15, v) ? v : -1) << "\n";
    t.put(25, 999);
    std::cout << "get(25) after update = " << (t.get(25, v) ? v : -1) << "\n";
    std::cout << "erase(5) = " << t.erase(5) << ", get(5) found? " << t.get(5, v) << "\n";
    std::cout << "get(25) still = " << (t.get(25, v) ? v : -1)
              << ", erase(100) = " << t.erase(100) << "\n";
    return 0;
}
```

Output

```
get(15) = 150
get(25) after update = 999
erase(5) = 1, get(5) found? 0
get(25) still = 999, erase(100) = 0
```

Keys 5, 15 and 25 all land in bucket 0 ($m = 5$). `put` scans the chain first, so the second `put(25, ...)` updates instead of duplicating. `erase` of an absent key returns `false` and changes nothing.

Q2 [Easy] Frequency of elements

Problem	Given an array, print every distinct value and how many times it occurs.
Input / output	Input: array in `main`. Output: one line per distinct value, sorted by value.
Constraints	$1 \le n \le 10^6$; values fit in `int` (negatives allowed).
Examples	`{3,1,3,2,1,3,-4}` → −4:1, 1:2, 2:1, 3:3.
Hint	`operator[]` creates a missing key with value 0, so `++freq[x]` counts. Sort the output; `unordered_map` has no order.
Hashing concept	`unordered_map` as a counter
Difficulty	Easy
Complexity	$O(n)$ expected time; $O(d)$ space for d distinct values

```
// Q2: frequency of every element
#include <algorithm>
#include <iostream>
#include <unordered_map>
#include <vector>

int main() {
    std::vector<int> a = {3, 1, 3, 2, 1, 3, -4};
    std::unordered_map<int, int> freq;
    for (int x : a) ++freq[x];                          // missing key starts at 0
    std::vector<std::pair<int, int>> out(freq.begin(), freq.end());
    std::sort(out.begin(), out.end());                  // unordered_map has no order
    for (auto& kv : out) std::cout << kv.first << " appears " << kv.second << " time(s)\n";
    return 0;
}
```

Output

```
-4 appears 1 time(s)
1 appears 2 time(s)
2 appears 1 time(s)
3 appears 3 time(s)
```

`++freq[x]` is the standard counting idiom. Printing straight from `unordered_map` would give an unspecified order, so the pairs are copied into a vector and sorted.

Q3 [Easy] Detect duplicates

Problem	Return `true` if any value appears at least twice.
Input / output	Input: array. Output: `true`/`false`.
Constraints	$0 \le n \le 10^6$.
Examples	`{1,2,3,1}` → true; `{1,2,3}` → false; `{}` → false.
Hint	`unordered_set::insert` returns a `pair` whose `second` is `false` if the value was already present.
Hashing concept	`unordered_set` membership
Difficulty	Easy
Complexity	$O(n)$ expected; $O(n)$ space

```cpp
// Q3: does the array contain a duplicate?
#include <iostream>
#include <unordered_set>
#include <vector>

bool hasDuplicate(const std::vector<int>& a) {
    std::unordered_set<int> seen;
    for (int x : a)
        if (!seen.insert(x).second) return true;         // insert fails: already present
    return false;
}

int main() {
    std::cout << std::boolalpha;
    std::cout << "{1,2,3,1}: " << hasDuplicate({1, 2, 3, 1}) << "\n";
    std::cout << "{1,2,3}:   " << hasDuplicate({1, 2, 3}) << "\n";
    std::cout << "{}:        " << hasDuplicate({}) << "\n";
    return 0;
}
```

Output

```
{1,2,3,1}: true
{1,2,3}:   false
{}:        false
```

One pass; we stop at the first failed insertion. The empty array is correctly `false`.

Q4 [Easy] Count duplicate values

Problem	Count how many **distinct** values occur more than once and how many surplus copies exist.
Input / output	Input: array. Output: two numbers.
Constraints	$1 \le n \le 10^6$.
Examples	`{4,4,4,2,2,9}` → 2 repeating values (4 and 2); surplus copies 2 + 1 = 3.
Hint	Count frequencies first, then scan the map once.
Hashing concept	frequency map, two-pass counting
Difficulty	Easy
Complexity	$O(n)$ expected time; $O(d)$ space

```cpp
// Q4: how many distinct values occur more than once, and how many surplus copies exist?
#include <iostream>
#include <unordered_map>
#include <vector>

int main() {
    std::vector<int> a = {4, 4, 4, 2, 2, 9};
    std::unordered_map<int, int> cnt;
    for (int x : a) ++cnt[x];
    int repeated = 0, surplus = 0;
    for (const auto& kv : cnt) {
        if (kv.second > 1) { ++repeated; surplus += kv.second - 1; }
    }
    std::cout << "distinct values that repeat: " << repeated << "\n";
    std::cout << "surplus copies: " << surplus << "\n";
    return 0;
}
```

```
Output
distinct values that repeat: 2
surplus copies: 3
```

The surplus of a value with count c is c-1: 4 occurs three times (2 surplus), 2 twice (1 surplus), 9 once (0).

Q5 [Easy] First non-repeating character

Problem	Return the index of the first character that occurs exactly once in a string, or −1.
Input / output	Input: string. Output: index.
Constraints	$0 \le \lvert s \rvert \le 10^5$.
Examples	"leetcode" → 0; "loveleetcode" → 2; "aabb" → −1.
Hint	Pass 1: count characters. Pass 2: scan the string in order and return the first character with count 1.
Hashing concept	frequency map plus order preserved by the original string
Difficulty	Easy
Complexity	$O(n)$ time; $O(\sigma)$ space (σ = alphabet size)

```cpp
// Q5: index of the first non-repeating character (-1 if none)
#include <iostream>
#include <string>
#include <unordered_map>

int firstUnique(const std::string& s) {
    std::unordered_map<char, int> cnt;
    for (char c : s) ++cnt[c];
    for (std::size_t i = 0; i < s.size(); ++i)
        if (cnt[s[i]] == 1) return static_cast<int>(i);
    return -1;
}

int main() {
    for (const char* w : {"leetcode", "loveleetcode", "aabb", ""})
        std::cout << "\"" << w << "\" -> " << firstUnique(w) << "\n";
    return 0;
}
```

```
Output
"leetcode" -> 0
"loveleetcode" -> 2
"aabb" -> -1
"" -> -1
```

The map has no order, so the second pass iterates the **string**, not the map. For lowercase letters an `int[26]` is an even faster direct-address table.

Q6 [Easy] Check whether two strings are anagrams

Problem	Return `true` if the strings use the same letters with the same multiplicities.
Input / output	Input: two strings. Output: `true`/`false`.
Constraints	$\lvert a \rvert, \lvert b \rvert \le 10^5$.
Examples	"listen","silent" → true; "rat","car" → false; "","" → true.
Hint	Different lengths can never be anagrams. Count letters of the first, then subtract for the second; fail on a missing or zero count.
Hashing concept	counting with a hash map; early exit
Difficulty	Easy
Complexity	$O(n)$ time; $O(\sigma)$ space

```cpp
// Q6: are two strings anagrams of each other?
#include <iostream>
#include <string>
#include <unordered_map>

bool isAnagram(const std::string& a, const std::string& b) {
    if (a.size() != b.size()) return false;
    std::unordered_map<char, int> cnt;
    for (char c : a) ++cnt[c];
    for (char c : b) {
        auto it = cnt.find(c);
        if (it == cnt.end() || it->second == 0) return false;   // extra or unknown letter
        --it->second;
    }
    return true;
}

int main() {
    std::cout << std::boolalpha;
    std::cout << "listen / silent: " << isAnagram("listen", "silent") << "\n";
    std::cout << "rat / car: " << isAnagram("rat", "car") << "\n";
    std::cout << "aab / abb: " << isAnagram("aab", "abb") << "\n";
    std::cout << "\"\" / \"\": " << isAnagram("", "") << "\n";
    return 0;
}
```

Output

```
listen / silent: true
rat / car: false
aab / abb: false
"" / "": true
```

The check `it->second == 0` catches a letter that occurs more often in `b` than in `a` (for example "aab" and "abb").

Q7 [Easy] Two Sum

Problem	Given an array and a target, return the indices $i < j$ with `a[i] + a[j] == target`.
Input / output	Input: array, target. Output: the two indices or "no solution".
Constraints	$2 \le n \le 10^5$; exactly one solution is guaranteed in the original problem (here the last case has none).
Examples	`{2,7,11,15}`, 9 → 0 and 1; `{3,3}`, 6 → 0 and 1.
Hint	For every `x` look up `target - x` among the values already seen; store `x` **after** the lookup so an element is not paired with itself.
Hashing concept	hash map from value to index
Difficulty	Easy
Complexity	$O(n)$ expected time; $O(n)$ space (a nested loop is $O(n^2)$)

```cpp
// Q7: Two Sum - indices of two numbers adding up to the target
#include <iostream>
#include <unordered_map>
#include <utility>
#include <vector>

// returns {-1, -1} if no pair exists
std::pair<int, int> twoSum(const std::vector<int>& a, int target) {
    std::unordered_map<int, int> where;                  // value -> index seen so far
    for (int i = 0; i < static_cast<int>(a.size()); ++i) {
        auto it = where.find(target - a[i]);
        if (it != where.end()) return {it->second, i};
        where[a[i]] = i;
    }
    return {-1, -1};
}

int main() {
    struct Case { std::vector<int> a; int t; };
    for (const Case& c : {Case{{2, 7, 11, 15}, 9}, Case{{3, 2, 4}, 6}, Case{{3, 3}, 6},
                          Case{{1, 2, 3}, 10}}) {
        auto r = twoSum(c.a, c.t);
        if (r.first < 0) std::cout << "target " << c.t << ": no solution\n";
        else std::cout << "target " << c.t << ": indices " << r.first << " and " << r.second << "\n";
    }
    return 0;
}
```

Output

```
target 9: indices 0 and 1
target 6: indices 1 and 2
target 6: indices 0 and 1
target 10: no solution
```

Storing the value after the lookup handles `{3,3}` correctly: the second 3 finds the first 3 but the first 3 did not find itself.

17.2 Medium Questions

Q8 [Medium] Group anagrams

Problem	Group the words into lists of mutual anagrams.
Input / output	Input: list of words. Output: groups (printed sorted).
Constraints	$n \le 10^4$ words, length ≤ 100.
Examples	`eat,tea,tan,ate,nat,bat` → [ate eat tea] [bat] [nat tan].
Hint	Anagrams become equal after sorting their letters: use the sorted word as the **hash key**.
Hashing concept	computed (canonical) key; `unordered_map<string, vector<string>>`
Difficulty	Medium
Complexity	$O(n \cdot L \log L)$ time for n words of length L

```cpp
// Q8: group words that are anagrams of each other
#include <algorithm>
#include <iostream>
#include <string>
#include <unordered_map>
#include <vector>

int main() {
    std::vector<std::string> words = {"eat", "tea", "tan", "ate", "nat", "bat"};
    std::unordered_map<std::string, std::vector<std::string>> groups;
    for (const std::string& w : words) {
        std::string key = w;
        std::sort(key.begin(), key.end());               // anagrams share the sorted word
        groups[key].push_back(w);
    }
    std::vector<std::vector<std::string>> out;
    for (auto& kv : groups) {
        std::sort(kv.second.begin(), kv.second.end());
        out.push_back(kv.second);
    }
    std::sort(out.begin(), out.end());                   // fixed order for printing
    for (const auto& g : out) {
        std::cout << "[";
        for (std::size_t i = 0; i < g.size(); ++i) std::cout << (i ? " " : "") << g[i];
        std::cout << "]\n";
    }
    return 0;
}
```

Output

```
[ate eat tea]
[bat]
[nat tan]
```

The map key is not the input but a **derived** canonical form. Output is sorted only to make the printout deterministic.

Q9 [Medium] Longest consecutive sequence

Problem	Find the length of the longest run of consecutive integers contained in an unsorted array, in $O(n)$.
Input / output	Input: array. Output: length.
Constraints	$n \le 10^5$; duplicates allowed.
Examples	`{100,4,200,1,3,2}` → 4 (1,2,3,4); `{0,3,7,2,5,8,4,6,0,1}` → 9; `{}` → 0.
Hint	Put everything in a set. Only start counting from numbers `x` where `x-1` is **not** in the set.
Hashing concept	hash set for $O(1)$ membership
Difficulty	Medium
Complexity	$O(n)$ expected (each element is visited at most twice); sorting would be $O(n \log n)$

```cpp
// Q9: length of the longest run of consecutive integers (any order), O(n)
#include <algorithm>
#include <iostream>
#include <unordered_set>
#include <vector>

int longestConsecutive(const std::vector<int>& a) {
    std::unordered_set<int> s(a.begin(), a.end());
    int best = 0;
    for (int x : s) {
        if (s.count(x - 1)) continue;                    // x is not the start of a run
        int len = 1;
        while (s.count(x + len)) ++len;
        best = std::max(best, len);
    }
    return best;
}

int main() {
    std::cout << longestConsecutive({100, 4, 200, 1, 3, 2}) << "\n";
    std::cout << longestConsecutive({0, 3, 7, 2, 5, 8, 4, 6, 0, 1}) << "\n";
    std::cout << longestConsecutive({}) << "\n";
    return 0;
}
```

Output

```
4
9
0
```

Skipping numbers that have a predecessor guarantees every run is counted once from its start, which is what makes the loop linear.

Q10 [Medium] Subarray sum equals k

Problem	Count the contiguous subarrays whose elements sum to k (the array may contain negatives, so two pointers fail).
Input / output	Input: array, k. Output: count.
Constraints	$n \le 2 \cdot 10^4$; values in int range.
Examples	`{1,1,1}`, 2 → 2; `{1,2,3}`, 3 → 2; `{1,-1,0}`, 0 → 3.
Hint	If `prefix[j] - prefix[i] = k` then `prefix[i] = prefix[j] - k`. Keep a map: prefix sum → occurrences. Seed it with `{0:1}`.
Hashing concept	hash map of prefix sums
Difficulty	Medium
Complexity	$O(n)$ expected; $O(n)$ space

```cpp
// Q10: number of contiguous subarrays whose sum equals k (negative numbers allowed)
#include <iostream>
#include <unordered_map>
#include <vector>

int subarraySum(const std::vector<int>& a, int k) {
    std::unordered_map<long long, int> seen;            // prefix sum -> how many times
    seen[0] = 1;                                        // empty prefix
    long long prefix = 0;
    int count = 0;
    for (int x : a) {
        prefix += x;
        auto it = seen.find(prefix - k);                // earlier prefix = prefix - k
        if (it != seen.end()) count += it->second;
        ++seen[prefix];
    }
    return count;
}

int main() {
    std::cout << subarraySum({1, 1, 1}, 2) << "\n";
    std::cout << subarraySum({1, 2, 3}, 3) << "\n";
    std::cout << subarraySum({1, -1, 0}, 0) << "\n";
    return 0;
}
```

Output

```
2
2
3
```

For `{1,-1,0}`, k=0: subarrays [1,-1], [0] and [1,-1,0]: three. The initial `seen[0] = 1` accounts for subarrays that start at index 0.

Q11 [Medium] Implement deletion correctly in linear probing

Problem	Implement insert, contains and erase for an open-addressing table such that erase never hides other keys and re-inserting an erased key works.
Input / output	Checks with `assert`; print "all checks passed".
Constraints	m = 11; keys that all share one home slot.
Examples	Insert 25, 36, 47 (home 3); erase 36; `contains(47)` must stay true; re-insert 36 reuses the tombstone.
Hint	Three states (empty, full, dead). Search stops only at empty; insert remembers the first dead slot and reuses it after the duplicate scan.
Hashing concept	tombstones
Difficulty	Medium
Complexity	O(1) expected probes at low load

```cpp
// Q11: linear probing table whose erase is correct (tombstones), checked with asserts
#include <cassert>
#include <iostream>
#include <vector>

class Table {
    enum class S { Empty, Full, Dead };
    struct Slot { int key = 0; S s = S::Empty; };
public:
    explicit Table(int m) : m_(m), t_(m) {}
    bool insert(int k) {
        int firstDead = -1;
        for (int i = 0; i < m_; ++i) {
            int j = (k % m_ + i) % m_;
            if (t_[j].s == S::Empty) { put(firstDead >= 0 ? firstDead : j, k); return true; }
            if (t_[j].s == S::Dead) { if (firstDead < 0) firstDead = j; }
            else if (t_[j].key == k) return false;       // duplicate
        }
        if (firstDead >= 0) { put(firstDead, k); return true; }
        return false;                                    // full
    }
    bool contains(int k) const {
        for (int i = 0; i < m_; ++i) {
            const Slot& x = t_[(k % m_ + i) % m_];
```

```
            if (x.s == S::Empty) return false;
            if (x.s == S::Full && x.key == k) return true;
        }
        return false;
    }
    bool erase(int k) {
        for (int i = 0; i < m_; ++i) {
            Slot& x = t_[(k % m_ + i) % m_];
            if (x.s == S::Empty) return false;
            if (x.s == S::Full && x.key == k) { x.s = S::Dead; return true; }
        }
        return false;
    }
private:
    void put(int j, int k) { t_[j].key = k; t_[j].s = S::Full; }
    int m_;
    std::vector<Slot> t_;
};

int main() {
    Table t(11);
    assert(t.insert(25) && t.insert(36) && t.insert(47));   // all home slot 3
    assert(t.erase(36));
    assert(t.contains(47));                                 // still reachable behind the tombstone
    assert(!t.contains(36));                                // deleted key is gone
    assert(t.insert(36));                                   // re-insertion reuses the tombstone
    assert(!t.insert(47));                                  // duplicate rejected
    assert(!t.erase(99));                                   // absent key
    assert(t.contains(25) && t.contains(36) && t.contains(47));
    std::cout << "all checks passed\n";
    return 0;
}
```

Output

```
all checks passed
```

The asserts encode the invariants from Part 6. If `erase` wrote EMPTY instead of Dead, `assert(t.contains(47))` would fail immediately.

Q12 [Medium] Custom hash for a struct

Problem	Store `Student{name, roll}` objects in an `unordered_set` so that equal students are not stored twice.
Input / output	Print whether a duplicate was added and the final size.
Constraints	Equality uses both fields.
Examples	Insert (Asha,1), (Ravi,2), (Asha,1) → size 2.
Hint	Provide `operator==` and a hash functor that **combines** the hashes of both members.
Hashing concept	custom hash and equality; equal objects ⇒ equal hashes
Difficulty	Medium
Complexity	$O(L)$ per operation for name length L

```cpp
// Q12: a custom hash and equality for a struct key
#include <functional>
#include <iostream>
#include <string>
#include <unordered_set>

struct Student {
    std::string name;
    int roll;
    bool operator==(const Student& o) const { return roll == o.roll && name == o.name; }
};

struct StudentHash {
    std::size_t operator()(const Student& s) const noexcept {
        std::size_t h1 = std::hash<std::string>{}(s.name);
        std::size_t h2 = std::hash<int>{}(s.roll);
        return h1 ^ (h2 + 0x9e3779b97f4a7c15ULL + (h1 << 6) + (h1 >> 2));   // combine, order matters
    }
};

int main() {
    std::unordered_set<Student, StudentHash> s;
    s.insert({"Asha", 1});
    s.insert({"Ravi", 2});
    bool added = s.insert({"Asha", 1}).second;            // duplicate
    std::cout << "duplicate added? " << added << ", size = " << s.size() << "\n";
    std::cout << "contains (Ravi,2): " << s.count({"Ravi", 2})
              << ", contains (Ravi,3): " << s.count({"Ravi", 3}) << "\n";
    return 0;
}
```

Output

```
duplicate added? 0, size = 2
contains (Ravi,2): 1, contains (Ravi,3): 0
```

The hash must use only fields that take part in `==`. Combining with a shift/add pattern keeps `(Asha,1)` and `(1,Asha)`-like swaps apart.

Q13 [Medium] LRU cache

Problem	Design a cache with capacity c supporting `get(key)` and `put(key, value)` in $O(1)$; when full, `put` evicts the **least recently used** key.
Input / output	Operations hard-coded.
Constraints	$c \le 3000$.
Examples	cap 2: put(1,1) put(2,2) get(1)=1 put(3,3) get(2)=-1 put(4,4) get(1)=-1 get(3)=3 get(4)=4.
Hint	Doubly linked list ordered by recency + hash map from key to list iterator. `splice` moves a node in $O(1)$.
Hashing concept	practical caching: hash map + linked list
Difficulty	Medium
Complexity	$O(1)$ per operation expected; $O(c)$ space

```cpp
// Q13: LRU cache with O(1) get and put (hash map + doubly linked list)
#include <iostream>
#include <list>
#include <unordered_map>
#include <utility>

class LRUCache {
public:
    explicit LRUCache(int capacity) : cap_(capacity) {}
    int get(int key) {
        auto it = pos_.find(key);
        if (it == pos_.end()) return -1;
        order_.splice(order_.begin(), order_, it->second);      // move to the front: most recent
        return it->second->second;
    }
    void put(int key, int value) {
        auto it = pos_.find(key);
        if (it != pos_.end()) {                          // update and refresh
            it->second->second = value;
            order_.splice(order_.begin(), order_, it->second);
            return;
```

```
        }
        if (static_cast<int>(order_.size()) == cap_) {   // evict the least recently used
            pos_.erase(order_.back().first);
            order_.pop_back();
        }
        order_.emplace_front(key, value);
        pos_[key] = order_.begin();
    }
private:
    int cap_;
    std::list<std::pair<int, int>> order_;               // front = most recently used
    std::unordered_map<int, std::list<std::pair<int, int>>::iterator> pos_;
};

int main() {
    LRUCache c(2);
    c.put(1, 1); c.put(2, 2);
    std::cout << "get(1) = " << c.get(1) << "\n";
    c.put(3, 3);                                         // evicts key 2
    std::cout << "get(2) = " << c.get(2) << "\n";
    c.put(4, 4);                                         // evicts key 1
    std::cout << "get(1) = " << c.get(1) << ", get(3) = " << c.get(3)
              << ", get(4) = " << c.get(4) << "\n";
    return 0;
}
```

Output

```
get(1) = 1
get(2) = -1
get(1) = -1, get(3) = 3, get(4) = 4
```

`get` moves the node to the front; `put` of a new key evicts `order_.back()`, whose key is erased from the map before the node is popped.

Q14 [Medium] Rehash a table

Problem	Given a linear-probing table of size 5, produce the equivalent table of size 11 and show that merely copying the array loses keys.
Input / output	Print both tables and lookup results.
Constraints	Keys are non-negative ints.
Examples	old `[25 . 12 17 .]` → new `[. 12 . 25 . . 17 . . . .]`.
Hint	Create a fresh array of the new size and **re-insert every key** with `key % newM`.
Hashing concept	rehashing versus copying
Difficulty	Medium
Complexity	$O(m_{\text{old}} + m_{\text{new}} + n)$

```
// Q14: rehash a linear-probing table into a larger table
#include <iostream>
#include <vector>

const int EMPTY = -1;

void insertLinear(std::vector<int>& t, int key) {
    int m = static_cast<int>(t.size());
    int j = key % m;
    while (t[j] != EMPTY) j = (j + 1) % m;
    t[j] = key;
}

std::vector<int> rehash(const std::vector<int>& old, int newM) {
    std::vector<int> fresh(newM, EMPTY);
    for (int k : old)
        if (k != EMPTY) insertLinear(fresh, k);          // recompute with the NEW size
    return fresh;
}

bool contains(const std::vector<int>& t, int key) {
    int m = static_cast<int>(t.size());
    for (int i = 0; i < m; ++i) {
        int v = t[(key % m + i) % m];
        if (v == EMPTY) return false;
```

```
        if (v == key) return true;
    }
    return false;
}

void show(const char* label, const std::vector<int>& t) {
    std::cout << label;
    for (int v : t) { if (v == EMPTY) std::cout << " ."; else std::cout << " " << v; }
    std::cout << "\n";
}

int main() {
    std::vector<int> old = {25, EMPTY, 12, 17, EMPTY};   // valid m = 5 table (17 sits in 3)
    show("old (m=5):   ", old);
    std::vector<int> naive(11, EMPTY);
    for (std::size_t i = 0; i < old.size(); ++i) naive[i] = old[i];
    std::cout << "naive copy finds 25? " << contains(naive, 25) << "\n";
    std::vector<int> fresh = rehash(old, 11);
    show("rehash (m=11):", fresh);
    std::cout << "rehash finds 25, 12, 17? " << contains(fresh, 25) << contains(fresh, 12)
              << contains(fresh, 17) << "\n";
    return 0;
}
```

Output

```
old (m=5):    25 . 12 17 .
naive copy finds 25? 0
rehash (m=11): . 12 . 25 . . 17 . . . .
rehash finds 25, 12, 17? 111
```

The naive copy keeps 25 in slot 0, but the new lookup looks at slot $25 \bmod 11 = 3$, so `contains` fails. The rehash puts 25 at slot 3, 12 at 1 and 17 at 6.

Q15 [Medium] Top-k frequent words

Problem	Print the k most frequent words; ties are broken alphabetically.
Input / output	Input: a sentence and k. Output: k lines `word count`.
Constraints	$n \le 10^5$ words.
Examples	"the day is sunny the the sunny is is", k=2 → is 3, the 3.
Hint	Count with `unordered_map`, copy to a vector and sort with a comparator (count descending, word ascending).
Hashing concept	frequency map + sorting
Difficulty	Medium
Complexity	$O(n + d \log d)$ for d distinct words

```
// Q15: k most frequent words (ties broken alphabetically)
#include <algorithm>
#include <iostream>
#include <sstream>
#include <string>
#include <unordered_map>
#include <vector>

int main() {
    std::string text = "the day is sunny the the sunny is is";
    int k = 2;
    std::unordered_map<std::string, int> cnt;
    std::istringstream in(text);
    for (std::string w; in >> w;) ++cnt[w];
    std::vector<std::pair<std::string, int>> v(cnt.begin(), cnt.end());
    std::sort(v.begin(), v.end(), [](const auto& a, const auto& b) {
        return a.second != b.second ? a.second > b.second : a.first < b.first;
    });
    for (int i = 0; i < k && i < static_cast<int>(v.size()); ++i)
        std::cout << v[i].first << " " << v[i].second << "\n";
    return 0;
}
```

```
Output
is 3
the 3
```

"is" and "the" both occur three times; the comparator puts "is" first alphabetically. For huge d and small k a min-heap of size k gives $O(d \log k)$.

17.3 Difficult Questions

Q16 [Difficult] Compare linear probing, quadratic probing and double hashing

Problem	Insert the same 8 keys into a table of 17 slots with each method and report (a) total probes for the insertions and (b) the average number of probes for a successful search.
Input / output	Print one line per method.
Constraints	$m = 17$ (prime); keys 5, 22, 39, 56 (home 5) and 7, 24, 41, 58 (home 7); quadratic uses $h + i^2$ (load 8/17 < 0.5, so insertion is guaranteed).
Examples	Expected: linear is worst, double hashing best (see output).
Hint	Write one `slotFor(strategy, key, i, m)` function and reuse the same insert and search loops.
Hashing concept	clustering; probe counts; the three open-addressing methods
Difficulty	Difficult
Complexity	$O(n \cdot p)$

```cpp
// Q16: linear vs quadratic vs double hashing on the same keys (m = 17, 8 keys)
#include <iostream>
#include <string>
#include <vector>

enum class Strategy { Linear, Quadratic, Double };

struct Result { int insertProbes; double avgSuccessProbes; };

int slotFor(Strategy s, int key, int i, int m) {
    int h = key % m;
    switch (s) {
        case Strategy::Linear:    return (h + i) % m;
        case Strategy::Quadratic: return (h + i * i) % m;                    // pure squares
        default:                  return (h + i * (1 + key % (m - 1))) % m; // double hashing
    }
}

Result run(Strategy s, int m, const std::vector<int>& keys) {
    std::vector<int> t(m, -1);
    int total = 0;
    for (int k : keys) {
        for (int i = 0; i < m; ++i) {
            int j = slotFor(s, k, i, m);
            ++total;
            if (t[j] == -1) { t[j] = k; break; }
        }
    }
    int succ = 0;
    for (int k : keys) {
        for (int i = 0; i < m; ++i) {
            ++succ;
            if (t[slotFor(s, k, i, m)] == k) break;
        }
    }
    return {total, static_cast<double>(succ) / keys.size()};
}

int main() {
    std::vector<int> keys = {5, 22, 39, 56, 7, 24, 41, 58};   // 4 keys with home 5, 4 with home 7
    const std::pair<const char*, Strategy> list[] = {
        {"linear", Strategy::Linear}, {"quadratic", Strategy::Quadratic},
        {"double", Strategy::Double}};
    for (const auto& e : list) {
        Result r = run(e.second, 17, keys);
        std::cout << e.first << ": insertion probes = " << r.insertProbes
                  << ", average successful search = " << r.avgSuccessProbes << "\n";
    }
```

```
    return 0;
}
```

```
Output
linear: insertion probes = 28, average successful search = 3.5
quadratic: insertion probes = 20, average successful search = 2.5
double: insertion probes = 14, average successful search = 1.75
```

Eight keys form two groups with identical home slots. Linear probing makes two merged clusters; pure squares separate the groups' paths; double hashing gives each key its own step. The numbers are checked against the simulator used for the figures of Parts 7–9.

Q17 [Difficult] Analyse pathological collision cases

Problem	Show quantitatively how keys that share a factor with the table size ruin hashing: for 9 keys compare the number of distinct home slots, the longest chain and the total probes of linear probing.
Input / output	Print one line per scenario.
Constraints	$m = 12$ or 13.
Examples	multiples of 4 with m=12 → only 3 home slots; the same keys with m=13 → 9 home slots.
Hint	Keys that are multiples of d reach only $m/\gcd(d, m)$ home slots. Count with an array of chain lengths and simulate linear probing with a `used` array.
Hashing concept	adversarial / patterned keys; why prime table sizes matter
Difficulty	Difficult
Complexity	$O(n)$ per scenario

```
// Q17: patterned keys versus spread keys: how bad can collisions get? (m = 12, 9 keys)
#include <algorithm>
#include <iostream>
#include <set>
#include <vector>

void analyse(const char* label, const std::vector<int>& keys, int m) {
    std::vector<int> chain(m, 0);
    std::set<int> homes;
    std::vector<bool> used(m, false);
    int probes = 0;
    for (int k : keys) {
        ++chain[k % m];
        homes.insert(k % m);
        int j = k % m;
        ++probes;
        while (used[j]) { j = (j + 1) % m; ++probes; }  // linear probing
        used[j] = true;
    }
    std::cout << label << ": distinct home slots = " << homes.size()
              << ", longest chain = " << *std::max_element(chain.begin(), chain.end())
              << ", linear-probing insertion probes = " << probes << "\n";
}

int main() {
    analyse("multiples of 4 (4..36)", {4, 8, 12, 16, 20, 24, 28, 32, 36}, 12);
    analyse("consecutive (1..9)     ", {1, 2, 3, 4, 5, 6, 7, 8, 9}, 12);
    analyse("multiples of 4, m = 13 ", {4, 8, 12, 16, 20, 24, 28, 32, 36}, 13);
    return 0;
}
```

```
Output
multiples of 4 (4..36): distinct home slots = 3, longest chain = 3, linear-probing insertion probes = 18
consecutive (1..9)     : distinct home slots = 9, longest chain = 1, linear-probing insertion probes = 9
multiples of 4, m = 13 : distinct home slots = 9, longest chain = 1, linear-probing insertion probes = 9
```

With $m = 12$ and keys 4, 8, …, 36 the homes are 4, 8, 0 repeating: longest chain 3 and 18 probes instead of 9. Switching to the prime $m = 13$ removes the pattern completely (9 probes).

Q18 [Difficult] Four-sum count

Problem	Given four arrays A, B, C, D of length n, count the tuples (i, j, k, l) with $A[i] + B[j] + C[k] + D[l] = 0$ in $O(n^2)$.
Input / output	Output: the count.
Constraints	$n \le 200$ (the answer needs 64-bit integers for larger n).
Examples	`{1,2},{-2,-1},{-1,2},{0,2}` → 2; all zeros with n=2 → 16.
Hint	Store all sums $A[i]+B[j]$ with their multiplicity in a map; for every $C[k]+D[l]$ add the multiplicity of the negated sum.
Hashing concept	meet-in-the-middle with a hash map of sums
Difficulty	Difficult
Complexity	$O(n^2)$ expected time and space (brute force $O(n^4)$)

```cpp
// Q18: count tuples (i,j,k,l) with A[i]+B[j]+C[k]+D[l] == 0 in O(n^2)
#include <iostream>
#include <unordered_map>
#include <vector>

long long fourSumCount(const std::vector<int>& A, const std::vector<int>& B,
                       const std::vector<int>& C, const std::vector<int>& D) {
    std::unordered_map<int, int> ab;                    // sum of A+B -> number of ways
    for (int a : A) for (int b : B) ++ab[a + b];
    long long count = 0;
    for (int c : C)
        for (int d : D) {
            auto it = ab.find(-(c + d));
            if (it != ab.end()) count += it->second;
        }
    return count;
}

int main() {
    std::cout << fourSumCount({1, 2}, {-2, -1}, {-1, 2}, {0, 2}) << "\n";
    std::cout << fourSumCount({0, 0}, {0, 0}, {0, 0}, {0, 0}) << "\n";
    std::cout << fourSumCount({1}, {1}, {1}, {1}) << "\n";
    return 0;
}
```

Output

```
2
16
0
```

Each pair-sum is stored once with a count, so equal sums contribute together. With all zeros every one of the $2 \times 2 \times 2 \times 2 = 16$ tuples qualifies.

Q19 [Difficult] Consistent hashing with virtual nodes

Problem	Build a hash ring with 100 virtual nodes per server. Assign 1000 keys to servers A, B, C, add server D, and report the load per server, how many keys moved, and how many moved **to D**.
Input / output	Print loads and counts.
Constraints	Use a 32-bit hash with a final mixing step.
Examples	Every moved key must move to D; compare with `hash mod N` going from 3 to 4 servers.
Hint	`std::map<uint32_t, string>` with `lower_bound`, wrapping to `begin()`.
Hashing concept	consistent hashing; minimal key movement; virtual nodes for balance
Difficulty	Difficult
Complexity	$O(\log N)$ to build, $O(\log (N\ v))$ per lookup

```cpp
// Q19: consistent hashing with virtual nodes; how many of 1000 keys move when a server joins?
#include <cstdint>
#include <iostream>
#include <map>
#include <string>
#include <vector>

std::uint32_t fnv1a32(const std::string& s) {
```

```
    std::uint32_t h = 2166136261u;
    for (unsigned char c : s) { h ^= c; h *= 16777619u; }
    h ^= h >> 16; h *= 0x85ebca6bu;                    // final mixing (avalanche) step so that
    h ^= h >> 13; h *= 0xc2b2ae35u; h ^= h >> 16;      // similar names land far apart on the ring
    return h;
}

class Ring {
public:
    void add(const std::string& server, int vnodes) {
        for (int v = 0; v < vnodes; ++v) ring_[fnv1a32(server + "#" + std::to_string(v))] = server;
    }
    const std::string& serverFor(const std::string& key) const {
        auto it = ring_.lower_bound(fnv1a32(key));     // first virtual node clockwise
        if (it == ring_.end()) it = ring_.begin();     // wrap around
        return it->second;
    }
private:
    std::map<std::uint32_t, std::string> ring_;
};

int main() {
    Ring r;
    for (const char* s : {"A", "B", "C"}) r.add(s, 100);
    std::vector<std::string> keys;
    for (int i = 0; i < 1000; ++i) keys.push_back("key" + std::to_string(i));
    std::vector<std::string> before;
    std::map<std::string, int> load;
    for (const auto& k : keys) { before.push_back(r.serverFor(k)); ++load[before.back()]; }
    std::cout << "load before: A=" << load["A"] << " B=" << load["B"] << " C=" << load["C"] << "\n";
    r.add("D", 100);
    int moved = 0, movedToD = 0;
    std::map<std::string, int> after;
    for (std::size_t i = 0; i < keys.size(); ++i) {
        const std::string& s = r.serverFor(keys[i]);
        ++after[s];
        if (s != before[i]) { ++moved; if (s == "D") ++movedToD; }
    }
    std::cout << "load after:  A=" << after["A"] << " B=" << after["B"] << " C=" << after["C"]
              << " D=" << after["D"] << "\n";
    std::cout << "keys moved: " << moved << ", of which moved to D: " << movedToD << "\n";
    int modMoved = 0;
    for (const auto& k : keys) if (fnv1a32(k) % 3 != fnv1a32(k) % 4) ++modMoved;
    std::cout << "with hash mod N, going from 3 to 4 servers would move about "
              << modMoved << " keys\n";
    return 0;
}
```

Output

```
load before: A=393 B=318 C=289
load after:  A=313 B=251 C=201 D=235
keys moved: 235, of which moved to D: 235
with hash mod N, going from 3 to 4 servers would move about 738 keys
```

Only the keys in the arcs now owned by D change server (235 of 1000, all to D), whereas `hash mod N` would reassign 738 keys. The mixing step matters: without it, similar names ("key0", "key1", ...) land close together on the ring and the load is badly uneven (an earlier draft without mixing gave loads of 240/595/165).

Q20 [Difficult] Cuckoo hashing with rehash on cycle

Problem	Implement cuckoo hashing with two tables. If an insertion evicts more than $2m$ times (a cycle), rebuild the table with a larger size and a different second hash function.
Input / output	Insert keys; print when a cycle triggers a rehash; verify every key is findable.
Constraints	$m = 11$ initially; $h_1 = k \bmod m$.
Examples	Keys 1, 122, 243 share both candidate slots for $m = 11$: three keys cannot fit in two slots.
Hint	Swap the key into its slot in table t, carry the evicted key to the other table. Count kicks. On failure collect all keys (including the one being carried) and re-insert them into new tables.
Hashing concept	cuckoo hashing; cycle detection; rehash with new hash functions
Difficulty	Difficult
Complexity	lookup $O(1)$ worst case (2 probes); insertion expected amortized $O(1)$

```cpp
// Q20: cuckoo hashing with cycle detection and rehash (new, larger table and new second hash)
#include <iostream>
#include <vector>

class Cuckoo {
public:
    Cuckoo(int m, int mult)
        : m_(m), mult_(mult), t_{std::vector<int>(m, -1), std::vector<int>(m, -1)} {}
    int h(int table, int k) const { return table == 0 ? k % m_ : (k / m_ * mult_ + k) % m_; }

    bool contains(int k) const { return t_[0][h(0, k)] == k || t_[1][h(1, k)] == k; }

    void insert(int k) {
        if (contains(k)) return;
        int table = 0;
        for (int kicks = 0; kicks < 2 * m_; ++kicks) {
            int i = h(table, k);
            std::swap(k, t_[table][i]);                  // place k, pick up the occupant
            if (k == -1) return;                         // nothing evicted: done
            table = 1 - table;
        }
        std::cout << "  cycle while inserting (carrying " << k << "): rehash\n";
        std::vector<int> keys;
        for (int t = 0; t < 2; ++t) for (int v : t_[t]) if (v != -1) keys.push_back(v);
        keys.push_back(k);                               // the key that was in the air
        m_ = 2 * m_ + 1;                                 // larger table
        mult_ += 2;                                      // different second hash function
        t_[0].assign(m_, -1);
        t_[1].assign(m_, -1);
        for (int key : keys) insert(key);
    }
    int capacity() const { return m_; }
private:
    int m_, mult_;
    std::vector<int> t_[2];
};

int main() {
    Cuckoo c(11, 1);
    for (int k : {20, 50, 53, 75, 100, 67, 105, 3, 36, 39}) c.insert(k);
    std::cout << "after 10 keys: capacity " << c.capacity() << "\n";
    for (int k : {1, 122, 243}) {              // same h1 AND same h2: 2 slots for 3 keys
        std::cout << "insert " << k << "\n";
        c.insert(k);
    }
    bool all = true;
    for (int k : {20, 50, 53, 75, 100, 67, 105, 3, 36, 39, 1, 122, 243}) all = all && c.contains(k);
    std::cout << "capacity " << c.capacity() << ", all 13 keys found: " << all
              << ", 7 found? " << c.contains(7) << "\n";
    return 0;
}
```

Output

```
after 10 keys: capacity 11
insert 1
insert 122
  cycle while inserting (carrying 100): rehash
insert 243
capacity 23, all 13 keys found: 1, 7 found? 0
```

The cycle is detected by the kick limit, not by remembering visited slots. The key that was "in the air" must be added to the rebuilt table, otherwise it is lost. After the rehash to 23 slots (second hash changed) all 13 keys are found and 7 is correctly absent.

Q21 [Difficult] Rabin–Karp substring search

Problem	Find all occurrences of a pattern in a text using a rolling hash: compute the hash of each window in $O(1)$ from the previous one, and compare the actual characters only when hashes match.
Input / output	Print the starting positions and the number of hash hits versus true matches.
Constraints	Pattern length $m \le$ text length n; the hash modulus is a small prime (101) so spurious hits are possible in general.
Examples	"abracadabra": "abra" → 0 and 7; "cad" → 4; "xyz" → none.
Hint	Window hash update: $h = ((h - t[s] \cdot B^{m-1})\ B + t[s+m]) \bmod M$; keep everything non-negative.
Hashing concept	rolling polynomial hash; verification after a hash match
Difficulty	Difficult
Complexity	expected $O(n + m)$; worst case $O(nm)$ if many hash matches are false

```cpp
// Q21: Rabin-Karp substring search with a rolling hash (verification avoids false matches)
#include <iostream>
#include <string>
#include <vector>

int main() {
    const std::string text = "abracadabra";
    const std::vector<std::string> patterns = {"abra", "cad", "xyz"};
    const long long B = 256, M = 101;                    // small prime so spurious hits can occur
    for (const std::string& p : patterns) {
        int n = static_cast<int>(text.size()), m = static_cast<int>(p.size());
        std::cout << "pattern \"" << p << "\":";
        if (m > n) { std::cout << " longer than text\n"; continue; }
        long long high = 1;                              // B^(m-1) mod M
        for (int i = 1; i < m; ++i) high = high * B % M;
        long long hp = 0, ht = 0;
        for (int i = 0; i < m; ++i) {
            hp = (hp * B + p[i]) % M;
            ht = (ht * B + text[i]) % M;
        }
        int hashHits = 0, found = 0;
        for (int s = 0; s + m <= n; ++s) {
            if (ht == hp) {
                ++hashHits;
                if (text.compare(s, m, p) == 0) { std::cout << " " << s; ++found; }
            }
            if (s + m < n)                               // roll: drop text[s], add text[s+m]
                ht = ((ht - text[s] * high % M + M) * B + text[s + m]) % M;
        }
        std::cout << (found ? "" : " none") << "   (hash hits " << hashHits << ", true matches "
                  << found << ")\n";
    }
    return 0;
}
```

Output

```
pattern "abra": 0 7   (hash hits 2, true matches 2)
pattern "cad": 4   (hash hits 1, true matches 1)
pattern "xyz": none   (hash hits 0, true matches 0)
```

The rolling update removes the leading character, shifts by one base-B digit and appends the new character. Every hash hit is **verified** with `compare`, so spurious hash collisions never produce a wrong answer; here hash hits equal true matches because the example is small.

> **NOTE.** Total coding questions in this part: 21. Easy 7, Medium 8, Difficult 6.

PART 18

Numerical, Dry-Run and Case-Study Questions

Worked problems with every calculation shown, practice problems with answers, and design case studies

Every table in the worked problems below was produced by running the book's simulator, so the arithmetic is exact. Do each problem on paper first, then compare.

18.1 Worked Problems

Problem 1: Computing indexes with different hash functions

Compute the index of keys 12345, 67890 and 31415 using (a) the division method with $m = 11$, (b) the multiplication method with $m = 16$ and $A = 0.6180339887$, (c) shift folding with 2-digit pieces and $m = 100$.

Key	(a) k mod 11	(b) $\lfloor 16 \cdot \mathrm{frac}(kA) \rfloor$	(c) 2-digit pieces	(c) sum → mod 100
12345	12345 = 11×1122 + 3 → **3**	7629.62959 → frac 0.62959 → 16× = 10.073 → **10**	1 + 23 + 45	69 → **69**
67890	67890 = 11×6171 + 9 → **9**	41958.32750 → frac 0.32750 → 16× = 5.240 → **5**	6 + 78 + 90	174 → **74**
31415	31415 = 11×2855 + 10 → **10**	19415.53776 → frac 0.53776 → 16× = 8.604 → **8**	3 + 14 + 15	32 → **32**

NOTE. The 2-digit pieces are taken from the **right**, so 12345 is split 1 | 23 | 45; the leading piece may be shorter.

Problem 2: Inserting keys that collide (chaining and linear probing)

Insert 14, 21, 28, 35, 42 into a table with $m = 7$, $h(k) = k$ mod 7. All keys have remainder 0.

Key	Chaining: bucket 0 after insertion	Linear probing: probes	Slot
14	14	0	0
21	21 → 14	0 1	1
28	28 → 21 → 14	0 1 2	2
35	35 → 28 → 21 → 14	0 1 2 3	3
42	42 → 35 → 28 → 21 → 14	0 1 2 3 4	4

Cost: chaining examined 0 + 1 + 2 + 3 + 4 = 10 existing nodes in total (duplicate checks); linear probing used 1 + 2 + 3 + 4 + 5 = 15 probes. Final linear table: [14, 21, 28, 35, 42, None, None].

Problem 3: Linear probing trace, search, deletion

$m = 10$, $h(k) = k$ mod 10, keys **89, 18, 49, 58, 69** in this order. Then search 58 and 99, delete 49, search 58 again, insert 79.

Operation	Home	Probe sequence	Result
insert 89	9	9	slot 9
insert 18	8	8	slot 8
insert 49	9	9 → 0	slot 0
insert 58	8	8 → 9 → 0 → 1	slot 1
insert 69	9	9 → 0 → 1 → 2	slot 2
search 58	8	8 → 9 → 0 → 1	found slot 1, 4 probes
search 99	9	9 → 0 → 1 → 2 → 3	absent, 5 probes (stopped at EMPTY slot 3)
delete 49	9	9 → 0	slot 0 := DEL

Operation	Home	Probe sequence	Result
search 58 again	8	8 → 9 → 0 → 1	found slot 1, 4 probes
insert 79	9	9 → 0 → 1 → 2 → 3	stored in tombstone slot 0 after 5 probes

Figure 18.1 shows the table states of Problem 3.

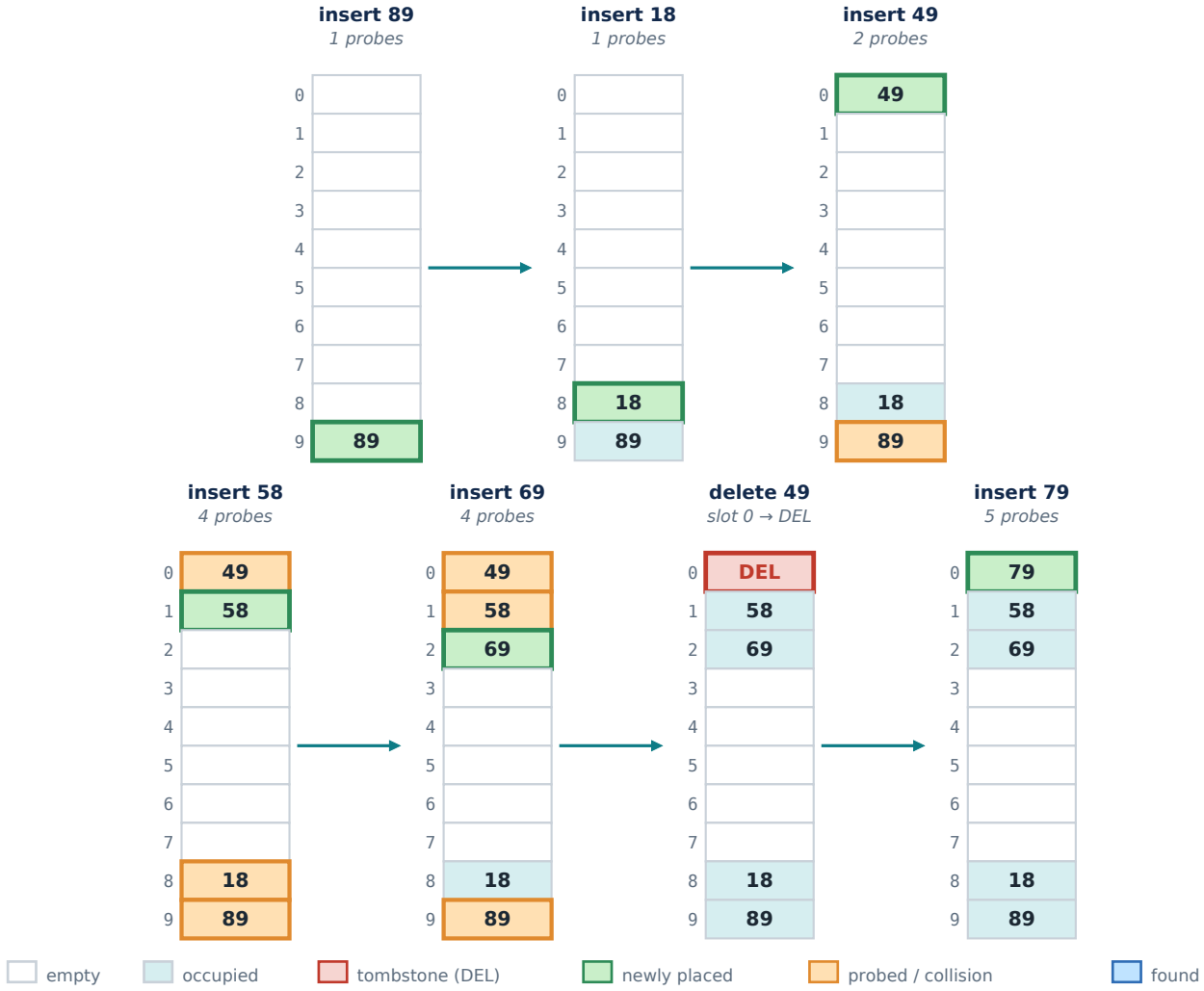


Figure 18.1 — Problem 3: table states. Insertion probe counts for 89, 18, 49, 58, 69: 1, 1, 2, 4, 4.

Reading the first table: 89 → slot 9; 18 → slot 8; 49 has home 9 (taken), next slot 0 (free); 58 has home 8 and walks 8 9 0 1; 69 has home 9 and walks 9 0 1 2. Total insertion probes = 12. Because 49 sat at slot 0 and was deleted with a tombstone, 58 (at slot 1) is **still found**; inserting 79 (home 9) walks 9 0 1 2 3, remembers the tombstone at slot 0 and, since the walk ends at an EMPTY slot, stores 79 there.

Problem 4: Quadratic probing trace

Same keys and $m = 10$ but probe slots $(h + i^2)$ mod 10:

Key	Home	Probes $h + i^2$	Slot
89	9	(9+0²=9)	9
18	8	(8+0²=8)	8
49	9	(9+0²=9) → (9+1²=0)	0
58	8	(8+0²=8) → (8+1²=9) → (8+2²=2)	2
69	9	(9+0²=9) → (9+1²=0) → (9+2²=3)	3

Final table (index: key): 0: 49, 2: 58, 3: 69, 8: 18, 9: 89. Total probes = 10. Key 58 jumps from slot 8 to 9 and then to $8 + 4 = 12 \to 2$; 69 goes 9, 0, then $9 + 4 = 13 \to 3$.

Problem 5: Double hashing trace

$m = 11$, $h_1 = k \bmod 11$, $h_2 = 1 + (k \bmod 10)$. Insert 79, 69, 98, 72, 14, 50.

Key	h_1	h_2	Probe sequence	Slot
79	2	10	2	2
69	3	10	3	3
98	10	9	10	10
72	6	3	6	6
14	3	5	3 → 8	8
50	6	1	6 → 7	7

Total probes = 8. Example: key 14 has $h_1 = 3$ and $h_2 = 5$; its probe sequence is 3 → 8, so it is stored in slot 8.

Problem 6: Counting probes and comparing the methods

Method (same 5 keys, m=10)	Total insertion probes
linear	12
quadratic	10
double ($h_2 = 7 - k \bmod 7$, m=10)	8

Linear probing is worst here because 49, 58 and 69 all pile into one cluster that started at slot 8; quadratic probing reaches free slots sooner. Double hashing with $h_2 = 7 - (k \bmod 7)$ on the **composite** $m = 10$ happens to work for these keys, but the coverage guarantee does not hold for composite m in general (Part 9).

Problem 7: Identifying clustering

Final table of Problem 3 (slots 0–9): [79, 58, 69, '-', '-', '-', '-', '-', 18, 89]. Occupied runs have lengths [3, 2]; the longest run has 3 slots. A key with home slot at any position inside that run must walk to its end, which is the signature of **primary clustering**.

Problem 8: Load factor

Situation	Calculation	Answer
45 keys, 60 slots (open addressing)	$45/60$	$\alpha = 0.75$
300 keys in 100 buckets (chaining)	$300/100$	$\alpha = 3$: average chain length 3
Table of 23 slots, linear probing, limit $\alpha \le 0.5$	$\lfloor 0.5 \times 23 \rfloor$	at most 11 keys; the 12th triggers a resize
11 live keys + 5 tombstones in 23 slots	$(11 + 5)/23$	effective load 0.70 although $n/m = 0.48$

Problem 9: Rehashing after a capacity change

A linear-probing table of $m = 7$ holds keys inserted in the order 50, 700, 76, 85, 92, 73, 101. It is rehashed to $m = 17$ (re-inserting in old slot order).

Key	Home (k mod 7)	Probes	Slot
50	1	1	1
700	0	0	0
76	6	6	6
85	1	1 → 2	2
92	1	1 → 2 → 3	3
73	3	3 → 4	4
101	3	3 → 4 → 5	5

Figure 18.2 shows the rehash of Problem 9.

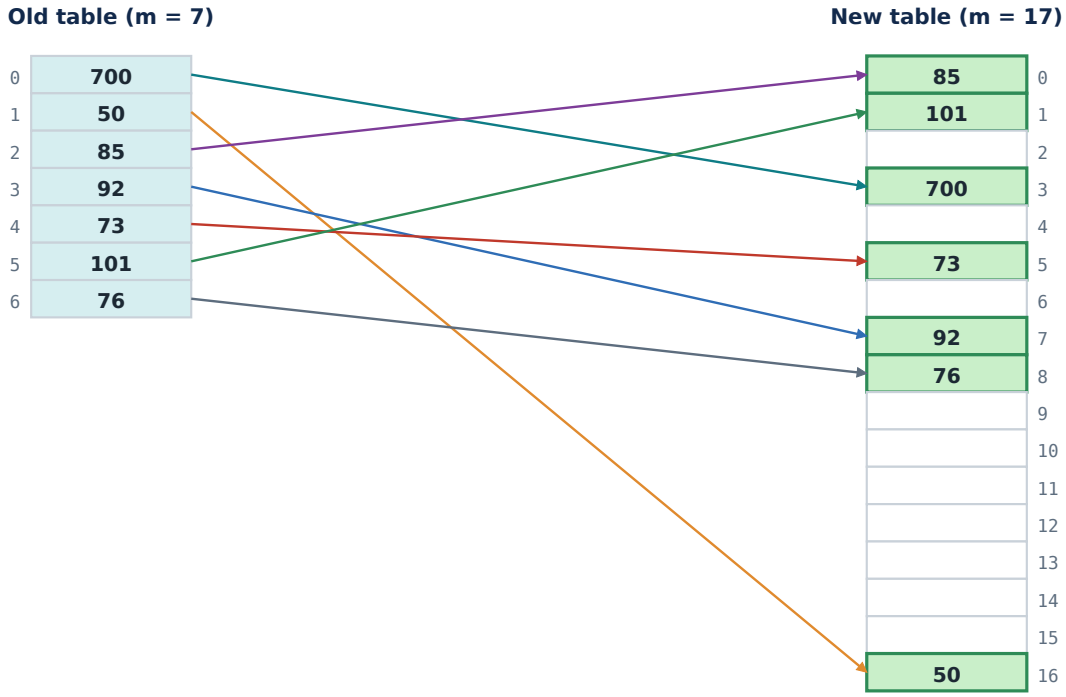


Figure 18.2 — Problem 9: the full table of 7 keys is rehashed into 17 slots; load factor drops from 1.00 to 0.41 and every key gets a new slot.

Key	Old slot	k mod 17	New slot
700	0	3	3
50	1	16	16
85	2	0	0
92	3	7	7
73	4	5	5
101	5	16	1
76	6	8	8

Problem 10: Average and worst-case reasoning

Question	Answer with working
Chaining, $\alpha = 2$: expected nodes compared (unsuccessful / successful)?	2 / 2.0
Linear probing, $\alpha = 0.75$: expected probes, unsuccessful?	$\frac{1}{2}[1 + 1/(0.25)^2] = \frac{1}{2}(1 + 16) = 8.5$
Double hashing, $\alpha = 0.75$: unsuccessful / successful?	$1/0.25 = 4$ / $(1/0.75)\ln 4 = 1.85$
Worst case of searching $n = 1000$ keys with a constant hash function	1000 comparisons (all keys in one chain or one probe path): $O(n)$
$n = 1000$ insertions into a table that doubles from $m = 8$ at $\alpha = 1$: total rehash moves?	$8 + 16 + \ldots + 512 = 1016$ (less than 1.1 moves per insertion)

Problem 11: Design for a real application

Task: a registrar stores 5,000 student records with integer roll numbers. Choose a hash table. **Solution:** target $\alpha \approx 0.7$ for linear probing gives $m \geq 5000/0.7 = 7143$; the next prime is **7151** (actual $\alpha = 0.699$). Expected probes: successful $\frac{1}{2}(1 + 1/(1-\alpha)) = 2.16$, unsuccessful $\frac{1}{2}(1 + 1/(1-\alpha)^2) = 6.03$. With double hashing the unsuccessful cost would be about 3.32; with chaining at $m = 2503$ ($\alpha \approx 2$) about 2 nodes.

18.2 Practice Problems with Answers

#	Problem	Answer
P1	Linear probing, $m = 11$: insert 25, 36, 47, 14. Slots?	25→3, 36→4, 47→5, 14→6
P2	Double hashing, $m = 13$, $h_2 = 1 + k \bmod 12$: insert 27, 40, 53. Slots?	27→1, 40→6, 53→7
P3	Triangular probing, $m = 8$: insert 8, 16, 24. Slots?	8→0, 16→1, 24→3
P4	Chaining, $m = 5$ (head insertion): insert 12, 7, 22, 17. Contents of bucket 2?	17 → 22 → 7 → 12
P5	A table with $m = 16$ has 12 keys. Load factor? Safe for linear probing?	$\alpha = 0.75$; borderline, expected unsuccessful probes 8.5: resize
P6	Rehash a table of $m = 5$ holding 12 and 25 (slots 2 and 0) to $m = 11$. New slots?	12 → slot 1, 25 → slot 3 (inserted in old slot order: 25 first, then 12)
P7	Why does quadratic probing with $h + i^2$ and $m = 8$ fail to use all slots?	Squares mod 8 take only the values 0, 1, 4: at most 3 distinct slots
P8	Key -13, $m = 5$: index in C++ with `%`, and the safe index?	`-13 % 5 = -3` (bad); safe index ((-13 % 5) + 5) % 5 = 2

Check P1 against Figure 7.5-style reasoning: 14 has home 3 (taken by 25), 36 sits at 4, 47 at 5, so 14 goes to slot 6.

18.3 Case Studies

Each case study states the requirement, the design decisions and the trade-offs. The numbers use the formulas of Parts 5–10 and are estimates, not measurements.

Case 1: Student record lookup

Requirement	Look up a student by roll number among 5,000 records (read-heavy, rare deletions, keys are integers with patterns: year prefix + serial).
Design decisions	Key = roll number. Division hash with a **prime** $m = 7151$ (breaks the year-prefix pattern; $\alpha \approx 0.7$). Open addressing with double hashing, or `unordered_map<int, Student>` in C++. Reserve capacity once.
Trade-offs and pitfalls	Chaining would simplify deletions (graduating students). A tombstone-heavy table needs periodic rebuilds. Do not use k mod 1000 on roll numbers that end in the same digits.

Case 2: Inventory management

Requirement	Maintain stock levels for 50,000 product codes (strings like "ELX-1043-B"): frequent updates, occasional new and discontinued products.
Design decisions	Key = SKU string; polynomial or library string hash. `unordered_map<string, Item>` (chaining) with `reserve(60000)`; update through `find` or `insert_or_assign`.
Trade-offs and pitfalls	Pitfall: `operator[]` on a mistyped SKU silently creates an item with stock 0. Use `find` for reads. Iteration order is unspecified, so sort when printing reports. If a sorted report by SKU is the main use, a `std::map` might be better.

Case 3: Word-frequency counting

Requirement	Count word occurrences in a 10 MB document and print the top 20.
Design decisions	`unordered_map<string,int>`; `++cnt[word]` per token; then copy to a vector and sort or use a heap of size 20 (Question 15). Expected $O(n)$ for n tokens.
Trade-offs and pitfalls	Normalise words first (lower-case, strip punctuation) or "Hash" and "hash" are different keys; or use a case-insensitive hash and equality (Part 13). Memory is proportional to distinct words, not tokens.

Case 4: Caching frequently requested data

Requirement	A web service caches the results of 10,000 expensive queries; the same queries repeat.
Design decisions	Hash map from query string to result, plus a recency list (LRU, Question 13) to evict. Capacity fixed; load factor controlled by `reserve`. Thread safety needs locks or sharded tables.
Trade-offs and pitfalls	Without eviction memory grows forever. Cache keys must include **all** parameters that affect the result. A rehash causes a latency spike: reserve in advance.

Case 5: Duplicate detection

Requirement	Detect duplicate email addresses while importing 1,000,000 rows.
Design decisions	`unordered_set<string>` of normalised addresses; `insert(...).second == false` signals a duplicate (Question 3). For data larger than memory, hash each row to a 64-bit digest and store digests, or partition by hash and process each partition separately.
Trade-offs and pitfalls	Digest-only storage risks a false "duplicate" if two different addresses have equal 64-bit digests (very unlikely, but compare the strings if correctness is critical).

Case 6: Database-style key lookup

Requirement	An index on a customer id column must answer equality lookups (not ranges) on a table that grows continually and lives on disk.
Design decisions	A **hash index** with dynamic hashing (extendible or linear hashing, Part 12): buckets are disk pages that split one at a time; a lookup costs one or two page reads.
Trade-offs and pitfalls	A hash index cannot answer range queries ("ids between 100 and 200") or ordered scans: a B-tree is the right tool for those. Skewed keys create overflow pages.

Case 7: Distributed-system partitioning

Requirement	Spread 10 million keys over a changing set of cache servers; servers can crash or be added.
Design decisions	Consistent hashing with virtual nodes (Part 12, Question 19). With `hash mod N` going from 3 to 4 servers about 738 of 1000 sample keys move; with the ring only 235 of 1000 (about 1/4) move, all to the new server.
Trade-offs and pitfalls	Plain consistent hashing without virtual nodes balances load poorly; the hash function needs good mixing; replication and rebalancing policies are separate design questions.

PART 19

LAST-MINUTE REVISION SHEET

Definitions, formulas, comparisons, complexity, C++ functions, traps and a syllabus checklist (details are in Parts 1–18)

19.1 Essential Definitions

Term	One-line definition
Hashing	computing an array index from a key with a hash function so that items are found without searching
Hash table	array of m slots/buckets + hash function + collision method; stores key–value pairs
Hash value / index	number returned by h (any size) / the value reduced to $0 \dots m\text{-}1$ (`% m`)
Collision	two different keys, same index (unavoidable)
Duplicate key	the same key inserted again (policy: reject, update, store many)
Separate chaining	each bucket holds a list of its keys
Open addressing	all keys in the array; probe a sequence of slots
Probe sequence	$h(k,0), h(k,1), \dots, h(k,m\text{-}1)$
Tombstone	DELETED marker: skipped by search, reusable by insert
Clustering	primary: runs of occupied slots (linear); secondary: same home ⇒ same probe path
Load factor α	n/m; average chain length (chaining) or fraction filled (open addressing)
Rehashing	rebuilding with a new m by recomputing every index
Perfect hashing	no collisions for a fixed key set

19.2 Important Formulas

Hash-function formulas

Method	Formula
Division	$h(k) = k \bmod m$; safe: `((k % m) + m) % m`
Multiplication	$h(k) = \lfloor m \cdot (\text{kA} \bmod 1) \rfloor$, $A \approx 0.6180339887$; integer: `(k * 2654435769u) >> (32 - p)`
Mid-square	middle digits of k^2 (4-digit key: `(k*k / 1000) % 100`)
Folding	add the digit pieces (shift) or add with every second piece reversed (boundary), then mod m
Polynomial string hash	$h = (\dots((c_0 B + c_1) B + c_2) \dots) + c_{n-1}$; loop `h = h * B + c` (unsigned)
Composite key	`h = 31*h(x) + h(y)` or `hash_combine`; never plain sum/XOR
Universal (Carter–Wegman)	$h_{a,b}(k) = ((\text{ak} + b) \bmod p) \bmod m$

Collision-resolution formulas

Method	Probe $h(k, i)$	Notes
Linear	$(h(k) + i) \bmod m$	primary clustering
Quadratic	$(h(k) + c_1\, i + c_2\, i^2) \bmod m$	prime m, $h+i^2$: first $(m+1)/2$ probes distinct (guaranteed if $\alpha < ½$); $m = 2^p$ with $c_1 = c_2 = ½$ visits all slots
Double hashing	$(h_1(k) + i \cdot h_2(k)) \bmod m$	$h_2 \neq 0$, $\gcd(h_2, m) = 1$; e.g. m prime, $h_2 = 1 + k \bmod (m\text{-}1)$
Chaining	bucket index $h(k) \bmod m$ + list	any α

Load factor and expected cost

Quantity	Formula
Load factor	$\alpha = n/m$ (open addressing: effective load $(n$ + deleted$)/m)$
Chaining	unsuccessful $\approx \alpha$ nodes; successful $\approx 1 + \alpha/2$ nodes
Uniform / double hashing	unsuccessful $\approx 1/(1-\alpha)$; successful $\approx (1/\alpha)\ \ln(1/(1-\alpha))$
Linear probing	unsuccessful $\approx$ ½$[1 + 1/(1-\alpha)^2]$; successful $\approx$ ½$[1 + 1/(1-\alpha)]$
Collisions	expected colliding pairs $n(n-1)/(2m)$; first collision after about $1.18\sqrt{m}$ keys
Coverage of a step d	visits m / gcd(d, m) slots
Amortised rehash	doubling: total moves $< 2n \Rightarrow O(1)$ per insertion

19.3 Rehashing Workflow

1. Before insertion check: (live keys + tombstones + 1) / $m > \alpha_{max}$?
2. Choose the new capacity: about $2m$, rounded up to a **prime** (growth by a factor, never by a constant).
3. Allocate a **fresh** array of EMPTY slots; set $m \leftarrow m_{new}$.
4. For every OCCUPIED old slot (or chain node): recompute the index **with the new** m and insert (probe/link). Tombstones are dropped.
5. Release the old array. Then insert the pending key. (Shrink only when α is far below the grow threshold.)

19.4 Comparison of Collision-resolution Techniques

	Chaining	Linear	Quadratic	Double hashing
Keys stored in	lists	array	array	array
Max α	> 1 ok	□ 0.7	< 0.5 (guarantee)	□ 0.8
Clustering	none	primary	secondary	minimal
Delete	unlink	tombstone	tombstone	tombstone
Cache	poor	best	medium	poor
Weak point	memory, pointers	clusters	may miss slots	two good hashes, prime m

19.5 Complexity Table

Operation	Average (expected)	Worst case	Remark
Insert / search / delete (chaining)	$O(1 + \alpha)$	$O(n)$	expected needs uniform hashing, bounded α
Insert / search / delete (open addressing)	$O(1)$ for $\alpha < 1$ fixed	$O(n)$	cost ~ $1/(1-\alpha)$
Insert with occasional resize	$O(1)$ amortised	$O(n)$ single insert	doubling
Rehash	$O(m_{old} + n)$	$O(n^2)$ if all collide	
Cuckoo / perfect hashing lookup	$O(1)$	$O(1)$	static or $\alpha <$ ½
Space	$O(m + n)$ chaining, $O(m)$ open addr.		

19.6 Important C++ Library Functions

Function	Purpose / trap
`insert`, `emplace`	add if absent; **no overwrite**; returns `pair<it, bool>`
`operator[]`	insert-or-access; **inserts a default value for a missing key**
`insert_or_assign`, `try_emplace` (C++17)	overwrite / insert only if missing
`at`, `find`, `count`, `contains` (C++20)	lookup without inserting (`at` throws)
`erase(key)`, `erase(it)`	remove; `it = m.erase(it)` inside loops
`bucket_count`, `load_factor`, `max_load_factor`	inspect or tune crowding (default max 1.0)
`reserve(n)`, `rehash(n)`	pre-size to avoid rehash; both rehash all elements
`bucket(k)`, `bucket_size(b)`	see collisions
`std::hash<T>`	unspecified algorithm; equal keys ⇒ equal hash; no hash for `pair`/`tuple`

19.7 Common Mistakes and Exam Traps

- `%` of a negative key is negative: use `((k % m) + m) % m`.
- Hash value ≠ index: always reduce modulo m.
- Never set a deleted slot to EMPTY in open addressing; use a tombstone.
- Search stops only at EMPTY (not at DELETED).
- Insert: check for a duplicate along the whole probe path **before** reusing a tombstone.
- Bound every probe loop by m.
- $h_2(k)$ must be non-zero and coprime to m.
- Quadratic probing may fail with empty slots left; it is guaranteed only for suitable m and load.
- Resizing = recompute **every** index with the new m; grow by a factor.
- $\mathrm{O}(1)$ is expected/amortised, not guaranteed; worst case $\mathrm{O}(n)$.
- Cryptographic hash ≠ table hash; never use a table hash for passwords.
- Equal keys must have equal hashes; use the same members in `==` and in the hash.

19.8 Frequently Confused Concepts

Pair	Difference
Collision vs duplicate key	different keys, same slot / the same key again
Collision resolution vs rehashing	one key's placement / rebuilding the whole table
Primary vs secondary clustering	runs of adjacent slots / identical probe paths for equal home slots
Open hashing vs open addressing	open hashing = chaining (outside the table); open addressing = inside the table
Load factor vs fill percentage	chaining load can exceed 1; open addressing cannot
Average vs amortised vs worst case	over a probability model / over a sequence of operations / over all inputs
Hash function vs hash table	the index-computing rule / the whole data structure
`unordered_map` vs `map`	hash table, unordered, expected $\mathrm{O}(1)$ / balanced tree, sorted, $\mathrm{O}(\log n)$
Checksum vs cryptographic hash	detects accidents / resists attackers

19.9 High-Value Viva Facts

- Collisions begin after about $\sqrt{m}$ keys, not m.
- Prime $m + h_2 = 1 + k \bmod (m\text{-}1)$ is the textbook double-hashing setup.
- A probe sequence must be a permutation of all slots to guarantee finding a free one.
- Linear probing: best cache behaviour, worst clustering; $\tfrac{1}{2}[1+1/(1-\alpha)^2]$ unsuccessful probes.
- Doubling the table makes insertion amortised $O(1)$ because the rehash costs form a geometric series.
- Chaining deletion is easy; open-addressing deletion needs tombstones.
- Consistent hashing moves only about $1/N$ of the keys when a server joins; `hash mod N` moves most.
- Perfect hashing needs a fixed key set; cuckoo hashing gives 2-probe lookups.
- `m[k]` on a missing key inserts it; rehash invalidates iterators but not references.

19.10 Syllabus Checklist

✓	Topic	Where
✓	Introduction, need, direct addressing, dictionaries, applications	Part 1
✓	Hash table internals, slot states, static vs dynamic, key types	Part 2
✓	Division, multiplication, mid-square, folding, digit extraction, truncation	3.2–3.7
✓	String and composite hashing, negative keys, table size, primes, poor functions	3.8–3.17
✓	Collisions, birthday paradox, clustering, duplicates, strategy map	Part 4
✓	Separate chaining: algorithms, C++ with pointers and STL, analysis	Part 5
✓	Open addressing fundamentals and tombstones	Part 6
✓	Linear probing	Part 7
✓	Quadratic probing (coverage and failure)	Part 8
✓	Double hashing (h_2 rules)	Part 9
✓	Load factor, resizing, rehashing, amortised analysis	Part 10
✓	Comparison of the four methods	Part 11
✓	Advanced: universal, perfect, extendible, linear, cuckoo, Robin Hood, hopscotch, coalesced, consistent, crypto, checksums, trees, tries	Part 12
✓	C++ implementation and STL library, custom hashes, invalidation, debugging	Part 13
✓	Complexity and performance	Part 14
✓	Seven complete programs	Part 15
✓	Viva questions	Part 16
✓	Coding questions	Part 17
✓	Numerical problems and case studies	Part 18

End of the chapter. Keep the tables of Sections 19.2 and 19.5 and the list of exam traps for the last hour before an examination.